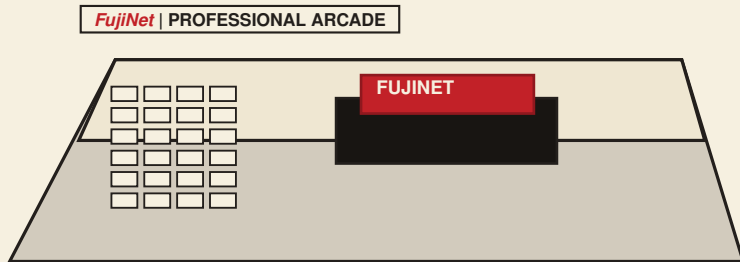


FujiNet[®] PROFESSIONAL ARCADE



PROGRAMMERS HANDBOOK

CONGRATULATIONS

The FujiNet cartridge is the first module of one of the most exciting new concepts in Arcade technology. Your Professional Arcade now reaches WiFi networks, disk hosts, web servers, and game lobbies the world over — and this handbook will teach you to program all of it, in Z80 assembly language and in C.

Congratulations, you have purchased the finest system available.

This handbook is a community work of the FujiNet Project, typeset in the style of the 1977 *Bally Home Library Computer Owners Manual* as a loving tribute. Bally, Astrocade, and Bally Professional Arcade are trademarks of their respective holders; no affiliation with or endorsement by any of them is implied. The Astrocade name is used throughout in its community-adopted sense for the Bally Arcade/Astrocade console family. Every address, opcode and listing is transcribed from the live FujiNet project sources and verified against the firmware; the sources-of-truth table with commit hashes is in the repository README.

TABLE OF CONTENTS

Page

WELCOME TO FUJINET.....	1	NETCAT.....	52
YOUR FUJINET CARTRIDGE.....	2	TEXAS HOLD'EM.....	54
THE MAILBOX.....	5	BATTLESHIP.....	57
FIRST CONTACT.....	12	FUJITZEE.....	59
THE Z80 LIBRARY.....	15	CONFIG.....	61
C ON THE PROFESSIONAL ARCADE.....	18	ALWAYS MORE TO COME.....	62
THE NETWORK DEVICE.....	22	APPENDIX A: MAILBOX QUICK REFERENCE.....	63
THE FUJI DEVICE.....	27	APPENDIX B: ERROR AND STATUS CODES.....	64
BOOT, SWAP, AND BANKING.....	29	APPENDIX C: PROGRAM LISTINGS.....	65
COMMAND REFERENCE: THE FUJI DEVICE.....	32	APPENDIX D: THE C DEMO.....	182
COMMAND REFERENCE: THE NETWORK DEVICE.....	46		

WELCOME TO FUJINET

Your Bally Professional Arcade — the Astrocade, as its friends call it — was sold in 1977 as “the exclusive computer video system that grows as you grow.” It has taken the world a few decades to deliver on that promise, but here we are: plug the FujiNet cartridge into the slot under the storage-compartment door, and your Arcade is on the network. WiFi. Web servers. Disk hosts. Telnet bulletin boards. Multiplayer game lobbies. All of it reachable from 1.789 megahertz of Z80.

FujiNet is a family of network adapters for classic computers and consoles. On every platform it presents the same two personalities: the *Fuji device*, which manages the adapter itself (WiFi, host slots, directories, mounting), and the *network device* — the famous **N**: — which opens connections to the outside world by URL. What differs per platform is only the plumbing between the machine and the adapter. On the Astrocade, that plumbing is the most delightfully strange in the whole family, and it is the subject of this book.

What this handbook teaches

You will learn, in order: how the FujiNet cartridge works and why the Astrocade edition is special (a cartridge port that cannot write!); the *mailbox* — the in-memory protocol every program uses to talk to the cartridge; how to drive it from Z80 assembly language with `fujilib.inc`, and from C with the Z88DK compiler; the network device and the Fuji device in working detail; a complete reference for every command the FujiNet answers on this platform; and then five real, shipping programs, explained end to end, with their complete listings in the appendix: **NETCAT**, **TEXAS HOLD'EM**, **BATTLESHIP**, **FUJITZEE**, and **CONFIG**.

What you will need

1. A Bally Arcade/Astrocade — or MAME's `astrocde` driver with the FujiNet cartridge device grafted in (the firmware tree's `pico/astrocade/emu/apply.sh` does the grafting).
2. A FujiNet cartridge — or, under MAME, a running *fujinet-pc* with its bus-over-IP listener on `127.0.0.1:9995`.
3. An assembler (the clients use *zmac 1.3*) and, for the C chapters, a *z88dk* checkout.
4. This handbook, a cup of coffee, and a television set. The automatic TV protection circuit will forgive long debugging sessions.

NOTE: Everything network-side — the 28 URL schemes the **N**: device speaks, from **TCP**: and **TELNET**: to **HTTPS**:, **JSON**:, **SSH**: and **GMAIL**: — is a book of its own: *The FujiNet Network Protocol Handbook*, from this same series. This handbook teaches you to reach the **N**: device; that one teaches you everything it can say once reached.

YOUR FUJINET CARTRIDGE

The Astrocade cartridge port is the most asymmetric in the FujiNet family. It carries thirteen address lines (A0–A12), eight data lines (D0–D7), one pre-decoded *Enable* that asserts for **reads** in the 2000H–3FFFH window, and power. That is the whole interface.

Count what is missing: no */RD*. No */WR*. No */IORQ*. No clock, no reset line. When your program stores a byte anywhere in 0000H–3FFFH, the console’s magic function generator eats the write — the cartridge never sees it. A cartridge on this port is, electrically, a read-only answering machine.

So how can a cartridge possibly be a *network adapter*? The FujiNet cartridge answers with a trick the Astrocade community has used for decades — the 512K homebrew mapper selects banks with reads at 3F80H–3FFFH, and AstroBASIC toggled its tape relay the same way:

1. The console talks by reading. Certain pages of the cartridge window are *hotspots*. When the console reads an address inside one, the cartridge does not care what byte it returns — it cares which address was read. **The low eight address bits are the payload.** Reading 3F41H “sends” the byte 41H.

2. The cartridge talks by repainting. The cartridge serves its window from RAM on its own processor. To answer, it simply changes the bytes it serves; by the time the console reads them, they are just ROM.

The tandem inside the cartridge

Two processors share the cartridge shell. An **RP2040** sits on the cartridge edge, serves the 8K window at Z80 bus speed, decodes the hotspot reads, and repaints the reply bytes. It speaks to an **ESP32-S3** — a stock FujiNet running the RS232 firmware build — over a

USB cable, using the same SLIP-framed FujiBus packets that FujiNet speaks over a serial port elsewhere. The ESP32-S3 is the actual network adapter: WiFi, TNFS, HTTP, TLS, the lot. The RP2040 is its translator into read-only cartridge language.

Under MAME there is no silicon, but the shape survives: the MAME cartridge device compiles the very same mailbox sources, and forwards the FujiBus packets over TCP (*bus-over-IP*) to a *fujinet-pc* process on your desk.



Figure 1 The three-tier stack. The mailbox is the only part your program ever sees.

The 8K window

The cartridge window is 8,192 bytes at console addresses 2000H–3FFFFH. Your program does not get all of it. The mailbox owns the top 1,280 bytes; the contract, from `fuji_mailbox.h` (the single source of truth, compiled into the RP2040 firmware and the MAME model alike):

1. Program code and data stay **below 3B00H** — that is 6,912 bytes for your program, cart offsets 0000H–1AFFH.
2. 3B00H–3CFFH is the **reply window** the cartridge repaints.
3. 3D00H–3FFFFH are the **hotspot pages** the console reads to talk.
4. At cart offset 1CFCH the image carries the four bytes `FUJI` — the *claim signature*, a promise that the image keeps 1B00H–1FFFFH free so the mailbox may stay alive after the image boots. An ordinary game (no claim) uses that space for its own bytes, and booting one shuts the mailbox down for the session.

A cartridge program, in brief

An Astrocade cartridge begins with the seven-byte on-board-menu header: the 55H sentinel, a chain word to `MENUST`, a pointer to the program's name, and a pointer to its entry. Power the console on and your program is one keypress away, listed on the SELECT GAME screen like it was 1978.

THE CARTRIDGE HEADER, AS EVERY CLIENT IN THIS BOOK WRITES IT

```
ORG    FIRSTC      ; 2000H
DB     55H          ; menued-cartridge sentinel
DW     MENUST       ; chain to the on-board SELECT GAME list
DW     PRGNAM
DW     PRGSTR
PRGNAM: DB "NETCAT"
DB     0
PRGSTR: DI          ; interrupts off, for the program's life
LD     SP,STACK     ; stack in high screen RAM
```

CAUTION

Programs that use the mailbox run with interrupts **disabled** — **DI** at entry, and no **EI**, ever. With interrupts off the Z80's I register keeps its reset value of zero, refresh cycles land harmlessly in on-board ROM space, and the refresh-stray hazard described in the next chapter never arises. Poll the keypad and handles directly from ports 10H–17H; the on-board SENTRY system needs interrupts you cannot give it.

THE MAILBOX

This chapter is the heart of the handbook. The mailbox is a small, strict, and rather beautiful protocol; learn its five moves and every FujiNet feature on this console is yours. All addresses below are **console addresses** — cart offset plus 2000H — exactly as your program uses them.

The two directions

Console to cartridge: hotspot reads. Three pages carry everything you will ever say. A read of `3D00H+r` *arms* mailbox register *r*. A read of `3E00H+v` delivers the value *v* to the armed register — one register write is therefore a *pair* of reads, back to back. A read of `3F00H+v` appends the byte *v* to the *TX stream*, the outgoing parameter-and-payload buffer.

Cartridge to console: repainted ROM. The reply window at 3B00H–3C0CH is plain memory you read whenever you like. 3B00H–3BFFH is a 256-byte *reply slice* 3C00H–3C0CH are thirteen status bytes, listed in Figure 3.

Instruction fetches are harmless. Your code executes from 2000H–3AFFH, far from the hotspot pages, so the thousands of reads the Z80 performs just running your program never trip anything.

Console address	Name	What lives there
2000H-3AFFH	—	your program: 6,912 bytes of code and data
3B00H-3BFFH	FNRDATA	reply slice: 256 bytes of the current reply
3C00H-3C0CH	FNACKSQ. . FNSECHO	thirteen painted status bytes
3CFCH-3CFFH	claim	FUJI signature (cart offset 1CFCH)
3D00H-3D7FH	FNREGSEL	hotspot: read +r arms register r
3D80H-3DEFH	FNBSSEL	hotspot: read +page selects a bank (v2)
3DFEH	FN_SWAP	hotspot: serve the staged boot image (armed only)
3E00H-3EFFH	FNREGDAT	hotspot: read +v delivers v to the armed register
3F00H-3FFFH	FNDATA	hotspot: read +v appends v to the TX stream

Figure 2 The cartridge window, as the console sees it.

Address	Name	Meaning
3C00H	FNACKSQ	echoes your SEQ when the reply is ready
3C01H	FNSTATS	bit 0 link up, bit 1 busy
3C02H	FNERR	transport verdict of the last transaction (Appendix B)
3C03H	FNREPLY	06H ACK or 15H NAK
3C04H	FNRXLL	reply length, low byte
3C05H	FNRXLH	reply length, high byte
3C06H	FNBSTAT	boot state: idle 0, transfer 1, ready 2, failed 80H
3C07H	FNBPCT	boot progress, 0–100
3C08H	FNBERR	boot error (Appendix B)
3C09H	FNMAFG	the letter F — cartridge presence check
3C0AH	FNMAFN	the letter N
3C0BH	FNPVER	mailbox protocol version (2 = banking)
3C0CH	FNSECHO	echoes the slice number, painted <i>last</i>

Figure 3 The thirteen painted status bytes.

The registers

Register numbers 00H–0FH mirror the FujiNet Odyssey² mailbox where the meanings match; 10H and up are Astrocade-specific.

00H DEVICE.FujiBus device id: 70H Fuji, 71H network
 01H COMMAND.FujiBus command id
 02H NPARAM.how many parameters lead the TX stream
 05H DATA RST.any value: rewind the TX write pointer
 06H RXSLICE.which 256-byte reply slice FNRDATA shows
 10H SEQ.nonzero and ≠ last ACK: launch the transaction
 11H BOOTLOCK.magic 0B5H arms the ROM swap
 12H/13H BOOTSEL. . . .magic 0B5H then 4AH: reboot cart to UF2 mode

The TX stream

Every transaction's outgoing bytes form one stream, at most **320 bytes**, with a fixed grammar: first the parameters — NPARAM of them, each a *size byte* (1, 2 or 4) followed by that many value bytes, little-endian — then the raw payload, whose length is simply whatever remains.

01	0C	01	00	N : H T T P S : / / ...
----	----	----	----	-------------------------

Figure 4 The TX stream for `NET_OPEN` with two one-byte parameters and a `devicespec` payload.

One whole transaction

Every exchange with the FujiNet — every one, from a WiFi scan to an HTTP GET — is the same five moves:

- 1. Begin.** Rewind the TX stream (register 05H), then set `DEVICE`, `COMMAND`, and `NPARAM`.

- 2. Stream.** Append the parameters and payload with `3F00H+v` reads.

- 3. Commit.** Read `FNACKSQ`, add one (wrapping 255 to 1 — zero is reserved), and write the result to the `SEQ` register. The cartridge sees a fresh sequence number and fires the whole stream at the ESP32-S3 as one FujiBus packet.

- 4. Wait.** Poll `FNACKSQ` until it equals the number you wrote. When it does, the reply is painted: check `FNERR` (the transport verdict), `FNREPLY` (ACK or NAK), and capture the reply length from `FNRXLL`/`FNRXLH` *immediately* — those bytes describe the most recent transaction only.

- 5. Read.** The first 256 reply bytes are already in `FNRDATA` (the cartridge republishes slice 0 after every transaction). For a longer reply, write 1–3 to the `RXSLICE` register and poll `FNSECHO` until it echoes your slice number — the cartridge paints that echo *last*, so when it matches, the slice is whole. Replies run to 1,024 bytes: four slices.

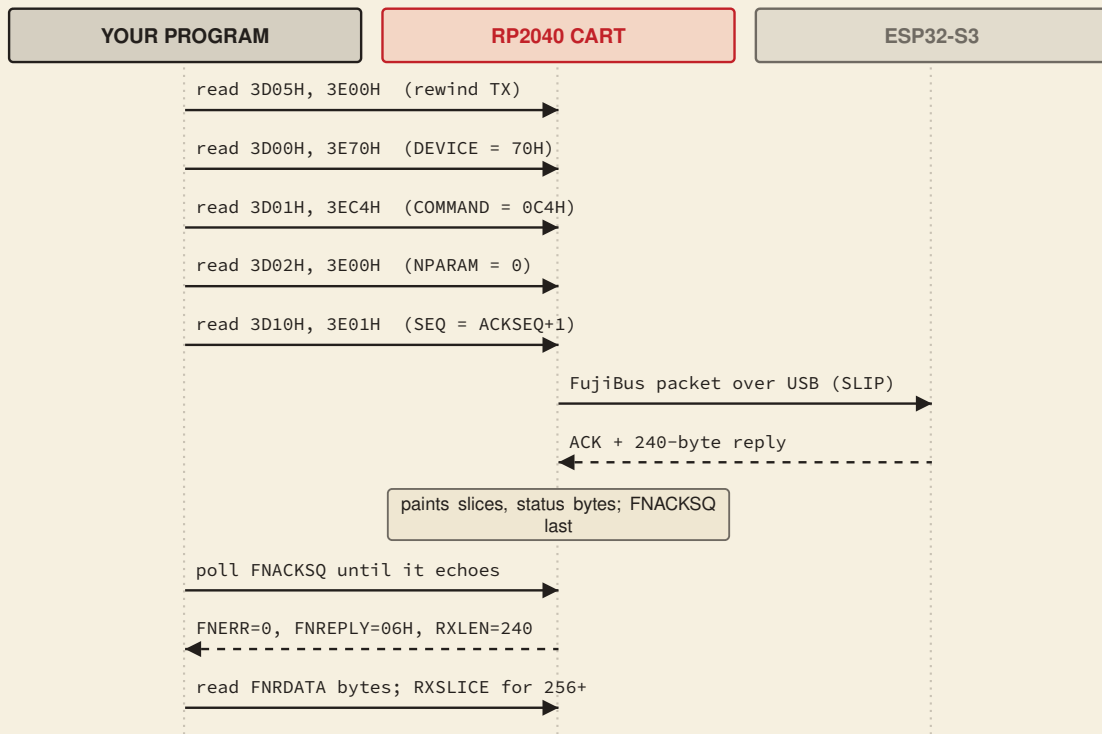


Figure 5 A `GET_ADAPTERCONFIG_EXTENDED` transaction, end to end.

The sequence rule

Why derive SEQ from the cartridge's own `FNACKSQ` instead of counting locally? Because the RESET button restarts *your program* — but not the cartridge. The cart edge carries no reset line. A program that kept its own counter would, after a RESET, re-send sequence number 1; the cartridge, having already acknowledged a 1 in this session, would ignore it forever. Read the cart's persisted ACKSEQ, add one, wrap 255 to 1, and RESET simply cannot desynchronize you. This rule is inherited from hard experience on the Intellivision and Odyssey² bring-ups; honor it.

Stray reads, and the defenses against them

On a port where reading is writing, the design hazard is the read you did not mean to make. The Z80's refresh machinery performs a memory read every instruction, with the I register on A8–A15 — if I ever pointed at 3DH–3FH while the console's Enable decode passed refresh cycles, your program would spray phantom hotspot reads. The defenses, in depth:

1. A REGDATA read with no immediately-preceding REGSEL read is a **no-op** — arming disarms after one use, so an isolated stray mutates nothing.

2. A transaction launches only on a SEQ value that is nonzero *and* differs from the last acknowledged one.

3. The ROM-swap trigger lives at offset 0FEH of the REGSEL page — bit 7 set, where the R register (which only counts through bit 6) can never wander — and fires only after BOOTLOCK armed it. Rebooting the cart to UF2 mode takes *two* full register writes with two different magics.

4. Your own program runs `DI` with `I = 0`, so refresh lands in on-board ROM and the whole question is moot.

Timeouts

The library waits in quanta of 65,536 polls of `FNACKSQ` — about two seconds each at 1.789 MHz. How many quanta to allow is a judgment the shipping clients have already made for you:

Operation	Quanta	Why
NET_OPEN	8	16 s: TLS handshakes and telnet negotiation are slow
NET_READ	8	16 s: a deferred HTTP GET performs the fetch here
NET_STATUS	4	8 s
NET_CLOSE	3	6 s
Fuji device commands	8	16 s (CONFIG's <code>CGO</code> helper)
MOUNT_IMAGE	60	2 min: the ROM push rides inside it, progress on <code>FNBPC</code>
slice select	1	a live cart repaints in microseconds

Figure 6 Timeout quanta used by the shipping clients.

CAUTION

The reply window is **never cleared** between transactions. A short reply leaves the stale tail of a longer, older one lying in `FNRDATA` after it. Capture `FNRXLL/FNRXLH` the moment a transaction acknowledges, believe only that many bytes, and validate what they claim to be before acting on them. Chapter “The Network Device” elevates this to a rule with the games’ `VALID8` idiom.

FIRST CONTACT

Enough theory — let us shake hands with the cartridge. The smallest useful FujiNet program does three things: proves the cartridge is there, runs one transaction, and shows what came back. This chapter builds it twice — once in Z80 assembly, once in C — and every program in this book, games included, is this program with a bigger middle.

Is anybody home?

The cartridge paints the letters **F** and **N** at 3C09H and 3C0AH. If they are not both there, you are running on a bare console — or a different cartridge — and must not touch the hotspot pages. Every client checks this first, and so should you.

One transaction, no parameters

`GET_ADAPTERCONFIG_EXTENDED` (command 0C4H to device 70H) is the perfect first transaction: it takes no parameters and no payload, works even before WiFi is configured, and returns a 240-byte structure whose best fields sit conveniently in slice 0: the SSID at offset 0, the firmware version string at offset 125, and the IP address, already in dotted-decimal text, at offset 140.

FIRST CONTACT, IN Z80 (AFTER FUJILIB.INC; THE SHAPE OF PICO/ASTROCADE/TESTROM/FUJITEST.ASM)

```
CALL    FNCHECK      ; Z set if 'F','N' painted
JP      NZ,NOCARD

LD      A,FNDFUJI     ; device 70H
LD      E,FCACFGX     ; GET_ADAPTERCONFIG_EXTENDED, 0C4H
LD      L,0           ; no parameters, no payload
CALL    FNBEGIN
LD      B,8           ; ~16 s: covers ESP32 enumeration
CALL    FNCOMMIT
JR      C,TMOUT       ; carry: the cart never answered
OR      A
JR      NZ,LERR       ; FNERR nonzero: no link, timeout...

; Slice 0 already shows the reply: ssid at +0, fn_version at +125,
; sLocalIP at +140 (AdapterConfigExtended).
LD      HL,FNRDATA+0   ; the SSID, NUL-terminated, 32 max
LD      B,32
CALL    SHOWSTR       ; your favorite string printer
```

THE SAME CONTACT, IN C (THE C-DEMO OF CHAPTER 6, VERBATIM)

```
if (!fn_check()) {
    txt_puts(0, 2, "NO FUJINET CART");
    for (;;) ;
}

fn_begin(FN_DEV_FUJI, FUJI_GET_ADAPTERCONFIG_EXTENDED, 0);
r = fn_commit(8);
rx_home();
if (r != FN_ERR_OK) { txt_puts(0, 3, "ERR"); for (;;) ; }
if (FN_REPLY != FN_ACK) { txt_puts(0, 3, "NAK"); for (;;) ; }

rx_strn(buf, 0, 20);      /* ssid[33] at offset 0      */
txt_puts(0, 4, buf);
rx_strn(buf, 125, 15);    /* fn_version[15] at 125    */
txt_puts(3, 5, buf);
rx_strn(buf, 140, 16);    /* sLocalIP[16] at 140      */
txt_puts(3, 6, buf);
```

What success looks like

Build either version, boot it, and press **1** at the SELECT GAME menu. Against a live FujiNet (or fujinet-pc under MAME) the screen answers in about a second:

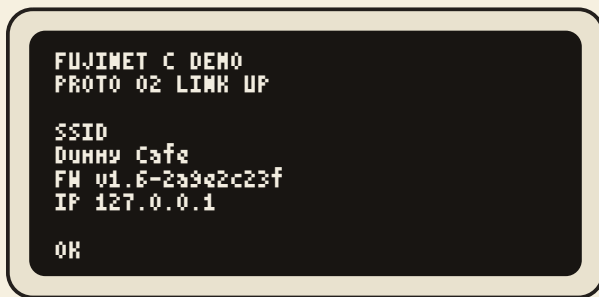


Figure 7 The C demo's first contact, exactly as captured from MAME against a live fujinet-pc.

Read the error paths as carefully as the happy one. **Carry set** from **FNCOMMIT** means the cartridge itself never acknowledged — almost always “no cartridge” or “the RP2040 never enumerated its ESP32.” A **nonzero FNERR** means the cartridge answered but the transport beneath it failed: **1** no link, **2** timeout, **3** bad frame, **4** too big. A **NAK in FNREPLY** means everything carried perfectly and the FujiNet itself refused the command — wrong parameters, an unmounted host, a file that is not there.

Three different layers, three different bytes, three different remedies. Programs that collapse them into one “error” spend their debugging sessions guessing; the shipping clients never do, and Chapter “The Network Device” shows the discipline they use instead.

THE Z80 LIBRARY

All five programs in this book share one transport file, `fujilib.inc` — 223 lines, byte-identical in every repository, and printed in full in Appendix C. This chapter walks its eight routines; know them and you know the mailbox by heart. The library hand-mirrors `fuji_mailbox.h` and is vendored per-program on purpose: a client builds with no particular firmware branch checked out.

FNREGWR — one register write

The atom everything else is made of: C names the register, A the value, and the routine issues the REGSEL/REGDATA read pair back to back. Note the idiom — the “read” whose result is thrown away is the message:

FNREGWR, FUJILIB.INC

```
; Write mailbox register: C = register number, A = value.
; Clobbers A,B,D,E.
FNREGWR:
    LD     B,A           ; hold the value
    LD     D,FNRSELP     ; 3DH: the REGSEL page
    LD     E,C
    LD     A,(DE)        ; arm the register
    LD     D,FNRDATP     ; 3EH: the REGDATA page
    LD     E,B
    LD     A,(DE)        ; deliver the value
    RET
```

`FNRSELP/FNRDATP/FNDATAP` hold the page bytes 3DH/3EH/3FH separately because `zmac 1.3` has no `HIGH` operator. Nothing requires the two reads to be adjacent instructions — ordinary instruction fetches in between are invisible to the hotspot decode — but the library keeps them adjacent as a matter of hygiene.

FNTXBYT, FNPARB, FNTXSTR — feeding the stream

`FNTXBYT` appends the byte in A with a single `3F00H+v` read. `FNPARB` wraps it for the parameter grammar: size byte 1, then the value. `FNTXSTR` streams a NUL-terminated string — the NUL itself is *not* sent; string payloads carry their length by the packet's edge, not by a terminator. A two-byte parameter is three `FNTXBYT` calls by hand: size byte 2, low, high. (`CONFIG` wraps that as `FNPARW` in its `fujicmd.inc`.)

FNBEGIN — opening the envelope

A = device, E = command, L = parameter count; the routine rewinds the TX pointer (register 05H) and writes DEVICE, COMMAND, NPARAM. After **FNBEGIN** returns, the stream is empty and waiting.

FNCOMMIT — the moment of truth

B = timeout in quanta of 65,536 polls (about 2 s each). It computes the sequence rule from Chapter “The Mailbox” — ACKSEQ + 1, wrap 255 to 1 — writes SEQ, and polls. On success it returns with A = **FNERR** and flags set from **OR A** (so **JR NZ,error** reads naturally); on timeout it returns with carry set.

FNCOMMIT, FUJILIB.INC -- THE SEQUENCE RULE IN THE FLESH

```
FNCOMMIT:
    LD     A,(FNACKSQ)
    INC    A
    JR     NZ,FNCMS
    INC    A                ; 255 wraps past the reserved 0
FNCMS:   LD     H,A          ; expected ACKSEQ
    LD     C,FRSEQ
    CALL  FNREGWR
FNCMO:   LD     DE,0         ; inner: 65536 polls, ~2 s at 1.789 MHz
FNCMI:   LD     A,(FNACKSQ)
    CP     H
    JR     Z,FNCMOK
    DEC    DE
    LD     A,D
    OR     E
    JR     NZ,FNCMI
    DJNZ   FNCMO
    SCF                    ; timed out
    RET
FNCMOK:  LD     A,(FNERR)
    OR     A                ; clears carry
    RET
```

FNSLICE — turning the reply window

A = slice 0–3. Writes the RXSLICE register, then polls `FNSECHO` until it echoes the slice number. The echo byte is painted *last* in every repaint, which is the whole reason it exists: on real hardware the RP2040 repaints asynchronously while the Z80 keeps running, and a client that read the slice without polling would race the paint. (The Odyssey² mailbox lacks this byte; its emulator model admits it cannot catch a client that forgets. Here, forgetting is impossible — there is nothing else to wait on.)

FNCHECK — is anybody home?

Compares 3C09H/3C0AH against `F,N`; returns with Z set when the cartridge is present. Call it once at boot, before any hotspot is touched.

RXGETB, RXGETW, RXSTRN — the flat window

`state.inc` (54 lines, shared by netcat and all three games) folds the four slices into one flat 0–1023 address space: `RXGETB` takes an offset in HL, switches slices only when the high byte changes (a one-byte cache, `CURSLC`), and returns the byte. `RXGETW` reads a little-endian word; `RXSTRN` copies a bounded, NUL-stopping string. One rule rides with the cache: **every transaction republishes slice 0**, so the round-trip code re-homes `CURSLC` to 0 after each commit.

Building an 8K client

The build is `zmac` plus forty lines of shell, and the layout contract is enforced, not hoped for: assemble, fail if the image crosses 1B00H, pad to exactly 8,192 bytes, stamp `FUJI` at 1CFCH, then run `checkrom.py` (size, sentinel, claim, and not one stray byte in the mailbox pages). NETCAT adds `checksize.py`, which reads the assembler listing and prints a per-module budget table — luxury living on a 6,912-byte budget.

C ON THE PROFESSIONAL ARCADE

Every shipping Astrocade client is assembly — the fujinet-lib C library has no Astrocade port, and the cross-platform game clients' shared C core does not fit this machine. But the mailbox itself is nothing but memory reads, and memory reads are something C does perfectly well. This chapter builds a complete, working C client with *z88dk*: the `c-demo` project, shipped whole in `listings/c-demo/` and printed in Appendix D. It compiles, boots, and talks — the screen in Figure 5 is its output, captured from MAME against a live fujinet-pc.

The target, honestly stated

z88dk ships an `astrocde` target — and marks it *incomplete*. What exists: a correct `crt0` (ORG 2000H, the 55H menu header, entry glue) and the HVGLIB equates. What is missing: the target C library, including the console driver the classic runtime insists every target provide. The gap is smaller than it sounds. Three additions make the target usable for mailbox work:

1. a one-line `fputc_cons_native` stub (the demo paints screen RAM itself);
2. a tiny `port_out` in assembly — *sccz80* has no port intrinsics here, and the screen needs five OUTs at boot;
3. pragmas that place things: BSS above the visible screen, the stack below the BIOS cells, and `DI` first thing.

THE WHOLE BUILD RECIPE (C-DEMO BUILD.SH)

```
zcc +astrocde main.c text.c fujinet.c support.asm \  
-o build/cdemo.raw -m \  
-pragma-define:CRT_ENABLE_EIDI=1 \  
-pragma-define:REGISTER_SP=0x4fc0 \  
-pragma-define:CRT_ORG_BSS=0x4d00 \  
-pragma-define:CRT_MODEL=1  
# then: pad to 8192, stamp FUJI at 0x1CFC, checkrom.py -- same as asm
```

`CRT_ENABLE_EIDI=1` makes the `crt0` execute `DI` before `main` — the mailbox contract from Chapter “Your FujiNet Cartridge.” `CRT_MODEL=1` selects the ROM model, so initialized data is copied from the cartridge image into RAM at startup. The demo compiles to about 2.7K of the 6,912-byte budget.

Reading as writing, in C

The C rendering of `FNREGWR` needs one idiom: the hotspot “write” is a read whose *address* matters and whose result does not. Assigning a *volatile* read to a *volatile* sink guarantees the compiler emits the load, never elides it, and never reorders it:

FUJINET.C -- THE TRANSPORT ATOMS

```
static volatile unsigned char fn_sink;

#define FN_HOT(page, v) (fn_sink = (page)[v])

void fn_regwr(unsigned char reg, unsigned char val)
{
    FN_HOT(FN_REGSEL, reg);    /* arm the register    */
    FN_HOT(FN_REGDATA, val);   /* deliver the value */
}

void fn_tx_byte(unsigned char v)
{
    FN_HOT(FN_DATA, v);
}
```

FUJINET.C -- FN_COMMIT, THE SEQUENCE RULE AGAIN

```
unsigned char fn_commit(unsigned char quanta)
{
    unsigned char want;
    unsigned int spin;

    want = FN_ACKSEQ + 1;
    if (want == 0)
        want = 1;
    fn_regwr(FN_REG_SEQ, want);
    while (quanta--) {
        spin = 0;
        do {
            if (FN_ACKSEQ == want)
                return FN_ERRBYTE;
        } while (--spin != 0); /* 65,536 polls per quantum */
    }
    return FN_ERR_CLIENT_TIMEOUT; /* 0xFF */
}
```

CAUTION

A compiler hazard, found while verifying this very chapter: sccz80 (z88dk 3bd06cad, July 2026) silently **miscompiles a constant subscript on a cast-constant pointer**. `FN_RDATA[2]` compiles to the constant 2 — not a memory read — with only a “value out of range” warning as the tell. A *variable* subscript compiles correctly, and so does a dereference of a constant sum. The demo’s `fujinet.h` therefore defines `FN_RDATA_B(i)` as `*(volatile unsigned char *) (0x3b00 + (i))` and uses it for every constant offset. Treat that warning as an error whenever you see it.

The rest of the demo

`fujinet.c` continues exactly along `fujilib.inc`’s lines — `fn_begin`, `fn_slice`, the flat-window readers `rx_getb/rx_getw/ rx_strn` with the same slice cache and the same re-home-after-commit rule — and then the N: round trip in netcat’s shape: `net_open`, `net_close`, `net_status` (filling `net_avail`, `net_connected`, `net_devstatus`), `net_read` (capturing `net_rxlen` immediately), and `net_write` with its count-rides-

twice rule. Chapter “The Network Device” explains those; the point here is that the C and assembly libraries are the *same library* in two spellings, and this handbook’s examples come in pairs for exactly that reason.

`text.c` is CONFIG’s byte-aligned 5×7 blitter in C: BIOS glyph data at 08E4H for 20H–63H, its own 26-glyph lowercase table, two

bytes per scanline at 2bpp, twenty columns. `main.c` is Chapter “First Contact.”

Running it

`make` builds `build/cdemo.bin` — 8,192 bytes, checkrom-clean. `make run` boots it in MAME against a fujinet-pc; `make smoke` does the same headless, presses **1** on the menu itself, and snapshots the result.

The demo appears on the SELECT GAME menu under the crt0’s built-in name, `z88dk` — renaming it means bringing your own crt0, a fine first exercise.

One honest limitation: a C quantum is a little *longer* than an assembly quantum — the poll loop runs through compiled code — so timeouts stretch by roughly half. The counts in this book are already generous; keep them.

THE NETWORK DEVICE

Device 71H is the first of eight network units — N1: through N8:, 71H–78H — each an independent connection to somewhere in the world. Every Astrocade client so far uses only N1:. You give it a *devicespec* — a URL with an **N:** prefix — and from then on it is a byte pipe with a status word.

```
N:TELNET://BBS.FOZZTEXX.COM/
```

```
N:HTTPS://fujitzee.carr-designs.com/state?bin=1
```

The scheme decides everything: **TCP:**, **TELNET:**, **HTTP:**, **HTTPS:**, **TNFS:**, **JSON:**, **SSH:**, **UDP:** and twenty more, each with its own grammar and behavior. This handbook deliberately does not repeat that material: it fills a book of its own, *The FujiNet Network Protocol Handbook*, and everything there applies to the Astrocade verbatim once you can drive the five commands below.

The lifecycle

OPEN 4FH.mode and translation params, devicespec payload
STATUS 53H.four reply bytes: bytes waiting, connected, device status
READ 52H.up to 1,024 bytes into the reply window
WRITE 57H.bytes out — NETCAT is the family's first user
CLOSE 43H.hang up — and *when* you send it is a doctrine

OPEN

Two one-byte client here uses is **0CH** — read/write for a socket, and the same 12 performs a GET on an HTTP adapter. The *translation* is **0** — none; the Astrocade's 4×6 font speaks ASCII natively and the games' wire format is binary anyway. NETCAT appends **?cols=40&rows=13** to

a bare TELNET spec — the TELNET adapter reads the terminal size off the query string and only then offers NAWS to the far end.

NOPEN, NETCAT NET.INC -- OPEN THE DEVICESPEC AT HL

```
; NOPEN: open the N: connection. HL = devicespec (NUL-terminated).
; Carry set on failure.
NOPEN:  PUSH    HL
        LD      A,NETDEV          ; 71H
        LD      E,NCOPEN         ; 'O'
        LD      L,2              ; two parameters
        CALL    FNBEGIN
        LD      A,NMRDWR         ; mode 0CH: read/write
        CALL    FNPARB
        LD      A,NTRNONE        ; translation 0: none
        CALL    FNPARB
        POP     HL
        CALL    FNTXSTR          ; the devicespec is the payload
        LD      B,8              ; 16 s: TLS handshakes are slow
        CALL    FNCOMMIT
        RET     C
        JR      NZ,NOPBAD
        LD      A,(FNREPLY)
        CP      FCACK
        RET     Z                ; open, carry clear
NOPBAD: SCF
        RET
```

STATUS

Two zero parameters (a quirk the firmware expects), and the reply is four bytes in slice 0:

avail lo	avail hi	connected	devstatus
----------	----------	-----------	-----------

Figure 8 The NET_STATUS reply.

`avail` is how many bytes wait to be read. `connected` is the socket truth. `devstatus` is the protocol adapter's verdict: 1 means SUCCESS, 136 end-of-file. The two flags earn their keep in different loops: the **games** gate rendering on `devstatus == 1` — an HTTP GET is performed *at read time*, so an error page still has a readable body and `connected` alone would happily render garbage — while **NETCAT**

watches `connected`, because on a live socket EOF is a hangup, not an error.

READ

One two-byte parameter: how many bytes you want, clamped to what STATUS said was waiting and to 1,024. On ACK, capture `FNRXLL/`

FNRLH *that instant* — the next transaction, any transaction, repaints them.

WRITE

NETCAT's contribution to the family. One two-byte parameter carries the count, then the bytes themselves follow as payload — the count rides *twice*, once as the parameter and once implicitly as the payload the cartridge measures. NETCAT caps a write at 128 bytes, comfortably inside the 320-byte TX stream with three bytes spent on the length parameter.

CLOSE

No parameters. The doctrine is *when*: see Rule 1.

The three rules

Every game client learned these the hard way; they are the difference between a demo and a product.

1. CLOSE at the start of the next request, never after the read.

Every transaction repaints the whole reply window, and a CLOSE reply is empty. The games render their screens straight out of the reply window between polls — close after read, and you have just erased the state you were about to draw. NETCAT is the deliberate exception: it consumes each read immediately and holds one connection open for the session.

2. **The settle loop.** The ESP32 reports STATUS the moment *some* of a response has arrived. A game polls STATUS until two consecutive *avail* readings agree (bounded at 20 tries, 3 frames apart), and only then reads. NETCAT skips this on purpose — a live stream never produces two equal readings, and a terminal wants bytes now, not settled.

3. **Validate before believing** — **VALID8**. The reply window is never cleared; a short reply leaves a stale tail. Check the captured

length against the format's minimum, check enums against their ranges, check counts against their caps — then act.

APICALL, BATTLESHIP NET.INC -- ONE WHOLE HTTP ROUND TRIP, RULES 1 AND 2 IN THE FLESH

```
APICALL: LD      (V_URL),A      ; which URL BLDURL builds
          CALL    NCLOSE        ; rule 1: the PREVIOUS connection
          CALL    NOPEN
          RET      C
          XOR      A
          LD      (PRVAVL),A
          LD      (PRVAVL+1),A
          LD      B,20          ; rule 2: settle-loop bound
APSET:    PUSH    BC
          CALL    NSTATUS
          POP      BC
          JR      C,APCFL
          LD      HL,(AVAIL)
          LD      A,H
          OR      L
          JR      Z,APAGN      ; nothing at all yet
          LD      DE,(PRVAVL)
          OR      A
          SBC     HL,DE
          JR      Z,APRD      ; two readings agree: it has landed
APAGN:    LD      HL,(AVAIL)
          LD      (PRVAVL),HL
          LD      DE,4000      ; ~3 frames between polls
APDLY:    DEC     DE
          LD      A,D
          OR      E
          JR      NZ,APDLY
          DJNZ    APSET
APCFL:    CALL    NCLOSE
          RET
APRD:    CALL    NREAD
          JR      C,APCFL
          ; falls through into VALID8 -- rule 3
```

NOTE: The `N:` device also speaks structured channels — JSON mode with `PARSE` and `QUERY`, `seek` and `tell` on file-like protocols, `rename`, `delete` and `mkdir` on filesystem ones. The *Astrocade* games chose raw binary (`?bin=1`) over JSON to keep parsing on the server, but the whole command set answers on this platform and Chapter “Command Reference: The Network Device” documents every command.

THE FUJI DEVICE

Device `70H` is the adapter itself: WiFi, host slots, directories, disk images, and a drawer of utilities. CONFIG is its natural habitat — the only shipping Astrocade client that speaks to it — and this chapter follows CONFIG’s road through it. The complete command cards are in Chapter “Command Reference: The Fuji Device”; here is how the roads fit together.

Getting on the air

CONFIG’s WiFi checkout is four commands long: `GET_WIFI_ENABLED` (`0EAH`) — is there a radio at all; `GET_WIFI_STATUS` (`0FAH`) — one reply byte, `3` means connected; `GET_SSID` (`0FEH`) — the current network and password, a 97-byte reply; and if you need a new network, `SCAN_NETWORKS` (`0FDH`) returns a count, `GET_SCAN_RESULT` (`0FCH`) takes an index parameter and returns 34 bytes — `ssid[33]` and an RSSI byte, a negative dBm you can bar-graph.

Joining is `SET_SSID` (`0FBH`), and it carries two famous sharp edges: the firmware demands **at least one parameter, whose value it ignores**, and the payload must be **exactly 97 bytes** — `ssid[33]` + `password[64]`, NUL-padded to their full widths.

The payload-shape rules

Behind those edges is one firmware fact worth internalizing: the transaction layer’s `transaction_get()` **fails a short read**. A command that expects a struct wants *all* of it:

1. `OPEN_DIRECTORY` and `SET_DEVICE_FULLPATH` always send a full **256-byte**, NUL-padded payload.
2. `SET_SSID` sends **exactly 97**.
3. `COPY_FILE` is the exception that proves the rule: its payload is read as a *string*, so it goes at **exact length** — one padding NUL would

land inside the destination filename. And its two host-slot parameters are **1-based**, unlike everywhere else.

Hosts and directories

Eight host slots of 32 bytes each — `READ_HOST_SLOTS` (`0F4H`) returns all 256 in one reply, which is to say: *exactly slice 0*. `MOUNT_HOST` (`0F9H`) takes the slot number; then `OPEN_DIRECTORY` (`0F7H`, host parameter, path + NUL + filter payload), `SET_DIRECTORY_POSITION` (`0E4H`, a two-byte position), `READ_DIR_ENTRY` (`0F6H`, maxlen and a flags parameter), and `CLOSE_DIRECTORY` (`0F5H`). Directory entries arrive as names with a trailing `/` on subdirectories; end of directory is **exactly two 7FH bytes**. Keep `maxlen` away from 31 — that magic value asks the firmware to append packed detail bytes meant for other platforms’ file pickers.

HWRITE, CONFIG HOSTS.INC -- THE REPLY WINDOW IS THE BUFFER

```
; Rename host slot A to the string at HL: a fresh READ_HOST_SLOTS for
; freshness, then WRITE_HOST_SLOTS streamed byte-for-byte out of the
; reply window itself, substituting only the edited slot. No 256-byte
; RAM mirror ever exists -- there is nowhere to put one. The one rule:
; no transaction may run between the read and the streamed write.
    CALL    CRHSL          ; READ_HOST_SLOTS -> slice 0
    RET     C
    LD      A,FNDFUJI
    LD      E,FCWHSLSL     ; WRITE_HOST_SLOTS
    LD      L,0
    CALL    FNBEGIN
    LD      DE,FNRDATA      ; stream 8 x 32 bytes back out of
    ...           ; the painted reply, swapping in the
    ...           ; edited slot as it passes
```

That fragment deserves its caution: CONFIG's on-screen editor is **transaction-free by contract** so that nothing can repaint the reply window between the freshness read and the streamed write. On a machine with 4K of RAM, total, the reply window is not a copy of the state — it *is* the state.

The utility drawer

The Fuji device also answers `GET_ADAPTERCONFIG_EXTENDED` (0C4H — Chapter “First Contact”), `RANDOM_NUMBER` (0D3H, four bytes of entropy), `GENERATE_GUID` (0BBH), app keys (0DCH–0DBH: per-app key/value storage on the adapter's SD card — TEXAS HOLD'EM already declares its lobby-key constants for the day it enrolls), base64 and hash engines (0D0H–0C2H: feed data in, compute, read the digest out — an SHA256 on call from a Z80!), and QR code rendering (0BCH–0BFH). Command cards for all of them follow in the reference.

BOOT, SWAP, AND BANKING

The finale of CONFIG's road: pick a program from a network host and *become* it. On other FujiNets that is “mount the disk and reboot.” On a console whose cartridge port cannot write, it is a small magic trick in three acts.

Act one: the push

`SET_DEVICE_FULLPATH` (0E2H: device slot 0, host, mode 1-read, path padded to 256) names the file. Then `MOUNT_IMAGE` (0F8H) — and here the Astrocade departs from every sibling. While your `MOUNT_IMAGE` transaction is *still outstanding*, the ESP32-S3 fetches the file and streams it to the RP2040 over a side channel: FujiBus frames addressed to device 0FFH (DBC) — an OPEN carrying a stream id and a 32-bit size, WRITE after WRITE of sector-sized chunks, then a CLOSE that commits (a CLOSE with a 1-byte payload aborts). Stream id 0 is the ROM; stream id 1, pushed first when one exists, is the image's `.cfg` sidecar.

Your program sees all this as theater on the status bytes: `FNBSTAT` walks idle → transfer → ready (or 80H, failed, with the reason in `FNBERR`), and `FNBPCT` climbs 0 to 100. CONFIG's `BCOMMIT` is `FNCOMMIT`'s shape with a 60-quantum patience and a progress bar painted from `FNBPCT` between polls — passive reads only; never touch a hotspot while the push is in flight.

Act two: the swap

When `FNBSTAT` says ready: write `BOOTLOCK` (register 11H) = 0B5H to arm the trigger, copy a stub into screen RAM, and jump to it. The stub must run from RAM for the best reason imaginable — the swap replaces **every byte of the cartridge window, including the code that triggers it.**

THE SWAP STUB, CONFIG BOOT.INC -- TEN BYTES THAT CHANGE THE GAME

```
BREADY: LD      A,100
        CALL    BARUPD          ; progress bar: done
        LD      C,FRBOOTL
        LD      A,FNBLMAG      ; 0B5H
        CALL    FNREGWR        ; arm the trigger
        LD      HL,BSTUB
        LD      DE,STUB        ; STUB EQU 4FE0H: top of screen RAM
        LD      BC,BSTUBL
        LDIR
        JP      STUB
BSTUB:  LD      A,(FNSWAP)      ; 3DFEH: the cart flips between this
        JP      0              ; read and the next fetch
BSTUBL EQU  $-BSTUB
```

`JP 0` cold-starts the on-board OS, which walks the freshly served image's 55H menu header, and the pushed program is one keypress away — delivered over WiFi to a 1978 console, wearing the OS's own menu like it came in a Videocade box.

Act three: after the swap

What the mailbox does next depends on what was pushed. An image carrying the `FUJI` claim at 1CFCH — every client in this book — keeps the mailbox alive: the OS menu itself is now served through it, and CONFIG can be re-entered by the BOOTLOCK road again. An ordinary game image makes no such promise, so the mailbox goes dark for the session; RESET all you like, the game keeps working exactly as a real cartridge would.

Banking: past 8K

Mailbox protocol version 2 (check `FNPVER`) adds two banking schemes, mutually exclusive by construction:

GAME banking — for pushed images of exactly 256K or 512K with no claim: the established homebrew mapper, byte-compatible with MAME's `rom_256k/rom_512k`. 2000H–2FFFFH is fixed to the image's last 4K bank; a read in 3FC0H–3FFFFH (256K; 3F80H up for 512K)

selects the bank *and returns the bank number as the data byte*. Those hotspots live inside the FNDATA page — which is exactly why game banking exists only with the mailbox dead.

APPBANK — for claimed images of 8K + k×4K, up to 112 extra pages: the mailbox stays fully alive. One read at `FNBKSEL+page` (3D80H+page) maps image page 0–111 into 2000H–2FFFFH; the high half never moves, because your code, the reply window and the hotspots live there. The select completes before the next read; the byte it returns is undefined. Two etiquette rules: never execute from 2000H–2FFFFH while switching away from the page the PC stands in, and stamp every selectable page with the 7-byte sentinel header whose start vector re-selects page 0 — console RESET does not, because the cartridge never hears it. The firmware's `tools/mkbanked.py` stamps exactly that.

CAUTION

On a cartridge serving a plain 8K image, `FNBKSEL` reads are no-ops — probing is safe. But `FNSWAP` is **armed-only** everywhere: without the BOOTLOCK magic, reading 3DFEH does nothing, and with it, it does everything. Arm it as the last act before the stub, never earlier.

COMMAND REFERENCE: THE FUJI DEVICE

Every command device `70H` answers on the Astrocade, verified against the firmware dispatch tables (`fujidevice.cpp`, its four mixins, and the RS232 fall-through switch). Conventions: *nparam* is the value for the NPARAM register; a *byte* parameter is `FNPARB` (size 1), a *word* is size 2, little-endian; replies land in the reply window and “slice 0” means the first 256 bytes. Any command not listed here is answered with a NAK on this platform — the closing table names them.

WiFi and the adapter

RESET

`0FFH`

DEVICE	70H	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	ACK, then the adapter reboots		

Reboots the ESP32-S3. The RP2040 and the mailbox stay up; the link bit in `FNSTATS` drops until the adapter re-enumerates (allow 16 s, the same 8 quanta as an OPEN).

GET_WIFI_ENABLED

`0EAH`

DEVICE	70H	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	1 byte: 1 = radio enabled		

CONFIG's first WiFi question (`wifi.inc`). A 0 here means the radio is administratively off — scanning and joining will refuse.

GET_WIFISTATUS

0FAH

DEVICE	70H	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	1 byte: 3 = connected; anything else = not (yet)		

The poll target while a join is in flight. CONFIG polls it after SET_SSID until it turns 3 or patience runs out.

GET_SSID

0FEH

DEVICE	70H	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	97 bytes: ssid[33] + password[64], NUL-padded		

The current network configuration. Yes, the password comes back in clear text — your Arcade is a trusted friend.

SCAN_NETWORKS

0FDH

DEVICE	70H	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	1 byte: number of networks found		

Starts a fresh scan and blocks until it finishes. Follow with GET_SCAN_RESULT once per index.

GET_SCAN_RESULT

0FCH

DEVICE	70H	NPARAM	1
PARAMS	byte: result index, 0-based		
PAYLOAD	none		
REPLY	34 bytes: ssid[33] + rssi (signed dBm)		

One scanned network. CONFIG bar-graphs the RSSI byte at offset 33 — remember it is negative dBm, so closer to zero is stronger.

SET_SSID

0FBH

DEVICE	70H	NPARAM	1
PARAMS	byte: any value — required but ignored		
PAYLOAD	exactly 97 bytes: <code>ssid[33]</code> + <code>password[64]</code> , NUL-padded to their full widths		
REPLY	ACK on join, NAK on refusal		

Joins a network and saves it as the default. The two sharp edges are in bold; a short payload fails the transaction outright (`transaction_get` refuses short reads).

GET_ADAPTERCONFIG

0E8H

DEVICE	70H	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	140 bytes: <code>ssid[33]</code> <code>hostname[64]</code> <code>localIP[4]</code> <code>gateway[4]</code> <code>netmask[4]</code> <code>dnsIP[4]</code> <code>mac[6]</code> <code>bssid[6]</code> <code>fn_version[15]</code>		

The compact adapter picture, addresses in binary. Prefer the extended form below — its text fields save you a print-an-IP routine.

GET_ADAPTERCONFIG_EXTENDED

0C4H

DEVICE	70H	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	240 bytes: the 140 above, then <code>sLocalIP[16]</code> at offset 140, <code>sGateway[16]</code> at 156, <code>sNetmask[16]</code> at 172, <code>sDnsIP[16]</code> at 188, <code>sMacAddress[18]</code> at 204, <code>sBssid[18]</code> at 222 — all dotted/colon text, NUL-terminated		

The first-contact command of Chapter 4. The three fields every status screen wants sit in slice 0: SSID at +0, `fn_version` at +125, IP text at +140.

STATUS

53H

DEVICE	70H	NPARAM	1
PARAMS	byte: request type — 0 connection error, 1 mount times, 2 netmask, 3 gateway, 4 DNS		
PAYLOAD	none		
REPLY	type 1: one mount timestamp per device slot (zero = unmounted); other types currently 4 zero bytes		

The RS232 build requires the parameter; omitting it is an error. Mostly a diagnostic.

DEVICE_READY

00H

DEVICE	70H	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	512 bytes, every one the letter A		

The bus test card. Two slices of solid 41H — if they arrive intact, your mailbox, the RP2040, the USB link and the ESP32-S3 are all telling the truth.

Hosts, slots, and mounting

READ_HOST_SLOTS

0F4H

DEVICE	70H	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	256 bytes: 8 slots × <code>hostname[32]</code> — exactly slice 0		

The host list, one transaction, no slicing needed. CONFIG renders its HOSTS screen straight from the reply window.

WRITE_HOST_SLOTS

0F3H

DEVICE	70H	NPARAM	0
PARAMS	none		
PAYLOAD	256 bytes: all 8 slots, full width		
REPLY	ACK		

All-or-nothing: there is no write-one-slot command. CONFIG's `HWRITE` streams the payload straight back out of a fresh `READ_HOST_SLOTS` reply, substituting the edited slot in flight — the “reply window IS the buffer” idiom of Chapter 8.

MOUNT_HOST

0F9H

DEVICE	70H	NPARAM	1
PARAMS	byte: host slot, 0-based		
PAYLOAD	none		
REPLY	ACK, or NAK if the host is unreachable		

Connects to the named host (TNFS, SMB, ...). Required before `OPEN_DIRECTORY` or a boot.

UNMOUNT_HOST

0E6H

DEVICE	70H	NPARAM	1
PARAMS	byte: host slot		
PAYLOAD	none		
REPLY	ACK		

Releases the connection. CONFIG never bothers; sessions are short.

SET_HOST_PREFIX / GET_HOST_PREFIX

0E1H / 0E0H

DEVICE	70H	NPARAM	1
PARAMS	byte: host slot; SET also carries the prefix string as payload		
PAYLOAD	none		
REPLY	GET: the prefix string		

A per-host working directory prepended to relative paths. The Astrocade clients keep absolute paths instead.

READ_DEVICE_SLOTS

0F2H

DEVICE	70H	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	38 bytes per device slot: <code>hostSlot mode filename[36]</code>		

What is (configured to be) in each virtual drive. The Astrocade boots images rather than serving drives, so CONFIG leaves this screenless — but the command answers.

WRITE_DEVICE_SLOTS

0F1H

DEVICE	70H	NPARAM	0
PARAMS	none		
PAYLOAD	the full device-slot table, as read		
REPLY	ACK		

The paired writer, same all-or-nothing contract as host slots.

SET_DEVICE_FULLPATH

0E2H

DEVICE	70H	NPARAM	3
PARAMS	bytes: device slot, host slot, mode (1 = read)		
PAYLOAD	256 bytes: the path, NUL-padded		
REPLY	ACK		

Names the file for a device slot. On the Astrocade this is the first half of booting: CONFIG points slot 0 at the chosen ROM (*boot.inc*).

GET_DEVICE_FULLPATH

0DAH

DEVICE	70H	NPARAM	1
PARAMS	byte: device slot		
PAYLOAD	none		
REPLY	the stored path		

Reads back what SET stored.

MOUNT_IMAGE

0F8H

DEVICE	70H	NPARAM	2
PARAMS	bytes: device slot, access mode (1 read, 2 write)		
PAYLOAD	none		
REPLY	ACK when the mount — and on this platform, the push — completes		

The second half of booting, and the Astrocade's most theatrical command: while it is outstanding, the adapter streams the image to the cartridge over the DBC side channel and *FNBSTAT/FNBPCT* narrate the progress. Allow 60 quanta and paint a progress bar (Chapter 9). Plain disk-style mounts also work — but nothing on this console reads sectors yet.

UNMOUNT_IMAGE

0E9H

DEVICE	70H	NPARAM	1
PARAMS	byte: device slot		
PAYLOAD	none		
REPLY	ACK		

Releases the image.

MOUNT_ALL

0D7H

DEVICE	70H	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	ACK		

Mounts every configured slot in one go — the other platforms' “boot my usual setup” button.

NEW_DISK

0E7H

DEVICE	70H	NPARAM	0
PARAMS	none		
PAYLOAD	262 bytes: numSectors (word) sectorSize (word) hostSlot deviceSlot filename[256]		
REPLY	ACK on creation		

Creates a blank image on a host. Full-struct payload, NUL-padded filename — the 256-pad rule again.

SET_BOOT_MODE / CONFIG_BOOT

0D6H / 0D9H

DEVICE	70H	NPARAM	1
PARAMS	byte: mode		
PAYLOAD	none		
REPLY	ACK		

Selects what the adapter offers at power-on on disk-serving platforms. CONFIG **deliberately never sends** CONFIG_BOOT — on this console the mailbox claim, not a config flag, decides who boots.

Directories

OPEN_DIRECTORY 0F7H

DEVICE	70H	NPARAM	1
PARAMS	byte: host slot		
PAYLOAD	256 bytes: path, NUL, then an optional filename filter (e.g. *.BIN), the rest NUL padding		
REPLY	ACK		

Opens a directory on a mounted host. The filter is applied server-side; CONFIG uses it for its file-type toggle.

READ_DIR_ENTRY 0F6H

DEVICE	70H	NPARAM	2
PARAMS	bytes: maxlen, flags (0)		
PAYLOAD	none		
REPLY	one entry name, NUL-terminated; directories arrive with a trailing /; end of directory is exactly two 7FH bytes		

One entry per transaction. Keep `maxLen` away from 31 — that magic value appends packed detail bytes (timestamp, size, media type) meant for other platforms' pickers, and with flags `0C0H` the whole reply switches to that block format.

SET_DIRECTORY_POSITION 0E4H

DEVICE	70H	NPARAM	1
PARAMS	word: absolute entry index		
PAYLOAD	none		
REPLY	ACK		

Seeks the open directory — CONFIG's paging in one command. Note the two-byte parameter: size byte 2, low, high.

GET_DIRECTORY_POSITION

0E5H

DEVICE	70H	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	word: the current index		

The matching tell.

CLOSE_DIRECTORY

0F5H

DEVICE	70H	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	ACK		

Closes the walk. Directories are a scarce resource on the adapter; close what you open.

Files

COPY_FILE

0D8H

DEVICE	70H	NPARAM	2
PARAMS	bytes: source host, destination host — both 1-based , alone among slot parameters		
PAYLOAD	exact length , no padding: <code>sourcepath destdir/destname, -separated</code>		
REPLY	ACK when the copy completes		

Server-to-server copy without a byte passing through the console. The string payload is the one place padding would corrupt (a NUL lands inside the destination name), hence the exact-length rule. CONFIG's copy screen (`copy.inc`) is this command plus a retry that brings the `.cfg` sidecar along.

App keys

Small key/value files on the adapter's SD card, namespaced by a 16-bit creator id, an app id and a key id — how FujiNet programs remember a player name between sessions. TEXAS HOLD'EM already declares the lobby's ids (creator 1, app 1, key 0 — the username shared by every lobby client) for the day it enrolls.

OPEN_APPKEY

0DCH

DEVICE	70H	NPARAM	0
PARAMS	none		
PAYLOAD	6 bytes: creator (word) app key mode (0 read, 1 write) reserved		
REPLY	ACK — NAK if no SD card is present		

Selects which key the next read or write touches.

READ_APPKEY

0DDH

DEVICE	70H	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	64 bytes: the value, NUL-padded		

Reads the selected key (open mode 0 first).

WRITE_APPKEY

0DEH

DEVICE	70H	NPARAM	0
PARAMS	none		
PAYLOAD	the value, up to 64 bytes		
REPLY	ACK		

Writes the selected key (open mode 1 first).

CLOSE_APPKEY

0DBH

DEVICE	70H	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	ACK		

Invalidates the selection.

Utilities

RANDOM_NUMBER

0D3H

DEVICE	70H	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	4 bytes: a random 32-bit value, little-endian		

Hardware entropy from the ESP32. The games keep their 16-bit LFSR for speed — but its seed could come from here.

GENERATE_GUID

0BBH

DEVICE	70H	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	37 bytes: a UUID in text, NUL-terminated		

A fresh unique id, ready to print.

BASE64 ENCODE: INPUT / COMPUTE / LENGTH / OUTPUT

0D0H / 0CFH / 0CEH / 0CDH

DEVICE	70H	NPARAM	INPUT and OUTPUT: 1; others: 0
PARAMS	word: byte count (INPUT: how many follow; OUTPUT: how many to return)		
PAYLOAD	INPUT: the bytes themselves		
REPLY	LENGTH: 4-byte little-endian size; OUTPUT: the encoded text		

A four-step pipeline on the adapter: feed raw bytes in (INPUT appends — call it repeatedly for long data), COMPUTE, ask LENGTH, then drain with OUTPUT. The buffer survives between transactions.

BASE64 DECODE: INPUT / COMPUTE / LENGTH / OUTPUT

0CCH / 0CBH / 0CAH / 0C9H

DEVICE	70H	NPARAM	as above
PARAMS	as above		
PAYLOAD	as above		
REPLY	as above, decoded bytes out		

The mirror pipeline, text in, bytes out.

HASH: INPUT / COMPUTE / COMPUTE_NO_CLEAR / LENGTH / OUTPUT / CLEAR

0C8H / 0C7H / 0C3H / 0C6H / 0C5H / 0C2H

DEVICE	70H	NPARAM	1 (CLEAR: 0)
PARAMS	INPUT: word, byte count. COMPUTE: byte, algorithm — 0 MD5, 1 SHA1, 2 SHA256, 3 SHA512, 4 SHA224, 5 SHA384. LENGTH and OUTPUT: byte, 1 = hex text, else raw		
PAYLOAD	INPUT: the bytes		
REPLY	LENGTH: 1 byte; OUTPUT: the digest		

SHA256 on call from a Z80. INPUT appends (COMPUTE clears the buffer afterward; COMPUTE_NO_CLEAR keeps it for incremental use); OUTPUT as hex costs twice the bytes but prints itself.

QR CODE: INPUT / ENCODE / LENGTH / OUTPUT

0BCH / 0BDH / 0BEH / 0BFH

DEVICE	70H	NPARAM	INPUT 1 · ENCODE 3 · LENGTH 1 · OUTPUT 1
PARAMS	INPUT: word, byte count. ENCODE: bytes — version (1–40), ECC level (0–3), shorten-URL flag. LENGTH: byte, output mode (0 binary). OUTPUT: word, byte count		
PAYLOAD	INPUT: the text to encode		
REPLY	LENGTH: 4-byte size; OUTPUT: the module bitmap		

The adapter renders a QR matrix you can blit — at 160×102 with 4×4-pixel cells, a version-1 code (21×21 modules) fits the Astrocade screen with room to spare. A weekend project waiting for its author.

Answered with a NAK on this platform

The remaining `fujiCommandID.h` opcodes fall through every dispatch table in this build and earn a NAK: `ENABLE_UDPSTREAM` (0F0H), `SET_BAUDRATE` (0EBH), `SET_HSIO_INDEX` (0E3H), `SET_SIO_EXTERNAL_CLOCK` (0DFH), `ENABLE/DISABLE_DEVICE` (0D5H/0D4H), `GET_TIME` (0D2H), `DEVICE_ENABLE_STATUS` (0D1H), `GET_HEAP` (0C1H), `GET_DEVICE1--10_FULLPATH` (0A0H–0A9H), `UPDATE_FIRMWARE` (90H), `HSIO_INDEX` (3FH), and the reply-plumbing codes `SEND_ERROR/SEND_RESPONSE` (02H/01H). Most are other platforms' bus tuning; none are needed here.

COMMAND REFERENCE: THE NETWORK DEVICE

Every command devices 71H–78H answer, verified against the RS232 network dispatcher (`lib/device/rs232/network.cpp`). Most opcodes are the ASCII letter of their name — 'O'pen, 'R'ead, 'W'rite — a habit inherited from the Atari. The connection itself, the devicespec grammar, and per-protocol behavior are Chapter “The Network Device” and the *Network Protocol Handbook* these are the cards.

The lifecycle five

NET_OPEN

4FH 'O'

DEVICE	71H	NPARAM	2
PARAMS	bytes: access mode, translation		
PAYLOAD	the devicespec, e.g. N:HTTPS://host/path		
REPLY	ACK when the connection (or deferred fetch) is established		

Modes: 4 read, 8 write, 12 read/write — and 12 doubles as GET on HTTP, whose fetch is deferred to the first READ. 13 POST, 14 PUT, 5 DELETE, 6 HEAD on the HTTP family. Translation: 0 none, 1 CR, 2 LF, 3 CR/LF — the Astrocade clients always send 0.

NET_STATUS

53H 'S'

DEVICE	71H	NPARAM	2
PARAMS	two zero bytes (the RS232 build expects both)		
PAYLOAD	none		
REPLY	4 bytes: avail low, avail high, connected, device status (1 SUCCESS, 136 END_OF_FILE)		

The heartbeat. Games gate on device status; NETCAT gates on connected; everyone reads avail. See the three rules, Chapter 7.

NET_READ

52H 'R'

DEVICE	71H	NPARAM	1
PARAMS	word: byte count — clamp to avail, and to the 1,024-byte reply window		
PAYLOAD	none		
REPLY	the bytes; capture FNRXLL/FNRXLH immediately		

On deferred HTTP, the first READ performs the fetch — give it OPEN-class patience (8 quanta).

NET_WRITE

57H 'W'

DEVICE	71H	NPARAM	1
PARAMS	word: byte count		
PAYLOAD	exactly that many bytes		
REPLY	ACK		

The count rides twice — as the parameter and as the payload length. NETCAT caps writes at 128 bytes to stay clear of the 320-byte TX stream.

NET_CLOSE

43H 'C'

DEVICE	71H	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	ACK — and an empty reply window		

Rule 1 of Chapter 7: send it at the *start* of the next request, never between a READ and the rendering.

Structured channels: JSON

NET_SET_CHANNEL_MODE

4DH 'M'

DEVICE	71H	NPARAM	2
PARAMS	bytes: 0 (ignored), then mode — 0 protocol (raw), 1 JSON		
PAYLOAD	none		
REPLY	ACK		

Switches an open HTTP-family connection between raw bytes and the JSON parser. (The dispatcher reads the mode from the *second* parameter; send a placeholder first.)

NET_PARSE

50H 'P'

DEVICE	71H	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	ACK when the document parsed		

In JSON mode: fetch and parse the reply body into a query-able tree. On the SGML channel mode it parses markup instead.

NET_QUERY

51H 'Q'

DEVICE	71H	NPARAM	0
PARAMS	none		
PAYLOAD	the query, e.g. <code>N:/players/0/name</code>		
REPLY	the value as text; then READ it		

Extracts one element from the parsed tree. The games skip this whole channel — their servers pre-chew binary with `?bin=1` — but it answers on this platform, and for a lightweight client it is the difference between parsing JSON on a Z80 and not.

File-shaped connections

NET_SEEK / NET_TELL

25H '%' / 26H '&'

DEVICE	71H	NPARAM	SEEK: 1 · TELL: 0
PARAMS	SEEK: 4-byte offset		
PAYLOAD	none		
REPLY	TELL: 4-byte position		

Random access where the protocol supports it (TNFS, HTTP with ranges).

NET_RENAME / NET_DELETE / NET_LOCK / NET_UNLOCK

20H / 21H '!' / 23H '#' / 24H '\$'

DEVICE	71H	NPARAM	0
PARAMS	none		
PAYLOAD	the target devicespec (RENAME: old,new)		
REPLY	ACK		

Filesystem verbs on protocols that have files.

NET_MKDIR / NET_RMDIR / NET_CHDIR / NET_GETCWD

2AH '*' / 2BH '+' / 2CH ',' / 30H '0'

DEVICE	71H	NPARAM	0
PARAMS	none		
PAYLOAD	MKDIR/RMDIR/CHDIR: the path		
REPLY	GETCWD: the current path		

Directory verbs, same family.

Sockets and servers

NET_CONTROL / NET_CLOSE_CLIENT

41H 'A' / 63H 'c'

DEVICE	71H	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	ACK		

TCP server duty: a listening `N:TCP://:port/` accepts a waiting client with 'A', and hangs up on that client with 'c' while keeping the listener. Your Astrocade can be the BBS.

NET_GET_REMOTE / NET_SET_DESTINATION

72H 'r' / 44H 'D'

DEVICE	71H	NPARAM	0
PARAMS	none		
PAYLOAD	SET_DESTINATION: <code>host:port</code> for subsequent writes		
REPLY	GET_REMOTE: the sender of the last datagram		

UDP's who-said-that and talk-to-them-instead.

NET_USERNAME / NET_PASSWORD

0FDH / 0FEH

DEVICE	71H	NPARAM	0
PARAMS	none		
PAYLOAD	the credential string		
REPLY	ACK		

Stored credentials for the next OPEN on protocols that log in (FTP, SSH, SMB). Send before OPEN.

Line discipline

NET_TRANSLATION

54H 'T'

DEVICE	71H	NPARAM	2
PARAMS	bytes: 0 (ignored), then the translation code		
PAYLOAD	none		
REPLY	ACK		

Changes end-of-line translation after OPEN. Same second-parameter quirk as SET_CHANNEL_MODE.

NET_SET_EOL

4CH 'L'

DEVICE	71H	NPARAM	2
PARAMS	bytes: the custom EOL pair; a first byte of 0 clears		
PAYLOAD	none		
REPLY	ACK		

Custom line terminators for odd servers.

NET_SET_INT_RATE

5AH 'Z'

DEVICE	71H	NPARAM	2
PARAMS	bytes: 0 (ignored), then the rate		
PAYLOAD	none		
REPLY	ACK		

Interrupt pacing for platforms with a bus interrupt line. The Astrocade polls; accepted and irrelevant here.

Answered with a NAK on this platform

Enum'd but not dispatched in this build: `NET_GET_DSTATS_VALUE` (0FFH), `NET_CHANNEL_MODE` (0FCH — the getter; the setter 4DH works), `NET_SET_PARAMETERS` (0FBH), `NET_SET_CHANNEL` (0FAH), `NET_SET_HSIO_INDEX` (0E3H), `NET_QUERY_ALT/NET_PARSE_ALT` (81H/80H), `NET_GET_ERROR` (45H — read the STATUS device-status byte instead), and `NET_HSIO_INDEX` (3FH).

NETCAT

"Dial, you tinhorn!" NETCAT is the family's terminal: give it any `N`: devicespec and it pumps bytes between the connection and a scrolling 40×13 pane, forever. Call a telnet BBS, watch a TCP echo server, talk to anything that talks back. It is also the family's teaching milestone three times over: the first Astrocade client to **hold a connection open** (everything before it was one-shot HTTP GET), the first to send **N: WRITE**, and the first with a pane that **scrolls**.

How to get NETCAT on your screen

Build with `./build.sh` (the default devicespec bakes in from `$ENDPOINT`; `N:TELNET://BBS.FOZZTEXX.COM/` out of the box) and run with `./run.sh` under MAME, or burn `build/nc.bin` to a cartridge. Press `[1]` at the SELECT GAME menu. The dial screen offers the devicespec for editing on a 32×3 grid keyboard — all 96 printable characters at once, steered with the hand control — and `[=]` dials.

Hand controls

Joy Stick.steer the grid-keyboard cursor
Trigger.type the highlighted character
Keypad `[=]`.dial · in session: compose-and-send line
Keypad `[C]`.backspace · in session: hang up



Figure 9 NETCAT in session.

Inside the program

2,144 lines, 3,181 bytes of ROM. The RAM budget is the design driver: `LINES=84` shows 14 rows of 4×6 text and leaves 736 bytes above the visible screen — the card games' 90-line screen would have left 496, and the terminal's shadow does not fit in that. The shadow (`TSHAD`, 160 bytes) is a character-level copy of rows 9–12, so the grid keyboard overlay can drop over a live session and the covered rows come back intact.

The session loop is Chapter 7's rules minus the settle loop, which a live stream must not have:

SESSN, NETCAT SCREENS.INC -- THE WHOLE TERMINAL, ONE LOOP

```
SESSN:  CALL    TINIT
        CALL    SSTAT
        CALL    TCURON
SESS1:  CALL    NSTATUS
        JR      C,SLOST
        LD      A,(NCONN)      ; a terminal watches CONNECTED --
        OR      A              ; EOF on a socket is a hangup,
        JR      Z,SEOF        ; not an error
        LD      HL,(AVAIL)
        LD      A,H
        OR      L
        JR      Z,SESSI      ; nothing waiting: check the keys
        CALL    TCUR0F
        CALL    NREAD
        JR      C,SLOST
        CALL    SREND        ; render RXLEN bytes through TPUTC
        CALL    TCURON
        JR      SESS2
SESSI:  CALL    DELAY10        ; ~11 ms idle pause
SESS2:  CALL    SPOLL        ; keys: compose, send, hang up
        JR      NC,SESS1
```

Scrolling is honest work on this hardware — there is no scroll register, so `TSCROL` moves the pane up one row with an `LDIR`, about 34 ms, and the escape-swallower in `term.inc` keeps ANSI-decorated BBSes from wrecking the pane. The 96-glyph font earns its keep the moment a BBS greets you in lowercase.

Files: `term.inc` (the pane, 274 lines), `edit.inc` (grid keyboard, 222), `screens.inc` (dial and session, 233), `net.inc` (the five N: commands including the family's only `NWRITE`, 267), plus the shared transport. Full listings: Appendix C. Palette: white text, green accent, gray, black.

TEXAS HOLD'EM

Pull up a chair — the table is world-wide. TEXAS HOLD'EM joins live multiplayer poker at th.carr-designs.com, against players on Ataris, Apples, ADAMs, Intellivisions, and the rest of the FujiNet family. The **server owns the rules**: every legal move arrives over the wire with its label, so the client contains no poker logic at all — and inherits rule changes without a re-burn.

How to play

Build, run, press **[1]**. Enter a name on the letter wheel, pick a table from the lobby, and the keypad's **[1]**-**[5]** press whichever move buttons the server dealt you — FOLD, CHECK, CALL, BET, RAISE, ALL IN as the moment demands. **[.]** shows help, **[E]** leaves.

The wire format

Every poll is one HTTP GET — `state, move/XX, or leave`, with `?table=...&player=...&bin=1` — and `bin=1` is the whole trick: the server serializes its JSON into a fixed binary layout, so the Z80 reads fields at offsets instead of parsing text. Words are little-endian; strings are NUL-padded records.

Offset	Field	Notes
0	<code>lastResult[81]</code>	the round's story, ready to print
81	<code>round</code>	0 waiting · 1 pre-flop · 2 flop · 3 turn · 4 river · 5 showdown
82	<code>pot (word)</code>	
84	<code>activePlayer</code>	signed; 0FFH = nobody
85	<code>moveTimer</code>	seconds on the shot clock
86	<code>viewing</code>	1 = you are a spectator
87	<code>community[11]</code>	the board cards, text
98	<code>validMoveCount</code>	
99	<code>validMoves[5×13]</code>	2-char code + 10-char label each
164	<code>playerCount</code>	
165	<code>players[n×33]</code>	name 0 · status 9 · bet 10 · move 12 · purse 20 · hand 22

Figure 10 The Game object, `?bin=1 (fujinet.inc offsets)`.

Replies are validated (rule 3) against `GMINLEN` 165 and `GMAXLEN` 429; the table list is its own format (36-byte records: id 0, name 9, players 30).

The simplest poll loop in the family

No screen classes, no staged-action enum — just one elegant trick: **a staged move rides the next request**. Press a move key and the client stores the 2-character code; the next poll requests `move/xx` instead of `state`, and *that reply is the new state*. One round trip, not two.

GAMELP, TEXAS SCREENS.INC

```
GAMELP: LD      A,(MOVBUF)      ; a staged move rides the next
      OR      A                ; request -- its reply IS the
      LD      A,1              ; next state
      JR      Z,GLREQ
      LD      A,2              ; V_URL 2: move/XX
GLREQ:  CALL    APICALL
      JR      C,GLFAIL
      XOR     A
      LD      (MOVBUF),A        ; consumed
      CALL    CHKNEW            ; what changed since last poll?
      CALL    RENDER
      LD      B,90              ; ~2 s of input scanning
GLWAIT: ...                    ; KEYSCN every 20 ms, DJNZ
```

The screen is 40×15 text over 160×90 pixels in four colors — felt green, black, red, white — with 12×17-pixel cards drawn on an 8-pixel pitch (*cards.inc*). The Lynx port supplied the screen plan; the Lynx is 160×102, an exact resolution cousin. A *DEMO=1* build assembles a static mock table for screenshots and box art.

2,977 lines, 4,607 bytes of ROM. Files of note: *screens.inc* (931 lines — table list, poll loop, renderer), *cards.inc*, *nament.inc* (the name wheel), *url.inc* (builds request URLs straight into the TX stream). Full listings: Appendix C.

BATTLESHIP

You sank my — well, you know the game. Up to four players, live over the network at battleship.carr-designs.com, on boards small enough that all of them fit your screen at once: four 10×10 quadrants, each cell 4×4 pixels — one byte per screen row, so **nothing ever shifts** when the renderer blits a cell. The right nineteen byte-columns are the status panel.

You are always quadrant 0: the server rotates its player list so every client sees itself first. Gameplay was transcribed from the Intellivision port, and the two clients meet happily at the same tables.

How to play

Name, table, then: place your five ships (sizes 5-4-3-3-2 — the placement screen rejects overlaps locally and offers a random legal seed), press ready, and when your turn lights up, steer the target cursor — painted simultaneously on *every* live enemy quadrant — and fire. Status 99 is the gun smoke clearing.

The staged-action machine

Where TEXAS stages a move code, BATTLESHIP stages an *action*: **PENDACT** holds 0–3 for none / ready / place / attack, and the poll loop maps it onto the request — 0 polls **state**, the rest ride **ready**, **place**/**P**, **P**, **P**, **P**, **P**, **attack**/**P** and are consumed on success. Every good poll also computes the turn edge (did **activePlayer** just become me?) from the previous poll's value, and **CHKNEW** dispatches one of four screen classes: lobby, placement, game, game-over.

The wire format, v2

?bin=1&v=2 — and the **v=2** is **mandatory**: v1 hides the winner index and sends only half the **myShips** array. Ships travel as one byte each, **position + 100×direction**; the game field is 100 cells of hit/miss/ship state, **y×10+x**.

Offset	Field	Notes
0	playerCount	
1	prompt[33]	the server's banner line
34	status	0 lobby · 1 place · 10 start · 11 miss · 12 hit · 13 sunk · 99 over
35	yourStatus	0 play · 1 dead · 2 view · 3 ready · 10 placing
36	activePlayer	0FFH nobody; at game-over: the winner
37	moveTimer	
38	lobby: serverName[21] · else lastAttackPos	0–99
39	myShips[10]	pos + 100×dir
49	players[n×115]	name 0 · status 9 · field[100] 10 · shipsLeft 110

Figure 11 The Game object, ?bin=1&v=2 (fujinet.inc offsets).

The 115-byte player records appear only from status 10 up; during placement they are 10 bytes — `VALID8` (rule 3) knows the difference and checks `GMINLEN` 38 against the status before trusting a byte.

The random seed is a 16-bit Galois LFSR (`rnd.inc`, polynomial 2DH) folded with the R register *at the output only* — fold R into the state and you would shorten the period.

3,354 lines, 5,598 bytes. Files of note: `place.inc` (335 lines), `target.inc` (time-sliced targeting, 5 s per input scan slice), `board.inc` (quadrant geometry and the wire-driven redraw), `screens.inc` (799). Palette: white, red, black, sea blue (hue 30 — the value that *reads* blue in old notes, 7AH, renders olive; `battle.asm` documents the MAME palette math that settles it). Full listings: Appendix C.

FUJITZEE

Five dice, thirteen rounds, and everybody you know online at fujitzee.carr-designs.com. FUJITZEE is the family's dice game, and on the Astrocade its 40 columns pay for themselves: the Intellivision port's two-column scorecard *plus* an always-visible standings panel, no screen-swapping.

How to play

Name, table, ready. On your turn: **R** rolls, the keypad's **1**–**5** toggle which dice to keep, and when the third roll is spent (or sooner, if you like what you see) pick a row on the card and bank it. The server scores; the client's only arithmetic is a running total.

Server-computed everything

The state object's masterstroke is `validScores[15]`: for every open row, **the exact points the current dice would earn there** — computed by the server, updated every roll. The client renders choices, never rules. A closed row is `0FFH`; scores travel as 16-bit words with `0FFFFH` for unset and `0FFFEH` marking a spectator.

Edges, not levels

Three sharp edge-detections make the UI honest, all computed per poll against the previous poll's values: the **turn edge** (active seat just became mine — ding!), the **view flag** (spectators never get a turn UI), and the subtle one, the **roll edge**: only a *decrease* in `rollsLeft` means dice actually landed — a same-value poll is just time passing, and the cross-screen sentinel `PRVRLS = 0FFH` keeps a stale comparison from firing the rattle. And rattle it does: FUJITZEE is the family's first client to use the noise generator, for a proper dice-cup shake.

Dice from thin air

No dice bitmaps exist. Each die face is three 3-bit pip masks — 18 bytes of data for all six faces — expanded procedurally into 12×12

dice (`dice.inc`). The ROM thanks you: 3,746 lines assemble to 6,070 bytes, the tightest fit in the family, 842 bytes under the ceiling.

Offset	Field	Notes
0	playerCount	spectators appended after players
1	serverName[21]	
22	prompt[41]	
63	round	0 lobby · 1–13 play · 99 over
64	rollsLeft	3..0 — the roll edge watches for decrease
65	activePlayer	0FFH nobody; 0 = your turn
66	moveTimer	
67	viewing	1 = spectator
68	dice[5] + NUL	ASCII 1–6
74	keep[5] + NUL	ASCII flags: 1 = re-roll this die
80	validScores[15]	signed; 0FFH = row closed; all zero in lobby
95	players[n×42]	name 0 · alias 9 · scores[16] words 10

Figure 12 The Game object, ?bin=1 (fujinet.inc offsets).

Requests: `state`, `ready`, `roll/MMMMM` (the keep mask, streamed verbatim from the wire's own ASCII flags), `score/N`, `leave` — the same staged-action machine as BATTLESHIP with the same `VALID8` discipline (`GMINLEN 95`, `GMAXLEN 599`, `PCMAX 12`).

Files of note: `turn.inc` (338 lines — the dice/score modes of your 5 s input slice), `card.inc` (scorecard + standings, 325), `dice.inc` (291), `sound.inc` (114 — tone A plus that noise rattle). `DEMO=1` builds a static mock play screen. Full listings: Appendix C.

CONFIG

CONFIG is the program your FujiNet cartridge wakes up in — the adapter’s face on the Astrocade, and the only client in this book that speaks to device 70H. WiFi setup with an on-screen password keyboard, eight host slots to list, mount and rename, a directory browser with paging and filters, host-to-host file copy, an info screen, the Game Lobby shortcut — and the boot road of Chapter 9, progress bar and all.

It is also the one program here that ships *inside* the cartridge: `make rom.h` packs `build/config.bin` into `fujiconfigrom.h`, a C array compiled into the RP2040 firmware, which serves it as the power-on image. The program you are reading about is the cartridge’s own boot ROM — and after it network-boots something else, the swap replaces it entirely.

Around the screens

Power on, press **1**. HOSTS lists the eight slots; pick one to mount and browse, **E** edits a slot’s name on the 16×4 character grid (with CASE toggle — SSIDs and passwords are case-sensitive, and CONFIG carries its own lowercase glyphs beside the BIOS font’s capitals). WIFI scans, bar-graphs RSSI, and joins. BROWSE pages through directories — subfolders, a text filter, `.cfg` files suppressed — and picking a ROM starts the boot. COPY walks source and destination and issues `COPY_FILE`. INFO is `GET_ADAPTERCONFIG_EXTENDED`, printed.

Program shape

2,859 lines, 5,501 bytes. Where the games are a poll loop, CONFIG is a classic screen-state machine — each screen owns its keys and its redraws, `browse.inc` (442 lines) is deliberately **stateless** (it re-reads every page from the adapter rather than caching), and `edit.inc`

(397 lines) is **transaction-free by contract** so the reply window survives an edit — the HWRITE idiom of Chapter 8 depends on it.

Deliberate scope, matching the Intellivision port: no device-slot screens, no eject, no read/write toggles, no new-disk, no appkeys — a boot console, not a disk manager. And `CONFIG_BOOT` is **never sent**: on this console the claim signature, not a config flag, decides who owns power-on.

Files of note: `fujicmd.inc` (the 70H wrappers — printed whole in Appendix C, it is the Fuji-device tutorial in 173 lines), `wifi.inc` (250), `hosts.inc` (321), `boot.inc` (224 — Chapter 9’s listings came from here), `font.inc` (162 — the byte-aligned 5×7 blitter the C demo’s `text.c` translates), `ui.inc`, `copy.inc`, `info.inc`, `input.inc`. Build: the staging Makefile copies sources and `lib/` flat into `build/stage/` because `zmac` resolves `INCLUDE` against the working directory. Palette: white, amber, blue, black.

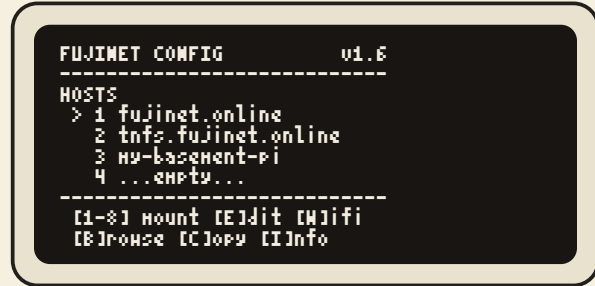


Figure 13 CONFIG’s hosts screen.

ALWAYS MORE TO COME

Your FujiNet cartridge not only ships five programs, it has virtually hundreds of program possibilities! Here is where to go next.

The Network Protocol Handbook

The companion volume from this series: all 28 URL schemes the N: device speaks — `TCP:`, `TELNET:`, `HTTP/HTTPS:`, `TNFS:`, `JSON:`, `SSH:`, `SMB:`, `FTP:`, `UDP:`, `GMAIL:`, `GCAL:` and more — with the complete devicespec grammar, every mode and error code, and worked examples. Everything in it runs on the Astrocade through Chapter 7's five commands.

The sources of truth

`fuji_mailbox.h`.....the mailbox spec — firmware `pico/astrocade/`
`fujilib.inc`.....the Z80 transport (Appendix C)
`fujiCommandID.h`.....every command opcode, canonically
`fujitest.asm`.....the minimal round trip, in the firmware tree
`fuji*.asm` tests.....boot, browse, and banking self-tests, same tree

The programs

Each program chapter's repository carries the full build system, the MAME smoke tests, and a README with its own war stories: `netcat`, `fujinet-texasHoldEm`, `fujinet-battleship`, `fujinet-fujitze`, `fujinet-config` (directory `astrocade/` in each), all under `github.com/FujiNetWIFI` and friends. The game servers are `carr-designs.com` productions; 5 CARD STUD, the games' common ancestor, has an Astrocade port too.

Sibling handbooks

The FujiNet manuals repository carries programming guides in this same spirit for the Apple II, the Coleco ADAM, the Atari 8-bits, and the Intellivision — the last of which shares this console's mailbox ancestry, one bring-up generation removed.

So do enjoy your FujiNet Professional Arcade... you now have a whole new dimension in home TV entertainment at your fingertips.

APPENDIX A: MAILBOX QUICK REFERENCE

Reply bytes (painted ROM)

Addr	Name	Meaning
3B00H	FNRRDATA	reply slice, 256 bytes
3C00H	FNACKSQ	echoes SEQ when ready
3C01H	FNSTATS	bit0 link · bit1 busy
3C02H	FNERR	transport verdict
3C03H	FNREPLY	06H ACK · 15H NAK
3C04H	FNRLLL	reply length lo
3C05H	FNRLH	reply length hi
3C06H	FNBSTAT	boot state
3C07H	FNBPCT	boot progress 0–100
3C08H	FNBERR	boot error
3C09H	FNMGF	F
3C0AH	FNMAGN	N
3C0BH	FNPPER	protocol version (2)
3C0CH	FNSECHO	slice echo, painted last

Hotspots (reading is writing)

Read	Effect
3D00H+r	arm register r (r < 80H)
3D80H+p	map image page p into 2000H (v2)
3DFEH	swap to staged image (BOOTLOCK-armed only)
3E00H+v	armed register ← v, disarm
3F00H+v	append v to the TX stream (max 320)

Registers

Reg	Name	Use
00H	DEVICE	70H Fuji · 71H–78H network
01H	COMMAND	the opcode
02H	NPARAM	parameters in the stream
05H	DATA RST	any value rewinds TX
06H	RXSLICE	reply slice 0–3
10H	SEQ	ACKSEQ+1, wrap 255→1, 0 reserved
11H	BOOTLOCK	0B5H arms the swap
12H/13H	BOOTSEL	0B5H then 4AH: cart to UF2

The five moves

1. BEGIN: rewind TX; DEVICE, COMMAND, NPARAM.
2. STREAM: params (size byte 1/2/4 + LE value) then payload.
3. COMMIT: SEQ = ACKSEQ+1 (wrap 255→1).
4. WAIT: poll FNACKSQ; then FNERR, FNREPLY, capture RXLEN.
5. READ: slice 0 is fresh; RXSLICE + FNSECHO poll for 1–3.

Layout contract (cart offsets)

Code below 1B00H · claim FUJI at 1CFCH · image exactly 8,192 bytes · reply max 1,024 · TX max 320 · APPBANK pages 4K × up to 112.

APPENDIX B: ERROR AND STATUS CODES

FNERR — transport verdict (3C02H)

Value	Name	Meaning
0	FN_ERR_OK	carried perfectly
1	FN_ERR_NOLINK	ESP32-S3 not enumerated
2	FN_ERR_TIMEOUT	adapter did not answer in time
3	FN_ERR_BADFRAME	SLIP/checksum failure
4	FN_ERR_TOOBIG	reply exceeded 1,024 bytes

FNREPLY — the FujiNet's answer (3C03H)

Value	Meaning
06H	ACK: command accepted, reply painted
15H	NAK: command refused — wrong parameters, missing target, unsupported opcode

FNBSTAT — boot state (3C06H)

Value	Name	Meaning
0	FN_BOOT_IDLE	no push in progress
1	FN_BOOT_XFER	image streaming; watch FNBPCT
2	FN_BOOT_READY	staged; arm BOOTLOCK and swap
80H	FN_BOOT_FAILED	see FNBERR

FNBERR — boot error (3C08H)

Value	Name	Meaning
1	TOOBIG	image exceeds the bank store
2	TRUNCATED	stream ended short of its declared size
3	NOMAP	size fits no supported mapping
4	STOREBUSY	the only fitting store is being served from

NET_STATUS byte 3 — device status

Value	Meaning
1	SUCCESS — the adapter is happy
136	END_OF_FILE
others	per-protocol error codes — the Network Protocol Handbook tabulates them

GET_WIFISTATUS reply

Value	Meaning
3	connected
6	disconnected
others	transitional — keep polling

APPENDIX C: PROGRAM LISTINGS

The five programs, complete, from the `listings/` snapshots — byte-identical to the source repositories at the commits in the README's sources-of-truth table. Shared files print once: the transport trio (C.1) is used by all five programs (CONFIG's `lib/` copies differ only in header comments), and the games' UI pair (C.2) by TEXAS HOLD'EM, BATTLESHIP and FUJITZEE. The games' 64-glyph fonts are byte-identical to the first 64 glyphs of NETCAT's 96-glyph `font.inc`, printed in C.3. Build scripts are described in Chapter 5; generated files (`build/endpoint.inc`, `build/flags.inc`) are not printed.

C.1 The shared transport

FUJILIB.INC -- THE MAILBOX TRANSPORT (ALL FIVE PROGRAMS)

```
1: fujilib.inc -- Z80 client half of the FujiNet cartridge mailbox.
2:
3: Copy of fujinet-firmware/pico/astrocade/testrom/fujilib.inc; keep the two
4: in step (that one is itself a hand-mirror of fujilib.inc).
5:
6: Hand-mirrors firmware/include/fuji_mailbox.h (the single source of truth);
7: every equate here is a console address, i.e. cart offset + 2000H. Keep the
8: two files in step.
9:
10: The cartridge port has no write or I/O visibility, so BOTH directions ride
11: reads: a "register write" is a read of FNRREGSEL+reg followed by a read of
12: FNRREGDAT+value (the low address byte IS the payload), and replies are
13: plain ROM bytes the cart repaints. A REGDATA read with no immediately
14: preceding REGSEL read is ignored by the cart -- that pairing rule is the
15: stray-read defense, so these helpers always issue the pair back to back.
16:
17: Clients run with DI and poll. With interrupts off the I register stays 0,
18: refresh cycles land in OS ROM space, and the refresh-stray hazard described
19: in fujilib_mailbox.h never arises at all.
20:
21: ---- reply bytes (repainted ROM) ----
22: FNRDATA EQU 3B00H ; 256-byte reply slice
23: FNACKSQ EQU 3C00H ; echoes SEQ when the reply is ready
24: FNSTATS EQU 3C01H ; bit0 link up
25: FNERR EQU 3C02H ; fb_status_t of the last transaction
26: FNRPLY EQU 3C03H ; 06H ACK / 15H NAK
27: FNRXL EQU 3C04H ; reply length, low
28: FNRXLH EQU 3C05H ; reply length, high
29: FNBSTAT EQU 3C06H ; boot state
30: FNBPC EQU 3C07H ; boot progress 0-100
31: FNBERR EQU 3C08H ; boot error
32: FNMAGF EQU 3C09H ; 'F'
33: FNMAGN EQU 3C0AH ; 'N'
34: FNPVER EQU 3C0BH ; protocol version
35: FNSSECHO EQU 3C0CH ; echoes RXSLICE after each repaint
36:
37: ---- hotspot pages ----
38: FNRREGSEL EQU 3D00H ; read +reg: arm register
39: FNRREGDAT EQU 3E00H ; read +val: armed register = val
40: FNDATA EQU 3F00H ; read +val: append val to TX stream
41: FNSWAP EQU 3DFEH ; armed-only: serve the staged image
42: FNRSEL EQU 3DH ; page bytes of the above (zmac 1.3 has no
43: FNRDATP EQU 3EH ; HIGH operator)
44: FNDATAP EQU 3FH
45:
46: ---- bank switching (protocol v2) ----
47: A banked app image is the ordinary 8K window plus 4K pages appended; one
48: read maps image page 0-111 into 2000H-2FFFH (the high half never moves --
49: ; client code and the mailbox live there; page 1 IS the high half). The
50: ; select is complete before the next read; the byte the select read returns
51: ; is undefined. Do not execute from 2000H-2FFFH while switching away from
52: ; the page the PC is in. On a cart serving a plain 8K image these reads are
53: ; no-ops, so probing is safe.
54: FNBKSEL EQU 3D00H ; read +page: map image page into 2000H-2FFFH
55: FNBKMAX EQU 112 ; pages 0..111; 3DF0H+ stays reserved
56: ; (3DFEH = FNSWAP)
57:
58: ; Selecting a page is one instruction, no helper needed (a macro here would
59: ; be useless anyway: this file is included after its callers, and zmac
60: ; resolves forward EQUs but not forward macros). Constant page n:
61: LD A,(FNBKSEL+n)
62: ; Runtime page number in A:
63: LD D,FNRSEL
64: ADD A,80H
65: LD E,A
66: LD A,(DE)
67:
68:
69: ---- register numbers ----
70: FRDEVIC EQU 00H
71: FRCMD EQU 01H
72: FRNPARM EQU 02H
73: FRDATRS EQU 05H
74: FRXSLC EQU 06H
75: FRSEQ EQU 10H
76: FRBOOTL EQU 11H ; BOOTLOCK; magic 0B5H arms the swap
77: FRSEL1 EQU 12H ; BOOTSEL pair: 0B5H then 4AH on FRBSEL2
78: FRBSEL2 EQU 13H
79:
80: ---- values ----
81: FNDFUJI EQU 70H ; FujiBus device id of the FujiNet itself
82: FCACFGX EQU 0C4H ; GET_ADAPTERCONFIG_EXTENDED
83: FCMHOST EQU 0F9H ; MOUNT_HOST
84: FCMIMG EQU 0F8H ; MOUNT_IMAGE
85: FCSDFP EQU 0E2H ; SET_DEVICE_FULLPATH
86: FCRHSL EQU 0F4H ; READ_HOST_SLOTS
87: FCDIR EQU 0F7H ; OPEN_DIRECTORY
88: FCDIR EQU 0F6H ; READ_DIR_ENTRY
89: FCCDIR EQU 0F5H ; CLOSE_DIRECTORY
90: FCDPS EQU 0E4H ; SET_DIRECTORY_POSITION
91: FCACK EQU 06H ; reply command: ACK
92: FCNAK EQU 15H ; reply command: NAK
93: FMRAD EQU 1 ; disk access mode: read
94: FEOK EQU 0 ; FNR values
95: FENOLNK EQU 1
96: FETMOUT EQU 2
```

```

97 FBIDLE EQU 0 ; FNBSTAT values
98 FBXFER EQU 1
99 FBREADY EQU 2
100 FBFAIL EQU 80H
101 FNBIMAG EQU 0B5H ; BOOTLOCK magic
102
103 ; ---- routines -----
104
105 ; Write mailbox register: C = register number, A = value.
106 ; Clobbers A,B,D,E.
107 FNREGWR:
108 LD B,A ; hold the value
109 LD D,FNRSELP
110 LD E,C
111 LD A,(DE) ; arm the register
112 LD D,FNRDATP
113 LD E,B
114 LD A,(DE) ; deliver the value
115 RET
116
117 ; Append A to the TX stream. Clobbers A,D,E.
118 FNTXBYT:
119 LD D,FNDATAP
120 LD E,A
121 LD A,(DE)
122 RET
123
124 ; Append a 1-byte parameter with value A. Clobbers A,B,D,E.
125 FNPARB:
126 LD B,A
127 LD A,1 ; size byte
128 CALL FNTXBYT
129 LD A,B
130 JP FNTXBYT
131
132 ; Append the 0-terminated string at HL to the TX stream.
133 ; Clobbers A,D,E,HL.
134 FNTXSTR:
135 LD A,(HL)
136 OR A
137 RET Z
138 CALL FNTXBYT
139 INC HL
140 JR FNTXSTR
141
142 ; Begin a transaction: A = device, E = command, L = nparam.
143 ; Rewinds the TX pointer; the caller then streams params and payload.
144 ; Clobbers A,B,C,D,E,H,L.
145 FNBEGIN:
146 PUSH HL
147 PUSH DE
148 PUSH AF
149 LD C,FRDATRS
150 XOR A
151 CALL FNREGWR
152 POP AF
153 LD C,FRDEVIC
154 CALL FNREGWR
155 POP DE
156 LD A,E
157 LD C,FRCMD
158 CALL FNREGWR
159 POP HL
160 LD A,L
161 LD C,FRNPARM
162 JP FNREGWR
163
164 ; Commit the transaction and wait for the cart to acknowledge.
165 ; B = timeout in ~2 second quanta. The next sequence number is derived
166 ; from the cart's own persisted ACKSEQ + 1 (never a local counter: RESET
167 ; restarts this program but not the cart), wrapping 255 -> 1, 0 reserved.

```

```

168 ; Returns: carry set on timeout; else A = FNERR value (0 = ok).
169 ; Clobbers A,B,C,D,E,H,L.
170 FNCOMMIT:
171 LD A,(FNACKSQ)
172 INC A
173 JR NZ,FNCMS
174 INC A ; 255 wraps past the reserved 0
175 FNCMS: LD H,A
176 LD C,FRSEQ
177 CALL FNREGWR
178 FNCMO: LD DE,0 ; inner: 65536 polls, ~2 s at 1.789 MHz
179 FNCMI: LD A,(FNACKSQ)
180 CP H
181 JR Z,FNCMOK
182 DEC DE
183 LD A,D
184 OR E
185 JR NZ,FNCMI
186 DJNZ FNCMO
187 SCF ; timed out
188 RET
189 FNCMOK: LD A,(FNERR)
190 OR A ; clears carry
191 RET
192
193 ; Select reply slice A (0-3) and wait for the repaint to complete: the cart
194 ; publishes FNSECHO last, so the slice is whole once it echoes.
195 ; Returns carry set on timeout. Clobbers A,B,C,D,E,H.
196 FNSLICE:
197 LD H,A
198 LD C,FRXSLOC
199 CALL FNREGWR
200 LD B,1 ; ~2 s; a live cart repaints in microseconds
201 FNSLO: LD DE,0
202 FNSLI: LD A,(FNSECHO)
203 CP H
204 JR Z,FNSLOK
205 DEC DE
206 LD A,D
207 OR E
208 JR NZ,FNSLI
209 DJNZ FNSLO
210 SCF
211 RET
212 FNSLOK: LD A
213 RET
214
215 ; Is a Fujinet cartridge present? Z set if the magic is there.
216 ; Clobbers A,B.
217 FNCHECK:
218 LD A,(FNMAGF)
219 CP 'F'
220 RET NZ
221 LD A,(FNMAGN)
222 CP 'N'
223 RET
224

```

HVGLIB.H -- HOME VIDEO GAME LIBRARY EQUATES (ALL FIVE)

```

1 ; HVGLIB.H -- Home Video Game Library equates for the Bally Astrocade,
2 ; FujiNet subset.
3 ;
4 ; Copy of fujinet-firmware/pico/astrocade/testrom/HVGLIB.H; keep in step.
5 ;
6 ; An original transcription of the facts in "Software and Hardware for the
7 ; Bally Arcade: A Technical Description" (Dave Nutting Associates, 1978):
8 ; system-call indexes, hardware ports, on-board ROM cells and SENTRY codes.
9 ; Only what the FujiNet clients use is here; it keeps the community file's
10 ; names so sources read like every other BallyAlley-convention program.
11 ; zmac syntax.
12 ;
13 ; ---- User Program Interface ----
14 ; A system call is RST 56 followed by one byte: routine index + option.
15 ; Option 0 takes arguments in registers; option 1 (the "suck" bit) reads
16 ; them inline after the call byte.
17 ;
18 SUCK EQU 1 ; inline-arguments option bit
19 ;
20 SYSTEM macro callno ; args in registers
21 RST 56
22 DB callno
23 endm
24 ;
25 SYSSUK macro callno ; args follow inline
26 RST 56
27 DB callno+SUCK
28 endm
29 ;
30 ; Inside the interpreter (entered with SYSTEM INTPC) each call is one byte:
31 DO macro callno ; interpreter call, args inline
32 DB callno+SUCK
33 endm
34 ;
35 DONT macro callno ; interpreter call, args in registers
36 DB callno
37 endm
38 ;
39 EXIT macro ; leave the interpreter
40 DB XINTC
41 endm
42 ;
43 ; ---- System-call routine indexes ----
44 INTPC EQU 00H ; begin interpreting
45 XINTC EQU 02H ; exit interpreter
46 RCALL EQU 04H ; call assembly subroutine
47 MCALL EQU 06H ; call interpreter subroutine
48 MRET EQU 08H ; return from interpreter subroutine
49 MJUMP EQU 0AH ; interpreter jump
50 ACTINT EQU 0EH ; activate interrupts
51 DECCTS EQU 10H ; decrement counter/timers
52 SETOUT EQU 16H ; set display ports (VERBL, HORCB, INMOD)
53 COLSET EQU 18H ; set color registers
54 FILL EQU 1AH ; fill area with constant
55 RECTAN EQU 1CH ; paint a rectangle
56 WRITR EQU 20H ; write pattern, relative
57 WRIT EQU 24H ; write pattern
58 BLANK EQU 2AH ; blank area
59 CHRDIS EQU 32H ; display character
60 STRDIS EQU 34H ; display string (0-terminated)
61 DISNUM EQU 36H ; display BCD number
62 KCTASC EQU 40H ; key code to ASCII
63 SENTRY EQU 42H ; sense transition
64 DOT EQU 44H ; respond to input transition
65 PIZBRK EQU 48H ; coffee break
66 MENU EQU 4AH ; display menu, branch on selection
67 GETPAR EQU 4CH ; get game parameter
68 GETNUM EQU 4EH ; get number
69 PAWS EQU 50H ; pause
70 MOVE EQU 5EH ; move bytes
71 RANGED EQU 76H ; ranged random number
72 MSKTD EQU 7EH ; joystick mask to deltas
73 ;
74 ; ---- Output ports ----
75 COLR EQU 00H ; color registers, right palette
76 COL1R EQU 01H
77 COL2R EQU 02H
78 COL3R EQU 03H
79 COL0L EQU 04H ; color registers, left palette
80 COL1L EQU 05H
81 COL2L EQU 06H
82 COL3L EQU 07H
83 CONCM EQU 08H ; consumer/commercial (low/high res)
84 HORCB EQU 09H ; horizontal color boundary
85 VERBL EQU 0AH ; vertical blank line
86 MAGIC EQU 0CH ; magic register
87 INFBK EQU 0DH ; interrupt feedback
88 INMOD EQU 0EH ; interrupt enable and mode
89 INLIN EQU 0FH ; interrupt line
90 TONMO EQU 10H ; master oscillator
91 VOLC EQU 15H ; tone C volume, noise mode
92 VOLAB EQU 16H ; tone A/B volumes
93 VOLN EQU 17H ; noise volume
94 XPAND EQU 19H ; expand register
95 ;
96 ; ---- Input ports ----
97 INTST EQU 08H ; intercept feedback
98 VERAFF EQU 0EH ; vertical address feedback
99 HORAF EQU 0FH ; horizontal address feedback
100 SW0 EQU 10H ; player 1 handle
101 SW1 EQU 11H
102 SW2 EQU 12H
103 SW3 EQU 13H
104 KEY0 EQU 14H ; keypad column 0 (right)
105 KEY1 EQU 15H
106 KEY2 EQU 16H
107 KEY3 EQU 17H ; keypad column 3 (left)
108 POT0 EQU 1CH ; player 1 knob
109 POT1 EQU 1DH
110 POT2 EQU 1EH
111 POT3 EQU 1FH
112 ;
113 ; ---- Memory map ----
114 FIRSTC EQU 2000H ; first byte of cartridge ROM
115 NORMEM EQU 4000H ; screen RAM (normal, non-magic image)
116 BYTEPL EQU 40 ; bytes per screen line
117 BITSPL EQU 160 ; pixels per screen line
118 ;
119 ; ---- On-board ROM cells ----
120 MENUST EQU 0218H ; head of the on-board menu chain
121 ALKEYS EQU 0214H ; keypad mask: all 24 keys
122 ;
123 ; ---- SENTRY return codes (register B) ----
124 SMUL EQU 00H ; nothing happened
125 SSEC EQU 11H ; seconds timer counted down
126 SKYU EQU 12H ; key up
127 SKYD EQU 13H ; key down
128 ST0 EQU 14H ; player 1 trigger
129 S30 EQU 15H ; player 1 joystick
130 SP0 EQU 1CH ; player 1 knob
131 ;

```

STATE.INC -- THE FLAT REPLY WINDOW (NETCAT AND THE GAMES)

```
1 ; state.inc -- reading the reply where it lies.
2 ;
3 ; The reply stays painted in the cart's 256-byte slice window (FNRRDATA)
4 ; until the next transaction, so nothing is ever buffered: these helpers
5 ; give the rest of the program a flat 0-1023 offset space over it, and
6 ; switch slices only when a read crosses a 256-byte boundary. CURSLC
7 ; caches the slice the cart is currently publishing; every transaction
8 ; ends with the cart republishing slice 0 and net.inc re-homing the cache.
9
10 ; RXGETB: A = reply byte at offset HL. Preserves HL, BC, DE.
11 RXGETB: PUSH    BC
12             PUSH    DE
13             PUSH    HL
14             LD      A,H                ; slice = offset high byte (0-3)
15             LD      HL,CURSLC
16             CP      (HL)
17             JR      Z,RXG1
18             LD      (HL),A
19             CALL    FNSLICE            ; repaints in microseconds; a timeout here
20 RXG1:  POP      HL                    ; means the cart is gone anyway
21             PUSH    HL
22             LD      D,3BH              ; FNRRDATA page
23             LD      E,L
24             LD      A,(DE)
25             POP      HL
26             POP      DE
27             POP      BC
28             RET
29
30 ; RXGETW: DE = little-endian word at offset HL. Preserves HL, BC.
31 RXGETW: CALL    RXGETB
32             LD      E,A
33             INC     HL
34             CALL    RXGETB
35             LD      D,A
36             DEC     HL
37             RET
38
39 ; RXSTRN: copy at most B bytes from offset HL into (DE), stopping at a
40 ; NUL; the destination is always NUL-terminated. Preserves HL, DE.
41 RXSTRN: PUSH    HL
42             PUSH    DE
43 RXS1:  CALL    RXGETB
44             OR      A
45             JR      Z,RXS9
46             LD      (DE),A
47             INC     DE
48             INC     HL
49             DJNZ    RXS1
50 RXS9:  XOR      A
51             LD      (DE),A
52             POP      DE
53             POP      HL
54             RET
55
```

C.2 The games' shared UI

GFX.INC -- 4X6 RENDERER VIA THE MAGIC EXPANDER (THE THREE GAMES)

```
1 ; gfx.inc -- the 4x6 renderer, fills, and raw blits.
```

```
2 ;
```

```

3 ; The whole port draws on one alignment trick: at 2bpp, 4 pixels are one
4 ; byte, and the character grid is 4 pixels wide -- so every glyph row is a
5 ; single byte write and nothing here ever shifts. Text goes through the
6 ; magic EXPANDER: OUT the fg/bg pair to XPAND, reset the expander's nibble
7 ; flip-flop by writing the MAGIC register, and each following write through
8 ; magic memory (0000H + screen offset) paints the HIGH nibble of the data
9 ; as 4 pixels in those two colors. One MAGIC write per row keeps the
10 ; flip-flop on the high nibble; MAME's fungen and the Nutting description
11 ; agree on that reset.
12
13 ; Everything runs with interrupts off and draws opaquely (bg included), so
14 ; redraw over live screen never flickers and never needs a clear first.
15
16 ; Char grid: 40 cols x 15 rows of 4x6 cells over 90 visible lines.
17 ; Colors (palette registers): 0 felt, 1 black, 2 red, 3 white.
18
19 MAGEXP EQU 08H ; MAGIC value: expand, no shift/rotate/or/xor
20
21 ; XPAND color pairs (fg*4 + bg).
22 XWONF EQU 0CH+0 ; white on felt (table text)
23 XKONW EQU 04H+3 ; black on white (card ranks)
24 XRONW EQU 08H+3 ; red on white
25 XKONF EQU 04H+0 ; black on felt
26 XRONF EQU 08H+0 ; red on felt
27 XFONW EQU 00H+3 ; felt on white (inverse: highlights)
28
29 ; -----
30 ; TXTAT: string at HL (NUL-terminated) at char col E, row D, colors C
31 ; (an XPAND pair). TXTN: exactly B cells of the string -- stops copying
32 ; at the NUL; the rest with spaces, so fixed-width fields redraw
33 ; opaquely. Lowercase folds to uppercase; anything else outside 20H-5FH
34 ; draws as '?'. Clobbers everything.
35 TXTAT: LD B,0 ; 0: no pad, stop at NUL
36 JR TXT0
37 LD A,B
38 OR A
39 RET Z
40 TXT0: LD A,C
41 OUT (XPAND),A
42 CALL TXTPOS
43 TXT1: LD A,(HL)
44 OR A
45 JR Z,TXT3 ; NUL: pad or stop
46 INC HL
47 CP 61H ; fold 'a'-'z'
48 JR C,TXT2
49 CP 7BH
50 JR NC,TXT2
51 SUB 20H
52 SUB 20H
53 CP 40H
54 JR C,TXT4
55 LD A,'?'-20H
56 TXT4: CALL GLYPH
57 LD A,B
58 OR A
59 JR Z,TXT1 ; unbounded (TXTAT)
60 DJNZ TXT1
61 TXT3: LD A,B
62 OR A
63 RET Z ; TXTAT: done at NUL
64 TXT5: XOR A ; TXTN: space-pad the remainder
65 CALL GLYPH
66 DJNZ TXT5
67 RET
68
69 ; TXTPOS: D,E char row/col -> shadow DE = magic address; preserves the
70 ; primary registers (HL string, B count, C colors).
71
72 TXTPOS: PUSH HL
73 PUSH BC

```

```

74 LD A,D
75 ADD A,A
76 LD H,0
77 LD L,A
78 LD BC,ROWTAB
79 ADD HL,BC
80 LD A,(HL)
81 INC HL
82 LD H,(HL)
83 LD L,A
84 LD B,0
85 LD C,E
86 ADD HL,BC ; HL = row*240 + col
87 PUSH HL
88 EXX
89 POP DE ; shadow DE = magic address
90 EXX
91 POP BC
92 POP HL
93 RET
94
95 ; GLYPH: draw glyph index A (0-3FH) at the shadow-DE magic address and
96 ; advance it one cell right. Preserves primary HL/BC (shadow DE is the
97 ; running magic address, so the *6 scratch uses shadow BC).
98 GLYPH: EXX
99 LD L,A
100 LD H,0
101 ADD HL,HL ; A*2
102 LD B,H
103 C,L
104 ADD HL,HL ; A*4
105 ADD HL,BC ; A*6
106 LD BC,FONT46
107 ADD HL,BC ; HL = glyph rows
108 LD B,6
109 GLYRW: LD A,MAGEXP
110 OUT (MAGIC),A ; reset the expander nibble flip-flop
111 LD A,(HL)
112 LD (DE),A ; one row: 4 pixels
113 INC HL
114 LD A,E
115 ADD A,BYTEPL
116 LD E,A
117 JR NC,GLYRW1
118 INC D
119 GLYRW1: DJNZ GLYRW ; back up 6 lines, over 1 col
120 LD A,E
121 SUB 6+BYTEPL-1
122 LD E,A
123 JR NC,GLYRW2
124 DEC D
125 GLYRW2: EXX
126 RET
127
128 ; -----
129 ; CLRSCR: the whole visible screen back to felt. Clobbers A,BC,DE,HL.
130 CLRSCR: LD HL,NORMEM
131 LD DE,NORMEM+1
132 LD BC,LINES*BYTEPL-1
133 LD (HL),0
134 LDIR
135 RET
136
137 ; -----
138 ; FILLRC: fill C bytes x B lines with byte D, top-left at screen address
139 ; HL (NORMEM-based). Clobbers A,B,E,HL.
140 FILLRC: PUSH BC
141 PUSH HL
142 FILL1: LD (HL),D
143 INC HL
144 DEC C

```



```

145      JR      NZ,FILL1
146      POP     HL
147      POP     BC
148      LD      A,L
149      ADD     A,BYTEPL
150      LD      L,A
151      JR      NC,FILL2
152      INC     H
153 FILL2: DJNZ   FILLRC
154      RET
155
156 ; RAWBLT: copy C bytes x B rows from HL to screen address DE
157 ; (NORMEM-based), advancing DE one screen line per row. Clobbers all.
158 RAWBLT: PUSH DE
159      PUSH    BC
160 RAWB1:  LD      A,(HL)
161      LD      (DE),A
162      INC     HL
163      INC     DE
164      DEC     C
165      JR      NZ,RAWB1
166      POP     BC
167      POP     DE
168      LD      A,E
169      ADD     A,BYTEPL
170      LD      E,A
171      JR      NC,RAWB2
172      INC     D
173 RAWB2:  DJNZ   RAWBLT
174      RET
175
176 ; -----
177 ; U16STR: HL as decimal into LINBUF, leading zeros suppressed, NUL
178 ; terminated. Clobbers everything.
179 U16STR: LD      DE,LINBUF
180      LD      BC,10000
181      CALL    U16DIG
182      LD      BC,1000
183      CALL    U16DIG
184      LD      BC,100
185      CALL    U16DIG
186      LD      BC,10
187      CALL    U16DIG
188      LD      A,L
189      ADD     A,'0',A
190      LD      (DE),A
191      INC     DE
192      XOR     A
193      LD      (DE),A
194      RET
195 U16DIG: XOR     A                ; digit for power BC
196 U16D1: SBC     HL,BC
197      JR      C,U16D2
198      INC     A
199      JR      U16D1
200 U16D2: ADD     HL,BC            ; undo the overshoot
201      OR      A
202      JR      NZ,U16D3          ; a real digit
203      LD      A,E
204      OR      A                ; suppress a leading zero: buffer still
205      RET     Z                ; empty? (LINBUF sits at 4F00H -- page
206      XOR     A                ; aligned -- so low byte 0 is the start)
207 U16D3: ADD     A,'0',A
208      LD      (DE),A
209      INC     DE
210      RET
211
212 ROWTAB: DW      0*240,1*240,2*240,3*240,4*240
213      DW      5*240,6*240,7*240,8*240,9*240
214      DW      10*240,11*240,12*240,13*240,14*240
215

```

INPUT.INC -- KEYPAD AND HAND CONTROL (THE THREE GAMES)

```
1 ; input.inc -- keypad and hand-controller scanning.
2 ;
3 ; The 24-key keypad reads as four column ports (KEY0 14H rightmost ..
4 ; KEY3 17H leftmost), active-high bits 0-5; the player 1 handle is port
5 ; 10H, active-high U/D/L/R/T in bits 0-4. Both are folded into one code
6 ; space: '0'-'9', 'E' (the CE key), '.', and U/D/L/R/T -- the keypad's own
7 ; up/down keys return the same 'U'/'D' as the stick, so list navigation
8 ; needs no special cases. Codes are edge-detected against V_KEY: a key
9 ; reports once per press.
10 ;
11 ; KEYSCN: A = freshly pressed code, or 0. Clobbers A,B,C,D,E,HL.
12 KEYSCN: CALL KEYRAW
13 LD HL,V_KEY
14 CP (HL)
15 JR NZ,KSNEW
16 XOR A ; held: already reported
17 RET
18 KSNEW: LD (HL),A
19 OR A
20 RET
21 ;
22 ; KEYRAW: A = currently pressed code, or 0. First match wins.
23 KEYRAW: LD HL,KEYMAP
24 LD B,3 ; three keypad columns carry mapped keys
25 KRCOL: LD C,(HL)
26 INC HL
27 IN A,(C)
28 LD D,A
29 LD E,6
30 KRBIT: LD A,(HL)
31 INC HL
32 SRL D
33 JR NC,KRB2
34 OR A
35 RET NZ ; pressed and mapped
36 KRB2: DEC E
37 JR NZ,KRBIT
38 DJNZ KRCOL
39 IN A,(SW0) ; the stick
40 AND 1FH
41 RET Z
42 LD D,A ; keep only the lowest set bit, so a
43 NEG ; diagonal cannot look like two keys
44 AND D
45 LD HL,STKMAP
46 KRSTK: SRL A
47 JR C,KRSTK1
48 INC HL
49 JR KRSTK
50 KRSTK1: LD A,(HL)
51 RET
52 ;
53 ; Column port, then the six bit codes (bit 0 first; 0 = unmapped).
54 KEYMAP: DB KEY3
55 DB 0,0,'7','4','1','E' ; C MR 7 4 1 CE
56 DB KEY2
57 DB 'U',0,'8','5','2','0' ; up MS 8 5 2 0
58 DB KEY1
59 DB 'D',0,'9','6','3','.' ; down CH 9 6 3 .
60 STKMAP: DB 'U','D','L','R','T'
61
```

C.3 NETCAT

NC.ASM

```
1  ; nc.asm -- netcat for the Bally Astrocade, over FujiNet.
2
3  ; The Family's minimal terminal: open an N: devicespec and then pump bytes
4  ; between the connection and the screen forever. This is the first Astrocade
5  ; FujiNet client that holds a connection open rather than doing a one-shot
6  ; HTTP GET, so it is also the first to send N: WRITE (net.inc) and the first
7  ; with a pane that scrolls (term.inc).
8
9  ; ROM budget: 0000H-1AFFH of the 8K window (6,912 bytes); 1B00H+ belongs to
10 ; the mailbox and build.sh stamps the "FUJI" claim at 1CFCH so the mailbox
11 ; stays alive when this image is booted over the network.
12
13 ; RAM is screen RAM, full stop. LINES=84 shows 14 rows of 4x6 text and
14 ; leaves 4D20H up -- 736 bytes -- to the program; the card games' LINES=90
15 ; would have left 496, and the terminal's shadow does not fit in that.
16
17 ; Interrupts stay off for the program's whole life (fujilib.inc's contract:
18 ; with I = 0, refresh strays land in OS ROM and never hit the hotspots).
19
20     INCLUDE "HVGLIB.H"
21     INCLUDE "fujinet.inc"
22
23 ; ---- geometry -----
24 LINES EQU 84 ; visible scanlines: 14 rows of 4x6
25 TCOLS EQU 40 ; columns; also the cols= we tell the BBS
26 TROWS EQU 13 ; scrolling terminal rows, 0..12
27 STATROW EQU 13 ; the status row
28 TSHTOP EQU 9 ; first terminal row the overlay covers
29 TSHROWS EQU 4 ; how many, hence the shadow's size
30 EDMAXL EQU TXMAX-8 ; longest devicespec or composed line: the
31 ; write cap less the CR LF and some slack, so
32 ; a full line plus its terminator still fits
33 ; the mailbox's TX stream
34
35 ; ---- RAM map -----
36 ; 4000H-4D1FH is the visible screen. Everything below is ours.
37 TSHAD EQU 4D20H ; 160: character shadow of rows 9-12
38 EDBUF EQU 4DC0H ; 128: devicespec / composed line. The two
39 ; are never live at the same time.
40 ; a one-byte write's staging cell
41 TXONE EQU 4E40H ; cursor column -- must sit immediately
42 TCOL EQU 4E41H ; below TROW: term.inc does LD DE, (TCOL)
43 TROW EQU 4E42H ; escape-sequence state, 0 = normal
44 TESC EQU 4E43H ; bytes swallowed in this sequence
45 TESCN EQU 4E44H ; last raw key, for edge detection
46 V_KEY EQU 4E45H ; auto-repeat countdown
47 V_RPT EQU 4E46H ; grid keyboard cursor
48 V_GX EQU 4E47H ; the editor's top row
49 V_GY EQU 4E48H ; the editor's length cap
50 EDITOPR EQU 4E49H ; value characters currently shown
51 EDMAX EQU 4E4AH ; grid cell being drawn
52 EDVN EQU 4E4BH ; word: the editor's hint string
53 EDCIX EQU 4E4CH ; reply slice the cart is publishing
54 EDHNT EQU 4E4EH ; word: NET STATUS bytes waiting
55 CURSLC EQU 4E50H ; NET_STATUS connected flag
56 AVAIL EQU 4E52H ; NET_STATUS nDevStatus_t
57 NCONN EQU 4E54H ; word: reply length, captured after a READ
58 NDEVST EQU 4E55H ; grows down; 4FC0H+ left to the BIOS cells
59 RXLEN EQU 4E56H
60 STACK EQU 4FC0H
61
62 ; MB_* labels are module fences for tools/checksize.py's budget table.
63 MB_MAIN: ORG FIRSTC
64 DB 55H
65
66 DW MENUST
67 DW PRGNAM
68 PRGSTR
69 DB "NETCAT"
70 DB 0
71
72 PRGSTR: DI
73 LD SP, STACK
74 LD INTPC
75 DO INTOUT
76 DB LINES*2 ; blank below the text
77 DB 0 ; HORCB 0: the whole line is the right palette
78 DB 8
79 DO COLSET
80 DW PALET
81 DO FILL
82 DW NORMEM
83 DW LINES*BYTEPL
84 DB 0 ; color 0 everywhere
85 EXIT
86
87 CALL FNCHECK
88 JP NZ, NOCARD
89
90 ; The keypad that picked us off the on-board menu is still down.
91 ; Drain it, or the dial screen's editor takes it as typing.
92 KWAIT: CALL KEYRAW
93 OR A
94 JR NZ, KWAIT
95
96 XOR A
97 LD (V_KEY), A
98 LD (CURSLC), A
99 LD A, 1 ; start the grid cursor on 'A', not on space
100 LD (V_GX), A
101 LD (V_GY), A
102 CALL SEEDU
103 CALL DIAL
104 CALL SESSN
105 JR MAINLP
106
107 ; ---- Errors -----
108 NOCARD: LD HL, SNOCART
109 LD D, 6
110 LD E, 3
111 LD C, XAONK
112 CALL TXTAT
113 HALTE: JR HALTE
114
115 ; ---- Data -----
116 MB_DATA:
117 ; COLSET stores descending, ports 7 down to 0, four bytes per palette in the
118 ; order color 3, 2, 1, 0; both halves are identical since HORCB is 0.
119 ; Byte = (hue < 3) | luminance, and hue 0 is the grayscale column.
120 ; 3 white, 2 green (the accent), 1 gray, 0 black.
121 PALET: DB 07H, 0A4H, 03H, 00H
122 DB 07H, 0A4H, 03H, 00H
123
124 INCLUDE "build/endpoint.inc"
125
126 MB_SCREEN:
127 INCLUDE "screens.inc"
128
129 MB_TERM:
130 INCLUDE "term.inc"
```

```

129 MB_EDIT:      INCLUDE "edit.inc"
130
131 MB_INPUT:      INCLUDE "input.inc"
132
133 MB_NET:        INCLUDE "net.inc"
134
135 MB_STATE:      INCLUDE "state.inc"
136
137 MB_GFX:        INCLUDE "gfx.inc"
138
139 MB_FONT:       INCLUDE "assets/font.inc"
140
141 MB_FUJILIB:    INCLUDE "fujilib.inc"
142
143 MB_END:
144

```

FUJINET.INC

```

1 ; fujinet.inc -- N: device commands and reply offsets.
2 ;
3 ; The N: command values are hand-synchronised with the other Astrocade
4 ; ports (fujinet-battleship/astrocade/fujinet.inc, whose o2/ family carries
5 ; the documentation of WHY each is what it is); the canonical list is
6 ; fujinet-firmware/include/fujiCommandID.h. The mailbox transport itself is
7 ; fujilib.inc.
8 ;
9 ; NCWRITE is new here: every Astrocade client so far has been a one-shot
10 ; HTTP GET, so none of them ever sent one. Its shape is transcribed from
11 ; netcat/intv/fujinet.bas's net_write, the same mailbox family.
12 ;
13 ; ---- Network device (N:) -----
14 NETDEV EQU 71H ; N1:
15 NCOPEN EQU 4FH ; 'O'
16 NCCLOSE EQU 43H ; 'C'
17 NCREAD EQU 52H ; 'R'
18 NCSTAT EQU 53H ; 'S'
19 NCWRITE EQU 57H ; 'W'
20
21 NMRDWR EQU 0CH ; open mode: read/write (the netcat mode:
22 ; the HTTP adapter reads the same 12 as GET)
23 NTRNONE EQU 0 ; translation: none -- we terminal ourselves
24
25 ; NET_STATUS reply: 4 bytes in slice 0.
26 NSAVLO EQU 0 ; bytes waiting, low
27 NSAVHI EQU 1 ; bytes waiting, high
28 NSCONN EQU 2 ; connected flag -- the session loop condition
29 NSDEVST EQU 3 ; nDevStatus_t: 1 SUCCESS, 136 END_OF_FILE.
30 ; NetworkProtocolTELNET derives from
31 ; NetworkProtocolTCP, whose status_client()
32 ; sets connected = client.connected() and
33 ; error = connected ? error : END_OF_FILE --
34 ; so NSCONN drives the loop and this only
35 ; tells a clean remote hangup from a fault.
36 NDSOK EQU 1 ; NDEV_STATUS::SUCCESS
37 NDSEOF EQU 136 ; NDEV_STATUS::END_OF_FILE
38
39 ; ---- Sizes -----
40 ; The cart publishes replies through four 256-byte slices; state.inc's
41 ; RXGETB gives a flat 0-1023 offset space over them.
42 RXMAX EQU 1024 ; FUJIMAIL_RX_MAX: the READ clamp
43 TXMAX EQU 128 ; our WRITE cap. FN_TX_MAX is 320 and a
44 ; write spends 3 bytes on the length
45 ; parameter, so this is far inside it.
46

```

NET.INC

```

1 ; net.inc -- the N: device, for a connection that is HELD open.
2 ;
3 ; Every other Astrocade FujiNet client is one-shot: OPEN an HTTP GET, poll
4 STATUS until the body has settled, READ it once, and CLOSE at the start of
5 the NEXT request. netcat opens once and then pumps STATUS/READ/WRITE for
6 the life of the session, so three of the card ports' rules change:
7 ;
8 ; * CLOSE goes where it reads. They deferred it because every transaction
9 repays the whole reply window and their screens render straight out of
10 that window between polls. We consume each READ into the terminal
11 immediately, so nothing is left pointing at the window; CLOSE belongs at
12 hangup and on failure.
13 ; * There is no settle loop. They polled STATUS until two consecutive
14 readings of 'avail' agreed, because an HTTP body arrives in pieces and
15 they wanted all of it. On a live stream that never terminates -- a
16 chatty server never produces two equal readings -- and it would add a
17 poll's latency to every echoed keystroke. We take whatever is there.
18 ; * NSCONN, not NSDEVST, is the loop condition (see fujinet.inc).
19 ;
20 ; Kept from them, because they are properties of the mailbox and not of the
21 request: FNCOMMIT derives the next SEQ from the cart's own persisted
22 ACKSEQ, and FNRLXLL/FNRLXLH belong to the most recent transaction, so the
23 reply length is captured the instant the READ returns.
24 ;
25 ; All routines return carry SET on failure, clear on success.
26 ;
27 ; NCMD: begin a network-device transaction. E = command, L = nparam.
28 NCMD: LD A,NETDEV
29 JP FNBEGIN
30 ;
31 ; SLC0: every transaction makes the cart republish slice 0, so re-home
32 ; state.inc's slice cache. Clobbers A.
33 SLC0: XOR A
34 LD (CURSLC),A
35 RET
36 ;
37 ;-----
38 ; NOPEN: OPEN the devicespec in EDBUF for read/write, no translation.
39 NOPEN: LD E,NCOPEN
40 LD L,2
41 CALL NCMD
42 LD A,NMRDWR
43 CALL FNPARB
44 LD A,NTRNONE
45 CALL FNPARB
46 CALL BLDURL ; a TCP connect plus telnet negotiation
47 LD B,8
48 CALL FNCOMMIT
49 RET C
50 JR NZ,NOBAD
51 CALL SLC0
52 LD A,(FNREPLY)
53 CP FACK
54 RET Z ; equal: carry already clear
55 NOBAD: SCF
56 RET
57 ;
58 ; NCLOSE: hang up. A close with nothing open earns a NAK; callers that are
59 ; already failing ignore the result.
60 NCLOSE: LD E,NCLOSE
61 LD L,0
62 CALL NCMD
63 LD B,3 ; ~6 s
64 CALL FNCOMMIT
65 JP SLC0
66 ;
67 ; NSTATUS: one poll. On success AVAIL, NCONN and NDEVST are fresh.

```

```

68 NSTATUS: LD E,NCSTAT
69 LD L,2
70 CALL NCMD
71 XOR A
72 CALL FNPARB
73 XOR A
74 CALL FNPARB
75 LD B,4
76 CALL FNCOMMIT
77 RET C
78 JR NZ,NSBAD
79 LD A,(FNREPLY)
80 CP FACK
81 JR NZ,NSBAD
82 LD A,(FNRRDATA+NSAVLO)
83 LD (AVAIL),A
84 LD A,(FNRRDATA+NSAVHI)
85 LD (AVAIL+1),A
86 LD A,(FNRRDATA+NSCONN)
87 LD (NCONN),A
88 LD A,(FNRRDATA+NSDEVST)
89 LD (NDEVST),A
90 CALL SLC0
91 XOR A ; clears carry
92 RET
93 NSBAD: SCF
94 RET
95 ;
96 ; NREAD: READ min(AVAIL, RXMAX) bytes into the reply window, then capture
97 ; the reply length -- the next transaction's would replace it -- and re-home
98 ; the slice cursor. The bytes stay in the window; state.inc reads them there.
99 NREAD: LD HL,(AVAIL)
100 LD DE,RXMAX
101 OR A
102 SBC HL,DE
103 JR C,NRD1 ; avail < clamp: take avail
104 LD H,D ; else take the clamp
105 LD L,E
106 JR NRD2
107 NRD1: ADD HL,DE ; undo the compare
108 NRD2: PUSH HL
109 LD E,NCREAD
110 LD L,1
111 CALL NCMD
112 LD A,2 ; one 2-byte parameter: the length, LE
113 CALL FNTXBYT
114 POP HL
115 PUSH HL
116 LD A,L
117 CALL FNTXBYT
118 POP HL
119 LD A,H
120 CALL FNTXBYT
121 LD B,8
122 CALL FNCOMMIT
123 RET C
124 JR NZ,NRBAD
125 LD A,(FNREPLY)
126 CP FACK
127 JR NZ,NRBAD
128 LD A,(FNRLXLL) ; capture NOW
129 LD (RXLEN),A
130 LD A,(FNRLXLH)
131 LD (RXLEN+1),A
132 JP SLC0 ; XOR A inside: carry clear
133 NRBAD: SCF
134 RET

```

```

135
136 ; NWRITE: send B bytes from (HL). B is at most TXMAX, so the length's high
137 ; byte is always zero. The count rides twice -- once as the parameter, once
138 ; as the payload, the card measures -- exactly as the Intellivision client
139 ; does it. Clobbers everything.
140 NWRITE: LD A,B
141 OR A
142 RET Z ; nothing to send, carry clear
143 PUSH HL
144 PUSH BC
145 LD E,NCWRITE
146 LD L,1
147 CALL NCMO ; clobbers HL and BC, hence the pushes
148 LD A,2 ; parameter size byte: a 2-byte length
149 CALL FNTXBYT
150 POP BC
151 PUSH BC
152 LD A,B
153 CALL FNTXBYT ; length, low
154 XOR A
155 CALL FNTXBYT ; length, high
156 POP BC
157 POP HL
158 NWR1: LD A,(HL) ; FNTXBYT touches only A,D,E
159 CALL FNTXBYT
160 INC HL
161 DJNZ NWR1
162 LD B,4
163 CALL FNCOMMIT
164 RET C
165 JR NZ,NWBAD
166 CALL SLC0
167 LD A,(FNREPLY)
168 CP FACK
169 RET Z
170 NWBAD: SCF
171 RET
172
173 ; NWRIT1: send the single byte in A.
174 NWRIT1: LD (TXONE),A
175 LD HL,TXONE
176 LD B,1
177 JP NWRITE
178
179 ; NWRSTR: send the NUL-terminated string at HL.
180 NWRSTR: PUSH HL
181 LD B,0
182 NWS1: LD A,(HL)
183 OR A
184 JR Z,NWS2
185 INC HL
186 INC B
187 JR NWS1
188 NWS2: POP HL
189 JP NWRITE
190
191 ;-----
192 ; BLDURL: stream the devicespec in EDBUF into the TX stream a character at
193 ; a time -- the card ports' way; it is never assembled anywhere else.
194
195 ; A bare spec gets "N:" prepended, and a TELNET spec with no query of its
196 ; own gets ?cols=40&rows=13 appended. NetworkProtocolTELNET::open() reads
197 ; cols/rows off the query and only then offers NAWs, and its default
198 ; terminal type is already "dumb" -- so this one string is what makes a BBS
199 ; wrap to this spec instead of to 80 columns. A spec that brought its own
200 ; query is left exactly alone, so ?term=ansi stays available.
201 BLDURL: LD A,(EDBUF)
202 CALL UPCASE
203 CP 'N'
204 JR Z,BURL1
205 LD A,'N'

```

```

206 CALL FNTXBYT
207 LD A','
208 CALL FNTXBYT
209 BURL1: LD HL,EDBUF
210 CALL FNTXSTR
211 CALL WANTSZ
212 RET NZ
213 LD HL,UQNAWS
214 JP FNTXSTR
215
216 ; WANTSZ: Z when EDBUF is a TELNET spec carrying no '?' of its own.
217 ; Clobbers A,B,C,D,E,H,L.
218 WANTSZ: LD HL,EDBUF
219 WSQ: LD A,(HL) ; any query at all disqualifies it
220 OR A
221 JR Z,WSQ9
222 CP '?'
223 JR Z,WSNO
224 INC HL
225 JR WSQ
226 WSQ9: LD HL,EDBUF ; step over a leading "N:" or "Nn:"
227 LD A,(HL)
228 CALL UPCASE
229 CP 'N'
230 JR NZ,WSCMP ; no unit prefix: the scheme starts here
231 INC HL
232 LD A,(HL)
233 CP ':'
234 JR Z,WSCOL ; "Nn:" -- skip the unit digit
235 INC HL
236 LD A,(HL)
237 CP ':'
238 JR NZ,WSNO
239 WSCOL: INC HL
240 WSCMP: LD DE,STELNET
241 LD B,6
242 WSC1: LD A,(DE)
243 LD C,A
244 LD A,(HL)
245 CALL UPCASE
246 CP C
247 JR NZ,WSNO
248 INC HL
249 INC DE
250 DJNZ WSC1
251 XOR A ; Z: append the window size
252 RET
253 WSNO: OR 1 ; NZ: leave the spec alone
254 RET
255
256 ; UPCASE: fold a-z in A to A-Z. Clobbers A only.
257 UPCASE: CP 'a'
258 RET C
259 CP 'z'+1
260 RET NC
261 SUB 20H
262 RET
263
264 STELNET: DB "TELNET"
265 UQNAWS: DB "?cols=", '0'+TCOLS/10, '0'+TCOLS-(TCOLS/10)*10
266 DB "&rows=", '0'+TROWS/10, '0'+TROWS-(TROWS/10)*10
267 DB 0
268

```

SCREENS.INC

```

1 ; screens.inc -- the dial screen and the session loop.
2
3 ; -----
4 ; DIAL: pick a devicespec and open it. Returns only once connected.
5 DIAL: CALL CLRSCR
6         LD HL,SBANRR
7         LD D,0
8         LD C,XAONK
9         CALL TXTAT
10        LD HL,SHELP1
11        LD D,2
12        LD E,0
13        LD C,XWONK
14        CALL TXTAT
15        LD HL,SHELP2
16        LD D,3
17        LD E,0
18        LD C,XWONK
19        CALL TXTAT
20        LD A,5
21        LD (EDTOPR),A
22        LD A,EDMAXL
23        LD (EDMAX),A
24        LD HL,SHDIAL
25        LD (EDHNT),HL
26        CALL EDIT
27        JR C,DDFLT ; C: put the compiled-in default back
28        CALL EDLEN
29        OR A
30        JR Z,DIAL ; nothing typed: ask again
31        LD HL,SCONN
32        CALL DMSG
33        CALL NOPEN
34        RET NC ; connected
35        CALL NCLOSE ; retire the half-open attempt
36        LD HL,SEFAIL
37        CALL DMSG
38        CALL INWAIT
39        JR DIAL
40        DDFLT: CALL SEEDU
41        JR DIAL
42
43 ; DMSG: message at HL on the dial screen's message row.
44 DMSG: LD D,11
45        LD E,0
46        LD C,XAONK
47        LD B,TCOLS
48        JP TXTN
49
50 ; SEEDU: copy the compiled-in default devicespec into EDBUF.
51 SEEDU: LD HL,UENDPT
52        LD DE,EDBUF
53        LD A,(HL)
54        LD (DE),A
55        OR A
56        RET Z
57        INC HL
58        INC DE
59        JR SEED1
60
61 ; -----
62 ; SESSN: pump the connection until it ends or the user hangs up.
63 SESSN: CALL INIT
64        CALL SSTAT
65        CALL TCURON
66
67 SESS1: CALL NSTATUS
68        JR C,SLOST
69        OR A,(NCONN)
70        OR A
71        JR Z,SEOF
72        LD HL,(AVAIL)
73        LD A,H
74        OR L
75        JR Z,SESSI
76        CALL TCUROF
77        CALL NREAD
78        JR C,SLOST
79        CALL SREND
80        CALL TCURON
81        JR SESS2
82 ; Nothing waiting: pause ~11 ms before asking again. Flat-out polling costs
83 ; several hundred mailbox round trips a second to learn nothing, and this
84 ; adds no delay at all to the path that matters -- once bytes ARE waiting the
85 ; loop runs at full speed until the stream is drained.
86 SESS1: CALL DELAY10
87 SESS2: CALL SPOLL
88        JR NC,SESS1
89        LD HL,SHUNG ; the user pressed C
90        JR SEND9
91
92 SEOF: LD HL,SDISC ; the peer closed: NSDEVST would read
93        JR SEND9 ; END_OF_FILE here
94 SLOST: LD HL,SLOSTM ; the transaction itself failed
95 SEND9: PUSH HL
96        CALL TCUROF
97        CALL NCLOSE
98        POP HL
99        CALL SSTAT
100       CALL INWAIT
101       RET
102
103 ; SSTAT: the session's status row.
104 SSTAT: LD HL,SHSESS
105 SSTAT: LD D,STATROW
106        LD E,0
107        LD C,XKONA
108        LD B,TCOLS
109        JP TXTN
110
111 ; SREND: put the whole reply through the terminal. The bytes are read where
112 ; they lie, out of the cart's slice window; RXLEN was captured by NREAD.
113 SREND: LD HL,0 ; flat offset into the reply
114 SRD1: LD DE,(RXLEN)
115        PUSH HL
116        OR A
117        SBC HL,DE
118        POP HL
119        RET NC ; consumed it all
120        PUSH HL
121        CALL RXGETB ; preserves HL
122        CALL TPUTC ; clobbers everything
123        POP HL
124        INC HL
125        JR SRD1
126
127 ; -----
128 ; SPOLL: one pass of the keyboard. Carry set if the user hung up.
129
130 ; Keys send immediately -- a BBS menu is one keypress, and making the user
131 ; open a composer for every "M" would be miserable. The stick sends cursor
132 ; keys, which is free: nothing else wants it while the overlay is closed.

```

```

133 SPOLL: CALL INPOLL
134 OR A
135 RET Z ; nothing pressed; carry clear
136 CP EVCLR
137 JR Z,SPHANG
138 CP EVTRIG
139 JR Z,SPCOMP
140 CP EVEQ
141 JR Z,SPCR
142 CP EVCE
143 JR Z,SPBS
144 CP EVMR
145 JR Z,SPESC
146 CP EVMS
147 JR Z,SPETX
148 CP EVUP
149 JR Z,SPAU
150 CP EVDOWN
151 JR Z,SPAD
152 CP EVLEFT
153 JR Z,SPAL
154 CP EVRIGHT
155 JR Z,SPAR
156 CALL EVTOCH
157 OR A
158 RET Z ; a key that types nothing
159 CALL NWRIT1
160 OR A ; a failed write is not a hangup: the next
161 RET ; NSTATUS decides that
162
163 SPHANG: SCF
164 RET
165 SPCR: LD HL,SCRLF
166 JR SPSTR
167 SPESC: LD A,27
168 JR SPBYT
169 SPETX: LD A,3
170 JR SPBYT
171 SPBS: LD A,8
172 SPBYT: CALL NWRIT1
173 OR A
174 RET
175 SPAU: LD HL,SARRU
176 JR SPSTR
177 SPAD: LD HL,SARRD
178 JR SPSTR
179 SPAL: LD HL,SARRL
180 JR SPSTR
181 SPAR: LD HL,SARRR
182 SPSTR: CALL NWRSTR
183 OR A
184 RET
185
186 ; SPCOMP: the grid keyboard, over the bottom of the pane. The rows it
187 ; covers come back from term.inc's shadow.
188 SPCOMP: CALL TCURUF
189 XOR A
190 LD (EDBUF),A ; each line starts empty
191 LD A,TSHTOP
192 LD (EDTOPR),A
193 LD A,EDMAXL
194 LD (EDMAX),A
195 LD HL,SHCOMP
196 LD (EDHNT),HL
197 CALL EDIT
198 PUSH AF ; EDIT's carry, across the repaint
199 CALL TRSHAD
200 CALL SSTAT
201 CALL TCURON
202 AF POP
203 JR C,SPC9 ; cancelled

```

```

204 CALL EDLEN
205 OR A
206 JR Z,SPC9
207 LD B,A
208 LD HL,EDBUF
209 CALL NWRITE
210 LD HL,SCRLF
211 CALL NWRSTR
212 SPC9: OR A ; not a hangup either way
213 RET
214
215 ; -----
216 SBANNR: DB "FujiNet netcat",0
217 SHELPI: DB "Stick and trigger pick characters.",0
218 SHELPI2: DB "the keypad types digits directly.",0
219 SHDIAL: DB "TRIG pick CE del = connect C dflt",0
220 SHCOMP: DB "TRIG pick CE del = send C esc",0
221 SHSESS: DB " keys type = enter C hang up",0
222 SCONN: DB "Connecting...",0
223 SEFAIL: DB "Connect failed - press a key",0
224 SDISC: DB "Remote host closed - press a key",0
225 SLOSTM: DB "Connection lost - press a key",0
226 SHUNG: DB "Disconnected - press a key",0
227 SNOCart: DB "No FujiNet cartridge",0
228
229 SCRLF: DB 13,10,0
230 SARRU: DB 27,"[A",0
231 SARRD: DB 27,"[B",0
232 SARRR: DB 27,"[C",0
233 SARRL: DB 27,"[D",0
234

```


TERM.INC

```

1 ; term.inc -- the scrolling terminal pane.
2
3 ; No other Astrocade FujiNet client has one: they all redraw opaquely from
4 ; the reply window every poll, so nothing ever had to move. This does.
5
6 ; SCROLLING. There is no display-base or scroll register on this hardware
7 ; (videoram is a flat mapping at 4000H, and the BIOS SCROLL call is itself a
8 ; software copy), so a line feed at the bottom of the pane is an LDIR of
9 ; (TROWS-1)*240 bytes. At 21 T-states a byte on a 1.789 MHz Z80 that is
10 ; ~34 ms, about two frames, and it happens once per line -- cheap enough
11 ; that the TV-typewriter fallback (re-home and clear forward) is not needed.
12
13 ; THE SHADOW. The grid keyboard opens over the bottom of the pane, so the
14 ; TSHROWS rows of it cover have to come back afterwards. A character shadow of
15 ; just those rows is 160 bytes; one of the whole pane would be 520 and RAM
16 ; here is 736 bytes total. TPUTC records into it, TSCROL shifts it by a row
17 ; in step with the screen, and TRSHAD repaints from it.
18
19 ; THE CURSOR is the cell's bottom scanline, set to 0FFH and cleared to 0 by
20 ; direct (non-magic) writes. It sits where the next character will land,
21 ; which in normal flow is blank, so clearing to zero is right; the one cost
22 ; is that a character the cursor previously rested on loses its bottom pixel
23 ; row. That is worth not spending 520 bytes on a full-pane shadow.
24
25 ; TINIT: clear the pane, home the cursor, drop any escape state.
26 ; Clobbers everything.
27 TINIT: XOR A
28 LD (TCOL),A
29 LD (TROW),A
30 LD (TESC),A
31 LD HL,TSHAD ; glyph index 0 is the space
32 LD DE,TSHAD+1
33 LD BC,TSHROWS*TCOLS-1
34 LD (HL),0
35 LDIR
36 XOR A
37 LD B,TROWS
38 JP CLRRWS
39
40 ;-----
41 ; TPUTC: put the byte in A through the terminal. Clobbers everything.
42 TPUTC: LD C,A ; C holds the byte throughout
43 LD A,(TESC)
44 OR A
45 JP NZ,TPESCS
46 LD A,C
47 CP 20H
48 JR C,TPCTL
49 CP 7FH
50 RET NC ; DEL, and 8-bit CP437 art: drop it rather
51 SUB 20H ; than print a wrong glyph
52 ; falls into TPGLY
53
54 ; TPGLY: draw glyph index A at the cursor and advance.
55 TPGLY: CALL TPDRAW
56 LD A,(TCOL)
57 INC A
58 CP TCOLS
59 JR C,TPG1
60 XOR A ; wrap
61 LD (TCOL),A
62 JR TNEWLN
63 TPG1: LD (TCOL),A
64 RET
65
66 ; TPDRAW: draw glyph index A at the cursor without moving it, recording it
67 ; in the shadow if the cursor is inside the shadowed band.
68 TPDRAW: PUSH AF
69 CALL TSHPUT
70 POP AF
71 LD DE,(TCOL) ; E = TCOL, D = TROW -- they are adjacent
72 LD C,XWONK
73 JP GLYPH1
74
75 ; TNEWLN: down one row, scrolling if already at the bottom.
76 TNEWLN: LD A,(TROW)
77 INC A
78 CP TROWS
79 JR NC,TSCROL
80 LD (TROW),A
81 RET
82
83 ; TSCROL: the whole pane up one row, screen and shadow together.
84 TSCROL: LD A,TROWS
85 CALL SCRLUP
86 LD A,TROWS-1
87 CALL CLRRW
88 LD HL,TSHAD+TCOLS
89 LD DE,TSHAD
90 LD BC,(TSHROWS-1)*TCOLS
91 LDIR
92 LD HL,TSHAD+(TSHROWS-1)*TCOLS
93 LD DE,TSHAD+(TSHROWS-1)*TCOLS+1
94 LD BC,TCOLS-1
95 LD (HL),0
96 LDIR
97 RET
98
99 ;-----
100 ; TPCTL: the control characters worth honouring. CR and LF are handled
101 ; independently, which gets CR LF right for free.
102 TPCTL: LD A,C
103 CP 13
104 JR Z,TPCR
105 CP 10
106 JR Z,TNEWLN
107 CP 8
108 JR Z,TPBS
109 CP 9
110 JR Z,TPTAB
111 CP 27
112 JR Z,TPESCB
113 RET ; BEL and everything else: ignore
114
115 TPCR: XOR A
116 LD (TCOL),A
117 RET
118
119 TPBS: LD A,(TCOL)
120 OR A
121 RET Z
122 DEC A
123 LD (TCOL),A
124 XOR A ; erase the cell we backed into
125 JR TPDRAW
126
127 TPTAB: LD A,(TCOL)
128 AND 0F8H
129 ADD A,8
130 CP TCOLS
131 JR C,TPT1
132 XOR A

```

```

133 LD (TCOL),A
134 JR TNEWLN
135 TPT1: LD (TCOL),A
136 RET
137
138 TPESCB: LD A,1
139 LD (TESC),A
140 XOR A
141 LD (TESCN),A
142 RET
143
144 ; -----
145 ; TPESCS: inside an escape sequence. A = state, C = the byte.
146 ;
147 ; A BBS told cols/rows still gets to decide it wants ANSI, so swallowing
148 ; these is what keeps [0;37;44m off a 40-column screen. Three states are
149 ; enough: 1 = just saw ESC, 2 = inside a CSI, 3 = swallow one more byte
150 ; (the character-set selectors, ESC ( B and friends).
151 TPESCS: CP 1
152 JR Z,TPES1
153 CP 2
154 JR Z,TPES2
155 JR TESCOF
156 ; state 3: this byte was the last of it
157
158 TPES1: LD A,C
159 CP '['
160 JR Z,TPCSIB
161 CP '('
162 JR Z,TPINT
163 CP ')'
164 JR Z,TPINT
165 CP '#'
166 JR Z,TPINT
167 TPCSIB: LD A,2
168 LD (TESC),A
169 RET
170 TPINT: LD A,3
171 LD (TESC),A
172 RET
173
174 TPES2: LD A,(TESCN) ; a CSI runs until a final byte in 40H-7EH
175 INC A
176 LD (TESCN),A
177 CP 32
178 JR NC,TESCOF
179 LD A,C
180 CP 40H
181 RET C ; parameter or intermediate byte
182 CP 7FH
183 RET NC
184 TESCOF: XOR A
185 LD (TESC),A
186 RET
187
188 ; -----
189 ; The shadow.
190 ;
191 ; TSHR00: HL = TSHAD + A*TCOLS. Clobbers A,D,E.
192 TSHR00: LD L,A
193 LD H,0
194 ADD HL,HL
195 ADD HL,HL
196 ADD HL,HL ; *8
197 LD D,H
198 LD E,L
199 ADD HL,HL
200 ADD HL,HL ; *32
201 ADD HL,DE ; *40
202 LD DE,TSHAD
203 ADD HL,DE

```

```

204 RET
205
206 ; TSHPUT: record glyph index A at the cursor, if the cursor row is covered.
207 ; Preserves A.
208 TSHPUT: PUSH AF
209 LD A,(TROW)
210 SUB TSHTOP
211 JR C,TSHP9 ; above the band: nothing to remember
212 CALL TSHR00
213 LD A,(TCOL)
214 LD E,A
215 LD D,0
216 ADD HL,DE
217 POP AF
218 LD (HL),A
219 RET
220 TSHP9: POP AF
221 RET
222
223 ; TRSHAD: repaint the shadowed rows -- what closing the overlay calls.
224 ; Clobbers everything.
225 TRSHAD: LD A,TSHTOP
226 LD B,TSHROWS
227 TRS1: PUSH BC
228 PUSH AF
229 CALL TRSR0W
230 POP AF
231 INC A
232 POP BC
233 DJNZ TRS1
234 RET
235
236 ; TRSR0W: repaint terminal row A from its shadow line.
237 TRSR0W: PUSH AF
238 SUB TSHTOP
239 CALL TSHR00
240 POP AF
241 LD D,A ; D = terminal row
242 LD E,0 ; E = column
243 LD B,TCOLS
244 TRSR1: LD C,XWONK
245 LD A,(HL)
246 PUSH HL
247 CALL GLYPH1 ; preserves BC, DE; HL for safety
248 POP HL
249 INC HL
250 INC E
251 DJNZ TRSR1
252 RET
253
254 ; -----
255 ; The cursor: the bottom scanline of the cell it sits on.
256 ;
257 ; TCURA: HL = screen address of that scanline. Clobbers A,D,E.
258 TCURA: LD A,(TROW)
259 CALL ROWADR
260 LD DE,5*BYTEPL
261 ADD HL,DE
262 LD A,(TCOL)
263 LD E,A
264 LD D,0
265 ADD HL,DE
266 RET
267
268 TCURON: CALL TCURA
269 LD (HL),0FFH ; a direct write: no expander, 4 white pixels
270 RET
271
272 TCUROF: CALL TCURA
273 LD (HL),0

```

EDIT.INC

```

1 ; edit.inc -- the on-screen grid keyboard.
2
3 ; Descended from fujinet-config/astrocade/edit.inc, but the 4x6 renderer
4 ; changes the shape of the problem: 40 columns fit all 96 printable
5 ; characters at once as 32x3, so config's CASE toggle and its whole action
6 ; row disappear. The keypad supplies the actions instead.
7
8 ; Five rows, drawn at a caller-supplied top row, so the dial screen and the
9 ; in-session composer are the same code at different heights:
10 ;
11 ; +0 the value line, tail-anchored, with an inverse cursor cell
12 ; +1..+3 the grid: cell (gx,gy) is character 20H + gy*32 + gx
13 ; +4 the hint line
14 ;
15 ; Contract, kept from config: the caller preloads EDBUF (NUL-terminated),
16 ; EDMAX, EDTOPR and EDHNT, and the editor runs NO transactions -- so a reply
17 ; window the caller is holding stays valid across an edit. Carry set on
18 ; cancel; EDBUF is mutated either way.
19
20 EGW EQU 32 ; grid cells per row
21 EGROWS EQU 3
22 EGX EQU (TCOLS-EGW)/2 ; centred: 4
23 EVWID EQU TCOLS-1 ; value characters shown; the last cell is
24 ; the cursor block
25
26 ; EDIT: run the editor. Carry set on cancel. Clobbers everything.
27 EDIT: CALL EDRAWA
28 ELOOP: CALL INWAIT
29 CP EVTRIG
30 JR Z,EPICK
31 CP EVLEFT
32 JR Z,EMVL
33 CP EVRIGHT
34 JR Z,EMVR
35 CP EVUP
36 JR Z,EMVU
37 CP EVDOWN
38 JR Z,EMVD
39 CP EVEQ
40 JR Z,EACC
41 CP EVCE
42 JR Z,EDEL
43 CP EVCLR
44 JR Z,ECAN
45 CALL EVTOCH ; a keypad key that types a character
46 OR A
47 JR Z,ELOOP
48 JR EINS
49
50 EACC: OR A ; accept: carry clear
51 RET
52 ECAN: SCF ; cancel
53 RET
54
55 ; ---- movement. A move repaints the whole grid rather than tracking the
56 ; cell it left: 96 glyphs is about 8 ms, well inside the repeat rate, and it
57 ; costs no state.
58 EMVL: LD A,(V_GX)
59 DEC A
60 JP P,EMVX
61 LD A,EGW-1
62 JR EMVX
63 EMVR: LD A,(V_GX)
64 INC A
65 CP EGW
66 JR C,EMVX
67 XOR A
68 EMVX: LD (V_GX),A
69 JR EDMOV
70 EMVU: LD A,(V_GY)
71 DEC A
72 JP P,EMVY
73 LD A,EGROWS-1
74 JR EMVY
75 EMVD: LD A,(V_GY)
76 INC A
77 CP EGROWS
78 JR C,EMVY
79 XOR A
80 EMVY: LD (V_GY),A
81 EDMOV: CALL EDGRID
82 ELOOP
83
84 ; ---- the buffer
85 EPICK: LD A,(V_GY) ; the cell under the cursor IS the character
86 ADD A,A
87 ADD A,A
88 ADD A,A
89 ADD A,A
90 ADD A,A ; gy*32
91 LD HL,V_GX
92 ADD A,(HL)
93 ADD A,20H
94 ; falls into EINS
95
96 ; EINS: append the character in A, if there is room.
97 EINS: LD C,A
98 CALL EDLEN
99 LD HL,EDMAX
100 CP (HL)
101 JR NC,EDRW1 ; full
102 LD E,A
103 LD D,0
104 LD HL,EDBUF
105 ADD HL,DE
106 LD (HL),C
107 INC HL
108 LD (HL),0
109 EDRW1: JR
110
111 EDEL: CALL EDLEN
112 OR A
113 JR Z,EDRW1
114 DEC A
115 LD E,A
116 LD D,0
117 LD HL,EDBUF
118 ADD HL,DE
119 LD (HL),0
120 EDRW1: CALL EDRAWV
121 JP
122
123 ; EDLEN: A = the length of EDBUF. Clobbers B,H,L.
124 EDLEN: LD HL,EDBUF
125 LD B,0
126 EDL1: LD A,(HL)

```

```

127      OR      A
128      JR      Z,EDL2
129      INC      HL
130      INC      B
131      JR      EDL1
132 EDL2:  LD      A,B
133      RET
134
135 ; ----- drawing
136 ; EDRAWA: the whole editor. Clobbers everything.
137 EDRAWA: LD      A,(EDTOPR)
138      LD      B,5
139      CALL     CLRRWS
140      CALL     EDGRID
141      CALL     EDHINT
142      ; falls into EDRAWV
143
144 ; EDRAWV: the value line. Shows the tail when the buffer is longer than the
145 ; row, so the end being typed is always visible.
146 EDRAWV: CALL     EDLEN
147      LD      HL,EDBUF
148      CP      EWVID
149      JR      C,EDV1
150      SUB      EWVID      ; scroll the window to the tail
151      LD      E,A
152      LD      D,0
153      ADD      HL,DE
154      LD      A,E,EWVID
155 EDV1:  LD      (EDVN),A
156      LD      A,(EDTOPR)
157      LD      D,A
158      LD      E,0
159      LD      C,XWONK
160      LD      B,TCOLS
161      CALL     TXTN      ; pads the rest of the row with spaces
162      LD      A,(EDTOPR)
163      LD      D,A
164      LD      A,(EDVN)
165      LD      E,A
166      LD      C,XKONW
167      XOR      A      ; a space in inverse: the cursor block
168      JP      GLYPH1
169
170 ; EDHINT: the hint line.
171 EDHINT: LD      A,(EDTOPR)
172      ADD      A,4
173      LD      D,A
174      LD      E,0
175      LD      C,XAONK
176      LD      HL,(EDHINT)
177      LD      B,TCOLS
178      JP      TXTN
179
180 ; EDGRID: all 96 cells, the selected one inverse.
181 EDGRID: LD      B,0
182 EDG1:  PUSH     BC
183      LD      A,B
184      CALL     EDCELL
185      POP      BC
186      INC      B
187      LD      A,B
188      CP      EGW*EGROWS
189      JR      C,EDG1
190      RET
191
192 ; EDCELL: draw grid cell index A (0-95). The cell index IS the glyph index,
193 ; because both spaces start at 20H.
194 EDCELL: LD      (EDCIX),A
195      LD      B,0
196 EDG1:  CP      EGW
197      JR      C,EDC2

```

```

198      SUB      EGW
199      INC      B
200      JR      EDC1
201 EDC2:  LD      C,A      ; B = gy, C = gx
202      LD      A,(V_GY)
203      CP      B
204      JR      NZ,EDC3
205      LD      A,(V_GX)
206      CP      C
207      JR      NZ,EDC3
208      LD      A,XKONW
209      JR      EDC4
210 EDC3:  LD      A,XWONK
211 EDC4:  PUSH     AF
212      LD      A,(EDTOPR)
213      INC      A
214      ADD      A,B
215      LD      D,A
216      LD      A,C
217      ADD      A,EGX
218      LD      E,A
219      POP      AF
220      LD      C,A
221      LD      A,(EDCIX)
222      JP      GLYPH1
223

```

INPUT.INC (NETCAT'S OWN)

```

1 ; input.inc -- keypad and hand-controller scanning.
2
3 ; The 24-key keypad reads as four column ports (KEY0 14H rightmost .. KEY3
4 ; 17H leftmost), active-high bits 0-5; the player 1 handle is port 10H,
5 ; active-high U/D/L/R/T in bits 0-4. Everything folds into one event space.
6
7 ; The card ports mapped only the three columns their games needed. netcat is
8 ; a terminal, so it wants every key it can get -- the whole matrix is here,
9 ; decoded from MAME's astrohome.cpp the way fujinet-config does it.
10
11 ; Pacing is a calibrated delay loop, not a frame counter: the program runs DI
12 ; for its whole life (fujilib's contract) and the vertical-address feedback
13 ; register is lightpen-latched and useless as a time base.
14
15 ; Two entry points because the two callers want different things. The session
16 ; loop takes INPOLL, edge only -- a key held down must not machine-gun the
17 ; BBS. The grid keyboard takes INPOLR, which auto-repeats so a thumb on a
18 ; direction walks the grid.
19
20 EVNONE EQU 0
21 EVUP EQU 1
22 EVDOWN EQU 2
23 EVLEFT EQU 3
24 EVRIGHT EQU 4
25 EVTRIG EQU 5
26 EVCE EQU 6 ; CE -- backspace / delete
27 EVEQ EQU 7 ; = -- accept / send
28 EVDDT EQU 8
29 EVCLR EQU 9 ; C -- hang up / cancel
30 EVMR EQU 0AH ; MR
31 EVMS EQU 0BH ; MS
32 EVCH EQU 0CH ; CH
33 EVPCT EQU 0DH ; %
34 EVDIV EQU 0EH ; %
35 EVMUL EQU 0FH ; X
36 EVMIN EQU 10H ; -
37 EVPLU EQU 11H ; +
38 EVDO EQU 20H ; digits: EVD0+n
39
40 INDLY EQU 30 ; ~300 ms before a held key repeats
41 INRATE EQU 8 ; ~80 ms between repeats
42
43 ; INPOLL: A = a freshly pressed event, or EVNONE. No auto-repeat.
44 INPOLL: CALL KEYRAW
45 LD HL,V_KEY
46 CP (HL)
47 JR NZ,IPNEW
48 XOR A
49 RET ; held: already reported
50
51 ; INPOLR: as INPOLL, but a held key repeats after INDLY at INRATE. Assumes
52 ; the caller paces itself with DELAY10.
53 INPOLR: CALL KEYRAW
54 LD HL,V_KEY
55 CP (HL)
56 JR NZ,IPNEW
57 OR A
58 RET
59 LD HL,V_RPT
60 DEC (HL)
61 JR Z,IPRPT
62 XOR A
63 RET ; still counting down -- and A must be zeroed
64 IPRPT: LD (HL),INRATE ; here, or the held key reads as a fresh press
65 LD A,(V_KEY)
66 CP EVRIGHT+1 ; only the directions repeat: a held trigger

```

```

67 RET C ; must not spray characters into the buffer
68 XOR A
69 RET
70
71 IPNEW: LD (HL),A ; HL = V_KEY
72 LD HL,V_RPT
73 LD (HL),INDLY
74 LD A,(V_KEY)
75 OR A
76 RET
77
78 ; INWAIT: block until an event arrives, with repeat. Returns it in A.
79 INWAIT: CALL DELAY10
80 CALL INPOLR
81 OR A
82 JR Z,INWAIT
83 RET
84
85 ; DELAY10: about 10 ms. 745 iterations of a 26 T-state loop at 1.789 MHz,
86 ; the figure fujinet-config calibrated. Clobbers A,D,E.
87 DELAY10: LD DE,745
88 DLY1: DEC DE
89 LD A,D
90 OR E
91 JR NZ,DLY1
92 RET
93
94 ; KEYRAW: A = the event for whatever is pressed right now, or EVNONE.
95 ; First match wins; a diagonal on the stick reduces to its lowest set bit so
96 ; it cannot look like two keys. Clobbers A,B,C,D,E,H,L.
97 KEYRAW: LD HL,KPTAB
98 LD B,4
99 KRCOL: LD C,(HL)
100 INC HL
101 IN A,(C)
102 LD D,A
103 LD E,6
104 KRBIT: LD A,(HL)
105 HL INC
106 SRL D
107 JR NC,KRB2
108 OR A
109 RET
110 KRB2: DEC E ; pressed, and mapped
111 JR NZ,KRBIT
112 DJNZ KRCOL
113 IN A,(SW0) ; the stick
114 AND 1FH
115 RET
116 LD D,A
117 NEG
118 AND D
119 LD HL,STKMAP
120 KIRSTK: SRL A
121 JR C,KIRSTK1
122 INC HL
123 JR KIRSTK
124 KIRSTK1: LD A,(HL)
125 RET
126
127 ; EVTOCH: A = event -> A = the character that key types, or 0 if it types
128 ; nothing. Shared by the terminal (which sends it) and the grid keyboard.
129 ; (which inserts it), so the keypad means the same thing in both places.
130 ; Clobbers D,E,H,L.
131 EVTOCH: CP EVDO
132 C,ETC1

```

```

133 SUB EVD0
134 CP 10
135 JR NC,ETCZ
136 ADD A,'0'
137 RET
138 ETC1: CP EVPLU+1
139 JR NC,ETCZ
140 LD HL,EVCHR
141 LD E,A
142 LD D,0
143 ADD HL,DE
144 LD A,(HL)
145 RET
146 ETCZ: XOR A
147 RET
148
149 ; Indexed by event code, EVNONE..EVPLU.
150 EVCHR: DB 0,0,0,0,0,0 ; none, up, down, left, right, trigger
151 DB 0 ; CE -- backspace, not a character
152 DB 0 ; = -- accept, not a character
153 DB ',' ;
154 DB 0 ; C -- cancel, not a character
155 DB 0 ; MR -- ESC in the terminal, nothing here
156 DB 0 ; MS -- Ctrl-C in the terminal
157 DB ',' ; CH
158 DB ' ','/','*','-','+',
159
160 ; Column port, then its six bit codes, bit 0 (bottom key) first.
161 KPTAB: DB KEY0
162 DB EVPCT,EVDIV,EVMUL,EVMIN,EVPLU,EVEQ ; % / x - + =
163 DB KEY1
164 DB EVDOWN,EVCH,EVD0+9,EVD0+6,EVD0+3,EVDOT ; v CH 9 6 3 .
165 DB KEY2
166 DB EVUP,EVMS,EVD0+8,EVD0+5,EVD0+2,EVD0 ; ^ MS 8 5 2 0
167 DB KEY3
168 DB EVCLR,EVMR,EVD0+7,EVD0+4,EVD0+1,EVCE ; C MR 7 4 1 CE
169 STKMAP: DB EVUP,EVDOWN,EVLEFT,EVRIGHT,EVTRIG
170

```

GFX.INC (NETCAT'S OWN)

```

1 ; gfx.inc -- the 4x6 renderer, fills and scrolls.
2 ;
3 ; Adapted from fujinet-battleship/astrocade/gfx.inc. The whole port draws on
4 ; one alignment trick: at 2bpp, 4 pixels are one byte, and the character grid
5 ; is 4 pixels wide -- so every glyph row is a single byte write and nothing
6 ; here ever shifts. Text goes through the magic EXPANDER: OUT the fg/bg pair
7 ; to XPAND, reset the expander's nibble flip-flop by writing the MAGIC
8 ; register, and each following write through magic memory (@000H + screen
9 ; offset) paints the HIGH nibble of the data as 4 pixels in those two colors.
10 ; One MAGIC write per row keeps the flip-flop on the high nibble.
11 ;
12 ; Everything runs with interrupts off and draws opaquely (bg included), so
13 ; redraw over live screen never flickers and never needs a clear first.
14 ;
15 ; Two changes from the card games' copy, both for the terminal:
16 ; * the glyph range is 20H-7FH, 96 glyphs, and lowercase is NOT folded to
17 ; * uppercase -- a BBS is mixed case and the font now has it;
18 ; * SCRLUP, which the games had no use for.
19 ;
20 ; Char grid: 40 cols x 14 rows of 4x6 cells over 84 visible lines.
21 ; Colors (palette registers): 0 black, 1 dim, 2 accent, 3 white.
22 ;
23 MAGEXP EQU 08H ; MAGIC value: expand, no shift/rotate/or/xor
24
25 ; XPAND color pairs (fg*4 + bg).
26 XWONK EQU 0CH+0 ; white on black -- terminal text
27 XKONW EQU 00H+3 ; black on white -- the cursor cell

```

```

28 XAONK EQU 08H+0 ; accent on black -- hints, headings
29 XKONA EQU 00H+2 ; black on accent -- the status bar
30 XDONK EQU 04H+0 ; dim on black -- the grid keyboard's ground
31
32 ;
33 ; TXTAT: string at HL (NUL-terminated) at char col E, row D, colors C
34 ; (an XPAND pair). TXTN: exactly B cells of the string -- stops copying
35 ; at the NUL and pads the rest with spaces, so fixed-width fields redraw
36 ; opaquely. Anything outside 20H-7FH draws as '?'. Clobbers everything.
37
38 TXTAT: LD B,0 ; 0: no pad, stop at NUL
39 JR TXT0
40 LD A,B
41 OR A
42 RET Z
43 LD A,C
44 OUT (XPAND),A
45 CALL TXTPOS
46 LD A,(HL)
47 OR A
48 JR Z,TXT3 ; NUL: pad or stop
49 INC HL
50 SUB 20H
51 CP 60H
52 JR C,TXT4
53 LD A,'?'-20H
54 TXT4: CALL GLYPH
55 LD A,B

```

```

55 OR A
56 JR Z,TXT1 ; unbounded (TXTAT)
57 DJNZ TXT1
58 RET
59 TXT3: LD A,B
60 OR A
61 RET Z ; TXTAT: done at NUL
62 TXT5: XOR A ; TXTN: space-pad the remainder
63 CALL GLYPH
64 DJNZ TXT5
65 RET
66
67 ; GLYPH1: one glyph, index A (0-5FH), at char row D col E in colors C.
68 ; The terminal's per-character path; TXTAT is for whole strings.
69 ; Clobbers everything except D,E,C.
70 GLYPH1: PUSH AF
71 LD A,C
72 OUT (XPAND),A
73 CALL TXTPOS
74 AF POP
75 JP GLYPH
76
77 ; TXTPOS: D,E char row/col -> shadow DE = magic address; preserves the
78 ; primary registers (HL string, B count, C colors).
79 TXTPOS: HL PUSH
80 LD BC
81 LD A,D
82 ADD A,A
83 LD H,0
84 LD L,A
85 LD BC,ROWTAB
86 ADD HL,BC
87 LD A,(HL)
88 INC HL
89 LD H,(HL)
90 LD L,A
91 LD B,0
92 LD C,E
93 ADD HL,BC ; HL = row*240 + col
94 PUSH HL
95 EXX
96 POP DE ; shadow DE = magic address
97 EXX
98 POP BC
99 POP HL
100 RET
101
102 ; GLYPH: draw glyph index A (0-5FH) at the shadow-DE magic address and
103 ; advance it one cell right. Preserves primary HL/BC (shadow DE is the
104 ; running magic address, so the *6 scratch uses shadow BC).
105 GLYPH: EXX
106 LD L,A
107 LD H,0
108 ADD HL,HL ; A*2
109 LD B,H
110 LD C,L
111 ADD HL,HL ; A*4
112 ADD HL,BC ; A*6
113 LD BC,FONT46
114 ADD HL,BC ; HL = glyph rows
115 LD B,6
116 GLYPH: LD A,MAGEXP
117 OUT (MAGIC),A ; reset the expander nibble flip-flop
118 LD A,(HL)
119 LD (DE),A ; one row: 4 pixels
120 INC HL
121 LD A,E ; down one line
122 ADD A,BYTEPL
123 LD E,A
124 JR NC,GLYRW1
125 INC D
126 GLYRW1: DJNZ GLYRW
127 LD A,E ; back up 6 lines, over 1 col
128 SUB 6*BYTEPL-1
129 LD E,A
130 JR NC,GLYRW2
131 DEC D
132 GLYRW2: EXX
133 RET
134
135 ;-----
136 ; CLRSCR: the whole visible screen back to color 0. Clobbers A,BC,DE,HL.
137 CLRSCR: LD HL,NORMEM
138 LD DE,NORMEM+1
139 LD BC,LINES*BYTEPL-1
140 LD (HL),0
141 LDIR
142 RET
143
144 ; CLRROW: clear char row A (all 40 columns) to color 0.
145 ; Clobbers A,BC,DE,HL.
146 CLRROW: CALL ROWADR ; HL = NORMEM + row*240
147 LD D,H
148 LD E,L
149 INC DE
150 LD (HL),0
151 LD BC,ROWBYT-1
152 LDIR
153 RET
154
155 ; CLRRWS: clear B char rows starting at row A. Clobbers A,BC,DE,HL.
156 CLRRWS: PUSH BC
157 PUSH AF
158 CALL CLRROW
159 POP AF
160 INC A
161 POP BC
162 DJNZ CLRRWS
163 RET
164
165 ; ROWADR: HL = NORMEM + row A * 240 (a real screen address, not magic).
166 ; Rides ROWTAB rather than multiplying. Clobbers A,DE.
167 ROWADR: ADD A,A ; word index into ROWTAB
168 LD H,0
169 LD L,A
170 LD DE,ROWTAB
171 ADD HL,DE
172 LD A,(HL)
173 INC HL
174 LD H,(HL)
175 LD L,A
176 LD DE,NORMEM
177 ADD HL,DE
178 RET
179
180 ;-----
181 ; SCRLUP: scroll char rows 0..A-1 up by one row, leaving the last row
182 ; untouched (the caller clears it). A is the row count of the pane.
183 ; A straight LDIR of (A-1)*240 bytes: ~34 ms for a 13-row pane at
184 ; 1.789 MHz, about two frames, and only on a line feed at the bottom.
185 ; Clobbers A,BC,DE,HL.
186 SCRLUP: DEC A ; rows to move
187 RET Z
188 LD B,A
189 LD HL,0
190 LD DE,ROWBYT
191 SCRL1: ADD HL,DE ; HL = (rows-1) * 240
192 DJNZ SCRL1
193 LD B,H ; BC = byte count
194 LD C,L
195 LD HL,NORMEM+ROWBYT
196 LD DE,NORMEM

```

```

197     LDIR
198     RET
199
200 ROWBYT EQU 6*BYTEPL ; 240 bytes in one char row
201
202 ROWTAB: DW 0*ROWBYT,1*ROWBYT,2*ROWBYT,3*ROWBYT,4*ROWBYT
203          DW 5*ROWBYT,6*ROWBYT,7*ROWBYT,8*ROWBYT,9*ROWBYT
204          DW 10*ROWBYT,11*ROWBYT,12*ROWBYT,13*ROWBYT
205

```

FONT.INC -- THE 96-GLYPH 4X6 FONT (GENERATED)

```

1 ; font.inc -- 4x6 font, 1bpp, GENERATED by tools/mkfont.py -- do not
2 ; edit; rerun 'make assets'.
3
4 ; 96 glyphs, codepoints 20H-7FH, 6 bytes each, top row first. 20H-5FH
5 ; are byte-identical to the card-game ports' font (same Lynx BMPs);
6 ; 61H-7AH are hand-drawn lowercase, so a BBS reads in mixed case.
7 ; Ink is the HIGH nibble, bit 7 leftmost: one magic-expander write
8 ; per row (the expander takes the high nibble first after a MAGIC
9 ; register write -- see gfx.inc).
10 FONT46:
11 ; 20H '20'
12 DB 000H ; ....
13 DB 000H ; ....
14 DB 000H ; ....
15 DB 000H ; ....
16 DB 000H ; ....
17 DB 000H ; ....
18 ; 21H '!'
19 DB 000H ; ....
20 DB 040H ; .#.
21 DB 040H ; .#.
22 DB 040H ; .#.
23 DB 000H ; ....
24 DB 040H ; .#.
25 ; 22H '"'
26 DB 000H ; ....
27 DB 0A0H ; .#.
28 DB 0A0H ; .#.
29 DB 000H ; ....
30 DB 000H ; ....
31 DB 000H ; ....
32 ; 23H '#'
33 DB 000H ; ....
34 DB 040H ; .#.
35 DB 0E0H ; ###
36 DB 040H ; .#.
37 DB 0E0H ; ###
38 DB 040H ; .#.
39 ; 24H '$'
40 DB 040H ; .#.
41 DB 0E0H ; ###
42 DB 080H ; #..
43 DB 0E0H ; ###
44 DB 020H ; .#.
45 DB 0E0H ; ###
46 ; 25H '%'
47 DB 000H ; ....
48 DB 080H ; .#.
49 DB 020H ; .#.
50 DB 040H ; .#.
51 DB 080H ; .#.
52 DB 020H ; .#.
53 ; 26H '&'
54 DB 000H ; ....
55 DB 000H ; ....
56 DB 040H ; .#.

```

```

57 DB 0E0H ; ###
58 DB 040H ; .#.
59 DB 000H ; ....
60 ; 27H '''
61 DB 000H ; ....
62 DB 040H ; .#.
63 DB 080H ; #..
64 DB 000H ; ....
65 DB 000H ; ....
66 DB 000H ; ....
67 ; 28H '('
68 DB 000H ; ....
69 DB 020H ; .#.
70 DB 040H ; .#.
71 DB 040H ; .#.
72 DB 040H ; .#.
73 DB 020H ; .#.
74 ; 29H ')'
75 DB 000H ; ....
76 DB 040H ; .#.
77 DB 020H ; .#.
78 DB 020H ; .#.
79 DB 020H ; .#.
80 DB 040H ; .#.
81 ; 2AH '*'
82 DB 000H ; ....
83 DB 040H ; .#.
84 DB 0E0H ; ###
85 DB 0E0H ; ###
86 DB 040H ; .#.
87 DB 000H ; ....
88 ; 2BH '+'
89 DB 000H ; ....
90 DB 000H ; ....
91 DB 040H ; .#.
92 DB 0E0H ; ###
93 DB 040H ; .#.
94 DB 000H ; ....
95 ; 2CH ','
96 DB 000H ; ....
97 DB 000H ; ....
98 DB 000H ; ....
99 DB 000H ; ....
100 DB 020H ; .#.
101 DB 040H ; .#.
102 ; 2DH '-'
103 DB 000H ; ....
104 DB 000H ; ....
105 DB 000H ; ....
106 DB 0E0H ; ###
107 DB 000H ; ....
108 DB 000H ; ....
109 ; 2EH '.'
110 DB 000H ; ....
111 DB 000H ; ....
112 DB 000H ; ....

```


113	DB	000H	;	184	DB	0A0H	; #. #.
114	DB	000H	;	185	DB	040H	; .#..
115	DB	040H	; .#..	186	; 39H '9'		
116	; 2FH '/'			187	DB	000H	;
117	DB	000H	;	188	DB	040H	; .#..
118	DB	000H	;	189	DB	0A0H	; #. #.
119	DB	020H	; .#.	190	DB	060H	; .###.
120	DB	040H	; .#..	191	DB	020H	; .#..
121	DB	040H	; .#..	192	DB	0C0H	; ###.
122	DB	080H	; #...	193	; 3AH ':'		
123	; 30H '0'			194	DB	000H	;
124	DB	000H	;	195	DB	000H	;
125	DB	040H	; .#..	196	DB	040H	; .#..
126	DB	0A0H	; #. #.	197	DB	000H	;
127	DB	0A0H	; #. #.	198	DB	040H	; .#..
128	DB	0A0H	; #. #.	199	DB	000H	;
129	DB	040H	; .#..	200	; 3BH ';'		
130	; 31H '1'			201	DB	000H	;
131	DB	000H	;	202	DB	000H	;
132	DB	040H	; .#..	203	DB	040H	; .#..
133	DB	0C0H	; ###.	204	DB	000H	;
134	DB	040H	; .#..	205	DB	040H	; .#..
135	DB	040H	; .#..	206	DB	080H	; #...
136	DB	0E0H	; ###.	207	; 3CH '<'		
137	; 32H '2'			208	DB	000H	;
138	DB	000H	;	209	DB	020H	; .#..
139	DB	0C0H	; ###.	210	DB	040H	; .#..
140	DB	020H	; .#.	211	DB	080H	; #...
141	DB	040H	; .#..	212	DB	040H	; .#..
142	DB	080H	; #...	213	DB	020H	; .#..
143	DB	0E0H	; ###.	214	; 3DH '='		
144	; 33H '3'			215	DB	000H	;
145	DB	000H	;	216	DB	000H	;
146	DB	0C0H	; ###.	217	DB	0E0H	; ###.
147	DB	020H	; .#.	218	DB	000H	;
148	DB	040H	; .#..	219	DB	0E0H	; ###.
149	DB	020H	; .#.	220	DB	000H	;
150	DB	0C0H	; ###.	221	; 3EH '>'		
151	; 34H '4'			222	DB	000H	;
152	DB	000H	;	223	DB	080H	; #...
153	DB	0A0H	; #. #.	224	DB	040H	; .#..
154	DB	0A0H	; #. #.	225	DB	020H	; .#.
155	DB	0E0H	; ###.	226	DB	040H	; .#..
156	DB	020H	; .#.	227	DB	080H	; #...
157	DB	020H	; .#.	228	; 3FH '?'		
158	; 35H '5'			229	DB	000H	;
159	DB	000H	;	230	DB	0C0H	; ###.
160	DB	0E0H	; ###.	231	DB	020H	; .#.
161	DB	080H	; #...	232	DB	040H	; .#..
162	DB	0C0H	; ###.	233	DB	000H	;
163	DB	020H	; .#.	234	DB	040H	; .#..
164	DB	0C0H	; ###.	235	; 40H '@'		
165	; 36H '6'			236	DB	000H	;
166	DB	000H	;	237	DB	040H	; .#..
167	DB	0C0H	; ###.	238	DB	0A0H	; #. #.
168	DB	080H	; #...	239	DB	0A0H	; #. #.
169	DB	0C0H	; ###.	240	DB	080H	; #...
170	DB	0A0H	; #. #.	241	DB	060H	; ###.
171	DB	0C0H	; ###.	242	; 41H 'A'		
172	; 37H '7'			243	DB	000H	;
173	DB	000H	;	244	DB	040H	; .#..
174	DB	0E0H	; ###.	245	DB	0A0H	; #. #.
175	DB	020H	; .#.	246	DB	0E0H	; ###.
176	DB	040H	; .#..	247	DB	0A0H	; #. #.
177	DB	040H	; .#..	248	DB	0A0H	; #. #.
178	DB	040H	; .#..	249	; 42H 'B'		
179	; 38H '8'			250	DB	000H	;
180	DB	000H	;	251	DB	0C0H	; ###.
181	DB	040H	; .#..	252	DB	0A0H	; #. #.
182	DB	0A0H	; #. #.	253	DB	0C0H	; ###.
183	DB	040H	; .#..	254	DB	0A0H	; #. #.

255	DB	0C0H	; ##..	326	; 4DH 'M'		
256	; 43H 'C'			327	DB	000H	;
257	DB	000H	;	328	DB	0A0H	; #.#.
258	DB	060H	; ##.#	329	DB	0E0H	; ###.
259	DB	080H	; #...	330	DB	0E0H	; ###.
260	DB	080H	; #...	331	DB	0A0H	; #.#.
261	DB	080H	; #...	332	DB	0A0H	; #.#.
262	DB	060H	; ##.	333	; 4EH 'N'		
263	; 44H 'D'			334	DB	000H	;
264	DB	000H	;	335	DB	0A0H	; #.#.
265	DB	0C0H	; ##.	336	DB	0E0H	; ###.
266	DB	0A0H	; #.#.	337	DB	0E0H	; ###.
267	DB	0A0H	; #.#.	338	DB	0E0H	; ###.
268	DB	0A0H	; #.#.	339	DB	0A0H	; #.#.
269	DB	0C0H	; ##..	340	; 4FH 'O'		
270	; 45H 'E'			341	DB	000H	;
271	DB	000H	;	342	DB	040H	; .#..
272	DB	0E0H	; ###.	343	DB	0A0H	; #.#.
273	DB	080H	; #...	344	DB	0A0H	; #.#.
274	DB	0C0H	; ##.	345	DB	0A0H	; #.#.
275	DB	080H	; #...	346	DB	040H	; .#..
276	DB	0E0H	; ###.	347	; 50H 'P'		
277	; 46H 'F'			348	DB	000H	;
278	DB	000H	;	349	DB	0C0H	; ##..
279	DB	0E0H	; ###.	350	DB	0A0H	; #.#.
280	DB	080H	; #...	351	DB	0C0H	; ###.
281	DB	0C0H	; ##..	352	DB	080H	; #...
282	DB	080H	; #...	353	DB	080H	; #...
283	DB	080H	; #...	354	; 51H 'Q'		
284	DB	000H	;	355	DB	000H	;
285	DB	000H	;	356	DB	040H	; .#..
286	DB	060H	; ##.	357	DB	0A0H	; #.#.
287	DB	080H	; #...	358	DB	0A0H	; #.#.
288	DB	0A0H	; #.#.	359	DB	0E0H	; ###.
289	DB	0A0H	; #.#.	360	DB	060H	; ##.
290	DB	060H	; ##.	361	; 52H 'R'		
291	; 48H 'H'			362	DB	000H	;
292	DB	000H	;	363	DB	0C0H	; ##..
293	DB	0A0H	; #.#.	364	DB	0A0H	; #.#.
294	DB	0A0H	; #.#.	365	DB	0C0H	; ##.
295	DB	0E0H	; ###.	366	DB	0A0H	; #.#.
296	DB	0A0H	; #.#.	367	DB	0A0H	; #.#.
297	DB	0A0H	; #.#.	368	; 53H 'S'		
298	; 49H 'I'			369	DB	000H	;
299	DB	000H	;	370	DB	060H	; #.#.
300	DB	0E0H	; ###.	371	DB	080H	; #...
301	DB	040H	; #.#.	372	DB	040H	; .#..
302	DB	040H	; #...	373	DB	020H	; .#..
303	DB	040H	; #...	374	DB	0C0H	; ##..
304	DB	0E0H	; ###.	375	; 54H 'T'		
305	; 4AH 'J'			376	DB	000H	;
306	DB	000H	;	377	DB	0E0H	; ###.
307	DB	020H	; .#.	378	DB	040H	; .#..
308	DB	020H	; .#.	379	DB	040H	; #...
309	DB	020H	; #.#.	380	DB	040H	; #.#.
310	DB	0A0H	; #.#.	381	DB	040H	; .#..
311	DB	040H	; #...	382	; 55H 'U'		
312	; 4BH 'K'			383	DB	000H	;
313	DB	000H	;	384	DB	0A0H	; #.#.
314	DB	0A0H	; #.#.	385	DB	0A0H	; #.#.
315	DB	0A0H	; #.#.	386	DB	0A0H	; #.#.
316	DB	0C0H	; ##..	387	DB	0A0H	; #.#.
317	DB	0A0H	; #.#.	388	DB	0E0H	; ###.
318	DB	0A0H	; #.#.	389	; 56H 'V'		
319	; 4CH 'L'			390	DB	000H	;
320	DB	000H	;	391	DB	0A0H	; #.#.
321	DB	080H	; #...	392	DB	0A0H	; #.#.
322	DB	080H	; #...	393	DB	0A0H	; #.#.
323	DB	080H	; #...	394	DB	0A0H	; #.#.
324	DB	080H	; #...	395	DB	040H	; .#..
325	DB	0E0H	; ###.	396	; 57H 'W'		

```

397      DB      000H      ; ....
398      DB      0A0H      ; #.#.
399      DB      0A0H      ; #.#.
400      DB      0E0H      ; ###.
401      DB      0E0H      ; ##.#.
402      DB      0A0H      ; #.#.
403 ; 58H 'X'
404      DB      000H      ; ....
405      DB      0A0H      ; #.#.
406      DB      0A0H      ; #.#.
407      DB      040H      ; #..
408      DB      0A0H      ; #.#.
409      DB      0A0H      ; #.#.
410 ; 59H 'Y'
411      DB      000H      ; ....
412      DB      0A0H      ; #.#.
413      DB      0A0H      ; #.#.
414      DB      040H      ; #..
415      DB      040H      ; #..
416      DB      040H      ; #..
417 ; 5AH 'Z'
418      DB      000H      ; ....
419      DB      0E0H      ; ###.
420      DB      020H      ; .#.
421      DB      040H      ; #..
422      DB      080H      ; #..
423      DB      0E0H      ; ###.
424 ; 5BH '['
425      DB      000H      ; ....
426      DB      060H      ; ##.#.
427      DB      040H      ; #..
428      DB      040H      ; #..
429      DB      040H      ; #..
430      DB      060H      ; ##.#.
431 ; 5CH '\ '
432      DB      000H      ; ....
433      DB      000H      ; ....
434      DB      080H      ; #..
435      DB      040H      ; #..
436      DB      040H      ; #..
437      DB      020H      ; #..
438 ; 5DH ']'
439      DB      000H      ; ....
440      DB      060H      ; ##.#.
441      DB      020H      ; #..
442      DB      020H      ; #..
443      DB      020H      ; #..
444      DB      060H      ; ##.#.
445 ; 5EH 'A'
446      DB      000H      ; ....
447      DB      040H      ; #..
448      DB      0A0H      ; #.#.
449      DB      000H      ; ....
450      DB      000H      ; ....
451      DB      000H      ; ....
452 ; 5FH 'I'
453      DB      000H      ; ....
454      DB      000H      ; ....
455      DB      000H      ; ....
456      DB      000H      ; ....
457      DB      000H      ; ....
458      DB      0F0H      ; #####
459 ; 60H 'I'
460      DB      000H      ; ....
461      DB      040H      ; #..
462      DB      020H      ; #..
463      DB      000H      ; ....
464      DB      000H      ; ....
465      DB      000H      ; ....
466 ; 61H 'a'
467      DB      000H      ; ....

```

```

468      DB      000H      ; ....
469      DB      0C0H      ; ##..
470      DB      060H      ; .##.
471      DB      0A0H      ; #.#.
472      DB      060H      ; .##.
473 ; 62H 'b'
474      DB      080H      ; #...
475      DB      080H      ; #...
476      DB      080H      ; #...
477      DB      0C0H      ; ##..
478      DB      0A0H      ; #.#.
479      DB      0C0H      ; ##..
480 ; 63H 'c'
481      DB      000H      ; ....
482      DB      000H      ; ....
483      DB      060H      ; .##.
484      DB      080H      ; #...
485      DB      080H      ; #...
486      DB      060H      ; .##.
487 ; 64H 'd'
488      DB      020H      ; ..#.
489      DB      020H      ; ..#.
490      DB      020H      ; ..#.
491      DB      060H      ; ##..
492      DB      0A0H      ; #.#.
493      DB      060H      ; ##..
494 ; 65H 'e'
495      DB      000H      ; ....
496      DB      000H      ; ....
497      DB      060H      ; ##.#.
498      DB      0A0H      ; #.#.
499      DB      0C0H      ; ##..
500      DB      060H      ; ##..
501 ; 66H 'f'
502      DB      060H      ; .##.
503      DB      040H      ; #..
504      DB      0E0H      ; ###.
505      DB      040H      ; #..
506      DB      040H      ; #..
507      DB      040H      ; #..
508 ; 67H 'g'
509      DB      000H      ; ....
510      DB      000H      ; ....
511      DB      060H      ; ##.#.
512      DB      0A0H      ; #.#.
513      DB      060H      ; ##..
514      DB      0C0H      ; ##..
515 ; 68H 'h'
516      DB      080H      ; #...
517      DB      080H      ; #...
518      DB      080H      ; #...
519      DB      0C0H      ; ##..
520      DB      0A0H      ; #.#.
521      DB      0A0H      ; #.#.
522 ; 69H 'i'
523      DB      040H      ; #...
524      DB      000H      ; ....
525      DB      040H      ; #...
526      DB      040H      ; #...
527      DB      040H      ; #...
528      DB      040H      ; #...
529 ; 6AH 'j'
530      DB      020H      ; ..#.
531      DB      000H      ; ....
532      DB      020H      ; ..#.
533      DB      020H      ; ..#.
534      DB      020H      ; ..#.
535      DB      0C0H      ; ##..
536 ; 6BH 'k'
537      DB      080H      ; #...
538      DB      080H      ; #...

```

539	DB	0A0H	; #. #.
540	DB	0C0H	; ##. .
541	DB	0C0H	; ##. .
542	DB	0A0H	; #. #.
543	; 6CH 'l'		
544	DB	0C0H	; ##. .
545	DB	040H	; .#..
546	DB	040H	; .#..
547	DB	040H	; .#..
548	DB	040H	; .#..
549	DB	060H	; .##.
550	; 6DH 'm'		
551	DB	000H	;
552	DB	000H	;
553	DB	0A0H	; #. #.
554	DB	0E0H	; ##. #.
555	DB	0A0H	; #. #.
556	DB	0A0H	; #. #.
557	; 6EH 'n'		
558	DB	000H	;
559	DB	000H	;
560	DB	0C0H	; ##. .
561	DB	0A0H	; #. #.
562	DB	0A0H	; #. #.
563	DB	0A0H	; #. #.
564	; 6FH 'o'		
565	DB	000H	;
566	DB	000H	;
567	DB	040H	; .#..
568	DB	0A0H	; #. #.
569	DB	0A0H	; #. #.
570	DB	040H	; .#..
571	; 70H 'p'		
572	DB	000H	;
573	DB	000H	;
574	DB	0C0H	; ##. .
575	DB	0A0H	; #. #.
576	DB	0C0H	; ##. .
577	DB	080H	; #. .
578	; 71H 'q'		
579	DB	000H	;
580	DB	000H	;
581	DB	060H	; .##.
582	DB	0A0H	; #. #.
583	DB	060H	; .##.
584	DB	020H	; ..#.
585	; 72H 'r'		
586	DB	000H	;
587	DB	000H	;
588	DB	0C0H	; ##. .
589	DB	0A0H	; #. #.
590	DB	080H	; #. .
591	DB	080H	; #. .
592	; 73H 's'		
593	DB	000H	;
594	DB	000H	;
595	DB	060H	; .##.
596	DB	080H	; #. .
597	DB	020H	; ..#.
598	DB	0C0H	; ##. .
599	; 74H 't'		
600	DB	040H	; .#..
601	DB	040H	; .#..
602	DB	0E0H	; ##. #.
603	DB	040H	; .#..
604	DB	040H	; .#..
605	DB	060H	; .##.
606	; 75H 'u'		
607	DB	000H	;
608	DB	000H	;
609	DB	0A0H	; #. #.

610	DB	0A0H	; #. #.
611	DB	0A0H	; #. #.
612	DB	060H	; .##.
613	; 76H 'v'		
614	DB	000H	;
615	DB	000H	;
616	DB	0A0H	; #. #.
617	DB	0A0H	; #. #.
618	DB	0A0H	; #. #.
619	DB	040H	; .#..
620	; 77H 'w'		
621	DB	000H	;
622	DB	000H	;
623	DB	0A0H	; #. #.
624	DB	0A0H	; #. #.
625	DB	0E0H	; ##. #.
626	DB	0A0H	; #. #.
627	; 78H 'x'		
628	DB	000H	;
629	DB	000H	;
630	DB	0A0H	; #. #.
631	DB	040H	; .#..
632	DB	040H	; .#..
633	DB	0A0H	; #. #.
634	; 79H 'y'		
635	DB	000H	;
636	DB	000H	;
637	DB	0A0H	; #. #.
638	DB	0A0H	; #. #.
639	DB	060H	; .##.
640	DB	0C0H	; ##. .
641	; 7AH 'z'		
642	DB	000H	;
643	DB	000H	;
644	DB	0E0H	; ##. #.
645	DB	020H	; ..#.
646	DB	040H	; .#..
647	DB	0E0H	; ##. #.
648	; 7BH '{'		
649	DB	000H	;
650	DB	020H	; ..#.
651	DB	040H	; .#..
652	DB	0C0H	; ##. .
653	DB	040H	; .#..
654	DB	020H	; ..#.
655	; 7CH ' '		
656	DB	040H	; .#..
657	DB	040H	; .#..
658	DB	040H	; .#..
659	DB	040H	; .#..
660	DB	040H	; .#..
661	DB	040H	; .#..
662	; 7DH '}'		
663	DB	000H	;
664	DB	0E0H	; ##. #.
665	DB	040H	; .#..
666	DB	060H	; .##.
667	DB	040H	; .#..
668	DB	080H	; #. .
669	; 7EH '~'		
670	DB	000H	;
671	DB	060H	; .##.
672	DB	080H	; #. .
673	DB	000H	;
674	DB	000H	;
675	DB	000H	;
676	; 7FH '7F'		
677	DB	000H	;
678	DB	000H	;
679	DB	000H	;
680	DB	000H	;

```

681 DB 000H ; ....
682 DB 000H ; ....
683

```

C.4 TEXAS HOLD'EM

TEXAS.ASM

```

1 ; texas.asm -- Texas Hold'em for the Bally Astrocade, over FujiNet.
2 ;
3 ; A standalone-assembly client in the o2/ mold: the shared C core cannot fit
4 ; this machine, so the game is written to it. The cartridge is the FujiNet
5 ; RP2040 cart (fujinet-firmware pico/astrocade); the graphics plan is the
6 ; Lynx port's, whose 168x102 screen this hardware matches exactly.
7 ;
8 ; Converted from fujinet-5cardstud/astrocade the way the other clients were:
9 ; the wire offsets shift +11 past viewing (community[11] at 87), each seat
10 ; holds 2 hole cards instead of 5, a 5-card community board owns the middle
11 ; of the table, and a street label names the round. The betting UI is
12 ; unchanged -- move codes and labels come from the server, so the client
13 ; still needs no poker rules.
14 ;
15 ; ROM budget: 0000H-1AFFH of the 8K window (6,912 bytes): 1B00H+ belongs to
16 ; the mailbox and build.sh stamps the "FUJIT" claim at 1CFCH so the mailbox
17 ; stays alive when this image is booted over the network. RAM is screen RAM
18 ; alone: 90 visible lines use 4000H-4E0FH and everything above is ours.
19 ;
20 ; Interrupts stay off for the program's whole life (fujilib.inc's contract:
21 ; with I = 0, refresh strays land in OS ROM and never hit the hotspots).
22 ;
23 INCLUDE "HVGLIB.H"
24 INCLUDE "fujinet.inc"
25 INCLUDE "build/flags.inc"
26 ;
27 ; --- RAM map -----
28 LINES EQU 90 ; 15 rows of 4x6 text
29 ;
30 ; What the last render drew. Rendering is opaque, so this is not a dirty
31 ; list -- it exists only for the two things overdraw cannot fix: content
32 ; that SHRANK, and edges (a new hand, your turn arriving).
33 PRVHND EQU 4E10H ; 8 bytes: cards each master seat showed
34 PRVCOM EQU 4E18H ; community cards drawn last poll
35 PRVRND EQU 4E19H
36 PRVACT EQU 4E1AH
37 PRVPC EQU 4E1BH
38 ;
39 ; Session state.
40 PLNBUF EQU 4E30H ; player name, 9 + NUL
41 TBLBUF EQU 4E3AH ; table id, 9 + NUL
42 MOVBUF EQU 4E44H ; staged move code, 2 chars; 0 = none
43 CURSLC EQU 4E46H ; reply slice the cart is publishing
44 AVAIL EQU 4E48H ; NET_STATUS bytes waiting, u16
45 PRIVAVL EQU 4E4AH ; previous reading, for the settle loop
46 RXLEN EQU 4E4CH ; reply length, captured after the READ
47 V_SEL EQU 4E4EH ; list cursor (tables / menus)
48 V_CNT EQU 4E4FH ; list entry count
49 V_KEY EQU 4E50H ; last keypad/handle scan state (edges)
50 CROWDAG EQU 4E54H ; card being drawn: magic-space address
51 CRDST EQU 4E56H ; .. and its direct screen address
52 V_URL EQU 4E58H ; request being built (0 tables, 1 state,
53 ; 2 move, 3 leave)
54 V_TMP EQU 4E59H ; loop index (player being rendered)
55 V_ACT EQU 4E5AH ; this poll's activePlayer
56 V_PC EQU 4E5BH ; this poll's playerCount
57 V_SEAT EQU 4E5CH ; master seat of the player being rendered
58 V_ST EQU 4E5DH ; that player's status byte
59 V_BASE EQU 4E5EH ; that player's record base offset (word)
60 BARCOL EQU 4E60H ; status-bar column cursor
61 V_TMP2 EQU 4E61H ; loop index (move menu entry)
62 V_NCUR EQU 4E62H ; name-entry cursor
63 V_NCNT EQU 4E63H ; cards the hand/board being drawn holds
64 V_MVL EQU 4E64H ; move-menu name width, from the move count
65 NAMEED EQU 4E6AH ; name-entry edit buffer, 8 slots
66 TBLNAM EQU 4E70H ; joined table's display name, 21 + NUL
67 HANDBUF EQU 4E88H ; hand or board being drawn, 11 + NUL
68 ;
69 LNBUFF EQU 4F00H ; display line being built (fujicfg style)
70 HEXBUF EQU 4F30H
71 STACK EQU 4FC0H ; grows down; 4FC0H+ left to the BIOS cells
72 ;
73 ; --- Display options (BIOS STRDIS, used until the 4x6 font lands) -----
74 OPTFB EQU 0CH ; fg color 3, bg color 0
75 ;
76 ; MB_* labels are module fences for tools/checksize.py's budget table.
77 ORG FIRSTC
78 MB_MAIN:
79 DB 55H
80 DW MENUST
81 DW PRGNAM
82 DW PRGSTR
83 DB "TEXAS HOLDEM"
84 DB 0
85 ;
86 PRGSTR: DI
87 LD SP,STACK
88 DO SYSTEM
89 SETOUT
90 DB LINES*2
91 DB 0 ; HORCB 0: the whole line is the right palette
92 DB 8
93 DO COLSET
94 DW PALET
95 DO FILL
96 DW NORMEM
97 DW LINES*BYTEPL
98 DB 0 ; color 0 everywhere: the felt
99 DO STRDIS
100 DB 32 ; "TEXAS HOLDEM", 12 chars x 8px, centred
101 DB 38
102 DB OPTFB
103 DW TTITLE
104 EXIT
105 ;
106 CALL FNCHECK
107 JP NZ,NOCARD
108 ;
109 IF DEMO
110 CALL DEMOSCR ; the static mock table (M1)
111 JR HALT0
112 ELSE
113 LD HL,DEFNAM ; player name until appkeys land
114 LD DE,PLNBUF
115 LD BC,6
116 LDIR

```

```

117     XOR     A
118     LD      (MOVBUF),A
119     LD      (V_KEY),A
120     CALL    NAMESCR      ; pick a name, then choose a table
121     JP      TBLSR
122     ENDIF
123
124 ; ---- Errors -----
125 NOCARD: SYSSUK STRDIS
126         DB      20
127         DB      56
128         DB      OPTFB
129         DW      ENOCART
130 HALTE:  JR      HALTE
131
132 ; ---- Data -----
133 MB_DATA:
134 TTITLE: DB      "TEXAS HOLD'EM",0
135 ENET:   DB      "NET ERR - ANY KEY RETRIES",0
136 ENOCART: DB      "NO FUJINET CART",0
137 DEFNAM: DB      "ASTRO",0
138
139 ; COLSET stores descending, ports 7 down to 0; both halves identical since
140 ; HORCB is 0. Right palette: 3 = white, 2 = red, 1 = black, 0 = felt green.
141 ; Byte = (hue << 3) | luminance; hue 0 is the grayscale column.
142 PALET:  DB      07H,52H,00H,0A0H
143         DB      07H,52H,00H,0A0H
144
145     INCLUDE "build/endpoint.inc"
146
147     IF      DEMO
148     ELSE
149 MB_SCREENS:
150     INCLUDE "screens.inc"
151 MB_NAMENT:
152     INCLUDE "nament.inc"
153 MB_SOUND:
154     INCLUDE "sound.inc"
155     ENDIF
156 MB_INPUT:
157     INCLUDE "input.inc"
158 MB_NET:
159     INCLUDE "net.inc"
160 MB_URL:
161     INCLUDE "url.inc"
162 MB_STATE:
163     INCLUDE "state.inc"
164 MB_GFX:
165     INCLUDE "gfx.inc"
166 MB_CARDS:
167     INCLUDE "cards.inc"
168     IF      DEMO
169 MB_DEMO:
170     INCLUDE "demo.inc"
171     ENDIF
172 MB_FONT:
173     INCLUDE "assets/font.inc"
174 MB_ART:
175     INCLUDE "assets/cardart.inc"
176 MB_FUJILIB:
177     INCLUDE "fujilib.inc"
178 MB_END:
179

```

FUJINET.INC -- THE WIRE FORMAT

```

1 ; fujinet.inc -- N: device, appkey commands, and the Texas Hold'em wire format.
2 ;

```

```

3 ; The command and offset values are hand-synchronised with o2/o2fujj.inc,
4 ; which carries the family's documentation of WHY each one is what it is;

```

```

5 ; this file only restates the numbers in zmac Z80 syntax. The mailbox
6 ; transport itself is fujilib.inc.
7
8 ; ---- Network device (N:) ----
9 NETDEV EQU 73H ; N1:
10 NCOPEX EQU 4FH ; 'O'
11 NCCLOSE EQU 43H ; 'C'
12 NCREAD EQU 52H ; 'R'
13 NCSTAT EQU 53H ; 'S'
14
15 NMHTTGP EQU 0CH ; open mode: HTTP GET
16 NTRNONE EQU 0 ; translation: none
17
18 ; NET_STATUS reply: 4 bytes in slice 0.
19 NSAVLO EQU 0 ; bytes waiting, low
20 NSAVHI EQU 1 ; bytes waiting, high
21 NSCONN EQU 2 ; connected flag
22 NSDEVST EQU 3 ; nDevStatus_t; 1 = SUCCESS. Checking this is
23 ; not optional: the GET is deferred until the
24 ; first STATUS, and an HTTP error page still
25 ; has a readable body (o2/net.inc).
26
27 ; ---- Fuji device appkeys ----
28 FCOKEY EQU 0DCH ; OPEN_APPKEY
29 FCCKEY EQU 0DBH ; CLOSE_APPKEY
30 FCRKEY EQU 0DDH ; READ_APPKEY
31 FCWKEY EQU 0DEH ; WRITE_APPKEY
32
33 ; Lobby appkey: creator 1 / app 1 / key 0 is the username shared by every
34 ; FujiNet client on this server (src/misc.h).
35 AKCRLO EQU 1
36 AKCRHI EQU 0
37 AKAPP EQU 1
38 AKKNAME EQU 0
39 AKMRO EQU 0
40 AKMWR EQU 1
41
42 ; ---- Wire format (?bin=1, uint16 little-endian) ----
43 ; /state, /move/XX, /leave -> Games; /tables -> Tables. Offsets are read in
44 ; place out of the reply slice window; nothing is buffered.
45
46 ; Texas Hold'em delta from 5 Card Stud: community[11] is inserted right
47 ; after viewing (offset 87), shifting everything from validMoveCount on by
48 ; +11. The offsets are locked on the other side by the _Static_asserts in
49 ; support/host-test/holdem_host_test.c against src/misc.h.
50 GOLASTR EQU 0 ; lastResult, 81 bytes
51 GOROUND EQU 81 ; 0 waiting, 1 PRE-FLOP, 2 FLOP, 3 TURN,
52 ; 4 RIVER, 5 SHOWDOWN
53 GOPOT EQU 82 ; u16
54 GOACTPL EQU 84 ; signed, OFFH = -1
55 GOMVTIM EQU 85 ; seconds
56 GOVIEW EQU 86
57 GOCOMM EQU 87 ; community board, 11 bytes: up to 5 cards
58 ; x 2 chars + NUL. Empty pre-flop, then 6 /
59 ; 8 / 10 chars at flop / turn / river.
60 GOMVCNT EQU 98
61 GOMOVES EQU 99 ; 5 slots, always
62 GOPLCNT EQU 164
63 GOPLYRS EQU 165
64
65 MVSTRID EQU 13
66 MVCODE EQU 0 ; 3 bytes; the first 2 go in the /move/XX path
67 MVNAME EQU 3 ; 10 bytes
68
69 PLSTRID EQU 33
70 PLNAME EQU 0 ; 9 bytes
71 PLSTAT EQU 9 ; 0 waiting, 1 playing, 2 folded, 3 left.
72 ; The bin=1 serializer folds all-in back
73 ; into 1, so 4 never reaches us.
74 PLBET EQU 10 ; u16
75 PLMOVE EQU 12 ; 8 bytes, the seat's last action

```

```

76 PLPURSE EQU 20 ; u16
77 PLHAND EQU 22 ; 11 bytes, but Hold'em uses 4: the 2 hole
78 ; cards. "???" masked, "??" folded, the
79 ; real chars at the showdown. The field
80 ; keeps 5 Card Stud's width on the wire.
81
82 TBSTRID EQU 36
83 TBID EQU 0 ; 9 bytes
84 TBNAME EQU 9 ; 21 bytes
85 TBPLYRS EQU 30 ; 6 bytes, literal "cur / max"
86
87 GMAXLEN EQU 429
88 GMINLEN EQU 165 ; fixed prefix through playerCount, 0 players
89 TMAXLEN EQU 361
90

```

NET.INC

```

1 ; net.inc -- the N: device round trip.
2 ;
3 ; A Z80 transcription of o2/net.inc's proven sequence, over fujilib's
4 ; mailbox. The rules it carries forward (learned the hard way there and on
5 ; the Intellivision):
6 ;
7 ; * The CLOSE belongs at the START of the next request, never after the
8 ; READ: every transaction repaints the whole reply window, a CLOSE
9 ; reply is empty, and every screen renders straight out of the window
10 ; between polls.
11 ; * FNRLX/FNRXLH belong to the most recent transaction, so the reply
12 ; length is captured immediately after the READ.
13 ; * The ESP32 reports STATUS as soon as SOME of the response has
14 ; arrived; poll until two consecutive readings agree (bounded) rather
15 ; than guessing a delay.
16 ; * The 4th STATUS reply byte is nDevStatus.t and checking it is not
17 ; optional: the HTTP GET is deferred until the first STATUS, and an
18 ; HTTP error page still has a readable body.
19 ; * Validate the reply length against what playerCount implies before
20 ; believing a byte of it.
21 ;
22 ; All routines return carry SET on failure, clear on success.
23 ;
24 ; NCMD: begin a network-device transaction. E = command, L = nparam.
25 NCMD: LD A,NETDEV
26 JP FNBEGIN
27 ;
28 ; NCLOSE: close the previous request's connection. A first call has
29 ; nothing to close and earns a NAK; both results are ignored.
30 NCLOSE: LD E,NCLOSE
31 LD L,0
32 CALL NCMD
33 LD B,3 ; ~6 s
34 JP FNCOMMIT
35 ;
36 ; NOPEN: OPEN the devicespec (V_URL) as HTTP GET, no translation.
37 NOPEN: LD E,NCOPEN
38 LD L,2
39 CALL NCMD
40 LD A,NMHTTGP
41 CALL FNPARB
42 LD A,NTRNONE
43 CALL FNPARB
44 CALL BLURL
45 LD B,8 ; HTTPS handshakes take a while
46 CALL FNCOMMIT
47 RET C
48 JR NZ,NOBAD
49 LD A,(FNREPLY)
50 CP FCACK
51 RET Z ; equal: carry already clear
52 NOBAD: SCF
53 RET
54 ;
55 ; NSTATUS: one STATUS poll. On success AVAIL = bytes waiting.
56 NSTATUS: LD E,NCSTAT
57 LD L,2
58 CALL NCMD
59 XOR A
60 CALL FNPARB
61 XOR A
62 CALL FNPARB
63 LD B,4
64 CALL FNCOMMIT
65 RET C
66 JR NZ,NSBAD
67 LD A,(FNREPLY)
68 CP FCACK
69 JR NZ,NSBAD
70 LD A,(FNRRDATA+NSDEVST)
71 CP 1 ; SUCCESS -- anything else is an HTTP error
72 JR NZ,NSBAD
73 LD A,(FNRRDATA+NSAVLO)
74 LD (AVAIL),A
75 LD A,(FNRRDATA+NSAVHI)
76 LD (AVAIL+1),A
77 OR A ; clear carry
78 RET
79 NSBAD: SCF
80 RET
81 ;
82 ; NREAD: READ min(AVAIL, GMAXLEN) bytes into the reply window, then
83 ; capture the reply length and re-home the slice cursor.
84 NREAD: LD HL,(AVAIL)
85 LD DE,GMAXLEN
86 OR A
87 SBC HL,DE
88 JR C,NRD1 ; avail < clamp: take avail
89 LD H,D
90 LD L,E
91 JR NRD2
92 ADD HL,DE
93 NRD2: PUSH HL
94 LD E,NCREAD
95 LD L,1
96 CALL NCMD
97 LD A,2 ; one 2-byte parameter: the length, LE
98 CALL FNTXBYT
99 POP HL
100 PUSH HL
101 LD A,L
102 CALL FNTXBYT
103 POP HL
104 LD A,H
105 CALL FNTXBYT
106 LD B,8
107 CALL FNCOMMIT
108 RET C
109 JR NZ,NRBD
110 LD A,(FNREPLY)
111 CP FCACK
112 JR NZ,NRBD
113 LD A,(FNRLXLL) ; capture NOW: the next transaction's
114 LD (RXLEN),A ; reply length would replace it
115 LD A,(FNRLXH)
116 LD (RXLEN+1),A
117 XOR A
118 LD (CURSLC),A ; the cart republished slice 0
119 RET ; carry clear (XOR A)
120 NRBD: SCF
121 RET
122 ;
123 ; -----
124 ; APICALL: A = request (0 tables, 1 state, 2 move, 3 leave). One full
125 ; round trip: on success (carry clear) the reply is in the window with
126 ; the slice cursor homed, and stays valid until the next APICALL.
127 APICALL: LD (V_URL),A
128 CALL NCLOSE ; the PREVIOUS request's connection
129 CALL NOPEN
130 RET C
131 XOR A
132 LD (PRVAVL),A

```



```

133     LD      (PRVAVL+1),A          203
134     LD      B,20                 204
135 APSET:  PUSH BC
136     CALL   NSTATUS
137     POP    BC
138     JR     C,APCFL
139     LD     HL,(AVAIL)
140     LD     A,H
141     OR     L
142     JR     Z,APAGN                ; nothing has arrived at all yet
143     LD     DE,(PRVAVL)
144     OR     A
145     SBC    HL,DE
146     JR     Z,APRD                 ; two readings agree: it has landed
147 APAGN:  LD     HL,(AVAIL)
148     LD     (PRVAVL),HL
149     LD     DE,4000                ; ~3 frames between polls
150 APDLY:  DEC    DE
151     LD     A,D
152     OR     E
153     JR     NZ,APDLY
154     DJNZ   APSET
155 APCFL:  CALL   NCLOSE              ; failed before the READ: nothing to keep
156     SCF
157     RET
158 APRD:   CALL   NREAD
159     JR     C,APCFL
160 ; Validation failure after the READ returns carry WITHOUT a close -- the
161 ; next APICALL's leading NCLOSE retires the connection, and closing here
162 ; would wipe the window a caller may still be rendering the last good
163 ; reply from.
164 ; falls through into VALID8
165
166 ; VALID8: sanity-check RXLEN for the request in V_URL. Tables: nonempty.
167 ; Game: at least the fixed prefix, and exactly 165 + 33*playerCount.
168 VALID8: LD     A,(V_URL)
169     OR     A
170     JR     NZ,VALGM
171     LD     HL,(RXLEN)              ; tables: any nonzero length; the count
172     LD     A,H                    ; byte bounds the walk
173     OR     L
174     RET    NZ                      ; OR cleared carry
175     SCF
176     RET
177 VALGM:  LD     HL,(RXLEN)
178     LD     DE,GMINLEN
179     OR     A
180     SBC    HL,DE
181     RET    C                      ; shorter than the fixed prefix
182     LD     HL,GOPLCNT
183     CALL   RXGETB
184     CP     9
185     JR     NC,VALBAD              ; more seats than the server has
186     LD     L,A                    ; expected = 165 + A*33
187     LD     H,0
188     LD     E,A
189     LD     D,0
190     ADD    HL,HL
191     ADD    HL,HL
192     ADD    HL,HL
193     ADD    HL,HL
194     ADD    HL,HL                  ; *32
195     ADD    HL,DE                  ; *33
196     LD     DE,GMINLEN
197     ADD    HL,DE
198     LD     DE,(RXLEN)
199     OR     A
200     SBC    HL,DE
201     RET    Z                      ; exact: carry clear
202 VALBAD: SCF

```

URL.INC

```
1 ; url.inc -- building the devicespec for a request.
2 ;
3 ; The URL is streamed straight into the mailbox's TX byte stream a
4 ; character at a time (o2/url.inc's way); it is never assembled in RAM.
5 ; The cart counts the payload itself, so nothing here tracks a length; a
6 ; whole /move request is ~90 bytes against the 320-byte TX ceiling.
7 ;
8 ; TBLBUF and PLNBUF are NUL-terminated copies of fixed-width wire fields
9 ; -- pushed up to the NUL, never through it (padding in the middle of a
10 ; URL corrupts the request on the wire).
11 ;
12 ; BLDURL: stream the devicespec for request (V_URL).
13 BLDURL: LD      A,(UENDPT)      ; "N:" unless the endpoint carries its own
14         CP      'N'
15         JR      Z,BURL1
16         LD      A,'N'
17         CALL    FNTXBYT
18         LD      A,':'
19         CALL    FNTXBYT
20 BURL1:  LD      HL,UENDPT
21         CALL    FNTXSTR
22         LD      A,(V_URL)
23         OR      A
24         JR      NZ,BURL2
25         LD      HL,UTABLS      ; tables: no table, no player
26         JP      FNTXSTR
27 BURL2:  CP      2
28         JR      C,BURLST
29         JR      Z,BURLMV
30         LD      HL,ULEAVE
31         CALL    FNTXSTR
32         JR      BURLO
33 BURLST: LD      HL,USTATE
34         CALL    FNTXSTR
35         JR      BURLO
36 BURLMV: LD      HL,UMOVE
37         CALL    FNTXSTR
38         LD      A,(MOVBUF)      ; the move code's first 2 characters
39         CALL    FNTXBYT
40         LD      A,(MOVBUF+1)
41         CALL    FNTXBYT
42 BURLO:  LD      HL,UQTAB
43         CALL    FNTXSTR
44         LD      HL,TBLBUF
45         CALL    FNTXSTR
46         LD      HL,UQPLR
47         CALL    FNTXSTR
48         LD      HL,PLNBUF
49         CALL    FNTXSTR
50         LD      HL,UQBIN
51         JP      FNTXSTR
52
53 UTABLS: DB      "tables?bin=1",0
54 USTATE: DB      "state",0
55 UMOVE:  DB      "move/",0
56 ULEAVE: DB      "leave",0
57 UQTAB:  DB      "?table=",0
58 UQPLR:  DB      "&player=",0
59 UQBIN:  DB      "&bin=1",0
60
```

SCREENS.INC

```

1 ; screens.inc -- table selection, the game screen, and the poll loop.
2
3 ; Rendering policy: every poll redraws everything, opaquely, straight out
4 ; of the reply window -- identical pixels rewritten are invisible, so
5 ; there is no flicker and no digest bookkeeping. The one thing opaque
6 ; overdraw cannot fix is content that SHRANK (a new hand dealt, a player
7 ; gone), so CHKNEW watches round/playerCount and answers with a full
8 ; clear+statics before the ordinary render.
9
10 ; Seat geometry is the Lynx port's, compressed to 15 rows: master seats
11 ; 0-7 run bottom-center, bottom-left, mid-left, top-left, top-center,
12 ; top-right, mid-right, bottom-right; PCIDX picks which masters serve a
13 ; given player count, with the requesting player (index 0) always at
14 ; master seat 0. Hands are 11 cols wide worst case; every hand draws
15 ; left-to-right whatever its side of the table.
16
17 ; =====
18 ; Table selection
19 ; =====
20 TBLSCR: CALL KEYRAW ; a key still held from the previous
21 LD (V_KEY),A ; screen must not fire here
22 CALL CLRSCR
23 SYSSUK STRDIS ; two-line title, BIOS font at 2x
24 DB 40 ; "TEXAS": 5 chars x 16px, centred on 160
25 DB 0
26 DB 4CH
27 DW TTEXAS
28 SYSSUK STRDIS
29 DB 32 ; "HOLDEM": 6 chars x 16px
30 DB 17
31 DB 4CH
32 DW THOLDM
33 LD HL,TCH00S
34 LD DE,6*256+2
35 LD C,XWONF
36 CALL TXTAT
37 TBLRF: XOR A ; fetch /tables
38 CALL APICALL
39 JR NC,TBL0K
40 LD HL,ENET
41 LD DE,14*256+0
42 LD C,XRONF
43 CALL TXTAT
44 TBLRW: CALL KEYSN
45 OR A
46 JR Z,TBLRW
47 JR TBLRF
48 TBL0K: LD HL,0
49 CALL RXGETB ; table count
50 OR A
51 JR Z,TBLRW ; none yet: wait for a key, refetch
52 CP 9
53 JR C,TBOK2
54 LD A,8 ; eight rows fit under the title
55 TBOK2: LD (V_CNT),A
56 XOR A
57 LD (V_SEL),A
58 CALL TBLDRAW
59 TBKEY: CALL KEYSN
60 OR A
61 JR Z,TBKEY
62 CP 'U'
63 JR NZ,TBK1
64 LD A,(V_SEL)
65 OR A
66 JR Z,TBKEY
67 DEC A
68 LD (V_SEL),A
69 CALL SNDKEY
70 CALL TBLDRAW
71 JR TBKEY

```

```

72 TBK1: CP 'D'
73 JR NZ,TBK2
74 LD A,(V_SEL)
75 INC A
76 LD HL,V_CNT
77 CP (HL)
78 JR NC,TBKEY
79 LD (V_SEL),A
80 CALL SNDKEY
81 CALL TBLDRAW
82 JR TBKEY
83 TBK2: CP 'E' ; CE: back to the name screen
84 JR NZ,TBK2A
85 CALL NAMESCR
86 JR TBLSCR
87 TBK2A: CP ' ' ; the . key: how to play
88 JR NZ,TBK2B
89 CALL HELPSCR
90 JP TBLSCR
91 TBK2B: CP 'I'
92 JR Z,TBJOIN
93 CP '0' ; digit shortcut: 1-9 then 0 = row 10
94 JR C,TBKEY
95 CP '0'+1
96 JR NC,TBKEY
97 SUB '1'
98 CP 0FFH ; '0' wrapped to FF: row 10
99 JR NZ,TBK3
100 LD A,9
101 TBK3: LD HL,V_CNT
102 CP (HL)
103 JR NC,TBKEY
104 LD (V_SEL),A
105 ; falls into the join
106 TBJOIN: CALL SNOSEL
107 LD A,(V_SEL) ; record base = 1 + sel*36
108 CALL TBOFF
109 LD DE,TBLBUF ; id, up to its NUL
110 LD B,9
111 CALL RXSTRN
112 LD DE,9 ; the name field, for the game header
113 ADD HL,DE
114 LD DE,TBLNAM
115 LD B,21
116 CALL RXSTRN
117 CALL CLRSCR
118 LD HL,TJOIN
119 LD DE,7*256+14
120 LD C,XWONF
121 CALL TXTAT
122 LD A,0FFH ; force the new-hand path on first poll
123 LD (PRVRND),A
124 LD (PRVACT),A ; and arm the your-turn cue
125 JP GAMELP
126
127 ; TBOFF: HL = window offset of table record A (1 + A*36, and 36 = 32+4).
128 ; Clobbers A,DE.
129 TBOFF: LD L,A
130 LD H,0
131 ADD HL,HL
132 ADD HL,HL ; *4
133 LD E,L
134 LD D,H
135 ADD HL,HL
136 ADD HL,HL
137 ADD HL,HL ; *32
138 ADD HL,DE ; *36
139 INC HL ; + the count byte
140 RET
141
142 ; TBLDRAW: the list rows: name and "cur / max", selected row inverse.

```

```

143 TDBDRAW: XOR      A
144           LD      (V_TMP),A
145 TBD1:   LD      A,(V_TMP)
146           LD      HL,V_CNT
147           CP      (HL)
148           RET     NC
149           CALL    TBOFF
150           PUSH    HL
151           LD      DE,9           ; the display name field
152           ADD     HL,DE
153           LD      DE,LINBUF
154           LD      B,21
155           CALL    RXSTRN
156           LD      A,(V_SEL)      ; row color
157           LD      HL,V_TMP
158           CP      (HL)
159           LD      C,XFONW
160           JR      Z,TBD2
161           LD      C,XWONF
162 TBD2:   LD      A,(V_TMP)      ; row y = 7 + i
163           ADD     A,7
164           LD      D,A
165           LD      E,2
166           LD      HL,LINBUF
167           LD      B,21
168           PUSH    BC
169           CALL    TXTN
170           POP     BC
171           POP     HL
172           LD      DE,30         ; the "cur / max" field
173           ADD     HL,DE
174           LD      DE,LINBUF
175           LD      B,6
176           CALL    RXSTRN
177           LD      A,(V_TMP)
178           ADD     A,7
179           LD      D,A
180           LD      E,25
181           LD      HL,LINBUF
182           LD      B,6
183           CALL    TXTN
184           LD      A,(V_TMP)
185           INC     A
186           LD      (V_TMP),A
187           JR      TBD1
188
189 ; =====
190 ; The game loop
191 ; =====
192 GAMELP: LD      A,(MOVBUFF)      ; a staged move rides the next request --
193           OR      A              ; its reply IS the next state
194           LD      A,1
195           JR      Z,GLREQ
196           LD      A,2
197 GLREQ:   CALL    APICALL
198           JR      C,GLFAIL
199           XOR     A
200           LD      (MOVBUFF),A    ; consumed
201           CALL    CHKNEW
202           CALL    RENDER
203           LD      B,90           ; ~2 s of input scanning between polls
204 GLWAIT:  PUSH    BC
205           CALL    KEYSCN
206           POP     BC
207           OR      A
208           JR      NZ,GLKEY
209           LD      DE,2500        ; ~20 ms
210 GLDLY:  DEC     DE
211           LD      A,D
212           OR      E
213           JR      NZ,GLDLY

214           DJNZ   GLWAIT
215           JR     GAMELP
216 GLKEY:  CP      'E'              ; CE leaves the table
217           JR     NZ,GLK1
218           LD      A,3
219           CALL    APICALL
220           JP      TBLSCR
221 GLK1:   CP      '.'              ; how to play, then a forced full redraw
222           JR     NZ,GLK1A
223           CALL    HELPSCR
224           LD      A,0FFH
225           LD      (PRVRND),A
226           JP      GAMELP
227 GLK1A:  CP      '1'
228           JR     C,GLK3          ; '0' (or below): refresh now
229           CP      '5'+1
230           JR     NC,GLK3
231           SUB     '1'            ; keypad 1-5 -> validMoves 0-4
232           LD      HL,GOMVCNT
233           LD      B,A
234           CALL    RXGETB
235           LD      C,A
236           LD      A,B
237           CP      C
238           JR     NC,GLK3        ; not on the menu right now
239           LD      L,A            ; move code offset = GOMOVES + n*13
240           LD      H,0
241           LD      E,A
242           LD      D,H
243           ADD     HL,HL
244           ADD     HL,DE          ; *3
245           ADD     HL,HL
246           ADD     HL,HL          ; *12
247           ADD     HL,DE          ; *13
248           LD      DE,GOMOVES
249           ADD     HL,DE
250           CALL    RXGETB
251           LD      (MOVBUFF),A
252           INC     HL
253           CALL    RXGETB
254           LD      (MOVBUFF+1),A
255           CALL    SNDSEL
256 GLK3:   JP      GAMELP          ; poll immediately (move or refresh)
257 GLFAIL: LD      HL,TBANG        ; a red mark in the corner; back off and
258           LD      DE,0+256+39    ; try again
259           LD      C,XRONF
260           CALL    TXTAT
261           LD      B,150
262 GLFW:   PUSH    BC
263           CALL    KEYSCN
264           POP     BC
265           CP      'E'
266           JR     NZ,GLFW1
267           LD      A,3
268           CALL    APICALL
269           JP      TBLSCR
270 GLFW1:  LD      DE,2500
271 GLFD:   DEC     DE
272           LD      A,D
273           OR      E
274           JR     NZ,GLFD
275           DJNZ   GLFW
276           JP      GAMELP
277
278           ; CHKNEW: a new hand (round dropped) or a changed seat count means the
279           ; screen holds cards that no longer exist: clear and redraw the statics.
280           CHKNEW: LD      HL,GOROUND
281           CALL    RXGETB
282           LD      HL,PRVRND
283           CP      (HL)
284           LD      (HL),A

```

```

285 JR C,CHKN1
286 LD HL,GOPLCNT
287 CALL RXGETB
288 LD HL,PRVPC
289 CP (HL)
290 LD (HL),A
291 RET Z
292 CHKN1: LD HL,GOPLCNT ; keep PRVPC honest on the round path too
293 CALL RXGETB
294 LD (PRVPC),A
295 ; falls through into GSTATIC
296
297 ; GSTATIC: clear + the parts of the game screen a poll never changes.
298 GSTATIC: CALL CLRSCR
299 LD A,XOR ; the clear took every card with it, so the
300 LD (PRVCOM),A ; shrink checks have nothing left to chase
301 LD B,8
302 LD HL,PRVHND
303 GSTZ: LD (HL),A
304 INC HL
305 DJNZ GSTZ
306 LD HL,TBLNAM
307 LD DE,0+256+0
308 LD C,XWONF
309 JP TXTAT
310
311 ; =====
312 ; Rendering a poll
313 ; =====
314 RENDER: LD HL,GOROUND ; the street, named above the board
315 CALL RXGETB
316 CP 6 ; VALID8 checks the reply LENGTH, never the
317 JR C,RNDOK ; round, and this indexes a table: clamp it
318 XOR A
319 RNDOK: ADD A,A ; STREETS is 8 space-padded bytes per round
320 ADD A,A
321 LD E,A
322 LD D,0
323 LD HL,STREETS
324 ADD HL,DE
325 LD DE,5*256+16
326 LD C,XWONF
327 LD B,8
328 LD TXTN ; TXTN pads the field, so a short label
329 CALL TXTN ; wipes a long one with no blanking pass
330 CALL BOARD
331 LD HL,GOACTPL
332 CALL RXGETB
333 LD (V_ACT),A
334 LD HL,PRVACT ; the your-turn cue, on the edge only
335 CP (HL)
336 LD (HL),A
337 JR Z,RENDA2
338 OR A
339 JR NZ,RENDA2
340 CALL SNDTRN
341 RENDA2: LD HL,GOPLCNT
342 CALL RXGETB
343 LD (V_PC),A
344 XOR A ; each seated player
345 LD (V_TMP),A
346 RENDP: LD A,(V_TMP)
347 LD HL,V_PC
348 CP (HL)
349 JR NC,RENDG
350 CALL SEAT1
351 LD A,(V_TMP)
352 INC A
353 LD (V_TMP),A
354 JR RENDP
355 RENDG: CALL PURSE

```

```

356 JP BAR
357
358 ; -----
359 ; SEAT1: render pPlayer (V_TMP): name, bet, hand.
360 A,(V_PC) ; master seat = PCIDX[(pc-2)*8 + i]
361 SEAT1: LD SUB
362 ADD A,A
363 ADD A,A
364 ADD A,A
365 LD HL,V_TMP
366 ADD A,(HL)
367 LD HL,PCIDX
368 LD E,A
369 LD D,0
370 ADD HL,DE
371 LD A,(HL)
372 LD (V_SEAT),A
373 LD A,(V_TMP) ; record base = GOPLYRS + i*33
374 LD L,A
375 LD H,0
376 LD E,A
377 LD D,H
378 ADD HL,HL
379 ADD HL,HL
380 ADD HL,HL
381 ADD HL,HL
382 ADD HL,HL ; *32
383 ADD HL,DE ; *33
384 LD DE,GOPLYRS
385 ADD HL,DE
386 LD (V_BASE),HL
387 LD DE,PLSTAT ; status
388 ADD HL,DE
389 CALL RXGETB
390 LD (V_ST),A
391 LD HL,(V_BASE) ; name
392 LD DE,LINBUF
393 LD B,9
394 CALL RXSTRN
395 LD A,(V_TMP) ; color: active inverse, folded/left dim
396 LD HL,V_ACT
397 CP (HL)
398 LD C,XFONW
399 JR Z,SEATN
400 LD A,(V_ST)
401 LD C,XKONF
402 LD NC,SEATN
403 LD C,XWONF
404 SEATN: LD A,(V_SEAT)
405 LD HL,SEATNR
406 CALL TBLK
407 LD D,A
408 LD A,(V_SEAT)
409 LD HL,SEATNC
410 CALL TBLK
411 LD E,A
412 LD HL,LINBUF
413 LD B,9
414 CALL TXTN
415 LD A,(V_SEAT) ; the bet chip; yours lives in the purse
416 OR A ; line instead
417 JR Z,SEATH
418 LD HL,(V_BASE)
419 LD DE,PLBET
420 ADD HL,DE
421 CALL RXGETW
422 LD A,D
423 OR E
424 JR Z,SEATB0
425 PUSH DE
426

```

```

427 CALL SEATBP ; D,E = bet row,col
428 CALL DRWCHP
429 CALL SEATBP
430 INC E ; amount right of the chip
431 POP HL
432 PUSH DE
433 CALL U16STR
434 POP DE
435 LD HL,LINBUF
436 LD B,4
437 LD C,XWONF
438 CALL TXTN
439 JR SEATH
440 SEATB0: CALL SEATBP ; no bet: blank the chip cell + amount
441 CALL CRDPOS
442 LD B,6
443 LD C,1
444 LD D,0
445 CALL FILLRC
446 SEATBP: CALL SEATBP
447 INC E
448 LD HL,TNUL
449 LD B,4
450 LD C,XWONF
451 SEATH: CALL TXTN
452 LD HL,(V_BASE) ; the hand: 2 hole cards
453 LD DE,PLHAND
454 ADD HL,DE
455 LD DE,HANDBUF
456 LD B,11
457 CALL RXSTRN
458 CALL HND CNT
459 LD A,(V_SEAT)
460 LD HL,SEATHR
461 CALL TBLK
462 LD D,A
463 LD A,(V_SEAT)
464 LD HL,SEATHC
465 CALL TBLK
466 LD E,A
467 LD A,(V_SEAT) ; what this seat showed last poll: a fold
468 LD HL,PRVHND ; turns "?????" into "???" mid-hand, and the
469 CALL TBLK ; card that went away is not overdrawn
470 LD C,A
471 CALL HND BLK
472 LD A,(V_SEAT)
473 LD HL,PRVHND
474 PUSH DE
475 LD E,A
476 LD D,0
477 ADD HL,DE
478 LD A,(V_NCNT)
479 LD (HL),A
480 POP DE
481 LD HL,HANDBUF
482 LD B,2
483 JP DRWHND
484
485 ; -----
486 ; BOARD: the community cards, centred at row 6 col 14. Five cards on the
487 ; 2-col pitch span cols 14-24, clear of the mid seats' hands at cols 0-4
488 ; and 29-33. Drawn every poll like any other field; only the shrink needs
489 ; help, because the server can empty the board without the round dropping.
490 BOARD: LD HL,GOCOMM
491 LD DE,HANDBUF
492 LD B,11
493 CALL RXSTRN
494 CALL HND CNT
495 LD A,(PRVCOM)
496 LD C,A
497 LD DE,6*256+14
498 CALL HND BLK
499 LD A,(V_NCNT)
500 LD (PRVCOM),A
501 LD HL,HANDBUF
502 LD B,5
503 JP DRWHND
504
505 ; HND CNT: (V_NCNT) = cards in HANDBUF -- two wire chars per card, and the
506 ; field is packed from the left, so it is just the length halved.
507 ; Clobbers A,HL.
508 HND CNT: LD HL,HANDBUF
509 CALL STRLEN
510 SRL A
511 LD (V_NCNT),A
512 RET
513
514 ; HND BLK: blank the card cells the last poll drew here that this one will
515 ; not repaint -- the one thing opaque overdraw cannot fix. C = cards drawn
516 ; last poll, (V_NCNT) = cards now, D = card row, E = start col. Only the
517 ; vacated tail is touched, so nothing that stays on screen ever flickers.
518 ; Preserves D,E; clobbers A,BC,HL.
519 HND BLK: LD A,(V_NCNT)
520 CP C
521 RET NC ; grew or held: the redraw covers it
522 PUSH DE
523 LD B,A
524 LD A,C
525 SUB B ; cards that went away...
526 ADD A,A ; ...at 2 cols each
527 LD C,A
528 LD A,B
529 OR A
530 JR Z,HNB0
531 ADD A,A ; the new art ends at col E + 2*count,
532 INC A ; so start one col past it
533 ADD A,E
534 LD E,A
535 JR HNB1
536 HNB0: INC C
537 HNB1: PUSH BC
538 CALL CRDPOS ; HL = screen address of row D, col E
539 POP BC
540 LD B,17 ; a card is 17 lines tall
541 LD D,0 ; back to felt
542 CALL FILLRC
543 POP DE
544 RET
545
546 ; SEATBP: D,E = bet row,col of seat (V_SEAT). Clobbers A,HL.
547 SEATBP: LD A,(V_SEAT)
548 LD HL,SEATBR
549 CALL TBLK
550 LD D,A
551 LD A,(V_SEAT)
552 LD HL,SEATBC
553 CALL TBLK
554 LD E,A
555 RET
556
557 ; TBLK: A = table(HL)[A]. Preserves DE (callers fill D and E from
558 ; consecutive lookups).
559 TBLK: PUSH DE
560 LD E,A
561 LD D,0
562 ADD HL,DE
563 LD A,(HL)
564 POP DE
565 RET
566
567 ; -----
568 ; PURSE: row 13 -- your purse and bet (or the viewing banner), and the

```

```

569 ; move clock at the right edge.
570 PURSE: LD HL,GOVIEW
571 CALL RXGETB
572 OR A
573 JR Z,PUR1
574 LD HL,TVIEW
575 LD DE,13*256+0
576 LD C,XWONF
577 LD B,22
578 CALL TXTN
579 JR PURT
580 PUR1: LD HL,TPURSE
581 LD DE,13*256+0
582 LD C,XWONF
583 CALL TXTAT
584 LD HL,GOPLYRS+PLPURSE
585 CALL RXGETW
586 EX DE,HL
587 CALL U16STR
588 LD HL,LINBUF
589 LD DE,13*256+6
590 LD C,XWONF
591 LD B,5
592 CALL TXTN
593 LD HL,TBET
594 LD DE,13*256+12
595 LD C,XWONF
596 CALL TXTAT
597 LD HL,GOPLYRS+PLBET
598 CALL RXGETW
599 EX DE,HL
600 CALL U16STR
601 LD HL,LINBUF
602 LD DE,13*256+16
603 LD C,XWONF
604 LD B,5
605 CALL TXTN
606 PURT: LD HL,GOPOT ; the pot, right of your purse and bet:
607 CALL RXGETW ; the board owns the middle of the table
608 EX DE,HL ; now, and cols 22-30 stay clear here even
609 CALL U16STR ; under the VIEWING banner (cols 0-21)
610 LD HL,TPOT
611 LD DE,13*256+22
612 LD C,XWONF
613 CALL TXTAT
614 LD HL,LINBUF
615 LD DE,13*256+26
616 LD C,XWONF
617 LD B,5
618 CALL TXTN
619 LD A,(V_ACT) ; the move clock
620 INC A
621 JR Z,PURT0 ; nobody active
622 LD HL,GOMVTIM
623 CALL RXGETB
624 LD L,A
625 LD H,0
626 CALL U16STR
627 LD HL,LINBUF
628 LD DE,13*256+37
629 LD C,XRONF
630 LD B,2
631 JP TXTN
632 PURT0: LD HL,TNUL
633 LD DE,13*256+37
634 LD C,XWONF
635 LD B,2
636 JP TXTN
637
638 -----
639 ; BAR: row 14, the white strip: your move menu when it is your turn,

```

```

640 ; otherwise the last-result banner.
641 BAR: LD HL,NORMEM+14*240
642 LD B,6
643 LD C,BYTEPL
644 LD D,0FFH
645 CALL FILLRC
646 LD A,(V_ACT)
647 OR A
648 JP NZ,BARLR ; not you (or nobody)
649 LD HL,GOVIEW
650 CALL RXGETB
651 OR A
652 JP NZ,BARLR
653 LD HL,GOMVCNT
654 CALL RXGETB
655 OR A ; RXGETB's LD A,(DE) leaves flags alone
656 JP Z,BARLR
657 CP 6
658 JR C,BARMC ; the wire says at most 5, but this indexes
659 A,5 ; a table and bounds a loop: do not trust it
660 BARMC: LD (V_CNT),A ; menu: "1 NAME 2 NAME ..."
661 LD HL,MVLEN-1 ; name width, scaled to the move count
662 LD E,A
663 LD D,0
664 ADD HL,DE
665 LD A,(HL)
666 LD (V_MVL),A
667 XOR A
668 LD (V_TMP2),A
669 LD (BARCOL),A
670 BARM: LD A,(V_TMP2)
671 LD HL,V_CNT
672 CP (HL)
673 RET NC
674 ADD A,'1' ; the digit
675 LD (HEXBUF),A
676 XOR A
677 LD (HEXBUF+1),A
678 LD A,(V_TMP2) ; the name: GOMOVES + n*13 + MVNAME
679 LD H,0
680 LD E,A
681 LD D,H
682 ADD HL,HL
683 ADD HL,DE ; *3
684 ADD HL,HL
685 ADD HL,HL ; *12
686 ADD HL,DE ; *13
687 LD DE,GOMOVES+MVNAME
688 LD HL,DE
689 ADD HL,DE
690 LD DE,LINBUF
691 LD B,10
692 CALL RXSTRN
693 LD A,(V_MVL)
694 CALL MVMCOMP
695 LD A,(BARCOL) ; digit
696 LD E,A
697 LD D,14
698 LD HL,HEXBUF
699 LD C,XRONW
700 CALL TXTAT
701 LD A,(BARCOL) ; name, in its fixed field
702 ADD A,2
703 LD E,A
704 LD D,14
705 LD HL,LINBUF
706 LD C,XKONW
707 LD A,(V_MVL)
708 LD B,A
709 CALL TXTN
710 LD A,(V_MVL) ; advance: digit + space + name + space

```

```

711 ADD A,3
712 LD HL,BARCOL
713 ADD A,(HL)
714 LD (HL),A
715 LD A,(V_TMP2)
716 INC A
717 LD (V_TMP2),A
718 JR BARM
719 ; Not your turn. The server leaves lastResult holding the PREVIOUS hand's
720 ; winner for the whole next hand, so showing it whenever the menu is down
721 ; parks a stale banner on screen all hand long. Show it only between
722 ; hands (round 0) and at the showdown; during a live hand, say whose
723 ; turn it is.
724 BARLR: LD HL,GOROUND
725 CALL RXGETB
726 OR A
727 JR Z,BARRES ; between hands: the result stands
728 CP 5
729 JR NC,BARRES ; showdown: the result is the news
730 LD A,(V_ACT)
731 INC A
732 RET Z ; FFH = nobody: leave the bar blank
733 LD HL,TAITON
734 LD DE,14*256+1
735 LD C,XKONW
736 CALL TXTAT
737 LD A,(V_ACT) ; the active player's name
738 LD L,A
739 LD H,0
740 LD E,A
741 LD D,H
742 ADD HL,HL
743 ADD HL,HL
744 ADD HL,HL
745 ADD HL,HL
746 ADD HL,HL ; *32
747 ADD HL,DE ; *33
748 LD DE,GOPLYRS
749 ADD HL,DE
750 LD DE,LINBUF
751 LD B,9
752 CALL RXSTRN
753 LD HL,LINBUF
754 LD DE,14*256+12
755 LD C,XKONW
756 LD B,9
757 JP TXTN
758 BARRES: LD HL,GOLASTR ; the last-result banner
759 LD DE,LINBUF
760 LD B,39
761 CALL RXSTRN
762 LD HL,LINBUF
763 LD DE,14*256+1
764 LD C,XKONW
765 LD B,39
766 JP TXTN
767 ; -----
768 ; HELPSCR: how to play. Any key returns.
769 HELPSCR: CALL KEYRAW
770 LD (V_KEY),A
771 CALL CLRSCR
772 LD HL,HELPTX
773 LD D,0
774 LD A,(HL) ; row; strings run in order, one per row,
775 HELP1: LD A,(HL) ; first byte = column (FF ends)
776 CP 0FFH
777 JR Z,HELP9
778 LD E,A
779 INC HL
780 DE PUSH HL
781
782 LD C,XWONF
783 CALL TXTAT
784 POP HL
785 HELP2: LD A,(HL) ; skip past the NUL
786 INC HL
787 OR A
788 JR NZ,HELP2
789 POP DE
790 INC D
791 JR HELP1
792 HELP9: CALL KEYSCN
793 OR A
794 JR Z,HELP9
795 RET
796
797 HELPTX: DB 13,"HOW TO PLAY",0
798 DB 0,"",0
799 DB 1,"EACH PLAYER GETS TWO HOLE CARDS.",0
800 DB 1,"FIVE SHARED CARDS ARE DEALT IN",0
801 DB 1,"THE MIDDLE - THREE, THEN ONE.",0
802 DB 1,"THEN ONE - WITH BETTING AFTER",0
803 DB 1,"EACH. THE BEST FIVE CARD HAND",0
804 DB 1,"TAKES THE POT.",0
805 DB 0,"",0
806 DB 1,"KEYPAD 1-5 CHOOSES A MOVE",0
807 DB 1,"0 POLLS NOW CE LEAVES",0
808 DB 1,"STICK PICKS TRIGGER JOINS",0
809 DB 1,". SHOWS THIS SCREEN",0
810 DB 0,"",0
811 DB 1,"TH.CARR-DESIGNS.COM",0
812 DB 0FFH
813 ; MVCOMP: squeeze the move name in LINBUF into A cells.
814 ;
815 ; 5 Card Stud laid the menu out at whatever width the names happened to
816 ; be, which was fine for its two or three moves. Hold'em routinely offers
817 ; four or five, and the server's labels run to nine characters ("Raise
818 ; 200", "All-in 10" -- capped by trimToWord in its util.go), so five of
819 ; them want 65 of the 40 columns this row has.
820 ;
821 ; So squeeze, the way the C clients' compactMoveName() does: if the name
822 ; has a space, keep its first letter and everything after that space
823 ; ("Call 100" -> "C100"); otherwise drop any '-' ("All-in" -> "ALLIN").
824 ; Every label the server actually emits lands inside 5 cells that way, so
825 ; nothing is truncated even at the tightest width. Clobbers everything.
826 MVCOMP: LD B,A ; the budget
827 LD HL,LINBUF
828 CALL STRLEN
829 CP B
830 RET C
831 RET Z ; it already fits: leave it in full
832 LD HL,LINBUF
833 LD A,(HL)
834 MVC1: OR A
835 OR A
836 JR Z,MVC3 ; no space: fall back to dropping '-'
837 CP ' '
838 JR Z,MVC2
839 INC HL
840 JR MCV1
841 MCV2: INC HL ; the amount, after the space
842 LD DE,LINBUF+1 ; kept behind LINBUF[0], the first letter
843 MCV2A: LD A,(HL)
844 LD (DE),A
845 OR A
846 RET Z
847 INC HL
848 INC DE
849 MCV3: LD HL,LINBUF
850 LD DE,LINBUF
851 LD A,(HL)
852 MCV3A:

```



```

853 INC HL
854 CP '!'
855 JR Z,MVC3A
856 LD (DE),A
857 INC DE
858 OR A
859 JR NZ,MVC3A
860 RET
861
862 ; Move-name width by move count, 1-5. Each entry costs the digit, a space,
863 ; the name and a trailing space, so the strides are 13/13/13/10/8 -- five
864 ; moves fill the 40-column row exactly and four leave it flush.
865 MVLEN: DB 10,10,10,7,5
866
867 ; STRLEN: A = Length of the string at HL. Clobbers HL.
868 STRLEN: PUSH BC
869 LD B,0
870 STRL1: LD A,(HL)
871 OR A
872 JR Z,STRL2
873 INC HL
874 INC B
875 JR STRL1
876 STRL2: LD A,B
877 POP BC
878 RET
879
880 ; -----
881 ; Seat geometry: master seats 0-7 (see the header comment).
882 ; A 2-card hand is 5 cols where a 5-card one was 11, so the hand columns
883 ; are re-anchored: left seats stay put, the two centre seats centre on
884 ; px 78 (the community board's own centre), and the right seats align to
885 ; the right edge of their name field instead of trailing off to its left.
886 SEATNC: DB 14,0,0,0,15,29,31,29 ; name col (31+9 is the screen edge)
887 SEATHR: DB 12,12,5,1,1,1,5,12 ; name row
888 SEATHC: DB 17,0,0,0,17,33,35,33 ; hand col
889 SEATHR: DB 9,9,6,2,2,2,6,9 ; hand row
890 ; The top seats' bets move off row 1 and down onto their own hand row.
891 ; Row 1 cannot hold three lots of name + chip + amount (3 x 14 = 42 cols in
892 ; 40): inherited from 5 Card Stud, seat 4's amount ran through seat 5's
893 ; chip and into its name. That only showed there when two particular seats
894 ; both had live bets, but in Hold'em every seat in the hand carries one on
895 ; every street, so it would paint over the table constantly. Seat 1's
896 ; amount likewise clipped the first cell of your own name on row 12.
897 SEATBC: DB 13,9,11,6,12,28,24,24 ; bet chip col (amount at +1)
898 SEATBR: DB 13,12,5,2,2,2,5,12 ; bet row
899
900 ; playerCountIndex: which masters serve 2..8 players (Lynx vars.c,
901 ; verbatim). The requesting player is always index 0 = master seat 0.
902 PCIDX: DB 0,4,0,0,0,0,0,0
903 DB 0,2,6,0,0,0,0,0
904 DB 0,2,4,6,0,0,0,0
905 DB 0,2,3,5,6,0,0,0
906 DB 0,2,3,4,5,6,0,0
907 DB 0,2,3,4,5,6,7,0
908 DB 0,1,2,3,4,5,6,7
909
910 TCHOOS: DB "CHOOSE A TABLE",0
911 TTEXAS: DB "TEXAS",0
912 THOLDM: DB "HOLD'EM",0
913 TJOIN: DB "JOINING...",0
914 TPOT: DB "POT",0
915 TVIEW: DB "VIEWING - CE TO LEAVE",0
916
917 ; Street names, 8 space-padded bytes per round value so round*8 indexes
918 ; straight in and TXTN's fixed-width field needs no terminator. Every cell
919 ; is rewritten opaquely each poll, so "PRE-FLOP" -> "FLOP" clears
920 ; itself with no separate blanking pass.
921 STREETS: DB "WAITING " ; 0 waiting (and the clamp target)
922 "PRE-FLOP" ; 1
923 "FLOP " ; 2

```

```

924 DB "TURN " ; 3
925 DB "RIVER " ; 4
926 DB "SHOWDOWN" ; 5
927 TPURSE: DB "PURSE",0
928 TWAITON: DB "WAITING ON",0
929 TBET: DB "BET",0
930 TBANG: DB "!",0
931 TNUL: DB 0
932

```

CARDS.INC

```

1 ; cards.inc -- drawing cards.
2 ;
3 ; A card is 12x17 pixels (3 bytes x 17 lines) whose top-left sits on a
4 ; char cell: col E (0-37), row D (0-12), pixel y = D*6. Hands lay cards
5 ; left to right on an 8-pixel (2-byte) pitch, so a covered card shows its
6 ; left 8 pixels: the black seam column at px0 (what separates overlapping
7 ; white cards), the rank at px4-7, and the suit's ink at px1-7. The last
8 ; card of a hand is fully visible; its right side is bare white with
9 ; rounded corners against the felt.
10 ;
11 ; The white body is painted by code; rank glyphs come from the 4x6 font
12 ; through the expander (red or black on white, matching the suit); the
13 ; ten and the suits are raw 2bpp stamps from assets/cardart.inc with the
14 ; seam baked in. Wire rank/suit characters are lowercase ASCII
15 ; ('as' = ace of spades, 't?' never occurs: ten is 't'; '??' = face down,
16 ; which DRWHND dispatches to DRWFDN).
17 ;
18 ; Card body row patterns (2bpp: felt 00, black 01, white 11).
19 CBTOP EQU 3FH ; .WWWWWWW (rounded corner row, byte 0)
20 CBTOPR EQU 0FCH ; WWWWWWWW.. (corner row, byte 2)
21 CBMID EQU 7FH ; KWWW: seam + white (byte 0)
22 CBWHT EQU 0FFH
23 ;
24 ;-----
25 ; DRWCRD: face-up card at char col E, row D; B = rank char, C = suit char.
26 ; Clobbers everything.
27 DRWCRD: PUSH BC
28 CALL CRDPOS ; HL = screen dst, and (CRDMAG) = magic dst
29 CALL CBODY
30 POP BC
31 ;
32 ; Suit lookup: C -> table walk (stamp address, colors). A 0 tag is the
33 ; catch-all row, so an unknown suit char cannot walk off the table.
34 ; A,C
35 LD HL,SUITTB
36 SUITL: LD E,(HL)
37 INC HL
38 CP E
39 JR Z,SUITF
40 INC E
41 DEC E
42 JR Z,SUITF ; tag 0: catch-all
43 INC HL
44 INC HL
45 INC HL
46 INC HL
47 JR SUITL
48 SUITF: LD E,(HL)
49 INC HL
50 LD D,(HL) ; DE = suit stamp
51 INC HL
52 LD C,(HL) ; C = XPAND pair for the rank
53 INC HL
54 LD A,(HL) ; A = ten stamp page flag (0 red, 1 black)
55 PUSH AF
56 PUSH BC
57 ;
58 ; Suit stamp: raw 12x6 at card rows 8-13.
59 PUSH DE
60 LD HL,(CRDDST)
61 LD DE,8*BYTEPL
62 ADD HL,DE
63 EX DE,HL
64 POP HL ; HL = stamp, DE = dst
65 LD B,6
66 LD C,3
67 CALL RAWBLT
68 ;
69 ; Rank: rows 1-6, Ten is a raw stamp; the rest are font glyphs at px4-7.
70 POP BC
71 POP AF
72 LD E,A
73 LD A,B
74 CP 't'
75 NZ,RANKG
76 LD HL,TENR
77 LD A,E
78 OR A
79 JR Z,RANK10
80 LD HL,TENB
81 RANK10: LD DE,(CRDDST)
82 LD A,E
83 ADD A,BYTEPL
84 LD E,A
85 NC,RANK11
86 INC D
87 RANK11: LD B,6
88 LD C,3
89 JP RAWBLT
90 ;
91 RANKG: LD A,C ; XPAND pair for this suit's color
92 OUT (XPAND),A
93 LD A,B ; rank char -> glyph index
94 CP 61H
95 JR C,RANKG1
96 SUB 20H
97 RANKG1: SUB 20H
98 PUSH AF
99 LD HL,(CRDMAG) ; glyph at byte 1, one line down
100 DE,BYTEPL+1
101 ADD HL,DE
102 PUSH HL
103 EXX
104 POP DE ; shadow DE = magic dst
105 EXX
106 POP AF
107 JP GLYPH
108 ;
109 ;-----
110 ; DRWFDN: face-down card at char col E, row D. Clobbers everything.
111 DRWFDN: CALL CRDPOS
112 EX DE,HL
113 LD HL,CARDBK
114 LD B,17
115 LD C,3
116 JP RAWBLT
117 ;
118 ; DRWCHP: bet chip (1 byte x 5 lines) at char col E, row D.
119 DRWCHP: CALL CRDPOS
120 EX DE,HL
121 LD HL,CHIP
122 LD B,5
123 LD C,1
124 JP RAWBLT
125 ;
126 ;-----
127 ; CRDPOS: char col E, row D -> HL = screen dst; stashes both address
128 ; forms for the stamp phases. Preserves DE.
129 CRDPOS: PUSH DE
130 LD A,D
131 ADD A,A
132 LD H,0

```

```

133 LD L,A
134 LD BC,ROWTAB
135 ADD HL,BC
136 LD A,(HL)
137 INC HL
138 LD H,(HL)
139 LD L,A
140 LD B,0
141 LD C,E
142 ADD HL,BC ; HL = magic-space offset
143 LD (CRDMAG),HL
144 LD BC,NORMEM
145 ADD HL,BC
146 LD (CRDDST),HL
147 POP DE
148 RET
149
150 ; CBODY: white card body with seam and rounded corners at HL. Clobbers
151 ; A,B,C,DE,HL.
152 CBODY: LD A,CBTOP
153 CALL CBROW3
154 LD B,15
155 CBMIDL: PUSH BC
156 LD A,CBMID
157 LD (HL),A
158 INC HL
159 LD A,CBWHT
160 LD (HL),A
161 INC HL
162 LD (HL),A
163 DEC HL
164 DEC HL
165 CALL CBNEXT
166 POP BC
167 DJNZ CBMIDL
168 LD A,CBTOP
169 ; falls through: bottom row = top row
170 CBROW3: LD (HL),A ; corner row: A, white, mirrored corner
171 INC HL
172 LD A,CBWHT
173 LD (HL),A
174 INC HL
175 LD A,CBTOPR
176 LD (HL),A
177 DEC HL
178 DEC HL
179 ; CBNEXT: HL down one screen line.
180 CBNEXT: LD A,L
181 ADD A,BYTEPL
182 LD L,A
183 RET NC
184 INC H
185 RET
186
187 -----
188 ; DRWHND: a hand of cards laid left to right on the 2-col (8px) pitch.
189 ; HL = wire card chars (2 per card), D = card row, E = start col,
190 ; B = max cards. Stops at NUL or space. "???" draws a full face-down card.
191 ; Serves both a seat's 2 hole cards and the 5-card community board.
192 ; Clobbers everything.
193
194 ; 5 Card Stud drew the FIRST "???" as a one-column sliver, because there it
195 ; was the hole card peeking out from under the four face-up ones. Hold'em
196 ; has no such card: an opponent in the hand shows "???" (two masked
197 ; cards) and a folded one "???", so every masked card here is a whole card.
198 DRWHND: LD A,(HL)
199 OR A
200 RET Z
201 CP ' '
202 RET Z
203 PUSH BC
204 PUSH HL
205 PUSH DE
206 CP '2'
207 JR NZ,DHFACE
208 CALL DRWFDN
209 POP DE
210 JR DHNEXT
211 DHFACE: LD B,A ; rank
212 INC HL
213 LD C,(HL) ; suit
214 CALL DRWCARD
215 POP DE
216 DHNEXT: INC E
217 INC E
218 POP HL
219 INC HL
220 INC HL
221 POP BC
222 DJNZ DRWHND
223 RET
224
225 ; suit char -> DW stamp, DB rank XPAND pair, DB ten flag (0 red, 1 black).
226 ; The last row is the catch-all (unknown suit draws as spades).
227 SUITTB: DB 'h'
228 DW SUITH
229 DB XRONW,0
230 DB 'd'
231 DW SUITD
232 DB XRONW,0
233 DB 'c'
234 DW SUITC
235 DB XKONW,1
236 DB 's'
237 DW SUITS
238 DB XKONW,1
239 DB 0 ; catch-all: unknown suits draw as spades
240 DW SUITS
241 DB XKONW,1
242

```

NAMENT.INC

```

1 ; nament.inc -- the name-entry screen.
2 ;
3 ; Eight slots, edited with the stick (or the keypad's arrow keys): up and
4 ; down turn the character wheel A-Z 0-9 then '-', left and right move the
5 ; cursor, trigger accepts. '-' is the blank -- the wheel has no space, so
6 ; a name can never need the '+' URL escape -- and everything from the
7 ; first '-' on is dropped at accept. An all-blank name keeps the default.
8
9 NAMESCR: CALL KEYRAW          ; no leaking presses from the OS menu
10 LD (V_KEY),A
11 CALL CLRSCR
12 SYSSUK STRDIS
13 DB 40          ; "TEXAS": 5 chars x 16px, centred on 160
14 DB 0
15 DB 4CH
16 DW TTEXAS
17 SYSSUK STRDIS
18 DB 32          ; "HOLDEM": 6 chars x 16px
19 DB 17
20 DB 4CH
21 DW THOLDM
22 LD HL,TNAME
23 LD DE,6*256+15
24 LD C,XWONF
25 CALL TXTAT
26 LD HL,TACCP
27 LD DE,12*256+7
28 LD C,XWONF
29 CALL TXTAT
30 LD HL,PLNBUF          ; edit buffer = name padded with blanks
31 LD DE,NAMEED
32 LD B,8
33 NEINI: LD A,(HL)
34 OR
35 JR Z,NEINI2
36 LD (DE),A
37 INC HL
38 INC DE
39 DJNZ NEINI
40 JR NEINI9
41 NEINI2: LD A,'-'
42 NEINI3: LD (DE),A
43 INC DE
44 DJNZ NEINI3
45 NEINI9: XOR A
46 LD (V_NCUR),A
47 CALL NEDRAW
48 NEKEY: CALL KEYSN
49 OR
50 JR Z,NEKEY
51 CP 'U'
52 JR NZ,NEK1
53 LD E,1          ; wheel forward
54 JR NEWHL
55 NEK1: CP 'D'
56 JR NZ,NEK2
57 LD E,36          ; wheel back (mod 37)
58 JR NEWHL
59 NEK2: CP 'L'
60 JR NZ,NEK3
61 LD A,(V_NCUR)
62 OR A
63 JR Z,NEKEY
64 DEC A
65 LD (V_NCUR),A
66 LD NEDRW
67 NEK3: CP 'R'
68 JR NZ,NEK4
69 LD A,(V_NCUR)
70 CP
71 JR NC,NEKEY
72 INC A
73 LD (V_NCUR),A
74 JR NEDRW
75 NEK4: CP 'I'
76 JR NZ,NEKEY
77 LD HL,NAMEED          ; accept: copy up to the first '-'
78 LD DE,PLNBUF
79 LD B,8
80 NEACC: LD A,(HL)
81 CP '-'
82 JR Z,NEACC9
83 LD (DE),A
84 INC HL
85 INC DE
86 DJNZ NEACC
87 NEACC9: LD A,E
88 CP PLNBUF-4E00H          ; empty name: keep what PLNBUF had
89 JR Z,NEKEY          ; (its low byte; the buffer is in 4Exx)
90 XOR A
91 LD (DE),A
92 JP SNDSSEL          ; confirm cue, then back to the caller
93
94 NEWHL: LD A,(V_NCUR)          ; step the wheel at the cursor by E
95 LD HL,NAMEED
96 PUSH DE
97 LD E,A
98 LD D,0
99 ADD HL,DE
100 POP DE
101 LD A,(HL)
102 PUSH HL
103 LD HL,WHEEL          ; find its wheel index
104 LD B,0
105 NEWH1: CP (HL)
106 JR Z,NEWH2
107 INC HL
108 INC B
109 JR NEWH1
110 NEWH2: LD A,B
111 ADD A,E          ; +1 or +36
112 NEWH3: CP 37
113 JR C,NEWH4
114 SUB 37
115 JR NEWH3
116 NEWH4: LD HL,WHEEL
117 LD E,A
118 LD D,0
119 ADD HL,DE
120 LD A,(HL)
121 POP HL
122 LD (HL),A
123 NEDRW: CALL SNDKEY
124 CALL NEDRAW
125 JP NEKEY
126
127 ; NEDRAW: the eight slots at row 8, cols 16-23, cursor cell inverse.
128 NEDRAW: XOR A
129 LD (V_TMP),A
130 NED1: LD A,(V_TMP)
131 CP 8
132 RET

```

```

133     LD     HL,NAMEED
134     LD     E,A
135     LD     D,0
136     ADD    HL,DE
137     LD     A,(HL)
138     LD     (LINBUF),A
139     XOR    A
140     LD     (LINBUF+1),A
141     LD     A,(V_TMP)
142     LD     HL,V_NCUR
143     CP     (HL)
144     LD     C,XFONW
145     JR     Z,NED2
146     LD     C,XWONF
147     LD     A,(V_TMP)
148     ADD    A,16
149     LD     E,A
150     LD     D,8
151     LD     HL,LINBUF
152     LD     B,1
153     CALL   TXTN
154     LD     A,(V_TMP)
155     INC    A
156     LD     (V_TMP),A
157     JR     NED1
158
159 WHEEL: DB     "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789--"
160 TNAME: DB     "YOUR NAME",0
161 TACCPT: DB    "STICK EDITS - TRIGGER ACCEPTS",0
162

```

SOUND.INC

```

1 ; sound.inc -- three cues, written straight to the music processor.
2 ;
3 ; Tone A only: set the master oscillator and tone A period, open the
4 ; volume, busy-wait, close it. Blocking is fine here -- every caller is
5 ; already in an input loop, and the longest cue is under a fifth of a
6 ; second.
7
8 TONEA EQU 11H ; tone A period (TONMO+1)
9
10 ; BEEP: A = tone A period (higher = lower pitch), B = duration, ~10 ms
11 ; units. Clobbers A,B,DE.
12 BEEP: PUSH BC
13     LD     C,A
14     LD     A,20H
15     OUT    (TONMO),A
16     LD     A,C
17     OUT    (TONEA),A
18     LD     A,08H
19     OUT    (VOLAB),A
20     POP    BC
21 BEEPW: LD     DE,600
22 BEEPD: DEC    DE
23     LD     A,D
24     OR     E
25     JR     NZ,BEEPD
26     DJNZ   BEEPW
27     XOR    A
28     OUT    (VOLAB),A
29     RET
30
31 SNDKEY: LD     A,18H ; key click
32     LD     B,2
33     JP     BEEP
34
35 SNDSEL: LD     A,28H ; select / join / move sent

```

```

36     LD     B,5
37     JP     BEEP
38
39 SNDTRN: LD     A,30H ; your turn: low then high
40     LD     B,6
41     CALL   BEEP
42     LD     A,18H
43     LD     B,8
44     JP     BEEP
45

```

DEMO.INC

```

1 ; demo.inc -- static mock table, assembled when DEMO=1.
2 ;
3 ; Exercises every drawing path with zero network: the layout, populated
4 ; with a full 8-player Hold'em table. This is where the palette, the card
5 ; art and the cell geometry get checked by screenshot, so the coordinates
6 ; here are the same ones screens.inc's tables carry -- if you move a seat
7 ; there, move it here.
8 ;
9 ; Rows: 0 table name | 1 top names | 2 top hands + their bets | 5 mid
10 ; names/bets + the street | 6 mid hands + the community board | 9 bottom
11 ; hands | 12 bottom names/bets | 13 purse, bet, pot, clock | 14 the bar.
12
13 DEMOSCR:
14     CALL    CLRSR
15 ; Row 0: the table name.
16     LD      HL,DTABLE
17     LD      DE,0*256+0
18     LD      C,XWONF
19     CALL    TXTAT
20
21 ; Top seats (3, 4, 5): names row 1, hands and bets row 2. Seat 4 is the
22 ; active player: inverse name.
23     LD      HL,DNAME3
24     LD      DE,1*256+0
25     LD      C,XWONF
26     CALL    TXTAT
27     LD      HL,DNAME4
28     LD      DE,1*256+15
29     LD      C,XWONF
30     CALL    TXTAT
31     LD      HL,DNAME5
32     LD      DE,1*256+29
33     LD      C,XWONF
34     CALL    TXTAT
35
36     LD      HL,DHAND3
37     LD      DE,2*256+0
38     LD      B,2
39     CALL    DRWHND
40     LD      HL,DHAND4
41     LD      DE,2*256+17
42     LD      B,2
43     CALL    DRWHND
44     LD      HL,DHAND5
45     LD      DE,2*256+33
46     LD      B,2
47     CALL    DRWHND
48
49     LD      DE,2*256+6
50     LD      HL,25
51     CALL    DBET
52     LD      DE,2*256+12
53     LD      HL,150
54     CALL    DBET
55     LD      DE,2*256+28
56     LD      HL,50
57     CALL    DBET
58
59 ; Middle seats (2 left, 6 right): names row 5, hands row 6. The street
60 ; label and the community board own the middle of both rows.
61     LD      HL,DNAME2
62     LD      DE,5*256+0
63     LD      C,XWONF
64     CALL    TXTAT
65     LD      HL,DNAME6
66     LD      DE,5*256+31
67     LD      C,XWONF
68     CALL    TXTAT
69     LD      HL,25
70     LD      C,XWONF
71     LD      DE,5*256+24
72     LD      HL,1250
73     LD      C,XWONF
74     LD      B,8
75     CALL    DBET
76     LD      HL,DSTREET
77     LD      DE,5*256+16
78     LD      C,XWONF
79     LD      B,8
80     CALL    DBET
81
82     LD      HL,DHAND2
83     LD      DE,6*256+0
84     LD      B,2
85     CALL    DRWHND
86     LD      HL,DHAND6
87     LD      DE,6*256+35
88     LD      B,2
89     CALL    DRWHND
90
91     LD      HL,DBOARD
92     LD      DE,6*256+14
93     LD      B,5
94     CALL    DRWHND
95
96 ; Bottom seats (1 left, 0 you center, 7 right): hands row 9, names row 12.
97 ; Your own hand is the only one face up; seat 1 has folded, so it shows a
98 ; single card back where the others show two.
99     LD      HL,DHAND1
100    LD      DE,9*256+0
101    LD      B,2
102    CALL    DRWHND
103    LD      HL,DHAND0
104    LD      DE,9*256+17
105    LD      B,2
106    CALL    DRWHND
107    LD      HL,DHAND7
108    LD      DE,9*256+33
109    LD      B,2
110    CALL    DRWHND
111    LD      HL,DNAME1
112    LD      DE,12*256+0
113    LD      C,XWONF
114    CALL    TXTAT
115    LD      HL,DNAME0
116    LD      DE,12*256+14
117    LD      C,XWONF
118    CALL    TXTAT
119    LD      HL,DNAME7
120    LD      DE,12*256+29
121    LD      C,XWONF
122    CALL    TXTAT
123    LD      DE,12*256+9
124    LD      HL,25
125    CALL    DBET
126    LD      DE,12*256+24
127    LD      HL,50
128    CALL    DBET
129
130 ; Row 13: your
131     LD      HL,DPURSE
132     LD      DE,13*256+0

```

```

133 LD C,XWONF
134 CALL TXTAT
135 LD HL,340
136 CALL U16STR
137 LD HL,LINBUF
138 LD DE,13*256+6
139 LD C,XWONF
140 LD B,5
141 CALL TXTN
142 LD HL, DBETL
143 LD DE,13*256+12
144 LD C,XWONF
145 CALL TXTAT
146 LD HL,25
147 CALL U16STR
148 LD HL,LINBUF
149 LD DE,13*256+16
150 LD C,XWONF
151 LD B,5
152 CALL TXTN
153 LD HL, DPOT
154 LD DE,13*256+22
155 LD C,XWONF
156 CALL TXTN
157 LD HL,1250
158 CALL U16STR
159 LD HL,LINBUF
160 LD DE,13*256+26
161 LD C,XWONF
162 LD B,5
163 CALL TXTN
164 LD HL, DCLOCK
165 LD DE,13*256+37
166 LD C,XWONF
167 CALL TXTN
168
169 ; Row 14: the status bar -- a white strip with the move menu in black.
170 ; DMENU is the worst case the real BAR can produce: five moves at the
171 ; 8-column stride, names already squeezed by MVCOMP, filling cols 0-39
172 ; exactly. If this fits, every shorter menu does.
173 LD HL,NORMEM+14*240
174 LD B,6
175 LD C,BYTEPL
176 LD D,0FFH
177 CALL FILLRC
178 LD HL,DMENU
179 LD DE,14*256+0
180 LD C,XKONW
181 CALL TXTN
182 RET
183
184 ; DBET: a chip at D,E with the amount in HL drawn just right of it.
185 DBET: PUSH HL
186 PUSH DE
187 CALL DRWCHP
188 POP DE
189 POP HL
190 INC E
191 PUSH DE
192 CALL U16STR
193 POP DE
194 LD HL,LINBUF
195 LD C,XWONF
196 LD B,4
197 JP TXTN
198
199 DTABLE: DB "AI ROOM - 7 BOTS",0
200 DPOT: DB "POT",0
201 DSTREET: DB "RIVER",0
202 DPURSE: DB "PURSE",0
203 DBETL: DB "BET",0

```

```

204 DCLOCK: DB "12",0
205 DMENU: DB "1 FOLD 2 CHECK 3 B10 4 B20 5 A500 ",0
206 DNAME0: DB "YOU",0
207 DNAME1: DB "MOZZWALD",0
208 DNAME2: DB "ERIC3",0
209 DNAME3: DB "THOM",0
210 DNAME4: DB "JEFFP",0
211 DNAME5: DB "ROGER",0
212 DNAME6: DB "NORM",0
213 DNAME7: DB "TSCHAK",0
214
215 DBOARD: DB "7dkcac9s2h",0 ; the full five-card board
216 DHAND0: DB "askh",0 ; yours, face up
217 DHAND1: DB "???",0 ; folded: one card back
218 DHAND2: DB "????",0
219 DHAND3: DB "?????",0
220 DHAND4: DB "?????",0
221 DHAND5: DB "?????",0
222 DHAND6: DB "?????",0
223 DHAND7: DB "tdjc",0 ; a showdown reveal
224

```

CARDART.INC -- CARD PIP ART (GENERATED)

```

1 ; cardart.inc -- raw 2bpp card stamps, GENERATED by tools/mkcards.py
2 ; -- do not edit; rerun 'make assets'. Pixel values are palette
3 ; registers (0 felt, 1 black, 2 red, 3 white); leftmost pixel is
4 ; bits 7-6. Proof art: K black, r red, space white, dot felt.
5 ; SUITH: 12x6
6 SUITH:
7     DB      07FH,0BBH,0FFH      ; K   r   r
8     DB      07EH,0AAH,0FFH      ; K   rrrrr
9     DB      07EH,0AAH,0FFH      ; K   rrrrr
10    DB      07FH,0ABH,0FFH      ; K   rrr
11    DB      07FH,0EFH,0FFH      ; K   r
12    DB      07FH,0FFH,0FFH      ; K
13 ; SUITD: 12x6
14 SUITD:
15    DB      07FH,0EFH,0FFH      ; K   r
16    DB      07FH,0ABH,0FFH      ; K   rrr
17    DB      07EH,0AAH,0FFH      ; K   rrrrr
18    DB      07FH,0ABH,0FFH      ; K   rrr
19    DB      07FH,0EFH,0FFH      ; K   r
20    DB      07FH,0FFH,0FFH      ; K
21 ; SUITS: 12x6
22 SUITS:
23    DB      07FH,0DFH,0FFH      ; K   K
24    DB      07FH,057H,0FFH      ; K   KKK
25    DB      07DH,055H,0FFH      ; K   KKKKK
26    DB      07DH,055H,0FFH      ; K   KKKKK
27    DB      07FH,0DFH,0FFH      ; K   K
28    DB      07FH,057H,0FFH      ; K   KKK
29 ; SUITC: 12x6
30 SUITC:
31    DB      07FH,0DFH,0FFH      ; K   K
32    DB      07FH,057H,0FFH      ; K   KKK
33    DB      07DH,075H,0FFH      ; K   KK KK
34    DB      07DH,055H,0FFH      ; K   KKKKK
35    DB      07FH,0DFH,0FFH      ; K   K
36    DB      07FH,057H,0FFH      ; K   KKK
37 ; TENR: 12x6
38 TENR:
39    DB      07FH,0FFH,0FFH      ; K
40    DB      07BH,0FBH,0FFH      ; K   r   r
41    DB      06BH,0EEH,0FFH      ; Krr   r r
42    DB      07BH,0EEH,0FFH      ; K   r   r r
43    DB      07BH,0EEH,0FFH      ; K   r   r r
44    DB      06AH,0FBH,0FFH      ; Krrr   r
45 ; TENB: 12x6
46 TENB:
47    DB      07FH,0FFH,0FFH      ; K
48    DB      077H,0F7H,0FFH      ; K K K
49    DB      057H,0DDH,0FFH      ; KKK K K
50    DB      077H,0DDH,0FFH      ; K K K K
51    DB      077H,0DDH,0FFH      ; K K K K
52    DB      055H,0F7H,0FFH      ; KKKK K
53 ; CARDBK: 12x17
54 CARDBK:
55    DB      03FH,0FFH,0FFH      ; .
56    DB      07BH,0BBH,0BBH      ; K   r r r r r r
57    DB      07EH,0EEH,0EFH      ; K   r r r r r
58    DB      07BH,0BBH,0BBH      ; K   r r r r r
59    DB      07EH,0EEH,0EFH      ; K   r r r r
60    DB      07BH,0BBH,0BBH      ; K   r r r r r
61    DB      07EH,0EEH,0EFH      ; K   r r r r r
62    DB      07BH,0BBH,0BBH      ; K   r r r r r
63    DB      07EH,0EEH,0EFH      ; K   r r r r
64    DB      07BH,0BBH,0BBH      ; K   r r r r r
65    DB      07EH,0EEH,0EFH      ; K   r r r r r
66    DB      07BH,0BBH,0BBH      ; K   r r r r r

```

```

67    DB      07EH,0EEH,0EFH      ; K   r r r r r
68    DB      07BH,0BBH,0BBH      ; K   r r r r r
69    DB      07EH,0EEH,0EFH      ; K   r r r r r
70    DB      07BH,0BBH,0BBH      ; K   r r r r r
71    DB      03FH,0FFH,0FFH      ; .
72 ; CHIP: 4x5
73 CHIP:
74    DB      028H                  ; .rr.
75    DB      0AAH                  ; rrrr
76    DB      0BEH                  ; r   r
77    DB      0AAH                  ; rrrr
78    DB      028H                  ; .rr.
79

```


C.5 BATTLESHIP

BATTLE.ASM

```
1 ; battle.asm -- Battleship for the Bally Astrocade, over FujiNet.
2
3 ; A standalone-assembly client in the texasHoldEm/astrocade mold: same
4 ; mailbox cartridge, same 4x6 renderer, same one-poll-loop shape. The
5 ; gameplay logic is transcribed from the Intellivision port (intv/), the
6 ; freshest client written directly against the binary wire format; the
7 ; screen plan is the Intellivision's four-quadrant board compressed onto
8 ; this machine's byte-aligned 2bpp bitmap.
9
10 ; The board geometry is the whole trick: a grid cell is 4x4 pixels = one
11 ; byte per row, so a 10x10 board is a 10-byte-wide, 40-line square and
12 ; nothing ever shifts. Two boards fit side by side with a divider byte,
13 ; two rows of boards fit in 84 lines, and the right 19 byte columns hold
14 ; the panel. You are always quadrant 0 (bottom-left) -- the server
15 ; rotates its player list so this client is index 0.
16
17 ; ROM budget: 0000H-1AFFH of the 8K window (6,912 bytes); 1B00H+ belongs
18 ; to the mailbox and build.sh stamps the "FUJII" claim at 1CFCH so the
19 ; mailbox stays alive when this image is booted over the network. RAM is
20 ; screen RAM alone; 90 visible lines use 4000H-4E0FH and everything above
21 ; is ours.
22
23 ; Interrupts stay off for the program's whole life (fujilib.inc's
24 ; contract; with I = 0, refresh strays land in OS ROM and never hit the
25 ; hotspots).
26
27         INCLUDE "HVGLIB.H"
28         INCLUDE "fujinet.inc"
29
30 ; --- RAM map
31 LINES   EQU    90          ; 15 rows of 4x6 text
32
33 ; What the last poll established. Rendering is opaque, so these are not a
34 ; dirty list but the things overdraw cannot fix:
35 ; content that SHRANK (a class or seat-count change clears the screen)
36 ; and edges (your turn arriving, a shot landing, the game ending).
37 PRVCLS EQU    4E10H        ; previous screen class (FF forces statics)
38 V_NEWS EQU    4E11H        ; the statics were redrawn this poll
39 PRVACT EQU    4E12H        ; previous activePlayer (the turn edge)
40 PRVPC EQU    4E13H        ; previous playerCount
41 PRVRST EQU    4E14H        ; previous result status } the pair that
42 PRVRPS EQU    4E15H        ; previous lastAttackPos } gates sounds
43 PENDACT EQU    4E16H        ; staged action: 0 none, 1 ready, 2 place,
44 ; 3 attack -- rides the next poll
45 ; staged /attack position
46 ATKPOS EQU    4E17H        ; 5 staged /place bytes, pos + 100*dir
47 MYSHP EQU    4E18H        ; targeting cursor; persists across turns
48 TGX EQU    4E1DH          ;
49 TGY EQU    4E1EH          ;
50 TGLEFT EQU    4E1FH        ; local countdown, reseeded each poll
51 TGRFM EQU    4E20H        ; ~20 ms ticks toward the next second
52 RNDST EQU    4E21H        ; LFSR state, 2 bytes
53 V_ST EQU    4E23H        ; scratch: a player's status byte
54 V_QD EQU    4E24H        ; quadrant being drawn
55 V_CX EQU    4E25H        ; cell being drawn
56 V_CY EQU    4E26H        ;
57 PCX EQU    4E27H        ; placement cursor
58 PCY EQU    4E28H        ;
59 PCDIR EQU    4E29H        ; 0 horizontal, 1 vertical
60 PCSIZ EQU    4E2AH        ;
61 PCNUM EQU    4E2BH        ; ship being placed, 0-4
62 V_FLAG EQU    4E2CH        ; targeting outcome: 0 slice out, 1 fired,
63 ; 2 leave, 3 help
64
65 V_BLNK EQU    4E2DH        ; cursor blink phase
66 V_CLS EQU    4E2EH        ; this poll's class (0 lobby, 1 place,
67 ; 2 game, 3 over)
68 V_TRNF EQU    4E2FH        ; turn-changed edge, this poll
69
70 ; Session state.
71 PLNBUF EQU    4E30H        ; player name, 9 + NUL (must stay in 4Exx:
72 ; nament tests its low byte)
73 TBLBUF EQU    4E3AH        ; table id, 9 + NUL
74 CURSLC EQU    4E46H        ; reply slice the cart is publishing
75 AVAIL EQU    4E48H        ; NET_STATUS bytes waiting, u16
76 PRVAVL EQU    4E4AH        ; previous reading, for the settle loop
77 RXLEN EQU    4E4CH        ; reply length, captured after the READ
78 V_SEL EQU    4E4EH        ; list cursor (tables)
79 V_CNT EQU    4E4FH        ; list entry count
80 V_KEY EQU    4E50H        ; last keypad/handle scan state (edges)
81 V_URL EQU    4E58H        ; request being built (0 tables, 1 state,
82 ; 2 ready, 3 place, 4 attack, 5 leave)
83 V_TMP EQU    4E59H        ; loop index (player being rendered)
84 V_ACT EQU    4E5AH        ; this poll's activePlayer
85 V_PC EQU    4E5BH        ; this poll's playerCount
86 V_NCUR EQU    4E5CH        ; name-entry cursor
87 V_BASE EQU    4E5EH        ; player record base offset (word)
88 TBLNAM EQU    4E60H        ; joined table's display name, 21 + NUL
89 NAMEED EQU    4E76H        ; name-entry edit buffer, 8 slots
90 OCCUPY EQU    4E80H        ; placement occupancy map, 100 bytes
91
92 LINBUF EQU    4F00H        ; display line being built; page-aligned
93 ; (U16STR's empty test reads the low byte)
94
95 HEXBUF EQU    4F30H        ;
96 STACK EQU    4FC0H        ; grows down; 4FC0H+ left to the BIOS cells
97
98 ; --- Display options (BIOS STRDIS, titles only) -----
99 OPTFB EQU    0CH          ; fg color 3, bg color 0
100
101 ; MB_* labels are module fences for tools/checksize.py's budget table.
102 ORG FIRSTC
103
104 MB_MAIN:
105     DB 55H
106     DW MENUST
107     DW PRGNAM
108     DW PRGSTR
109     PRGNAM: DB "BATTLESHIP"
110     DB 0
111     PRGSTR: DI SP,STACK
112     DI SYSTEM
113     DI INTPC
114     DI SETOUT
115     DI LINES*2
116     DB 8
117     DB 0
118     DB COLSET
119     DB PALET
120     DB FILL
121     DB NORMEM
122     DB LINES*BYTEPL
123     DB 0
124     DB STRDIS
125     DB 38
126     DB OPTFB
127
128 ; HORCB 0: the whole line is the right palette
```

```

125 DW      TTITLE
126 EXIT
127
128 CALL    FNCHECK
129 JP      NZ, NOCARD
130
131 LD      HL, DEFNAM      ; player name until appkeys land
132 LD      DE, PLNBUF
133 LD      BC, 6
134 LDIR
135 XOR      A
136 LD      (PENDACT), A
137 LD      (V_KEY), A
138 LD      A, 4            ; targeting cursor starts mid-board
139 LD      (TGX), A
140 LD      (TGY), A
141 LD      HL, 1           ; LFSR seed; RNDNXT mixes R in per call
142 LD      (RNDST), HL
143 CALL    NAMESCR        ; pick a name, then choose a table
144 JP
145
146 ; ---- Errors -----
147 NOCARD: SYSSUK STRDIS
148 DB      20
149 DB      56
150 DB      OPTFB
151 DW      ENOCART
152 HALTE: JR      HALTE
153
154 ; ---- Data -----
155 MB_DATA:
156 TTITLE: DB      "BATTLESHIP", 0
157 ENET:   DB      "NET ERR - ANY KEY RETRIES", 0
158 ENOCART: DB      "NO FUJINET CART", 0
159 DEFNAM: DB      "ASTRO", 0
160
161 ; COLSET stores descending, ports 7 down to 0; both halves identical since
162 ; HORCB is 0. Right palette: 3 = white, 2 = red, 1 = black, 0 = sea blue.
163 ; Byte = (hue << 3) | luminance. Blue lives at the top of the hue circle
164 ; (NAME's astro: palette: by = 1.15*cos(hue angle)); 0F2H = hue 30
165 ; luma 2 = a deep sea blue. Checked against the palette math, not a
166 ; guess -- 7AH (hue 15) reads "blue" in old notes but renders olive.
167 PALET: DB      07H, 52H, 00H, 0F2H
168 DB      07H, 52H, 00H, 0F2H
169
170 INCLUDE "build/endpoint.inc"
171
172 MB_SCREENS:
173 INCLUDE "screens.inc"
174 MB_PLACE:
175 INCLUDE "place.inc"
176 MB_TARGET:
177 INCLUDE "target.inc"
178 MB_BOARD:
179 INCLUDE "board.inc"
180 MB_NAMENT:
181 INCLUDE "nament.inc"
182 MB_SOUND:
183 INCLUDE "sound.inc"
184 MB_INPUT:
185 INCLUDE "input.inc"
186 MB_NET:
187 INCLUDE "net.inc"
188 MB_URL:
189 INCLUDE "url.inc"
190 MB_STATE:
191 INCLUDE "state.inc"
192 MB_GFX:
193 INCLUDE "gfx.inc"
194 MB_RND:
195 INCLUDE "rnd.inc"

```

```

196 MB_FONT:
197 INCLUDE "assets/font.inc"
198 MB_FUJILIB:
199 INCLUDE "fujilib.inc"
200
201 MB_END:

```

FUJINET.INC -- THE V2 WIRE FORMAT

```

1 ; fujinet.inc -- N: device commands and the Battleship wire format.
2
3 ; The N: command values are hand-synchronised with the texasHoldEm port
4 ; (whose o2/ family carries the documentation of WHY which is what it is).
5 ; The wire offsets are verified against the server's serializeResults()
6 ; (servers/fujinet-game-system/battleship/util.go) and the Intellivision
7 ; client's byte-exact map (intv/state.bas). The mailbox transport itself
8 ; is fujilib.inc.
9
10 ; --- Network device (N:) ---
11 NETDEV EQU 71H ; N1:
12 NCOPEX EQU 4FH ; 'O'
13 NCCLOSE EQU 43H ; 'C'
14 NCREAD EQU 52H ; 'R'
15 NCSTAT EQU 53H ; 'S'
16
17 NMHTTPG EQU 0CH ; open mode: HTTP GET
18 NTRNONE EQU 0 ; translation: none
19
20 ; NET_STATUS reply: 4 bytes in slice 0.
21 NSAVLO EQU 0 ; bytes waiting, low
22 NSAVHI EQU 1 ; bytes waiting, high
23 NSCONN EQU 2 ; connected flag
24 NSDEVST EQU 3 ; nDevStatus, 1 = SUCCESS. Checking this is
25 ; not optional: the GET is deferred until the
26 ; first STATUS, and an HTTP error page still
27 ; has a readable body.
28
29 ; --- Wire format (?bin=1&v=2) ---
30 ; /state, /ready, /place/., /attack/., /leave -> the same state blob;
31 ; /tables -> Tables. Offsets are read in place out of the reply slice
32 ; window; nothing is buffered. All strings are lowercased, NUL-padded,
33 ; one byte longer than their content width.
34
35 ; v=2 is mandatory: v1 hides the winner (activePlayer forced to -1 at
36 ; gameover) and sends only 5 myShips bytes instead of 10.
37
38 ; Header, always present (38 bytes):
39 GOPC EQU 0 ; playerCount
40 GOPRMP EQU 1 ; prompt[33]
41 GOSTAT EQU 34 ; game status, STxxxx below
42 GOPSTAT EQU 35 ; YOUR status, PSxxxx below
43 GOACTPL EQU 36 ; signed; FFH = -1 nobody, 0 = your turn
44 ; (the server rotates you 12 index 0);
45 ; at gameover (v=2) the winner's index
46 GOMVTIM EQU 37 ; seconds left, 0-250
47
48 ; status = 0 (lobby): the header is followed by
49 GOSRVNM EQU 38 ; serverName[21]
50 GOLPLRS EQU 59 ; playerCount x 10-byte records
51 LPSTRID EQU 10 ; name[9]
52 LPNAME EQU 0 ; the player's PSxxxx status (3 = ready)
53 LPSTAT EQU 9
54
55 ; status <> 0: the header is followed by
56 GOLATK EQU 38 ; lastAttackPos, 0-99
57 GOMYSH EQU 39 ; myShips[10]: [0-4] yours, [5-9] the
58 ; winner's (gameover only); each byte is
59 ; pos * 100 + dir (dir 0 horiz, 1 vert)
60 GOMPLRS EQU 49 ; playerCount x 115-byte records ONCE THE
61 ; GAMEFIELDS EXIST (status >= 10). During
62 ; PLACE_SHIPS the records are 10 bytes
63 ; (name + status only) -- never index
64 ; player records in the place phase.
65 PLSTRID EQU 115 ; name[9]
66 PLNAME EQU 0

```

```

67 PLSTAT EQU 9 ; the player's PSxxxx status
68 PLFIELD EQU 10 ; gamefield[100], y*10+x: 0 unknown,
69 ; 1 hit, 2 miss
70 PLSHFT EQU 110 ; shipsLeft[5]: 1 afloat, 0 sunk, per
71 ; SHPSIZ index
72
73 ; /tables (?bin=1) -- byte-identical to the card games' Tables struct.
74 TBSTRID EQU 36 ; table[9]
75 TBID EQU 0 ; name[21]
76 TBNAME EQU 9 ; players[6], literal "cur / max"
77 TBPLRS EQU 30
78
79 ; Game status values.
80 STLOBBY EQU 0
81 STPLACE EQU 1
82 STSTART EQU 10
83 STMISS EQU 11
84 STHIT EQU 12
85 STSUNK EQU 13
86 STOVER EQU 99
87
88 ; Player status values.
89 PSPLAY EQU 0
90 PSDEAD EQU 1
91 PSVIEW EQU 2
92 PSREADY EQU 3
93 PSPLACE EQU 10
94
95 GMAXLEN EQU 509 ; 49 + 4*115; the NREAD clamp, and it fits
96 ; the cart's four 256-byte reply slices
97 GMINLEN EQU 38 ; the fixed header
98 TMAXLEN EQU 361
99

```

NET.INC

```

1 ; net.inc -- the N: device round trip.
2 ;
3 ; A Z80 transcription of the card-game ports' proven sequence, over
4 ; Fujilib's mat'low. The rules it carries forward (learned the hard way
5 ; there and on the Intellivision):
6 ;
7 ; * The CLOSE belongs at the START of the next request, never after the
8 ; READ: every transaction repaints the whole reply window, a CLOSE
9 ; reply is empty, and every screen renders straight out of the window
10 ; between polls.
11 ; * FNRLX/FNRXLH belong to the most recent transaction, so the reply
12 ; length is captured immediately after the READ.
13 ; * The ESP32 reports STATUS as soon as SOME of the response has
14 ; arrived; poll until two consecutive readings agree (bounded) rather
15 ; than guessing a delay.
16 ; * The 4th STATUS reply byte is nDevStatus.t and checking it is not
17 ; optional: the HTTP GET is deferred until the first STATUS, and an
18 ; HTTP error page still has a readable body.
19 ; * Validate the reply length against what playerCount and the status
20 ; imply before believing a byte of it: FN_RX is never cleared, so a
21 ; short read leaves stale bytes from a bigger previous reply.
22 ;
23 ; All routines return carry SET on failure, clear on success.
24 ;
25 ; NCMD: begin a network-device transaction. E = command, L = nparam.
26 NCMD: LD A,NETDEV
27 JP FNBEGIN
28 ;
29 ; NCLOSE: close the previous request's connection. A first call has
30 ; nothing to close and earns a NAK; both results are ignored.
31 NCLOSE: LD E,NCLOSE
32 LD L,0
33 CALL NCMD
34 LD B,3 ; -6 s
35 JP FNCOMMIT
36 ;
37 ; NOPEN: OPEN the devspec (V_URL) as HTTP GET, no translation.
38 NOPEN: LD E,NCOPEN
39 LD L,2
40 CALL NCMD
41 LD A,NMHTPG
42 CALL FNPARB
43 LD A,NTRNONE
44 CALL FNPARB
45 CALL BLDURL
46 LD B,8 ; HTTPS handshakes take a while
47 CALL FNCOMMIT
48 RET C
49 JR NZ,NOBAD
50 LD A,(FNREPLY)
51 CP FCACK
52 RET Z ; equal: carry already clear
53 NOBAD: SCF
54 RET
55 ;
56 ; NSTATUS: one STATUS poll. On success AVAIL = bytes waiting.
57 NSTATUS: LD E,NCSTAT
58 LD L,2
59 CALL NCMD
60 XOR A
61 CALL FNPARB
62 XOR A
63 CALL FNPARB
64 LD B,4
65 CALL FNCOMMIT
66 RET C
67 JR NZ,NSBAD
68 LD A,(FNREPLY)
69 CP FCACK
70 JR NZ,NSBAD
71 LD A,(FNRRDATA+NSDEVST)
72 CP 1 ; SUCCESS -- anything else is an HTTP error
73 JR NZ,NSBAD
74 LD A,(FNRRDATA+NSAVLO)
75 LD (AVAIL),A
76 LD A,(FNRRDATA+NSAVHI)
77 LD (AVAIL+1),A
78 OR A ; clear carry
79 RET
80 NSBAD: SCF
81 RET
82 ;
83 ; NREAD: READ min(AVAIL, GMAXLEN) bytes into the reply window, then
84 ; capture the reply length and re-home the slice cursor.
85 NREAD: LD HL,(AVAIL)
86 LD DE,GMAXLEN
87 OR A
88 SBC HL,DE
89 JR C,NRD1 ; avail < clamp: take avail
90 LD H,D
91 LD L,E
92 JR NRD2
93 NRD1: ADD HL,DE
94 NRD2: PUSH HL
95 LD E,NCREAD
96 LD L,1
97 CALL NCMD
98 LD A,2 ; one 2-byte parameter: the length, LE
99 CALL FNTXBYT
100 POP HL
101 PUSH HL
102 LD A,L
103 CALL FNTXBYT
104 POP HL
105 LD A,H
106 CALL FNTXBYT
107 LD B,8
108 CALL FNCOMMIT
109 RET C
110 JR NZ,NRBAD
111 LD A,(FNREPLY)
112 CP FCACK
113 JR NZ,NRBAD
114 LD A,(FNRLX) ; capture NOW: the next transaction's
115 LD (RXLEN),A ; reply length would replace it
116 LD A,(FNRLX)
117 LD (RXLEN+1),A
118 XOR A
119 LD (CURSLC),A ; the cart republished slice 0
120 RET ; carry clear (XOR A)
121 NRBAD: SCF
122 RET
123 ;
124 ; -----
125 ; APICALL: A = request (0 tables, 1 state, 2 ready, 3 place, 4 attack,
126 ; 5 leave). One full round trip; on success (carry clear) the reply is in
127 ; the window with the slice cursor homed, and stays valid until the next
128 ; APICALL.
129 APICALL: LD (V_URL),A
130 NCLOSE
131 CALL NOPEN ; the PREVIOUS request's connection
132 RET C

```

```

133 XOR A
134 LD (PRVAVL),A
135 LD (PRVAVL+1),A
136 LD B,20 ; settle-loop bound
137 APSET: PUSH BC
138 CALL NSTATUS
139 POP BC
140 JR C,APCFL
141 LD HL,(AVAIL)
142 LD A,H
143 OR L
144 JR Z,APAGN ; nothing has arrived at all yet
145 LD DE,(PRVAVL)
146 OR A
147 SBC HL,DE
148 JR Z,APRD ; two readings agree: it has landed
149 APAGN: LD HL,(AVAIL)
150 LD (PRVAVL),HL
151 LD DE,4000 ; ~3 frames between polls
152 APDLY: DEC DE
153 LD A,D
154 OR E
155 JR NZ,APDLY
156 DJNZ APSET
157 APCFL: CALL NCLOSE ; failed before the READ: nothing to keep
158 SCF
159 RET
160 APRD: CALL NREAD
161 JR C,APCFL
162 ; Validation failure after the READ returns carry WITHOUT a close -- the
163 ; next APICALL's leading NCLOSE retires the connection, and closing here
164 ; would wipe the window a caller may still be rendering the last good
165 ; reply from.
166 ; falls through into VALID8
167
168 ; VALID8: sanity-check the reply for the request in V_URL, transcribed
169 ; from the Intellivision port's validate_state. Tables: nonempty. State:
170 ; at least the fixed header, playerCount <= 4, a status byte on the enum,
171 ; and at least the length that status and playerCount imply --
172 ; lobby: 59 + 10*pc place: 49 (records are name+status only)
173 ; game/over: 49 + 11*pc
174 ; Minimum, not exact: the serializer's per-player append makes the place
175 ; phase variable and a longer reply is harmless; a SHORTER one means the
176 ; fixed offsets would read stale bytes.
177 VALID8: LD A,(V_URL)
178 OR A
179 JR NZ,VALGM
180 LD HL,(RXLEN) ; tables: any nonzero length; the count
181 LD A,H ; byte bounds the walk
182 OR L
183 RET NZ ; OR cleared carry
184 SCF
185 RET
186 VALGM: LD HL,(RXLEN)
187 LD DE,GMINLEN
188 OR A
189 SBC HL,DE
190 RET C ; shorter than the fixed header
191 LD HL,GOPC
192 CALL RXGETB
193 CP 5
194 JR NC,VALBAD ; more seats than the game has
195 LD C,A ; C = playerCount
196 LD HL,GOSTAT
197 CALL RXGETB
198 LD HL,STATTAB ; status must be on the enum: everything
199 LD B,7 ; downstream dispatches on it
200 VALS1: CP (HL)
201 JR Z,VALS2
202 HL TNC
203 DJNZ VALS1

```

```

204 JR VALBAD
205 VALS2: OR A ; A = status
206 JR NZ,VALS3
207 LD B,C ; lobby: 59 + 10*pc
208 HL,59
209 LD DE,LPSTRID
210 JR VALS5
211 VALS3: CP STPLACE
212 JR NZ,VALS4
213 LD HL,49 ; place: header + lastAttack + myShips
214 JR VALS6
215 VALS4: LD B,C ; game/over: 49 + 11*pc
216 LD HL,49
217 LD DE,PLSTRID
218 VALS5: LD A,B
219 OR A
220 JR Z,VALS6
221 VALSA: ADD HL,DE
222 DJNZ VALSA
223 VALS6: EX DE,HL ; DE = expected minimum
224 LD HL,(RXLEN)
225 OR A
226 SBC HL,DE ; carry set exactly when RXLEN < expected
227 RET
228 VALBAD: SCF
229 RET
230
231 STATTAB: DB STLOBBY,STPLACE,STSTART,STMISS,STHIT,STSUNK,STOVER
232

```

URL.INC

```

1 ; url.inc -- building the devicespec for a request.
2
3 ; The URL is streamed straight into the mailbox's TX byte stream a
4 ; character at a time (the card-game ports' way); it is never assembled
5 ; in RAM. The cart counts the payload itself, so nothing here tracks a
6 ; length; the worst case (/place with five 3-digit positions) is ~95
7 ; bytes against the 320-byte TX ceiling.
8
9 ; The staged move is raw bytes, not text -- MYSHP[5] and ATKPOS -- and
10 ; BYTDEC turns each into decimal digits on the way out, so /place needs
11 ; no 20-character move buffer.
12
13 ; TBLBUF and PLNBUF are NUL-terminated copies of fixed-width wire fields
14 ; -- pushed up to the NUL, never through it (padding in the middle of a
15 ; URL corrupts the request on the wire).
16
17 ; BLDURL: stream the devicespec for request (V_URL).
18 BLDURL: LD A,(UENDPT) ; "N:" unless the endpoint carries its own
19 CP 'N'
20 JR Z,BURL1
21 LD A,'N'
22 CALL FNTXBYT
23 LD A,0
24 CALL FNTXBYT
25 BURL1: LD HL,UENDPT
26 CALL FNTXSTR
27 LD A,(V_URL)
28 OR A
29 JR NZ,BURL2
30 LD HL,UTABLS ; tables: no table, no player
31 JP FNTXSTR
32 BURL2: CP 1
33 JR NZ,BURL3
34 LD HL,USTATE
35 CALL FNTXSTR
36 JR BURLQ
37 BURL3: CP 2
38 JR NZ,BURL4
39 LD HL,UREADY ; bare /ready toggles, matching the
40 CALL FNTXSTR ; Intellivision client
41 JR BURLQ
42 BURL4: CP 3
43 JR NZ,BURL5
44 LD HL,UPLACE ; place/P,P,P,P,P
45 CALL FNTXSTR
46 LD HL,MYSHP
47 LD B,5
48 BURLP: LD A,(HL)
49 HL PUSH
50 BC PUSH
51 CALL BYTDEC
52 POP BC
53 POP HL
54 INC HL
55 LD A,B
56 DEC A
57 JR Z,BURLQ ; no comma after the last
58 LD A,','
59 CALL FNTXBYT ; clobbers only A,D,E
60 DJNZ BURLP
61 BURL5: CP 4
62 JR NZ,BURL6
63 LD HL,UATTKC ; attack/P
64 CALL FNTXSTR
65 LD A,(ATKPOS)
66 CALL BYTDEC
67
68 JR BURLQ
69 LD HL,UQTAB
70 CALL FNTXSTR
71 LD HL,TBLBUF
72 CALL FNTXSTR
73 LD HL,UQPLR
74 CALL FNTXSTR
75 LD HL,PLNBUF
76 CALL FNTXSTR
77 LD HL,UQBIN2
78 JP FNTXSTR
79
80
81 ; BYTDEC: stream A as 1-3 decimal digits, no leading zeros. Rides on
82 ; U16STR (which builds in LINBUF); safe mid-transaction -- it touches
83 ; only RAM and registers, never the mailbox pages.
84 BYTDEC: LD L,A
85 LD H,0
86 CALL U16STR
87 LD HL,LINBUF
88 JP FNTXSTR
89
90 UTABLS: DB "tables?bin=1",0
91 USTATE: DB "state",0
92 UREADY: DB "ready",0
93 UPLACE: DB "place/",0
94 UATTKC: DB "attack/",0
95 ULEAVE: DB "leave",0
96 UQTAB: DB "?table=",0
97 UQPLR: DB "&player=",0
98 UQBIN2: DB "&bin=1&v=2",0 ; v=2: winner + ship reveal at gameover
99

```

SCREENS.INC

```

1 ; screens.inc -- table selection, the poll loop, and the per-class
2 ; renderers.
3
4 ; Rendering policy (the card-game ports!): every poll redraws everything,
5 ; opaquely, straight out of the reply window -- identical pixels
6 ; rewritten are invisible, so there is no flicker and no digest
7 ; bookkeeping. The one thing opaque overdraw cannot fix is content that
8 ; SHRANK, so CHKNEW watches the screen class (lobby / place / game /
9 ; over) and the seat count, and answers a change with a full clear +
10 ; statics before the ordinary render.
11 ;
12 ; The poll loop is the Intellivision tl_loop reshaped into the texas
13 ; GAMELP: one staged action (PENDACT) rides the next request in place of
14 ; /state endpoint returns the same state blob -- and the reply
15 ; is dispatched by class. The turn edge (V_TRNF) is computed from
16 ; PRVACT at the top of every successful poll, lobby and placement
17 ; included, so the lobby->play transition is caught by the same
18 ; comparison as any other turn change.
19
20 ; =====
21 ; Table selection
22 ; =====
23 TBLSCR: CALL KEYRAW ; a key still held from the previous
24 ; (V_KEY),A ; screen must not fire here
25 CALL CLRSCR
26 SYSSUK STRDIS ; title, BIOS font at 2x
27 DB 0 ; "BATTLESHIP": 10 chars x 16px = the row
28 DB 0
29 DB 4CH
30 DW TTITLE
31 LD HL,TCH00S
32 LD DE,6*256+2
33 LD C,XWONF
34 CALL TXTAT
35 TBLRF: XOR A ; fetch /tables
36 CALL APICALL
37 JR NC,TBLOK
38 LD HL,ENET
39 LD DE,14*256+0
40 LD C,XRONF
41 CALL TXTAT
42 TBLRW: CALL KEYSN
43 OR A
44 JR Z,TBLRW
45 JR TBLRF
46 TBLOK: LD HL,0
47 CALL RXGETB ; table count
48 OR A
49 JR Z,TBLRW ; none yet: wait for a key, refetch
50 CP 9
51 JR C,TBOK2
52 LD A,8 ; eight rows fit under the title
53 TBOK2: LD (V_CNT),A
54 XOR A
55 LD (V_SEL),A
56 CALL TDBRAW
57 TBKEY: CALL KEYSN
58 OR A
59 JR Z,TBKEY
60 CP 'U'
61 JR NZ,TBK1
62 LD A,(V_SEL)
63 OR A
64 JR Z,TBKEY
65 DEC A
66 LD (V_SEL),A
67 CALL SNDKEY
68 CALL TDBRAW
69 JR TBKEY
70 CP 'D'
71 JR NZ,TBK2
72 LD A,(V_SEL)
73 INC A
74 LD HL,V_CNT
75 CP (HL)
76 JR NC,TBKEY
77 LD (V_SEL),A
78 CALL SNDKEY
79 CALL TDBRAW
80 JR TBKEY
81 TBK2: CP 'E' ; CE: back to the name screen
82 JR NZ,TBK2A
83 CALL NAMESCR
84 JP TBLSCR
85 TBK2A: CP ' ' ; the . key: how to play
86 JR NZ,TBK2B
87 CALL HELPSCR
88 JP TBLSCR
89 TBK2B: CP 'T'
90 JR Z,TBJOIN
91 CP '0' ; digit shortcut: 1-9 then 0 = row 10
92 JR C,TBKEY
93 CP '9'+1
94 JR NC,TBKEY
95 SUB '1'
96 CP 0FFH ; '0' wrapped to FF: row 10
97 JR NZ,TBK3
98 LD A,9
99 TBK3: LD HL,V_CNT
100 CP (HL)
101 JR NC,TBKEY
102 LD (V_SEL),A
103 ; falls into the join
104 TBJOIN: CALL SNDSEL
105 LD A,(V_SEL) ; record base = 1 + sel*36
106 CALL TBOFF
107 LD DE,TBLBUF ; id, up to its NUL
108 LD B,9
109 CALL RASTRN
110 LD DE,TBID+TBNAME ; the name field, for the lobby header
111 ADD HL,DE
112 LD DE,TBLNAM
113 LD B,21
114 CALL RASTRN
115 CALL CLRSCR
116 LD HL,TJOIN
117 LD DE,7*256+14
118 LD C,XWONF
119 CALL TXTAT
120 LD A,0FFH
121 LD (PRVCLS),A ; force the statics on the first poll
122 LD (PRVACT),A ; and arm the your-turn cue
123 LD (PRVRST),A ; and the result-sound edge
124 XOR A
125 LD (PENDACT),A
126 JP GAMELP
127
128 ; TBOFF: HL = window offset of table record A (1 + A*36, and 36 = 32+4).
129 ; Clobbers A,DE.
130 TBOFF: LD L,A
131 LD H,0
132 ADD HL,HL

```

```

133 ADD HL,HL ; *4
134 LD E,L
135 LD D,H
136 ADD HL,HL
137 HL,HL
138 ADD HL,HL ; *32
139 ADD HL,DE ; *36
140 INC HL ; + the count byte
141 RET
142
143 ; TBCDRAW: the list rows: name and "cur / max", selected row inverse.
144 TBCDRAW: XOR A
145 LD (V_TMP),A
146 TBCD1: LD A,(V_TMP)
147 LD HL,V_CNT
148 CP (HL)
149 RET NC
150 CALL TBOFF
151 PUSH HL
152 DE,TBNAME ; the display name field
153 ADD HL,DE
154 LD DE,LINBUF
155 LD B,21
156 CALL RXSTRN
157 LD A,(V_SEL) ; row color
158 LD HL,V_TMP
159 CP (HL)
160 LD C,XFONW
161 JR Z,TB02
162 LD C,XWONF
163 TBCD2: LD A,(V_TMP) ; row y = 7 + i
164 ADD A,7
165 LD D,A
166 LD E,2
167 LD HL,LINBUF
168 LD B,21
169 PUSH BC
170 CALL TXTN
171 POP BC
172 HL POP
173 LD DE,TBPLYRS ; the "cur / max" field
174 ADD HL,DE
175 LD DE,LINBUF
176 LD B,6
177 CALL RXSTRN
178 LD A,(V_TMP)
179 ADD A,7
180 LD D,A
181 LD E,25
182 LD HL,LINBUF
183 LD B,6
184 CALL TXTN
185 LD A,(V_TMP)
186 INC A
187 LD (V_TMP),A
188 JR TBCD1
189
190 ; =====
191 ; The poll loop
192 ; =====
193 GAMLP: LD A,(PENDACT) ; a staged action rides the next request:
194 INC A ; 0 none -> 1 /state, 1 -> 2 /ready,
195 CALL APICALL ; 2 -> 3 /place, 3 -> 4 /attack
196 JP C,GLFAIL
197 XOR A
198 LD (PENDACT),A ; consumed
199 LD HL,GOACTPL ; this poll's header, cached
200 CALL RXGETB
201 LD (V_ACT),A
202 LD HL,PRVACT ; the turn edge -- computed on EVERY good
203 CP (HL) ; poll (lobby included), so the

```

```

204 LD (HL),A ; lobby->play transition rides it too
205 LD A,0
206 JR Z,GLTE1
207 INC A
208 LD (V_TRNF),A
209 HL,GOPC
210 CALL RXGETB
211 LD (V_PC),A
212 CALL CHKNW
213 LD A,(V_CLS) ; dispatch by class
214 OR A
215 JP Z,GLLOB
216 CP 1
217 JP Z,GLPLC
218 CP 2
219 JP Z,GLGAM
220 CALL RENDGO ; class 3: game over
221 LD B,150 ; ~3 s; the server resets to the lobby
222 JP GLWAIT
223
224 GLLOB: CALL LOBBYR
225 LD B,90 ; ~2 s between lobby polls
226 JP GLWAIT
227
228 GLPLC: LD HL,GOPSTAT
229 CALL RXGETB
230 CP PSPACE
231 JR NZ,GLPLW
232 CALL PLACESCR ; blocking; fills MYSH
233 LD A,2
234 LD (PENDACT),A ; stage /place
235 JP GAMLP
236 GLPLW: LD D,14 ; placed already: show the server prompt
237 CALL PRMP ; ("waiting for others")
238 LD B,45
239 JP GLWAIT
240
241 GLGAM: CALL RENDER
242 LD A,(V_ACT)
243 OR A
244 JR NZ,GLGW ; not your turn
245 LD HL,GOPSTAT
246 CALL RXGETB
247 CP PSPACE
248 LD A,(V_TRNF) ; defeated or viewing: watch only
249 OR A ; announce BEFORE targeting -- it holds
250 CALL NZ,SNDTRN ; the loop for up to a slice, and a cue
251 LD A,(V_FLAG) ; after it is no alert at all
252 CALL TARGET
253 LD A,(V_FLAG)
254 CP 1
255 JR NZ,GLG2
256 LD A,3 ; fired: stage /attack
257 LD (PENDACT),A
258 JP GAMLP
259 GLG2: CP 2
260 JP Z,GLLEAV ; CE inside targeting
261 CP 3
262 JR NZ,GLG3
263 CALL HELPSCR ; '.' inside targeting
264 LD A,0FFH
265 LD (PRVCLS),A
266 GLG3: JP GAMLP ; slice burned either way: poll now
267 GLGW: LD B,45 ; ~1 s between game polls
268 JP GLWAIT
269
270 ; The between-polls input scan. The lobby ready toggle lives HERE, in the
271 ; input path -- edge-detected by KEYSCN, so one press cannot ready and
272 ; un-ready across two polls, and the trigger that joined the table cannot
273 ; toggle on arrival while still held (the Intellivision lesson).
274 GLWAIT: PUSH BC

```



```

275 CALL KEYSN
276 POP BC
277 OR A
278 JR NZ, GLKEY ; ~20 ms
279 LD DE, 2500
280 GLDLY: DEC DE
281 LD A, D
282 OR E
283 JR NZ, GLDLY
284 DJNZ GLWAIT
285 JP GAMELP
286 GLKEY: CP 'E' ; CE leaves the table
287 JR Z, GLLEAV
288 CP 'I'
289 JR NZ, GLK1
290 CALL HELPSCR
291 LD A, 0FFH
292 LD (PRVCLS), A ; force the statics on return
293 JP GAMELP
294 GLK1: CP 'I'
295 JR NZ, GLK2
296 LD A, (V_CLS)
297 OR A
298 JR NZ, GLWAIT ; the trigger only means READY in the lobby
299 CALL SNOSEL
300 LD A, 1
301 LD (PENDACT), A ; stage /ready
302 JP GAMELP
303 GLK2: CP '0'
304 JP Z, GAMELP ; poll now
305 JP GLWAIT
306 GLLEAV: LD A, 5
307 CALL APICALL ; best effort; we are going regardless
308 JP TBLSCR
309 GLFAIL: LD HL, TBANG ; a red mark in the corner; back off and
310 LD DE, 0*256+39 ; try again
311 LD C, XRONF
312 CALL TXSTAT
313 LD B, 150
314 GLFW: PUSH BC
315 CALL KEYSN
316 POP BC
317 CP 'E'
318 JR Z, GLLEAV
319 LD DE, 2500
320 GLFD: DEC DE
321 LD A, D
322 OR E
323 JR NZ, GLFD
324 DJNZ GLFW
325 JP GAMELP
326
327 ; -----
328 ; CHKNEW: map the status to a screen class (0 lobby, 1 place, 2 game,
329 ; 3 over). A class count change means the screen holds content
330 ; that no longer exists: clear, redraw the statics, and reset the edges
331 ; that must not fire across the boundary. V_NEWS = 1 when that happened
332 ; (RENDGO's cue plays on it).
333 CHKNEW: LD HL, GOSTAT
334 CALL RXGETB
335 OR A
336 JR Z, CHKC ; 0 -> lobby
337 CP STPLACE
338 JR Z, CHKC ; 1 -> place
339 CP STOVER
340 JR Z, CHKC3
341 LD A, 2
342 LD CHKC
343 CHKC3: LD A, 3
344 CHKC: LD (V_CLS), A
345 LD HL, PRVCLS
346 CP (HL)
347 LD (HL), A
348 JR NZ, CHKN1
349 LD A, (V_PC)
350 LD HL, PRVPC
351 CP (HL)
352 LD (HL), A
353 JR NZ, CHKN1
354 XOR A
355 LD (V_NEWS), A
356 RET
357 CHKN1: LD A, (V_PC) ; keep PRVPC honest on the class path too
358 LD (PRVPC), A
359 LD A, 0FFH ; a stale result pair must not swallow
360 LD (PRVRST), A ; the new screen's first result
361 LD A, 1
362 LD (V_NEWS), A
363 CALL CLRSCR
364 LD A, (V_CLS)
365 OR Z
366 RET ; lobby: LOBBYR draws everything
367 JP BRDSTAT ; boards: the dividers
368
369 ; PRMPT: the server prompt (33 wire bytes) as a fixed-width field at
370 ; row D, col 0. Clobbers everything.
371 PRMPT: PUSH DE
372 LD HL, GOPRMPT
373 LD DE, LINBUF
374 LD B, 33
375 CALL RXSTRN
376 POP DE
377 LD E, 0
378 LD HL, LINBUF
379 LD C, XWONF
380 LD B, 34
381 JP TXTN
382
383 ; =====
384 ; The lobby
385 ; =====
386 ; LOBBYR: a full-text screen. Fixed-width TXTN fields make every row
387 ; self-clearing; the ready toggle itself is sampled in GLWAIT (the input
388 ; path), never here -- this runs only once per poll.
389 LOBBYR: LD HL, TBLNAM
390 LD DE, 0*256+0
391 LD C, XWONF
392 LD B, 21
393 CALL TXTN
394 LD HL, GOSRVNM
395 LD DE, LINBUF
396 LD B, 21
397 CALL RXSTRN
398 LD HL, LINBUF
399 LD DE, 1*256+0
400 LD C, XWONF
401 LD B, 21
402 CALL TXTN
403 XOR A
404 LD (V_TMP), A
405 LOBP: LD A, (V_TMP)
406 CP 4
407 JR NC, LOBP9
408 LD HL, V_PC
409 CP (HL)
410 JR NC, LOBLBK ; row beyond the seat count: blank it
411 LD L, A ; record base = GOLPLRS + i*10
412 LD H, 0
413 ADD HL, HL
414 LD E, L
415 LD D, H
416 ADD HL, HL

```

```

417 ADD HL,HL ; *8
418 ADD HL,DE ; *10
419 LD DE,GOLPLRS
420 ADD HL,DE
421 HL PUSH
422 LD DE,LINBUF ; the name
423 LD B,9
424 CALL RXSTRN
425 LD A,(V_TMP)
426 ADD A,3 ; list rows 3-6
427 LD D,A
428 LD E,2
429 LD HL,LINBUF
430 LD C,XWONF
431 LD B,9
432 CALL TTXN
433 POP HL
434 LD DE,LPSTAT ; the seat's status word
435 ADD HL,DE
436 CALL RXGETB
437 LD HL,TREADY
438 LD C,XFONW
439 CP PSREADY
440 JR Z,LOBS1
441 LD HL,TVIEWW
442 LD C,XWONF
443 CP PSVIEW
444 JR Z,LOBS1
445 LD HL,TNUL
446 LD A,(V_TMP)
447 ADD A,3
448 LD D,A
449 LD E,13
450 LD B,6
451 CALL TTXN
452 JR LOBNXT
453 LOBBLK: ADD A,3 ; vacated row: blank name + status
454 LD D,A
455 LD E,2
456 LD HL,TNUL
457 LD C,XWONF
458 LD B,17
459 CALL TTXN
460 LOBNXT: LD A,(V_TMP)
461 INC A
462 LD (V_TMP),A
463 JR LOBP
464 LOBP9: LD D,8
465 CALL PRMPT
466 LD HL,TLOBK1
467 LD DE,10*256+0
468 LD C,XWONF
469 CALL TTXAT
470 LD HL,TLOBK2
471 LD DE,12*256+0
472 LD C,XWONF
473 JP TTXAT
474
475 ; =====
476 ; Rendering a game poll
477 ; =====
478 RENDER: XOR A ; the quadrants, in wire order (walks the
479 LD (V_TMP),A ; window forward: fewest slice flips)
480 RENQ: LD A,(V_TMP)
481 LD HL,V_PC
482 CP (HL)
483 JR NC,RENQ9
484 CALL QUADRW
485 LD A,(V_TMP)
486 INC A
487 LD (V_TMP),A

488 JR RENQ
489 RENQ9: LD HL,GOPSTAT ; own-ship overlay on quadrant 0 -- the
490 CALL RXGETB ; wire is authoritative over what
491 CP PSVIEW ; placement guessed (covers reconnects);
492 JR Z,REN9 ; a viewer has no ships
493 LD DE,GOMYSH
494 XOR A ; quadrant 0
495 LD C,0FFH ; per cell: your own field marks damage
496 CALL SHIPSET
497 CALL PANEL
498 LD D,14
499 CALL PRMPT
500 CALL CLOCK
501 LD HL,GOSTAT ; the result edge: (status, lastAttack)
502 CALL RXGETB ; as a pair -- the status repeats across
503 CP STMISS ; polls when nobody has fired since
504 RET C ; GAMESTART: no result yet
505 CP STSUNK+1
506 RET NC
507 LD B,A
508 LD HL,GOLATK
509 CALL RXGETB
510 LD C,A
511 LD A,(PRVRST)
512 CP B
513 JR NZ,REN1
514 LD A,(PRVRPS)
515 CP C
516 RET Z ; the same event as last poll
517 REN1: LD A,B
518 LD (PRVRST),A
519 LD A,C
520 LD (PRVRPS),A
521 LD A,B
522 CP STMISS
523 JP Z,SNDMISS
524 CP STHIT
525 JP Z,SNDHIT
526 JP SNDSUNK
527
528 ; CLOCK: the move timer, red, in the prompt row's last three cells --
529 ; three digits, because single-player runs to 250.
530 CLOCK: LD A,(V_ACT)
531 INC A ; FFH = nobody: blank the field
532 JR Z,CLKB
533 LD HL,GOMVTIM
534 CALL RXGETB
535 LD L,A
536 LD H,0
537 CALL UI6STR
538 LD HL,LINBUF
539 LD DE,14*256+37
540 LD C,XRONF
541 LD B,3
542 JP TTXN
543 CLKB: LD HL,TNUL
544 LD DE,14*256+37
545 LD C,XWONF
546 LD B,3
547 JP TTXN
548
549 ; =====
550 ; PANEL: the right-hand chrome, char cols 22-39 (col 21 is the gutter).
551 ; Two rows per player: the quadrant tag + name (inverse when active,
552 ; black when defeated) + a red TURN tag, then five ship pips from
553 ; shipsLeft ('#' afloat, '.' sunk) with SUNK when none remain. Absent
554 ; seats blank themselves via the fixed-width fields. Row 13 carries the
555 ; your-turn banner (game class only).
556 PANEL: XOR A
557 LD (V_TMP),A
558 PANL: LD A,(V_TMP)

```

```

559 CP 4
560 JP NC,PAN9
561 LD HL,V_PC
562 CP (HL)
563 JP NC,PANBLK
564 ADD A,A ; the quadrant tag
565 LD HL,QTAG
566 LD E,A
567 LD D,0
568 ADD HL,DE
569 LD A,(HL)
570 LD (LINBUF),A
571 INC HL
572 LD A,(HL)
573 LD (LINBUF+1),A
574 XOR A
575 LD (LINBUF+2),A
576 LD A,(V_TMP)
577 LD D,A
578 ADD A,A
579 ADD A,D ; row = i*3
580 LD D,A
581 LD E,22
582 LD HL,LINBUF
583 LD C,XWONF
584 LD B,2
585 CALL TXTN
586 LD A,(V_TMP) ; the name and its status color
587 CALL PLBASE
588 HL PUSH
589 LD DE,LINBUF
590 LD B,9
591 CALL RXSTRN
592 POP HL
593 HL PUSH
594 LD DE,PLSTAT
595 ADD HL,DE
596 CALL RXGETB
597 LD (V_ST),A
598 LD A,(V_TMP)
599 LD HL,V_ACT
600 CP (HL)
601 LD C,XFONW ; active: inverse
602 JR Z,PANNC
603 LD A,(V_ST)
604 CP PSDEAD
605 LD C,XKONF ; defeated: black
606 JR Z,PANNC
607 LD C,XWONF
608 LD A,(V_TMP)
609 LD D,A
610 ADD A,A
611 ADD A,D
612 LD D,A
613 LD E,25
614 LD HL,LINBUF
615 LD B,9
616 CALL TXTN
617 LD A,(V_TMP) ; the TURN tag
618 LD HL,V_ACT
619 CP (HL)
620 LD HL,TURN
621 JR Z,PANT1
622 LD HL,TNUL
623 LD A,(V_TMP)
624 LD D,A
625 ADD A,A
626 ADD A,D
627 LD D,A
628 LD E,35
629 LD C,XRONF
630 LD B,4
631 CALL TXTN
632 POP HL ; ship pips from shipsLeft
633 LD DE,PLSHLFT
634 ADD HL,DE
635 LD DE,LINBUF
636 LD C,0 ; afloat count
637 LD B,5
638 PANP1: CALL RXGETB
639 OR A
640 LD A,', '
641 JR Z,PANP2
642 LD A,'# '
643 PANP2: INC C
644 LD (DE),A
645 INC DE
646 INC HL
647 DJNZ PANP1
648 XOR A
649 LD (DE),A
650 LD A,C
651 LD (V_ST),A ; the afloat count, for the SUNK tag
652 LD A,(V_TMP)
653 LD D,A
654 ADD A,A
655 ADD A,D
656 INC A
657 LD D,A ; row = i*3 + 1
658 LD E,25
659 LD HL,LINBUF
660 LD C,XWONF
661 LD B,5
662 CALL TXTN
663 LD A,(V_ST)
664 OR A
665 LD HL,TSUNK
666 JR Z,PANS1
667 LD HL,TNUL
668 PANP1: LD A,(V_TMP)
669 LD D,A
670 ADD A,A
671 ADD A,D
672 INC A
673 LD D,A
674 LD E,31
675 LD C,XRONF
676 LD B,4
677 CALL TXTN
678 PANBLK: JR PANNXT
679 LD A,(V_TMP) ; absent seat: blank both rows whole
680 LD D,A
681 ADD A,A
682 ADD A,D
683 LD D,A
684 LD E,22
685 LD HL,TNUL
686 LD C,XWONF
687 LD B,18
688 CALL TXTN
689 LD A,(V_TMP)
690 LD D,A
691 ADD A,A
692 ADD A,D
693 INC A
694 LD D,A
695 LD E,22
696 LD HL,TNUL
697 LD C,XWONF
698 LD B,18
699 CALL TXTN
700 PANNXT: LD A,(V_TMP)

```

```

701 INC A
702 LD (V_TMP),A
703 JP PANL
704 PAN9: LD A,(V_CLS) ; the your-turn banner, game class only
705 CP
706 JR NZ,PAN9A
707 LD A,(V_ACT)
708 OR A
709 JR NZ,PAN9A
710 LD HL,GOPSTAT
711 CALL RXGETB
712 CP PSPLAY
713 JR NZ,PAN9A
714 LD HL,TFIRE
715 JR PAN9B
716 PAN9A: LD HL,TNUL
717 PAN9B: LD DE,13*256+22
718 LD C,XRONF
719 LD B,17
720 JP TXTN
721
722 ; -----
723 ; RENDGO: the final boards plus the winner's ships revealed in the SUNK
724 ; pattern on their quadrant (v=2's myShips[5..9]). The cue plays once,
725 ; on the edge; the server auto-resets to the lobby and that class
726 ; change rides CHKNEW like any other.
727 RENDGO: CALL RENDER ; final fields; the prompt carries the
728 LD A,(V_NEWS) ; winner message
729 OR A
730 CALL NZ,SNDOVER
731 LD HL,TGOVER
732 LD DE,13*256+22
733 LD C,XRONF
734 LD B,17
735 CALL TXTN
736 LD A,(V_ACT) ; v=2: activePlayer = the winner
737 CP 4
738 RET NC ; -1 (or junk): nothing to reveal
739 LD DE,COMYSH+5
740 LD C,PSUNK
741 JP SHIPSET
742
743 ; -----
744 ; HELPSCR: how to play. Any key returns.
745 HELPSCR: CALL KEYRAW
746 LD (V_KEY),A
747 CALL CLRSCR
748 LD HL,HELPTX
749 LD D,0 ; row; strings run in order, one per row,
750 HELP1: LD A,(HL) ; first byte = column (FF ends)
751 CP 0FFH
752 JR Z,HELP9
753 LD E,A
754 INC HL
755 DE PUSH HL
756 PUSH HL
757 LD C,XWONF
758 CALL TXSTAT
759 POP HL
760 HELP2: LD A,(HL) ; skip past the NUL
761 INC HL
762 OR A
763 JR NZ,HELP2
764 POP DE
765 INC D
766 JR HELP1
767 HELP9: CALL KEYSCN
768 OR A
769 JR Z,HELP9
770 RET
771

```

```

772 HELPTX: DB 15,"BATTLESHIP",0
773 DB 0,"",0
774 DB 1,"PLACE FIVE SHIPS, THEN TAKE",0
775 DB 1,"TURNS FIRING. ONE SHOT HITS",0
776 DB 1,"THAT CELL ON EVERY ENEMY",0
777 DB 1,"BOARD AT ONCE. SINK EVERY",0
778 DB 1,"ENEMY FLEET TO WIN.",0
779 DB 0,"",0
780 DB 1,"STICK MOVES TRIGGER FIRES",0
781 DB 1,"0 ROTATES A SHIP / POLLS NOW",0
782 DB 1,"TRIGGER READIES IN THE LOBBY",0
783 DB 1,"CE LEAVES . THIS SCREEN",0
784 DB 0,"",0
785 DB 1,"BATTLESHIP.CARR-DESIGNS.COM",0
786 DB 0FFH
787
788 TCHOOS: DB "CHOOSE A TABLE",0
789 TJOIN: DB "JOINING...",0
790 TREADY: DB "READY",0
791 TVIEWW: DB "VIEW",0
792 TTURN: DB "TURN",0
793 TSUNK: DB "SUNK",0
794 TFIRE: DB "YOUR TURN - FIRE",0
795 TGOVER: DB "GAME OVER",0
796 TLOBK1: DB "TRIGGER = READY ON/OFF",0
797 TLOBK2: DB "CE LEAVES . HELP 0 REFRESH",0
798 TBANG: DB "!",0
799 TNUL: DB 0
800

```

PLACE.INC

```

1 ; place.inc -- ship placement, transcribed from the Intellivision
2 ; handle placement. Each ship (sizes 5,4,3,3,2, fixed order) seeds at a
3 ; random legal spot, the stick nudges it, keypad 0 rotates, and the
4 ; trigger confirms -- rejected locally when it overlaps an
5 ; already-confirmed ship, so no server round trip is spent finding that
6 ; out. The pending ship blinks ring <-> solid; confirmed hull is solid.
7 ;
8 ; Blocking, like the Intellivision's: bots place instantly and humans
9 ; are the wait, the waiter's own myShips makes the next poll
10 ; authoritative if the server somehow moved on without us.
11 ;
12 ; Fills MYSHIP[0..4] (pos + 100*dir); the caller stages /place.
13
14 PLACESC: CALL KEYRAW ; no leaking presses
15 LD (V_KEY),A
16 LD HL,OCCUPY ; only the occupancy map needs clearing:
17 LD B,100 ; CHKNEW just painted the boards sea
18 PLCZ: LD (HL),0
19 INC HL
20 DJNZ PLCZ
21 XOR A
22 LD (PCNUM),A
23 PLSHIP: LD A,(PCNUM)
24 CP 5
25 RET NC ; all placed
26 LD HL,SHPSIZ
27 LD E,A
28 LD D,0
29 ADD HL,DE
30 LD A,(HL)
31 LD (PCSIZ),A
32 ; Seed a random legal spot. The long axis draws 0..10-size, the other
33 ; 0..9, so bounds are legal by construction; only overlap can reject.
34 PLSEED: CALL RNDNXT
35 AND 1
36 LD (PCDIR),A
37 LD A,(PCSIZ)
38 LD B,A
39 LD A,11
40 SUB B
41 CALL RNDMOD ; A = rand mod (11-size): the long axis
42 PUSH AF ; RNDMOD clobbers C -- the first result
43 LD A,10 ; must not ride there (it did once: every
44 CALL RNDMOD ; long axis came back 10, the modulus)
45 LD B,A ; B = the free axis
46 POP AF
47 LD C,A ; C = the long axis
48 LD A,(PCDIR)
49 OR A
50 JR NZ,PLSV ; horizontal: x rides the long axis
51 LD A,C
52 LD (PCX),A
53 LD A,B
54 LD (PCY),A
55 JR PLSF
56 PLSV: LD A,B ; vertical: y does
57 LD (PCX),A
58 LD A,C
59 LD (PCY),A
60 PLSF: CALL PLCFIT
61 JR C,PLSEED
62 LD A,(PCNUM) ; the prompt: SHIP n/5 ...
63 ADD A,1
64 LD (HEXBUF),A
65 XOR A
66 LD (HEXBUF+1),A
67 LD HL,TSHIP
68 LD DE,14*256+0
69 LD C,XWONF
70 CALL TXTAT
71 LD HL,HEXBUF
72 LD DE,14*256+5
73 LD C,XWONF
74 CALL TXTAT
75 LD HL,TSHIP2
76 LD DE,14*256+6
77 LD C,XWONF
78 LD B,28
79 CALL TXTN
80 XOR A
81 LD (V_BLNK),A
82 CALL PLCDRW ; the pending ship, ring pattern
83 PLL0OP: CALL KEYSCHN
84 OR A
85 JP NZ,PLKEY
86 LD DE,2500 ; ~20 ms
87 PLDL: DEC DE
88 LD A,D
89 OR E
90 JR NZ,PLDL
91 LD HL,V_BLNK ; ring <-> solid every ~320 ms
92 INC (HL)
93 LD A,(HL)
94 AND 0FH
95 JR NZ,PLL0OP
96 LD A,(HL)
97 AND 10H
98 JR Z,PLBL1
99 CALL PLCDRW
100 PLL0OP: CALL PLCDRS
101 PLBL1: CALL PLL0OP
102 JR PLKEY: CP 'U'
103 LD A,(PCY)
104 LD A
105 OR A
106 JR Z,PLL0OP
107 CALL PLCLERS
108 LD HL,PCY
109 DEC (HL)
110 PLMV: CALL SNDKEY
111 CALL PLCDRW
112 JP PLL0OP
113 PLK1: CP 'D'
114 JR NZ,PLK2
115 CALL PLYMAX
116 LD HL,PCY
117 CP (HL)
118 JR Z,PLL002 ; already at the edge
119 JR C,PLL002
120 CALL PLCLERS
121 LD HL,PCY
122 INC (HL)
123 JR PLMV
124 PLK2: CP 'L'
125 JR NZ,PLK3
126 LD A,(PCX)
127 OR A
128 JR Z,PLL002
129 CALL PLCLERS
130 LD HL,PCX
131 DEC (HL)
132

```

```

133 JR PLMV
134 PLK3: CP 'R'
135 JR NZ,PLK4
136 CALL PLXMAX
137 LD HL,PCX
138 CP (HL)
139 JR Z,PLLO02
140 JR C,PLLO02
141 CALL PLCLERS
142 LD HL,PCX
143 INC (HL)
144 JR PLMV
145 PLK4: CP '0' ; rotate, clamping the origin so the
146 JR NZ,PLK5 ; ship stays on the board
147 CALL PLCLERS
148 LD A,(PCDIR)
149 XOR 1
150 LD (PCDIR),A
151 CALL PLXMAX
152 LD HL,PCX
153 CP (HL)
154 JR NC,PLR1
155 LD (HL),A
156 PLR1: CALL PLYMAX
157 LD HL,PCY
158 CP (HL)
159 JR NC,PLMV
160 LD (HL),A
161 JR PLMV
162 PLK5: CP 'T'
163 JR NZ,PLLO02
164 CALL PLCFIT
165 JR NC,PLOK
166 CALL SNDBAD ; overlaps a placed ship
167 PLLO02: JP PLL00P
168 PLOK: CALL PLCMRK ; occupy + repaint solid
169 CALL SNDPLC ; MYSHP[i] = x + y*10 + 100*dir
170 LD A,(PCY)
171 CALL MUL10
172 LD HL,PCX
173 ADD A,(HL)
174 LD HL,PCDIR
175 BIT 0,(HL)
176 JR Z,PLOK1
177 LD A,100
178 PLOK1: LD C,A
179 LD A,(PCNUM)
180 LD HL,MYSHP
181 LD E,A
182 LD D,0
183 ADD HL,DE
184 LD (HL),C
185 LD A,(PCNUM)
186 INC A
187 LD (PCNUM),A
188 JP PLSHIP
189
190 ; PLXMAX/PLYMAX: A = the cursor's max origin on that axis (the long
191 ; axis stops at 10-size). Clobbers B.
192 PLXMAX: LD A,(PCDIR)
193 OR A
194 JR NZ,PLXM1 ; vertical: x is free to 9
195 LD A,(PCSIZ)
196 LD B,A
197 LD A,10
198 SUB B
199 RET
200 PLXM1: LD A,9
201 LD RET
202 PLYMAX: LD A,(PCDIR)
203 OR A
204 JR Z,PLXM1 ; horizontal: y is free to 9
205 LD A,(PCSIZ)
206 LD B,A
207 LD A,10
208 SUB B
209 RET
210
211 ; PLCELL: A = occupancy index (y*10+x) of pending cell B (0-based along
212 ; the ship). Preserves B,DE; clobbers C.
213 PLCELL: PUSH HL
214 LD A,(PCDIR)
215 OR A
216 JR NZ,PLCV
217 LD A,(PCY) ; horizontal: (x+i, y)
218 CALL MUL10
219 LD HL,PCX
220 ADD A,(HL)
221 ADD A,B
222 POP HL
223 RET
224 PLCV: LD A,(PCY) ; vertical: (x, y+i)
225 ADD A,B
226 CALL MUL10
227 LD HL,PCX
228 ADD A,(HL)
229 POP HL
230 RET
231
232 ; PLCFIT: carry set when the pending ship overlaps a confirmed one.
233 ; Bounds need no check - the seed and the clamps keep them legal.
234 PLCFIT: LD A,(PCSIZ)
235 LD B,A
236 LD A,B
237 DEC A
238 PUSH BC
239 LD B,A
240 CALL PLCELL
241 POP BC
242 LD HL,OCCUPY
243 LD E,A
244 LD D,0
245 ADD HL,DE
246 LD A,(HL)
247 OR A
248 JR NZ,PLCFX
249 DJNZ PLCF1
250 OR A ; carry clear: it fits
251 RET
252 PLCFX: SCF
253 RET
254
255 ; PLCMRK: mark the pending cells occupied, then fall through to the
256 ; solid repaint.
257 PLCMRK: LD A,(PCSIZ)
258 LD B,A
259 LD A,B
260 DEC A
261 PUSH BC
262 LD B,A
263 CALL PLCELL
264 POP BC
265 LD HL,OCCUPY
266 LD E,A
267 LD D,0
268 ADD HL,DE
269 LD (HL),1
270 DJNZ PLCM1
271 ; falls through: repaint the (now confirmed) ship solid
272 PLCDRS: LD A,PSHIP ; the pending ship, solid phase
273 LD RET
274 PLCDRW: LD A,PCURSR ; the pending ship, ring phase

```

```

275  PLCPAT: LD      (V_ST),A
276          LD      A,(PCSIZE)
277          LD      B,A
278  PLCP1:  LD      A,B
279          DEC      A
280          PUSH     BC
281          LD      B,A
282          CALL     PLCELL
283          POP      BC
284          PUSH     BC
285          CALL     DIV10      ; B = y, A = x
286          LD      (V_CX),A
287          LD      A,B
288          LD      (V_CV),A
289          XOR      A          ; quadrant 0
290          CALL     BRDPOS
291          LD      A,(V_ST)
292          EX       DE,HL
293          CALL     CELLDRAW
294          POP      BC
295          DJNZ     PLCP1
296          RET
297
298  ; PLCERS: repaint the pending cells to their truth -- sea, or a
299  ; confirmed ship's hull where the cursor is passing over one.
300  PLCERS: LD      A,(PCSIZE)
301          LD      B,A
302  PLCE1:  LD      A,B
303          DEC      A
304          PUSH     BC
305          LD      B,A
306          CALL     PLCELL
307          POP      BC
308          PUSH     BC
309          PUSH     AF
310          LD      HL,OCCUPY
311          LD      E,A
312          LD      D,0
313          ADD      HL,DE
314          LD      A,(HL)
315          OR      A
316          LD      A,PWATER
317          JR       Z,PLCE2
318          LD      A,PSHIP
319  PLCE2:  LD      (V_ST),A
320          POP      AF
321          CALL     DIV10
322          LD      (V_CX),A
323          LD      A,B
324          LD      (V_CV),A
325          XOR      A
326          CALL     BRDPOS
327          LD      A,(V_ST)
328          EX       DE,HL
329          CALL     CELLDRAW
330          POP      BC
331          DJNZ     PLCE1
332          RET
333
334  TSHIP:  DB      "SHIP",0
335  TSHIP2: DB      "/5 STICK MOVES 0=ROT T=OK",0
336

```

TARGET.INC

```

1  ; target.inc -- your turn. One cursor, in lockstep over every live enemy
2  ; quadrant: the server's core multiplayer rule is that a single shot
3  ; fires at that coordinate on every other playing board at once, so

```

```

4  ; there is only ever one position to choose, not one per opponent.
5  ;
6  ; Unlike the Intellivision's fully-blocking loop, TARGET runs one

```

```

7 ; bounded ~5 s slice and returns no-fire; GAMELP re-polls and re-enters
8 ; while it is still our turn. The client keeps polling even inside the
9 ; 250-second single-player clock, so a game that ends (or a table that
10 ; empties) is noticed within a slice. TGX/TGY persist -- the cursor
11 ; resumes where it left off, across slices and turns alike. A keypress
12 ; restarts the slice: an actively-aiming player is not interrupted, and
13 ; KEYSCN's edge detection means holding a key cannot spin it.
14 ;
15 ; The countdown reseeds from the wire's movetime on every entry, so the
16 ; calibrated-delay, clock self-corrects each poll; at zero it bails --
17 ; the server has already skipped the turn, and only the next poll can
18 ; discover where the game went (the Intellivision lesson).
19 ;
20 ; V_FLAG on return: 0 slice out / timeout, 1 fired (ATKPOS staged),
21 ; 2 leave, 3 help.
22
23 TARGET: XOR      A
24 LD              (V_FLAG),A
25 LD              (TGFRM),A
26 LD              (V_BLNK),A
27 LD              HL,GOMVTIM
28 CALL           RXGETB
29 LD              (TGLEFT),A
30 CALL           TGDWR
31 LD              B,250
32 TGLP:  PUSH     BC
33 CALL           KEYSCN
34 POP            BC
35 OR             A
36 JP             NZ,TGKEY
37 PUSH          BC
38 LD             DE,2500
39 TGDLY:  DEC     DE
40 LD             A,D
41 OR             E
42 JR             NZ,TGDLY
43 LD             HL,V_BLNK
44 INC            (HL)
45 LD             A,(HL)
46 AND            OFH
47 JR             NZ,TGSEC
48 LD             A,(HL)
49 AND            10H
50 JR             Z,TGBL1
51 CALL           TGDWR
52 JR             TGSEC
53 TGBL1:  CALL     TGRS
54 TGSEC:  LD       HL,TGFRM
55 INC     (HL)
56 LD       A,(HL)
57 CP       50
58 JR       C,TGNX
59 LD       (HL),0
60 LD       A,(TGLEFT)
61 OR       A
62 JR       Z,TGOUT
63 DEC      A
64 LD       (TGLEFT),A
65 PUSH     AF
66 LD       L,A
67 LD       H,0
68 CALL     U16STR
69 LD       HL,LINBUF
70 LD       DE,14*256+37
71 LD       C,XRONF
72 LD       B,3
73 CALL     TXTN
74 POP      AF
75 OR       A
76 JR       Z,TGOUT
77 CP       6

```

; ~5 s slice of 20 ms ticks

; ~20 ms

; ring <-> truth every ~320 ms

; the local second hand

; 50 x 20 ms = 1 s

; already zero: bail now

; repaint the clock field

; hit zero: the server has skipped us

```

78 CALL     C,SNDTICK
79 TGNX:  POP      BC
80 DJNZ    TGLP
81 JP      TGRS
82 TGOUT:  POP      BC
83 JP      TGRS
84 TGKEY:  CP       'U'
85 JR      NZ,TGK1
86 LD      A,(TGY)
87 OR      A
88 JR      Z,TGLP2
89 CALL     TGRS
90 LD      HL,TGY
91 DEC     (HL)
92 TGMV:  CALL     SNDKEY
93 CALL     TGDWR
94 TGLP2: LD      B,250
95 JP      TGLP
96 TGK1:  CP       'D'
97 JR      NZ,TGK2
98 LD      A,(TGY)
99 CP       9
100 JR      NC,TGLP2
101 CALL     TGRS
102 LD      HL,TGY
103 INC     (HL)
104 JR      TGMV
105 TGK2:  CP       'L'
106 JR      NZ,TGK3
107 LD      A,(TGX)
108 OR      A
109 JR      Z,TGLP2
110 CALL     TGRS
111 LD      HL,TGX
112 DEC     (HL)
113 JR      TGMV
114 TGK3:  CP       'R'
115 JR      NZ,TGK4
116 LD      A,(TGX)
117 CP       9
118 JR      NC,TGLP2
119 CALL     TGRS
120 LD      HL,TGX
121 INC     (HL)
122 JR      TGMV
123 TGK4:  CP       'T'
124 JR      NZ,TGK5
125 CALL     TGRS
126 LD      A,(TGY)
127 CALL     MUL10
128 LD      HL,TGX
129 ADD     A,(HL)
130 LD      (ATKPOS),A
131 LD      A,1
132 LD      (V_FLAG),A
133 JP      SNDSEL
134 TGK5:  CP       'E'
135 JR      NZ,TGK6
136 LD      A,2
137 LD      (V_FLAG),A
138 JP      TGRS
139 TGK6:  CP       '.'
140 JR      NZ,TGLP2
141 LD      A,3
142 LD      (V_FLAG),A
143 JP      TGRS
144
145 ; TGDWR: the cursor ring on every live enemy quadrant at (TGX,TGY).
146 ; TGRS: those cells back to their wire truth -- the reply window IS
147 ; the shadow board. Both clobber everything.
148 TGDWR: LD      A,PCURSR

```

; a press restarts the slice

; fire

; leave -- handed back to GAMELP so the
stack stays balanced

; help, same hand-back


```

149      JR      TGPAT
150 TGRS: LD      A,0FFH
151 TGPAT: LD      (V_ST),A
152      LD      A,1
153      LD      (V_TMP),A
154 TQ1:  LD      A,(V_TMP)
155      CP      4
156      RET     NC
157      LD      HL,V_PC
158      CP      (HL)
159      RET     NC          ; beyond the seat count
160      CALL    PLBASE      ; only PLAYING boards carry the cursor
161      LD      DE,PLSTAT
162      ADD     HL,DE
163      CALL    RXGETB
164      CP      PSPLAY
165      JR      NZ,TQON
166      LD      A,(V_ST)      ; the pattern: ring, or the wire truth
167      CP      0FFH
168      JR      NZ,TQ2
169      LD      A,(V_TMP)
170      CALL    PLBASE
171      LD      DE,PLFIELD
172      ADD     HL,DE
173      LD      A,(TGY)
174      CALL    MUL10
175      LD      E,A
176      LD      A,(TGX)
177      ADD     A,E
178      LD      E,A
179      LD      D,0
180      ADD     HL,DE
181      CALL    RXGETB
182      CALL    CELLST
183 TQ2:  PUSH    AF
184      LD      A,(TGX)
185      LD      (V_CX),A
186      LD      A,(TGY)
187      LD      (V_CY),A
188      LD      A,(V_TMP)
189      CALL    BRDPOS
190      POP     AF
191      EX      DE,HL
192      CALL    CELLDRAW
193 TQON: LD      A,(V_TMP)
194      INC     A
195      LD      (V_TMP),A
196      JR      TQ1
197

```

BOARD.INC

```

1 ; board.inc -- the four 10x10 boards: geometry, cell patterns, and the
2 ; wire-driven redraw.
3
4 ; A board cell is 4x4 pixels = one byte per row at 2bpp, so a cell
5 ; paints as four byte writes and nothing shifts. Quadrant layout
6 ; (player i = quadrant i; the server rotates you to index 0):
7
8 ;   bytes 0-9 | 10 | 11-20   lines 0-39 q1 (TL)  q2 (TR)
9 ;   board   div  board       lines 40-43 divider
10
11 ;   bytes 44-83 q0 (YOU) q3 (BR)
12
13 ; The reply window is the shadow board: erasing a cursor or a pending
14 ; ship reads the cell's truth back out of the wire (or OCCUPY during
15 ; placement), so no 100-byte RAM copies exist.
16 ; Cell pattern indexes.

```

```

17 PWATER EQU 0
18 PHIT   EQU 1
19 PHISS  EQU 2
20 PSHIP  EQU 3
21 PSHPT  EQU 4
22 PSUNK  EQU 5
23 PCURSR EQU 6
24
25 ;

```

```

26 ; BRDPOS: DE = screen address of cell (V_CX,V_CY) on quadrant A.
27 ; line = QDROW[q] + 4*y; addr = NORMEM + line*40 + QDCOL[q] + x.
28 ; Clobbers A,BC,HL.
29 BRDPOS: LD      C,A
30          LD      B,0
31          LD      HL,QDROW
32          ADD     HL,BC

```

```

33     LD      A,(V_CY)
34     ADD     A,A
35     ADD     A,A          ; 4*y
36     LD      A,(HL)      ; + the quadrant's top line (max 80)
37     LD      L,A
38     LD      H,0
39     ADD     HL,HL
40     ADD     HL,HL
41     ADD     HL,HL        ; *8
42     LD      E,L
43     LD      D,H
44     ADD     HL,HL
45     ADD     HL,HL        ; *32
46     ADD     HL,DE        ; *40
47     EX      DE,HL       ; DE = line*40
48     LD      HL,QDCOL
49     ADD     HL,BC
50     LD      A,(HL)
51     LD      HL,V_CX
52     ADD     A,(HL)
53     LD      L,A
54     LD      H,0
55     ADD     HL,DE
56     LD      DE,NORMEM
57     ADD     HL,DE
58     EX      DE,HL
59     RET
60
61 ; CELLDRW: paint pattern A at cell screen address HL (4 rows of 1
62 ; byte). Preserves HL, DE, BC.
63 CELLDRW: PUSH HL
64         PUSH DE
65         PUSH BC
66         ADD A,A
67         ADD A,A          ; *4
68         LD E,A
69         LD D,0
70         PUSH HL
71         LD HL,CELLPAT
72         ADD HL,DE
73         EX DE,HL        ; DE = the pattern rows
74         POP HL
75         LD B,4
76 CDR1:   LD A,(DE)
77         LD (HL),A
78         INC DE
79         LD A,L
80         ADD A,BYTEPL
81         LD L,A
82         JR NC,CDR2
83         INC H
84 CDR2:   DJNZ CDR1
85         POP BC
86         POP DE
87         POP HL
88         RET
89
90 ; CELLST: wire cell byte -> pattern index. The wire says 0-2, but this
91 ; indexes a table: clamp everything else to sea.
92 CELLST: CP 1
93         JR Z,CST1
94         CP 2
95         JR Z,CST2
96         LD A,PWATER
97         RET
98 CST1:   LD A,PHIT
99         RET
100 CST2:   LD A,PMISS
101         RET
102
103 ; PLBASE: HL = wire offset of player record A (GOPLYRS + A*115). Only
104 ; valid once the gamefields exist (status >= 10). Clobbers B,DE.
105 PLBASE: LD HL,GOPLYRS
106         OR A
107         RET Z
108         LD B,A
109         LD DE,PLSTRID
110 PLB1:   ADD HL,DE
111         DJNZ PLB1
112
113
114 ; MUL10: A -> A*10 (callers keep A <= 25). Clobbers C.
115 MUL10: LD C,A
116         ADD A,A
117         ADD A,A          ; *4
118         ADD A,C          ; *5
119         ADD A,A          ; *10
120         RET
121
122 ; DIV10: A -> B = A/10, A = A mod 10.
123 DIV10: LD B,0
124 DIV1:   CP 10
125         RET C
126         SUB 10
127         INC B
128         JR DIV1
129
130 ; -----
131 ; QUADRRW: quadrant A's 100 cells from player A's wire gamefield,
132 ; walked forward so the slice window flips at most three times.
133 QUADRRW: LD (V_QD),A
134         CALL PLBASE
135         LD DE,PLFIELD
136         ADD HL,DE        ; HL = wire offset of cell 0
137         XOR A
138         LD (V_CY),A
139 QDRY:   XOR A
140         LD (V_CX),A
141         PUSH HL
142         LD A,(V_QD)
143         CALL BRDPOS      ; DE = screen address of the row start
144         POP HL
145         LD B,10
146 QDRX:   CALL RXGETB      ; preserves HL,BC,DE
147         CALL CELLST
148         LD DE,HL
149         CALL CELLDRW
150         EX DE,HL
151         INC HL            ; next wire cell
152         INC DE            ; next cell is the next byte right
153         DJNZ QDRX
154         LD A,(V_CY)
155         INC A
156         LD (V_CY),A
157         CP 10
158         JR C,QDRY
159         RET
160
161 ; -----
162 ; SHIPSET: overlay the 5-ship set at wire offset DE on quadrant A.
163 ; C = the pattern to draw, or 0FFH for "your own gamefield decides": a
164 ; hit cell shows PSHPHT inside the hull. Each ship byte is
165 ; pos + 100*dir; a byte of 0 is a real carrier at the top-left, not an
166 ; empty slot, so callers only come here when the set is known valid --
167 ; and a byte that decodes off the board is dropped, because it indexes
168 ; the screen. Resets the placement cursor cells (never live at once).
169 SHIPSET: LD (V_QD),A
170         LD (V_BASE),DE    ; wire offset of ship[0]
171         LD A,C
172         LD (V_ST),A      ; the pattern mode
173         XOR A
174         LD (PCNUM),A

```

```

175 SHS1: LD A,(PCNUM)
176 CP 5
177 RET NC
178 LD HL,(V_BASE) ; the ship byte
179 LD E,A
180 LD D,0
181 ADD HL,DE
182 CALL RXGETB
183 LD D,0
184 CP 100 ; decode the direction
185 JR C,SHS2
186 SUB 100
187 INC D
188 SHS2: CP 100
189 JR NC,SHSN ; junk byte: drop the whole ship
190 LD E,A
191 LD A,D
192 LD (PCDIR),A
193 LD A,E
194 CALL DIV10 ; B = y, A = x
195 LD (PCX),A
196 LD A,B
197 LD (PCY),A
198 LD A,(PCNUM) ; the size
199 HL,SHPSIZ
200 LD E,A
201 LD D,0
202 ADD HL,DE
203 LD A,(HL)
204 LD (PCSIZ),A
205 SHSC: LD A,(PCSIZ) ; per cell, walking the ship
206 OR A
207 JR Z,SHSN
208 DEC A
209 LD (PCSIZ),A
210 LD A,(PCX)
211 CP 10
212 JR NC,SHSN ; ran off the board: junk, stop here
213 LD (V_CX),A
214 LD A,(PCY)
215 CP 10
216 JR NC,SHSN
217 LD (V_CY),A
218 LD A,(V_ST) ; the pattern
219 CP 0FFH
220 JR NZ,SHSP
221 LD A,(PCY) ; your own field: damaged or whole?
222 CALL MUL10
223 LD HL,PCX
224 ADD A,(HL)
225 LD L,A
226 LD H,0
227 LD DE,GOPLYRS+PLFIELD
228 ADD HL,DE
229 CALL RXGETB
230 CP 1
231 LD A,PSHPHT
232 JR Z,SHSP
233 LD A,PSHIP
234 SHSP: PUSH AF
235 LD A,(V_QD)
236 CALL BRDPOS ; DE = screen address
237 POP AF
238 EX DE,HL
239 CALL CELLDRAW
240 LD A,(PCDIR) ; advance along the ship
241 OR A
242 LD HL,PCX
243 JR Z,SHSA
244 LD HL,PCY
245 SHSA: INC (HL)

```

```

246 JR SHSC
247 SHSN: LD A,(PCNUM)
248 INC A
249 LD (PCNUM),A
250 JP SHS1
251
252 ; -----
253 ; BRDSTAT: the board dividers -- the only game-screen statics.
254 BRDSTAT: LD HL,NORMEM+10 ; vertical, byte column 10, lines 0-83
255 B,84
256 LD C,1
257 LD D,0FFH
258 CALL FILLRC
259 LD HL,NORMEM+40+BYTEPL ; horizontal, lines 40-43, bytes 0-20
260 B,4
261 LD C,21
262 LD D,0FFH
263 JP FILLRC
264
265 ; -----
266 QDROW: DB 44,0,0,44 ; quadrant top line
267 QDCOL: DB 0,0,11,11 ; quadrant left byte column
268 QTAG: DB "BLTLTRBR" ; panel tags, 2 chars per quadrant
269 SHPSIZ: DB 5,4,3,3,2 ; fixed order; the index is also the
; shipsLeft / myShips slot
270
271 ; Cell patterns: 4 rows x 1 byte each, 2bpp pixels left to right in
272 ; bits 7-6 .. 1-0 (00 sea, 01 black, 10 red, 11 white).
273 CELLPAT: DB 000H,000H,000H,000H ; WATER .... flat sea
274 DB 0AAH,0AAH,0AAH,0AAH ; HIT rrrr solid red
275 DB 000H,03CH,03CH,000H ; MISS .ww. splash dot
276 DB 0FFH,0FFH,0FFH,0FFH ; SHIP wwww solid hull
277 DB 0FFH,0EBH,0EBH,0FFH ; SHIPHIT wrwr red core in hull
278 DB 0AAH,0BEH,0BEH,0AAH ; SUNK rwrw white core in red
279 DB 0FFH,0C3H,0C3H,0FFH ; CURSOR w..w ring: targeting
280 ; and the pending placement
281
282

```

NAMENT.INC

```

1 ; nament.inc -- the name-entry screen.
2 ;
3 ; Eight slots, edited with the stick (or the keypad's arrow keys): up and
4 ; down turn the character wheel A-2 0-9 then '-', left and right move the
5 ; cursor, trigger accepts. '-' is the blank -- the wheel has no space, so
6 ; a name can never need the '+' URL escape -- and everything from the
7 ; first '-' on is dropped at accept. An all-blank name keeps the default.
8
9 NAMESCR: CALL KEYRAW          ; no leaking presses from the OS menu
10 LD      (V_KEY),A
11 CALL    CLRSCR
12 SYSSUK  STRDIS
13 DB      0                      ; "BATTLESHP": 10 chars x 16px = the row
14 DB      0
15 DB      4CH
16 DW      TTITLE
17 LD      HL,TNAME
18 LD      DE,6*256+15
19 LD      C,XWONF
20 CALL    TXTAT
21 LD      HL,TACCP
22 LD      DE,12*256+5
23 LD      C,XWONF
24 CALL    TXTAT
25 LD      HL,PLNBUF          ; edit buffer = name padded with blanks
26 LD      DE,NAMEED
27 LD      B,8
28 NEINI:  LD      A,(HL)
29 OR      A
30 JR      Z,NEINI2
31 LD      (DE),A
32 INC     HL
33 INC     DE
34 DJNZ   NEINI
35 JR      NEINI9
36 NEINI2: LD      A,'-'
37 NEINI3: LD      (DE),A
38 INC     DE
39 DJNZ   NEINI3
40 NEINI9: XOR     A
41 LD      (V_NCUR),A
42 CALL    NEDRAW
43 NEKEY:  CALL    KEYSCN
44 OR      A
45 JR      Z,NEKEY
46 CP      'U'
47 JR      NZ,NEK1
48 LD      E,1
49 JR      NEWHL
50 NEK1:   CP      'D'
51 JR      NZ,NEK2
52 LD      E,36
53 JR      NEWHL
54 NEK2:   CP      'L'
55 JR      NZ,NEK3
56 LD      A,(V_NCUR)
57 OR      A
58 JR      Z,NEKEY
59 DEC     A
60 LD      (V_NCUR),A
61 JR      NEDRW
62 NEK3:   CP      'R'
63 JR      NZ,NEK4
64 LD      A,(V_NCUR)
65 CP      A
66 JR      NC,NEKEY
67 INC     A
68 LD      (V_NCUR),A
69 JR      NEDRW
70 NEK4:   CP      'T'
71 JR      NZ,NEKEY
72 LD      HL,NAMEED
73 LD      DE,PLNBUF
74 LD      B,8
75 LD      A,(HL)
76 NEACC:  CP      A
77 JR      Z,NEACC9
78 LD      (DE),A
79 INC     HL
80 INC     DE
81 DJNZ   NEACC
82 NEACC9: LD      A,E
83 CP      PLNBUF-4E00H
84 JR      Z,NEKEY
85 XOR     A
86 LD      (DE),A
87 JP      SNDSEL
88
89 NEWHL:  LD      A,(V_NCUR)
90 LD      HL,NAMEED
91 PUSH    DE
92 LD      E,A
93 LD      D,0
94 ADD     HL,DE
95 POP     DE
96 LD      A,(HL)
97 PUSH    HL
98 LD      HL,WHEEL
99 LD      B,0
100 NEWH1: CP      (HL)
101 JR      Z,NEWH2
102 INC     B
103 INC     HL
104 LD      A,B
105 NEWH2: LD      (HL),A
106 ADD     A,E
107 NEWH3: CP      A
108 JR      C,NEWH4
109 SUB     37
110 JR      NEWH3
111 NEWH4: LD      HL,WHEEL
112 LD      E,A
113 LD      D,0
114 ADD     HL,DE
115 LD      A,(HL)
116 POP     HL
117 LD      (HL),A
118 NEDRW: CALL    SNDKEY
119 CALL    NEDRAW
120 JP      NEKEY
121
122 ; NEDRAW: the eight slots at row 8, cols 16-23, cursor cell inverse.
123 NEDRAW: XOR     A
124 LD      (V_TMP),A
125 NED1:   LD      A,(V_TMP)
126 CP      8
127 RET     NC
128 LD      HL,NAMEED
129 LD      E,A
130 LD      D,0
131 ADD     HL,DE
132 LD      A,(HL)

```

```

133     LD      (LINBUF),A
134     XOR     A
135     LD      (LINBUF+1),A
136     LD      A,(V_TMP)
137     LD      HL,V_NCUR
138     CP      (HL)
139     LD      C,XFONW
140     JR      Z,NED2
141     LD      C,XWONF
142     LD      A,(V_TMP)
143     ADD     A,16
144     LD      E,A
145     LD      D,8
146     LD      HL,LINBUF
147     LD      B,1
148     CALL    TXTN
149     LD      A,(V_TMP)
150     INC     A
151     LD      (V_TMP),A
152     JR      NED1
153
154 WHEEL: DB      "ABCDEFGH IJKLMN OPQRST UVWXYZ0123456789--"
155 TNAME: DB      "YOUR NAME",0
156 TACAPT: DB      "STICK EDITS - TRIGGER ACCEPTS",0
157

```

SOUND.INC

```

1 ; sound.inc -- cues, written straight to the music processor.
2 ;
3 ; Tone A only: set the master oscillator and tone A period, open the
4 ; volume, busy-wait, close it. Blocking is fine here -- every caller is
5 ; already in an input loop, and the longest cue is under half a second.
6
7 TONEA EQU 11H ; tone A period (TONMO+1)
8
9 ; BEEP: A = tone A period (higher = lower pitch), B = duration, ~10 ms
10 ; units. Clobbers A,B,DE.
11 BEEP: PUSH BC
12     LD C,A
13     LD A,20H
14     OUT (TONMO),A
15     LD A,C
16     OUT (TONEA),A
17     LD A,88H
18     OUT (VOLAB),A
19     POP BC
20 BEEPW: LD DE,600
21 BEEPD: DEC DE
22     LD A,D
23     OR E
24     JR NZ,BEEPD
25     DJNZ BEEPW
26     XOR A
27     OUT (VOLAB),A
28     RET
29
30 SNDKEY: LD A,18H ; key click
31     LD B,2
32     JP BEEP
33
34 SNDSEL: LD A,28H ; select / join / fire sent
35     LD B,5
36     JP BEEP
37
38 SNDTRN: LD A,30H ; your turn: low then high
39     LD B,6
40     CALL BEEP

```

```

41     LD A,18H
42     LD B,8
43     JP BEEP
44
45 SNDMISS: LD A,50H ; a dull splash
46     LD B,6
47     JP BEEP
48
49 SNDHIT: LD A,40H ; impact
50     LD B,4
51     CALL BEEP
52     LD A,58H
53     LD B,6
54     JP BEEP
55
56 SndsUNK: LD A,28H ; a descending triple: down she goes
57     LD B,4
58     CALL BEEP
59     LD A,40H
60     LD B,4
61     CALL BEEP
62     LD A,60H
63     LD B,8
64     JP BEEP
65
66 SndBAD: LD A,70H ; the overlap buzz
67     LD B,5
68     JP BEEP
69
70 SndPLC: LD A,20H ; ship confirmed
71     LD B,3
72     JP BEEP
73
74 SndTICK: LD A,10H ; the last-seconds tick
75     LD B,1
76     JP BEEP
77
78 SndOVER: LD A,40H ; game over
79     LD B,5
80     CALL BEEP

```

```

81      LD      A,30H
82      LD      B,5
83      CALL    BEEP
84      LD      A,20H
85      LD      B,10
86      JP      BEEP
87

```

RND.INC

```

1 ; rnd.inc -- a 16-bit Galois LFSR for the placement seeds.
2
3 ; The state is a pure maximal LFSR (poly mask 2DH, period 65535),
4 ; stepped a full 8 bits per call so consecutive outputs are independent
5 ; PRNG bytes. The R register is folded into the OUTPUT only, never the
6 ; state: R advances with every opcode fetch, so it varies with input
7 ; and network timing and makes runs differ -- but XORing it into the
8 ; state (the first cut here) correlated consecutive calls whenever the
9 ; code path between them was fixed, and every placement seeded dir=0,
10 ; x=0. State out, output in.
11
12 ; RNDNXT: A = the next pseudo-random byte. Clobbers B,HL.
13 RNDNXT: LD    HL,(RNDST)
14          LD    B,8
15 RNDST:  ADD    HL,HL          ; one LFSR bit: shift left, tap on carry
16          JR    NC,RNDST2
17          LD    A,L
18          XOR    2DH          ; maximal 16-bit polynomial
19          LD    L,A
20 RNDST2: DJNZ   RNDST
21          LD    A,H
22          OR     L
23          JR    NZ,RNDST3
24          INC    L          ; never park on zero
25 RNDST3: LD     (RNDST),HL
26          LD    A,R
27          XOR    L
28          RET
29
30 ; RNDMOD: A = a random value 0 .. A-1 (A >= 1). Clobbers B,C,HL.
31 RNDMOD: LD    C,A
32          CALL   RNDNXT
33 RNDM1:  SUB    C
34          JR    NC,RNDM1
35          ADD    A,C
36          RET
37

```

C.6 FUJITZEE

FUJITZEE.ASM

```

1 ; fujitzee.asm -- Fujitzee for the Bally Astrocade, over FujiNet.
2
3 ; A standalone-assembly client in the battleship/astrocade mold: same
4 ; mailbox cartridge, same 4x6 renderer, same one-poll-loop shape. The
5 ; gameplay logic is transcribed from the Intellivision port (intv/), the
6 ; freshest client written directly against the binary wire format; the
7 ; screen plan traces the Intellivision's 20-column squeeze for this
8 ; machine's 40 columns: the compact two-column scorecard keeps its shape
9 ; and the reclaimed right half becomes an always-visible standings panel,
10 ; so the seat strip and the hold-to-view standings overlay disappear.
11 ;
12 ; The server owns the whole game: every endpoint returns the same state
13 ; blob, validScores[] carries the exact points each open row would earn,
14 ; and the client's only arithmetic is the running total (the wire's grand
15 ; total is empty until game over). You are always player index 0.
16 ;
17 ; ROM budget: 0000H-1AFFH of the 8K window (6,912 bytes); 1B00H+ belongs
18 ; to the mailbox and build.sh stamps the "FUJI" claim at 1CFH so the
19 ; mailbox stays alive when this image is booted over the network. RAM is
20 ; screen RAM alone: 90 visible lines use 4000H-4E0FH and everything above
21 ; is ours.
22 ;

```

```

23 ; Interrupts stay off for the program's whole life (fujilib.inc's
24 contract: with I = 0, refresh strays land in OS ROM and never hit the
25 hotspots).
26
27 INCLUDE "HVGLIB.H"
28 INCLUDE "fujinet.inc"
29 INCLUDE "build/flags.inc"
30
31 ; --- RAM map ---
32 LINES EQU 90 ; 15 rows of 4x6 text
33
34 ; What the last poll established. Rendering is opaque, so these are not a
35 ; dirty list -- they exist only for the things overdraw cannot fix:
36 ; content that SHRANK (a class or seat-count change clears the screen)
37 ; and edges arriving, the game ending, a roll landing, the game ending.
38 PRVCLS EQU 4E10H ; previous screen class (FF forces statics)
39 V_NEWS EQU 4E11H ; the statics were redrawn this poll
40 PRVACT EQU 4E12H ; previous activePlayer (the turn edge)
41 PRVPC EQU 4E13H ; previous playerCount
42 PRVRLS EQU 4E14H ; previous rollsLeft (FF = sentinel: no
43 ; roll edge across a screen boundary)
44 PENDACT EQU 4E15H ; staged action: 0 none, 1 ready, 2 roll,
45 ; 3 score -- rides the next poll
46 SCOREIX EQU 4E16H ; staged /score category, 0-14
47 KEEPBF EQU 4E17H ; staged roll mask, 5 ascii '0'/'1' + NUL
48 ; ('!' = re-roll); streamed verbatim
49 UIMODE EQU 4E1DH ; my-turn input mode: 0 dice, 1 score
50 DCUR EQU 4E1EH ; dice cursor 0-5 (0 = the ROLL tile)
51 SCUR EQU 4E1FH ; score cursor = category index 0-5, 8-14
52 VIEWIDX EQU 4E20H ; whose scorecard is shown
53 TGLEFT EQU 4E21H ; local countdown, reseeded each poll
54 TGFRM EQU 4E22H ; ~20 ms ticks toward the next second
55 RNDST EQU 4E23H ; LFSR state, 2 bytes
56 V_CLS EQU 4E25H ; this poll's class (0 lobby, 1 play,
57 ; 2 over)
58 V_TRNF EQU 4E26H ; turn-changed edge, this poll
59 V_MYTRN EQU 4E27H ; my turn and not a spectator, this poll
60 V_FLAG EQU 4E28H ; turn-input outcome: 0 slice out, 1 roll,
61 ; 2 score, 3 leave, 4 help
62 V_ST EQU 4E29H ; scratch
63 V_TOT EQU 4E2AH ; running-total scratch, word
64 V_WIN EQU 4E2CH ; gameover: the leader's seat
65 V_CAT EQU 4E2DH ; scorecard loop: category being drawn
66 V_FACE EQU 4E2EH ; dice loop scratch
67 V_TM2 EQU 4E2FH ; inner loop index
68
69 ; Session state.
70 PLNBUF EQU 4E30H ; player name, 9 + NUL (must stay in 4Exx:
71 ; nament tests its low byte)
72 TBLBUF EQU 4E3AH ; table id, 9 + NUL
73 CURSLC EQU 4E46H ; reply slice the cart is publishing
74 AVAIL EQU 4E48H ; NET_STATUS bytes waiting, u16
75 PRVAVL EQU 4E4AH ; previous reading, for the settle loop
76 RXLEN EQU 4E4CH ; reply length, captured after the READ
77 V_SEL EQU 4E4EH ; list cursor (tables)
78 V_CNT EQU 4E4FH ; list entry count
79 V_KEY EQU 4E50H ; last keypad/handle scan state (edges)
80 V_URL EQU 4E58H ; request being built (0 tables, 1 state,
81 ; 2 ready, 3 roll, 4 score, 5 leave)
82 V_TMP EQU 4E59H ; loop index (player being rendered)
83 V_ACT EQU 4E5AH ; this poll's activePlayer
84 V_PC EQU 4E5BH ; this poll's playerCount
85 V_NCUR EQU 4E5CH ; name-entry cursor
86 V_BASE EQU 4E5EH ; player record base offset (word)
87 TBLNAM EQU 4E60H ; joined table's display name, 21 + NUL
88 NAMED EQU 4E76H ; name-entry edit buffer, 8 slots
89 BSTTOT EQU 4E80H ; gameover: the leading total, word
90
91 LINBUF EQU 4F00H ; display line being built; page-aligned
92 ; (UI6STR's empty test reads the low byte)
93 STACK EQU 4FC0H ; grows down; 4FC0H+ left to the BIOS cells

```

```

94
95 ; --- Display options (BIOS STRDIS, titles only) -----
96 OPTFB EQU 0CH ; fg color 3, bg color 0
97
98 ; MB_* labels are module fences for tools/checksize.py's budget table.
99 ORG FIRSTC
100 MB_MAIN:
101 DB 55H
102 DW MENUST
103 DW PRGNAM
104 DW PRGSTR
105 PRGNAM: DB "FUJITZEE"
106 DB 0
107
108 PRGSTR: DI
109 LD SP,STACK
110 SYSTEM INTPC
111 DO SETOUT
112 DB LINES*2
113 DB 8 ; HORCB 0: the whole line is the right palette
114 DO COLSET
115 DW PALET
116 DO FILL
117 DW NORMEM
118 DW LINES*BYTEPL
119 DB 0 ; color 0 everywhere: the felt
120 DO STRDIS
121 DB 48 ; "FUJITZEE", 8 chars x 8px, centred
122 DB 38
123 DB OPTFB
124 DW TTITLE
125 EXIT
126
127 CALL FNCHECK
128 JP NZ,NOCARD
129
130 IF
131 CALL DEMO
132 JR HALT0 ; the static mock table (M1)
133 ELSE
134 LD HL,DEFNAM ; player name until appkeys land
135 LD DE,PLNBUF
136 LD BC,6
137 LDIR
138
139 XOR A
140 LD (PENDACT),A
141 LD (V_KEY),A
142 LD HL,1
143 LD (RNDST),HL ; LFSR seed; RNDNXT mixes R in per call
144 CALL NAMESCR
145 JP TBLSCR ; pick a name, then choose a table
146 ENDIF
147
148 ; --- Errors -----
149 NOCARD: SYSSUK STRDIS
150 DB 20
151 DB 56
152 DB OPTFB
153 DW ENOCART
154 HALTE: JR HALTE
155
156 ; --- Data -----
157 MB_DATA:
158 TITLE: DB "FUJITZEE",0
159 ENET: DB "NET ERR - ANY KEY RETRIES",0
160 ENOCART: DB "NO FUJINET CART",0
161 DEFNAM: DB "ASTRO",0
162
163 ; COLSET stores descending, ports 7 down to 0; both halves identical since
164 ; HORCB is 0. Right palette: 3 = white, 2 = red, 1 = black, 0 = felt green

```

```

165 ; (the texasHoldEm table color: hue 20, luma 0).
166 ; Byte = (hue < 3) | luminance; hue 0 is the grayscale column.
167 PALET: DB 07H,52H,00H,0A0H
168 DB 07H,52H,00H,0A0H
169
170 INCLUDE "build/endpoint.inc"
171
172 IF DEMO
173 MB_DICE: INCLUDE "dice.inc"
174
175 MB_CARDST: INCLUDE "cardst.inc"
176
177 MB_DEMO: INCLUDE "demo.inc"
178
179 MB_SOUND: INCLUDE "sound.inc"
180
181 MB_INPUT: INCLUDE "input.inc"
182
183 MB_STATE: INCLUDE "state.inc"
184
185 MB_GFX: INCLUDE "gfx.inc"
186
187 MB_RND: INCLUDE "rnd.inc"
188
189 ELSE
190 MB_SCREENS: INCLUDE "screens.inc"
191
192 MB_CARD: INCLUDE "card.inc"
193
194 MB_CARDST: INCLUDE "cardst.inc"
195
196 MB_TURN: INCLUDE "turn.inc"
197
198 MB_DICE: INCLUDE "dice.inc"
199
200 MB_NAMENT: INCLUDE "nament.inc"
201
202 MB_SOUND: INCLUDE "sound.inc"
203
204 MB_INPUT: INCLUDE "input.inc"
205
206 MB_NET: INCLUDE "net.inc"
207
208 MB_URL: INCLUDE "url.inc"
209
210 MB_STATE: INCLUDE "state.inc"
211
212 MB_GFX: INCLUDE "gfx.inc"
213
214 MB_RND: INCLUDE "rnd.inc"
215
216 ENDF
217 MB_FONT: INCLUDE "assets/font.inc"
218
219 MB_FUJILIB: INCLUDE "fujilib.inc"
220
221 MB_END:
222

```

FUJINET.INC -- THE WIRE FORMAT

```

1 ; fujinet.inc -- N: device commands and the Fujitzee wire format.
2 ;
3 ; The N: command values are hand-synchronised with the battleship port
4 ; (whose o2/ family carries the documentation of WHY each is what it is).
5 ; The wire offsets are verified against the shared C client's packed
6 ; structs (src/misc.h) and the Intellivision client's byte-exact map
7 ; (intv/state.bas). The mailbox transport itself is fujilib.inc.

```

```

8
9 ; --- Network device (N:) -----
10 NETDEV EQU 71H ; N1:
11 NCCOPEN EQU 4FH ; 'O'
12 NCCLOSE EQU 43H ; 'C'
13 NCREAD EQU 52H ; 'R'
14 NCSTAT EQU 53H ; 'S'

```



```

15
16 NMHTTGP EQU    0CH      ; open mode: HTTP GET
17 NTRNONE EQU    0        ; translation: none
18
19 ; NET_STATUS reply: 4 bytes in slice 0.
20 NSAVLO EQU      0        ; bytes waiting, low
21 NSAVHI EQU      1        ; bytes waiting, high
22 NSCONN EQU      2        ; connected flag
23 NSDEVST EQU      3        ; nDevStatus_t; 1 = SUCCESS. Checking this is
24                          ; not optional; the GET is deferred until the
25                          ; first STATUS, and an HTTP error page still
26                          ; has a readable body.
27
28 ; ---- Wire format (?bin=1) -----
29 ; /state, /ready, /oll/..., /score/..., /leave -> the same state blob;
30 ; /tables -> Tables. Offsets are read in place out of the reply slice
31 ; window; nothing is buffered. ALL strings are lowercased, NUL-padded,
32 ; one byte longer than their content width.
33
34 ; Header, always present (95 bytes):
35 GPC EQU         0        ; playerCount (spectators appended)
36 GOSRVNM EQU     1        ; serverName[21]
37 GPRMPT EQU      22       ; prompt[41]
38 GORND EQU       63       ; round: 0 lobby, 1-13 play, 99 gameover
39 GORLEFT EQU     64       ; rollsLeft this turn (3 down to 0)
40 GOACTPL EQU     65       ; signed; FFH = -1 nobody, 0 = your turn
41                          ; (the server rotates you to index 0)
42 GOMVTIM EQU     66       ; seconds left, 0-250
43 GOVIEW EQU      67       ; 1 = YOU are a spectator: input disabled
44 GODOCE EQU      68       ; 5 ascii '1'-'6' + NUL; NULs at turn start
45 GOKEEP EQU      74       ; 5 ascii '0'/'1' + NUL; '1' = re-roll
46 GOVALID EQU     80       ; validScores[15], signed bytes: FFH = -1
47                          ; closed row, else the points that row
48                          ; would earn. ALL ZERO in the lobby --
49                          ; only rounds 1-13 may read it.
50 GOPLYRS EQU     95       ; playerCount x 42-byte records
51 PLSTRID EQU     42
52 PLNAME EQU      0        ; name[9]
53 PLALIAS EQU     9        ; index of the highlight char in name
54 PLSORS EQU      10       ; scores[16], u16 LE each: FFFFH = -1
55                          ; unset, FFFEH = -2 spectator. scores[0]
56                          ; doubles as the lobby ready flag.
57
58 ; Score slots. 0-5 ones..sixes and 8-14 are the 13 selectable
59 ; categories; 6, 7 and 15 are server-computed and never selectable.
60 SCUPTOT EQU     6        ; upper total
61 SCBONUS EQU     7        ; upper bonus (35 at 63+)
62 SCSET3 EQU      8        ;
63 SCFUJI EQU     14        ; five of a kind
64 SCGRAND EQU    15        ; grand total -- filled only at gameover
65
66 ; Round values.
67 RNLOBBY EQU     0
68 RNFINAL EQU     13
69 RNOVER EQU      99
70
71 ; /tables (?bin=1) -- byte-identical to the card games' Tables struct.
72 TBSTRID EQU     36
73 TBID EQU        0        ; table[9]
74 TBNAME EQU      9        ; name[21]
75 TBPLYRS EQU     30       ; players[6], literal "cur / max"
76
77 GMAXLEN EQU     599       ; 95 + 12*42; the NREAD clamp, and it fits
78                          ; the cart's four 256-byte reply slices
79 GMTINLEN EQU    95       ; the fixed header
80 TMAXLEN EQU     361
81 PCMAX EQU       12       ; the serializer's seat + spectator cap
82

```

NET.INC

```

1 ; net.inc -- the N: device round trip.
2
3 ; A Z80 transcription of the card-game ports' proven sequence, over
4 ; fujilib's mailbox. The rules it carries forward (learned the hard way
5 ; there and on the Intellivision):
6
7 * The CLOSE belongs at the START of the next request, never after the
8 ; READ: every transaction repaints the whole reply window, a CLOSE
9 ; reply is empty, and every screen renders straight out of the window
10 ; between polls.
11 * FNRXLX/FNRXLH belong to the most recent transaction, so the reply
12 ; length is captured immediately after the READ.
13 * The ESP32 reports STATUS as soon as SOME of the response has
14 ; arrived; poll until two consecutive readings agree (bounded) rather
15 ; than guessing a delay.
16 * The 4th STATUS reply byte is nDevStatus.t and checking it is not
17 ; optional: the HTTP GET is deferred until the first STATUS, and an
18 ; HTTP error page still has a readable body.
19 * Validate the reply length against what playerCount and the round
20 ; imply before believing a byte of it: FN_RX is never cleared, so a
21 ; short read leaves stale bytes from a bigger previous reply.
22 ; All routines return carry SET on failure, clear on success.
23
24 ; NCMD: begin a network-device transaction. E = command, L = nparam.
25 NCMD: LD A,NETDEV
26 JP FNBEGIN
27
28 ; NCLOSE: close the previous request's connection. A first call has
29 ; nothing to close and earns a NAK; both results are ignored.
30 NCLOSE: LD E,NCLOSE
31 LD L,0
32 CALL NCMD
33 LD B,3 ; -6 s
34 JP FNCOMMIT
35
36 ; NOPEN: OPEN the devicespec (V_URL) as HTTP GET, no translation.
37 NOPEN: LD E,NOPEN
38 LD L,2
39 CALL NCMD
40 LD A,NMHTTPG
41 CALL FNPARB
42 LD A,NTRNONE
43 CALL FNPARB
44 CALL BLDURL
45 LD B,8 ; HTTPS handshakes take a while
46 CALL FNCOMMIT
47 RET
48
49 JR NZ,NOBAD
50 LD A,(FNREPLY)
51 CP FCACK
52 RET Z ; equal: carry already clear
53 NOBAD: SCF
54 RET
55
56 ; NSTATUS: one STATUS poll. On success AVAIL = bytes waiting.
57 NSTATUS: LD E,NCSTAT
58 LD L,2
59 CALL NCMD
60 XOR A
61 CALL FNPARB
62 XOR A
63 CALL FNPARB
64 LD B,4
65 CALL FNCOMMIT
66 RET C
67 JR NZ,NSBAD
68
69 LD A,(FNREPLY)
70 CP FCACK
71 JR NZ,NSBAD
72 LD A,(FNRRDATA+NSDEVST)
73 CP 1 ; SUCCESS -- anything else is an HTTP error
74 JR NZ,NSBAD
75 LD A,(FNRRDATA+NSAVLO)
76 LD A,(AVAIL),A
77 LD A,(FNRRDATA+NSAVHI)
78 LD A,(AVAIL+1),A ; clear carry
79 OR A
80 RET
81 NSBAD: SCF
82 RET
83 ; NREAD: READ min(AVAIL, GMAXLEN) bytes into the reply window, then
84 ; capture the reply length and re-home the slice cursor.
85 NREAD: LD HL,(AVAIL)
86 LD DE,GMAXLEN
87 OR A
88 SBC HL,DE
89 JR C,NRD1 ; avail < clamp: take avail
90 LD H,D
91 LD L,E
92 JR NRD2
93 ADD HL,DE
94 PUSH HL
95 LD E,NCREAD
96 LD L,1
97 CALL NCMD
98 LD A,2 ; one 2-byte parameter: the length, LE
99 CALL FNTXBYT
100 POP HL
101 PUSH HL
102 LD A,L
103 CALL FNTXBYT
104 POP HL
105 LD A,H
106 CALL FNTXBYT
107 LD B,8
108 CALL FNCOMMIT
109 RET C
110 JR NZ,NRBD
111 LD A,(FNREPLY)
112 CP FCACK
113 JR NZ,NRBD
114 LD A,(FNRXLX) ; capture NOW: the next transaction's
115 LD A,(RXLEN),A ; reply length would replace it
116 LD A,(FNRXLH)
117 LD A,(RXLEN+1),A
118 XOR A
119 LD A,(CURSLC),A ; the cart republished slice 0
120 RET ; carry clear (XOR A)
121 NRBD: SCF
122 RET
123
124 ;-----
125 ; APICALL: A = request (0 tables, 1 state, 2 ready, 3 roll, 4 score,
126 ; 5 leave). One full round trip; on success (carry clear) the reply is in
127 ; the window with the slice cursor homed, and stays valid until the next
128 ; APICALL.
129 APICALL: LD A,(V_URL),A
130 CALL NCLOSE ; the PREVIOUS request's connection
131 CALL NOPEN
132 RET C
133 XOR A
134 LD A,(PRVAVL),A

```

```

135     LD      (PRVAVL+1),A
136     LD      B,20          ; settle-loop bound
137 APSET:  PUSH BC
138     CALL   NSTATUS
139     POP    BC
140     JR     C,APCFL
141     LD     HL,(AVAIL)
142     LD     A,H
143     OR     L
144     JR     Z,APAGN        ; nothing has arrived at all yet
145     LD     DE,(PRVAVL)
146     OR     A
147     SBC    HL,DE
148     JR     Z,APRD         ; two readings agree: it has landed
149 APAGN:  LD     HL,(AVAIL)
150     LD     (PRVAVL),HL
151     LD     DE,4000        ; ~3 frames between polls
152 APDLY:  DEC    DE
153     LD     A,D
154     OR     E
155     JR     NZ,APDLY
156     DJNZ   APSET
157 APCFL:  CALL   NCLOSE      ; failed before the READ: nothing to keep
158     SCF
159     RET
160 APRD:   CALL   NREAD
161     JR     C,APCFL
162 ; Validation failure after the READ returns carry WITHOUT a close -- the
163 ; next APICALL's leading NCLOSE retires the connection, and closing here
164 ; would wipe the window a caller may still be rendering the last good
165 ; reply from.
166 ; falls through into VALID8
167
168 ; VALID8: sanity-check the reply for the request in V_URL, transcribed
169 ; from the Intellivision port's validate_state. Tables: nonempty. State:
170 ; at least the fixed header, playerCount <= 12, the round on its enum
171 ; (0-13 or 99), and at least 95 + 42*playerCount bytes.
172 ; Minimum, not exact: a longer reply is harmless; a SHORTER one means
173 ; the fixed offsets would read stale bytes.
174 VALID8: LD     A,(V_URL)
175     OR     A
176     JR     NZ,VALGM
177     LD     HL,(RXLEN)      ; tables: any nonzero length; the count
178     LD     A,H            ; byte bounds the walk
179     OR     L
180     RET    NZ             ; OR cleared carry
181     SCF
182     RET
183 VALGM:  LD     HL,(RXLEN)
184     LD     DE,GMINLEN
185     OR     A
186     SBC    HL,DE
187     RET    C              ; shorter than the fixed header
188     LD     HL,GOPC
189     CALL   RXGETB
190     CP     PCMAX+1
191     JR     NC,VALBAD      ; more seats than the serializer allows
192     LD     C,A            ; C = playerCount
193     LD     HL,GORND
194     CALL   RXGETB
195     CP     RNOVER
196     JR     Z,VALS2
197     CP     RNFINAL+1
198     JR     NC,VALBAD
199 VALS2:  LD     HL,GMINLEN ; expected minimum = 95 + 42*pc
200     LD     DE,PLSTRID
201     LD     A,C
202     OR     A
203     JR     Z,VALS6
204     LD     B,C
205 VALSA:  ADD     HL,DE

```

```

206     DJNZ   VALSA
207 VALS6:  EX     DE,HL      ; DE = expected minimum
208     LD     HL,(RXLEN)
209     OR     A
210     SBC    HL,DE        ; carry set exactly when RXLEN < expected
211     RET
212 VALBAD: SCF
213     RET
214

```

URL.INC

```

1 ; url.inc -- building the devicespec for a request.
2
3 ; The URL is streamed straight into the mailbox's TX byte stream a
4 ; character at a time (the card-game ports' way): it is never assembled
5 ; in RAM. The cart counts the payload itself, so nothing here tracks a
6 ; length; the worst case (/score/14 with the query) is ~90 bytes against
7 ; the 320-byte TX ceiling.
8
9 ; The roll mask needs no conversion at all: KEEPBF is staged as the five
10 ; ascii '0'/'1' bytes the wire wants ('1' = re-roll), NUL-terminated, so
11 ; it streams through FNTXSTR verbatim. The score category is a raw byte
12 ; that BYTDEC turns into decimal digits on the way out.
13
14 ; TBLBUF and PLNBUF are NUL-terminated copies of fixed-width wire fields
15 ; -- pushed up to the NUL, never through it (padding in the middle of a
16 ; URL corrupts the request on the wire).
17
18 ; BLDURL: stream the devicespec for request (V_URL).
19 BLDURL: LD A,(UENDPT) ; "N:" unless the endpoint carries its own
20 CP 'N'
21 JR Z,BURL1
22 LD A,'N'
23 CALL FNTXBYT
24 LD A,':'
25 CALL FNTXBYT
26 BURL1: LD HL,UENDPT
27 CALL FNTXSTR
28 LD A,(V_URL)
29 OR A
30 JR NZ,BURL2
31 LD HL,UTABLS ; tables: no table, no player
32 JP FNTXSTR
33 BURL2: CP 1
34 JR NZ,BURL3
35 LD HL,USTATE
36 CALL FNTXSTR
37 JR BURLQ
38 BURL3: CP 2
39 JR NZ,BURL4
40 LD HL,UREADY ; bare /ready toggles, matching the
41 CALL FNTXSTR ; Intellivision client
42 JR BURLQ
43 BURL4: CP 3
44 JR NZ,BURL5
45 LD HL,UROLL ; roll/MMMM -- the staged mask verbatim
46 CALL FNTXSTR
47 LD HL,KEEPBF
48 CALL FNTXSTR
49 JR BURLQ
50 BURL5: CP 4
51 JR NZ,BURL6
52 LD HL,USCORE ; score/N
53 CALL FNTXSTR
54 LD A,(SCOREIX)
55 CALL BYTDEC
56 JR BURLQ
57 BURL6: LD HL,ULEAVE
58 CALL FNTXSTR
59 BURLQ: LD HL,UQTAB
60 CALL FNTXSTR
61 LD HL,TBLBUF
62 CALL FNTXSTR
63 LD HL,UQPLR
64 CALL FNTXSTR
65 LD HL,PLNBUF
66 CALL FNTXSTR
67 LD HL,UQBIN
68 JP FNTXSTR
69
70 ; BYTDEC: stream A as 1-3 decimal digits, no leading zeros. Rides on
71 ; U16STR (which builds in LINBUF); safe mid-transaction -- it touches
72 ; only RAM and registers, never the mailbox pages.
73 BYTDEC: LD L,A
74 LD H,0
75 CALL U16STR
76 LD HL,LINBUF
77 JP FNTXSTR
78
79 UTABLS: DB "tables?bin=1",0
80 USTATE: DB "state",0
81 UREADY: DB "ready",0
82 UROLL: DB "roll",0
83 USCORE: DB "score/",0
84 ULEAVE: DB "leave",0
85 UQTAB: DB "?table=",0
86 UQPLR: DB "&player=",0
87 UQBIN: DB "&bin=1",0
88

```

SCREENS.INC

```

1 ; screens.inc -- table selection, the poll loop, and the per-class
2 ; renderers.
3 ;
4 ; Rendering policy (the card-game ports!): every poll redraws everything,
5 ; opaquely, straight out of the reply window -- identical pixels
6 ; rewritten are invisible, so there is no flicker and no digest
7 ; bookkeeping. The one thing opaque overdraw cannot fix is content that
8 ; SHRANK, so CHKNEW watches the screen class (lobby / play / over) and
9 ; the seat count, and answers a change with a full clear + statics
10 ; before the ordinary render.
11 ;
12 ; The poll loop is the battleship GAMELP reshaped for Fujitzee: one
13 ; staged action (PENDACT) rides the next request in place of /state --
14 ; every endpoint returns the same state blob -- and the reply is
15 ; dispatched by class. The turn edge (V_TRNF) is computed from PRVACT at
16 ; the top of every successful poll, lobby included, so the lobby->play
17 ; transition is caught by the same comparison as any other turn change.
18 ; The roll edge rides PRVRLS the same way: rollLeft DECREASING means a
19 ; roll landed (a new turn resets it upward), and that is what triggers
20 ; the dice animation -- for your rolls and everyone else's alike.
21 ;
22 ; =====
23 ; Table selection
24 ; =====
25 TBLSCR: CALL KEYRAW ; a key still held from the previous
26 LD (V_KEY),A ; screen must not fire here
27 CALL CLRSCR
28 SYSSUK STRDIS ; title, BIOS font at 2x
29 DB 16 ; "FUJITZEE": 8 chars x 16px, centred
30 DB 0
31 DB 4CH
32 DW TTITLE
33 LD HL,TCH00S
34 DE,6*256+2
35 LD C,XWONF
36 CALL TXTAT
37 TBLRF: XOR A ; fetch /tables
38 CALL APICALL
39 JR NC,TBLOK
40 LD HL,ENET
41 LD DE,14*256+0
42 LD C,XRONF
43 CALL TXTAT
44 TBLRW: CALL KEYSCN
45 OR A
46 JR Z,TBLRW
47 JR TBLRF
48 TBLOK: LD HL,0
49 CALL RXGETB
50 OR A
51 JR Z,TBLRW
52 CP 9
53 JR C,TBOK2
54 LD A,8
55 TBOK2: LD (V_CNT),A
56 XOR A
57 LD (V_SEL),A
58 CALL TBDRW
59 TBKEY: CALL KEYSCN
60 OR A
61 JR Z,TBKEY
62 CP 'U'
63 JR NZ,TBK1
64 LD A,(V_SEL)
65 OR A
66 JR Z,TBKEY
67 DEC A
68 LD (V_SEL),A
69 CALL TBDRW
70 TBK1: JR TBKEY
71 CP 'D'
72 JR NZ,TBK2
73 LD A,(V_SEL)
74 INC A
75 LD HL,V_CNT
76 CP (HL)
77 JR NC,TBKEY
78 LD (V_SEL),A
79 CALL SNDKEY
80 CALL TBDRW
81 JR TBKEY
82 TBK2: CP 'E' ; CE: back to the name screen
83 JR NZ,TBK2A
84 CALL NAMESCR
85 JP TBLSCR
86 TBK2A: CP ',' ; the . key: how to play
87 JR NZ,TBK2B
88 CALL HELPSCR
89 JP TBLSCR
90 TBK2B: CP 'T'
91 JR Z,TBJOIN
92 CP '0' ; digit shortcut: 1-9 then 0 = row 10
93 JR C,TBKEY
94 CP '9'+1
95 JR NC,TBKEY
96 SUB '1'
97 CP 0FFH ; '0' wrapped to FF: row 10
98 JR NZ,TBK3
99 LD A,9
100 TBK3: LD HL,V_CNT
101 CP (HL)
102 JR NC,TBKEY
103 LD (V_SEL),A
104 ; falls into the join
105 TBJOIN: CALL SNDSEL
106 LD A,(V_SEL) ; record base = 1 + sel*36
107 CALL TBOFF
108 LD DE,TBLBUF ; id, up to its NUL
109 LD B,9
110 CALL RXSTRN
111 LD DE,TBID+TBNAME ; the name field, for the lobby header
112 ADD HL,DE
113 LD DE,TBLNAM
114 LD B,21
115 CALL RXSTRN
116 CALL CLRSCR
117 LD HL,TJOIN
118 LD DE,7*256+14
119 LD C,XWONF
120 CALL TXTAT
121 LD A,0FFH
122 LD (PRVCLS),A ; force the statics on the first poll
123 LD (PRVACT),A ; and arm the your-turn cue
124 LD (PRVRLS),A ; and mute the roll edge across the join
125 XOR A
126 LD (PENDACT),A
127 LD (VIEWIDX),A
128 LD (UTIMODE),A
129 LD (OCUR),A
130 CALL KEEPST
131 JP GAMELP

```

```

133 ; TBOFF: HL = window offset of table record A (1 + A*36, and 36 = 32+4).
134 ; clobbers A,DE.
135 TBOFF: LD L,A
136 LD H,0
137 ADD HL,HL
138 ADD HL,HL ; *4
139 LD E,L
140 LD D,H
141 ADD HL,HL
142 ADD HL,HL ; *32
143 ADD HL,HL ; *36
144 INC HL ; + the count byte
145 RET
146
147 ; TBDRAW: the list rows: name and "cur / max", selected row inverse.
148 TBDRAW: XOR A
149 LD (V_TMP),A
150 LD A,(V_TMP)
151 LD HL,V_CNT
152 CP (HL)
153 RET NC
154 CALL TBOFF
155 HL PUSH
156 LD DE,TBNAME ; the display name field
157 ADD HL,DE
158 LD DE,LINBUF
159 LD B,21
160 CALL RXSTRN
161 LD A,(V_SEL) ; row color
162 LD HL,V_TMP
163 CP (HL)
164 LD C,XFONW
165 JR Z,TBD2
166 LD C,XWONF
167
168 TBD2: LD A,(V_TMP) ; row y = 7 + i
169 ADD A,7
170 LD D,A
171 LD E,2
172 LD HL,LINBUF
173 LD B,21
174 PUSH BC
175 CALL TXTN
176 BC POP
177 HL POP
178 LD DE,TBPLYRS ; the "cur / max" field
179 ADD HL,DE
180 LD DE,LINBUF
181 LD B,6
182 CALL RXSTRN
183 LD A,(V_TMP)
184 ADD A,7
185 LD D,A
186 LD E,25
187 LD HL,LINBUF
188 LD B,6
189 CALL TXTN
190 LD A,(V_TMP)
191 INC A
192 LD (V_TMP),A
193 JR TBD1
194
195 ; =====
196 ; The poll loop
197 ; =====
198
199 GAMELP: LD A,(PENDACT) ; a staged action rides the next request:
200 INC A ; 0 none -> 1 /state, 1 -> 2 /ready,
201 CALL APICALL ; 2 -> 3 /roll, 3 -> 4 /score
202 JP C,GLFAIL
203 XOR A

```

```

204 LD (PENDACT),A ; consumed
205 LD HL,GOACTPL ; this poll's header, cached
206 CALL RXGETB
207 LD (V_ACT),A
208 LD HL,PRVACT
209 CP (HL) ; the turn edge -- computed on EVERY good
210 LD (HL),A ; poll (lobby included), so the
211 LD A,0 ; lobby->play transition rides it too
212 JR Z,GLTE1
213 INC A
214 GLTE1: LD (V_TRNF),A
215 LD HL,GOPC
216 CALL RXGETB
217 LD (V_PC),A
218 XOR A ; my turn = active seat 0 and not viewing
219 LD (V_MYTRN),A
220 LD A,(V_ACT)
221 OR A
222 JR NZ,GLMT1
223 LD HL,GOVIEW
224 CALL RXGETB
225 OR A
226 JR NZ,GLMT1
227 LD A,1
228 LD (V_MYTRN),A
229 GLMT1: CALL CHKNEW
230 LD A,(V_CLS) ; dispatch by class
231 OR A
232 JP Z,GLLOB
233 CP 1
234 JP Z,GLGAM
235 CALL GOVERR ; class 2: game over
236 LD B,150 ; ~3 s; the server resets to the lobby
237 JP GLWAIT
238
239 GLLOB: CALL LOBBYR
240 LD B,90 ; ~2 s between lobby polls
241 JP GLWAIT
242
243 ; ---- A play-class poll -----
244 GLGAM: LD HL,GORLEFT ; the roll edge: rollLeft DECREASED
245 CALL RXGETB
246 LD (V_ST),A
247 LD A,(PRVRLS)
248 CP (HL) ; sentinel: no edge across a boundary
249 JR Z,GLGR3
250 LD HL,V_ST
251 CP (HL) ; PRVRLS vs new
252 JR Z,GLGR3 ; unchanged
253 JR C,GLGR3 ; increased: a fresh turn, not a roll
254 CALL ROLLANM ; a roll landed: rattle the dice
255 LD A,(V_MYTRN)
256 OR A
257 JR Z,GLGR3
258 CALL KEEPSYN
259 LD A,(V_ST) ; restage the mask the server echoed
260 OR A ; out of rolls: jump to the best row
261 JR NZ,GLGR3
262 LD A,1
263 LD (UIMODE),A
264 CALL PICKBEST
265 GLGR3: LD A,(V_ST)
266 LD (PRVRLS),A ; the turn edge
267 LD A,(V_TRNF)
268 OR A
269 JR Z,GLGT2
270 LD A,(V_MYTRN)
271 OR A
272 JR Z,GLGT1
273 XOR A ; my turn arrived: dice mode, cursor on
274 LD (UIMODE),A ; ROLL, my own card

```

```

275 LD (DCUR),A
276 LD (VIEWIDX),A
277 HL,GODICE ; a fresh turn stages roll-everything; a
278 CALL RXGETB ; mid-turn rejoin restages the wire's
279 OR A ; mask instead
280 JR Z,GLGT0
281 CALL KEEPSYN
282 JR GLGT0A
283 GLGT0: CALL KEEPST
284 GLGT0A: LD A,(V_ST) ; rejoined with the rolls spent: straight
285 OR A ; to the score rows
286 JR NZ,GLGT0B
287 LD A,1
288 LD (UIMODE),A
289 CALL PICKCALST
290 GLGT0B: CALL SNDRN ; announce BEFORE the input slice
291 JR GLGT2
292 GLGT1: LD A,(V_ACT) ; someone else's turn: follow the roller
293 INC A
294 JR Z,GLGT2 ; FFH: nobody
295 DEC A
296 LD HL,V_PC
297 CP (HL)
298 JR NC,GLGT2
299 LD (VIEWIDX),A
300 GLGT2: CALL PLAYRD
301 LD A,(V_MYTRN)
302 OR A
303 JR Z,GLGW ; not my turn: watch only
304 CALL TURNEN
305 LD A,(V_FLAG)
306 CP 1
307 JR NZ,GLG2
308 LD A,2 ; rolled: stage /roll
309 LD (PENDACT),A
310 JP GAMP
311 GLG2: CP 2
312 JR NZ,GLG3
313 LD A,3 ; scored: stage /score
314 LD (PENDACT),A
315 JP GAMP
316 GLG3: CP 3
317 JP Z,GLLEAV ; CE inside the turn
318 CP 4
319 JR NZ,GLG4
320 CALL HELPSCR ; '.' inside the turn
321 LD A,0FFH
322 LD (PRVCLS),A
323 GLG4: JP GAMP
324 GLGW: LD B,45 ; slice burned either way: poll now
325 JP GLWAIT ; -1 s between game polls
326
327 ; The between-polls input scan. The lobby ready toggle lives HERE, in the
328 ; input path - edge-detected by KEYSCN, so one press cannot ready and
329 ; unready across two polls, and the trigger that joined the table cannot
330 ; toggle on arrival while still held (the Intellivision lesson). The
331 ; scorecard paging while it is someone else's turn lives here too.
332 GLWAIT: PUSH BC
333 CALL KEYSCN
334 POP BC
335 OR A
336 JR NZ,GLKEY
337 LD DE,2500 ; ~20 ms
338 GLDLY: DEC DE
339 LD A,D
340 OR E
341 JR NZ,GLDLY
342 DJNZ GLWAIT
343 JP GAMP
344 GLKEY: CP 'E' ; CE leaves the table
345 Z,GLLEAV

```

```

346 CP '.'
347 JR NZ,GLK1
348 CALL HELPSCR
349 LD A,0FFH
350 LD (PRVCLS),A ; force the statics on return
351 JP GAMP
352 GLK1: CP 'T'
353 JR NZ,GLK2
354 LD A,(V_CLS)
355 OR A
356 JP NZ,GLWAIT ; the trigger only means READY in the lobby
357 CALL SNDSSEL
358 LD A,1
359 LD (PENDACT),A ; stage /ready
360 GAMP
361 GLK2: CP '0'
362 JP Z,GAMP
363 CP 'L'
364 LD E,0FFH
365 JR Z,GLPAGE
366 CP 'R'
367 LD E,1
368 JR Z,GLPAGE
369 JP GLWAIT
370 GLPAGE: LD A,(V_CLS) ; page the viewed card: play class,
371 CP 1 ; someone else's turn, more than one seat
372 JP NZ,GLWAIT
373 LD A,(V_MYTRN)
374 OR A
375 JP NZ,GLWAIT
376 LD A,(V_PC)
377 CP 2
378 JP C,GLWAIT
379 LD A,(VIEWIDX)
380 ADD A,E ; +1 or -1
381 CP 0FFH
382 JR NZ,GLPG1
383 LD A,(V_PC) ; wrapped below 0
384 DEC A
385 GLPG1: LD HL,V_PC
386 CP (HL)
387 JR C,GLPG2
388 XOR A ; wrapped past the end
389 GLPG2: LD (VIEWIDX),A
390 PUSH BC
391 CALL SNDKEY
392 CALL HDRDRW ; immediate redraw from the cached reply
393 CALL CARDRW
394 CALL PANEL
395 POP BC
396 JP GLWAIT
397 GLLEAV: LD A,5
398 CALL APICALL ; best effort; we are going regardless
399 LD J,TBLSCR
400 GLFAIL: LD DE,TBANG ; a red mark in the corner; back off and
401 LD DE,0*256+39 ; try again
402 LD C,XRONF
403 CALL TXTAT
404 LD B,150
405 GLFW: PUSH BC
406 CALL KEYSCN
407 POP BC
408 CP 'E'
409 JR Z,GLLEAV
410 LD DE,2500
411 GLFD: DEC DE
412 LD A,D
413 OR E
414 JR NZ,GLFD
415 DJNZ GLFW
416 JP GAMP

```

```

417 ; -----
418 ; CHKNEW: map the round to a screen class (0 lobby, 1 play, 2 over). A
419 ; class or seat-count change means the screen holds content that no
420 ; longer exists: clear, redraw the statics, and reset the edges that
421 ; must not fire across the boundary. V_NEWS = 1 when that happened
422 ; (GOVERR's cue plays on it).
423 CHKNEW: LD HL,GORND
424 CALL RXGETB
425 OR A
426 ; Z,CHKC ; 0 -> lobby
427 JR Z,RNOVER
428 JR NZ,CHKC1
429 LD A,2
430 JR CHKC
431 LD A,1
432 CHKC1: LD (V_CLS),A
433 CHKC: LD HL,PRVCLS
434 LD CP (HL),A
435 LD HL,PRVPC
436 LD CP (HL),A
437 LD NZ,CHKN1
438 LD A,(V_PC)
439 LD HL,PRVPC
440 LD CP (HL),A
441 LD NZ,CHKN1
442 XOR A
443 LD (V_NEWS),A
444 RET
445 CHKN1: LD A,(V_PC) ; keep PRVPC honest on the class path too
446 LD (PRVPC),A
447 LD A,0FFH ; a roll edge must not fire across the
448 LD (PRVRLS),A ; boundary
449 LD A,1
450 LD (V_NEWS),A
451 CALL CLRSCR
452 LD A,(V_CLS)
453 LD CP 1
454 RET NZ ; lobby/gameover draw everything per poll
455 JP CARDST ; play statics: the scorecard labels
456
457 ; PRMPT: the server prompt (41 wire bytes) as a fixed-width field at
458 ; row 0, col 0 -- 36 cells; the clock owns the last three. Clobbers
459 ; everything.
460 PRMPT: PUSH DE
461 LD HL,GOPRMT
462 LD DE,LINBUF
463 LD B,37
464 CALL RXSTRN
465 DE POP
466 LD E,0
467 LD HL,LINBUF
468 LD C,XWONF
469 LD B,36
470 JP TXTN
471
472 ; CLOCK: the move timer, red, in the prompt row's last three cells.
473 CLOCK: LD A,(V_ACT)
474 INC A ; FFH = nobody: blank the field
475 JR Z,CLKB
476 LD HL,GOMVTIM
477 CALL RXGETB
478 LD L,A
479 LD H,0
480 CALL UI6STR
481 LD HL,LINBUF
482 LD DE,14*256+37
483 LD C,XRONF
484 LD B,3
485 JP TXTN
486 CLKB: LD HL,TNUL

```

```

488 LD DE,14*256+37
489 LD C,XWONF
490 LD B,3
491 JP TXTN
492
493 ; =====
494 ; The lobby
495 ; =====
496 ; LOBBYR: a full-text screen. Fixed-width TXTN fields make every row
497 ; self-clearing; the ready toggle itself is sampled in GLWAIT (the input
498 ; path), never here -- this runs only once per poll. The ready flag is
499 ; the seat's scores[0] word: 1 ready, 0 not, FFEH spectator.
500 LOBBYR: LD HL,TBLNAM
501 LD DE,0*256+0
502 LD C,XWONF
503 LD B,21
504 CALL TXTN
505 LD HL,GOSRVNM
506 LD DE,LINBUF
507 LD B,21
508 CALL RXSTRN
509 LD HL,LINBUF
510 LD DE,1*256+0
511 LD C,XWONF
512 LD B,21
513 CALL TXTN
514 XOR A
515 LD (V_TMP),A
516 LOBP: LD A,(V_TMP) ; list rows 3-10
517 CP 8
518 JR NC,LOBP9
519 LD HL,V_PC
520 CP (HL)
521 JR NC,LOBBLK ; row beyond the seat count: blank it
522 CALL PLBASE
523 PUSH HL
524 LD DE,LINBUF ; the name
525 LD B,9
526 CALL RXSTRN
527 LD A,(V_TMP)
528 ADD A,3
529 LD D,A
530 LD E,2
531 LD HL,LINBUF
532 LD C,XWONF
533 LD B,9
534 CALL TXTN
535 POP HL
536 LD C,0
537 CALL RDRSCOR ; the seat's ready word, scores[0]
538 LD HL,TREADY ; DE = word
539 LD C,XFONW
540 LD A,D
541 OR A
542 JR NZ,LOBS0
543 LD A,E
544 CP 1
545 JR Z,LOBS1 ; 0001: ready
546 LOBS0: LD HL,TVIEWW
547 LD C,XWONF
548 LD A,D
549 CP 0FFH
550 JR NZ,LOBS2
551 LD A,E
552 CP 0FEH
553 JR Z,LOBS1 ; FFFE: a spectator
554 LOBS2: LD HL,TDOT ; anything else: not ready yet
555 LD C,XWONF
556 LOBS1: LD A,(V_TMP)
557 ADD A,3
558 LD D,A

```



```

559 LD E,15
560 LD B,6
561 CALL TXTN
562 JR LOBNXT
563 LOBLK: ADD A,3 ; vacated row: blank name + status
564 LD D,A
565 LD E,2
566 LD HL,TNUL
567 LD C,XWONF
568 LD B,19
569 CALL TXTN
570 LOBNXT: LD A,(V_TMP)
571 INC A
572 LD (V_TMP),A
573 JR LOBP
574 LOBP9: LD D,11
575 CALL PRMPT
576 LD HL,GOPRMT ; "starting in N..." begins with 's':
577 CALL RXGETB ; tick along with the countdown
578 CP 73H
579 CALL Z,SNOTICK
580 LD HL,TLOBK1
581 LD DE,12*256+0
582 LD C,XWONF
583 CALL TXSTAT
584 LD HL,TLOBK2
585 LD DE,14*256+0
586 LD C,XWONF
587 JP TXSTAT
588
589 ; =====
590 ; Rendering a play poll
591 ; =====
592 ; PLAYRD: header, scorecard, standings panel, banner, dice, prompt,
593 ; clock -- all straight out of the reply window.
594 PLAYRD: CALL HDRDRW
595 CALL CARDRW
596 CALL PANEL
597 LD A,(V_MYTRN) ; the your-turn banner, row 9
598 OR A
599 LD HL,TYOUT
600 JR NZ,PLYB1
601 LD HL,TNUL
602 PLYB1: LD DE,9*256+6
603 LD C,XRONF
604 LD B,28
605 CALL TXTN
606 LD A,(V_MYTRN) ; the paging hint, row 8 right
607 OR A
608 LD HL,TNUL
609 JR NZ,PLYH1
610 LD A,(V_PC)
611 CP 2
612 JR C,PLYH1
613 LD HL,TPAGE
614 PLYH1: LD DE,8*256+32
615 LD C,XWONF
616 LD B,8
617 CALL TXTN
618 CALL DICEROW
619 CALL DICEHUD
620 LD D,14
621 CALL PRMPT
622 JP CLOCK
623
624 ; HDRDRW: row 0 -- the viewed player's name and the round counter.
625 HDRDRW: LD A,(VIEWIDX)
626 CALL PLBASE
627 LD DE,LINBUF
628 LD B,9
629 CALL RXSTRN
630 LD HL,LINBUF
631 LD DE,0*256+0
632 LD C,XWONF
633 LD B,9
634 CALL TXTN
635 LD HL,TROUND
636 LD DE,0*256+10
637 LD C,XWONF
638 CALL TXSTAT
639 LD HL,GORND
640 CALL RXGETB
641 LD L,A
642 LD H,0
643 LD B,2
644 CALL DECFD
645 LD HL,LINBUF
646 LD DE,0*256+13
647 LD C,XWONF
648 LD B,2
649 CALL TXTN
650 LD HL,TOF13
651 LD DE,0*256+15
652 LD C,XWONF
653 JP TXSTAT
654
655 ; =====
656 ; Game over (class 2)
657 ; =====
658 ; GOVRR: every player with their final total (the running-total sum --
659 ; at gameover all nine summed slots are filled, so it equals the wire's
660 ; grand total), the leader inverse, the winner text in the prompt. The
661 ; cue plays once, on the class edge; the server auto-resets to the lobby
662 ; and that class change rides CHKNEW like any other.
663 GOVRR: LD HL,IGOVER
664 LD DE,0*256+11
665 LD C,XRONF
666 CALL TXSTAT
667 XOR A ; find the leader
668 LD (V_WIN),A
669 LD (V_TMP),A
670 LD HL,0
671 LD (BSTTOT),HL
672 GOVF1: LD A,(V_TMP)
673 LD HL,V_PC
674 CP (HL)
675 JR NC,GOVF9
676 CALL RUNTOT ; HL = the player's total
677 LD DE,(BSTTOT)
678 OR A
679 SBC HL,DE
680 JR C,GOVF2
681 JR Z,GOVF2 ; first max wins
682 ADD HL,DE
683 LD (BSTTOT),HL
684 LD A,(V_TMP)
685 LD (V_WIN),A
686 GOVF2: LD A,(V_TMP)
687 INC A
688 LD (V_TMP),A
689 JR GOVF1
690 GOVF9: XOR A ; the list, rows 2-9
691 LD (V_TMP),A
692 GOVL1: LD A,(V_TMP)
693 CP 8
694 JR NC,GOVL9
695 LD HL,V_PC
696 CP (HL)
697 JR NC,GOVL9
698 CALL PLBASE
699 LD HL,LINBUF
700 LD B,9

```

```

701 CALL RXSTRN
702 LD A,(V_TMP) ; the leader's row inverse
703 LD HL,V_TMP
704 CP (HL)
705 LD C,XFONW
706 JR Z,GOVL2
707 LD C,XWONF
708 GOVL2: LD A,(V_TMP)
709 ADD A,2
710 LD D,A
711 LD E,4
712 LD HL,LINBUF
713 LD B,9
714 PUSH BC
715 CALL TXTN
716 POP BC
717 LD A,(V_TMP)
718 CALL RUNTOT
719 LD B,4
720 CALL DECFLD
721 LD A,(V_WIN) ; the row color again -- RUNTOT and
722 LD HL,V_TMP ; DECFLD clobbered C
723 CP (HL)
724 LD C,XFONW
725 JR Z,GOVL3
726 LD C,XWONF
727 GOVL3: LD A,(V_TMP)
728 ADD A,2
729 LD D,A
730 LD E,16
731 LD HL,LINBUF
732 LD B,4
733 CALL TXTN
734 LD A,(V_TMP)
735 INC A
736 LD (V_TMP),A
737 JR GOVL1
738 GOVL9: LD D,11
739 CALL PRMPT
740 LD HL,TGOK
741 LD DE,14*256+0
742 LD C,XWONF
743 CALL TXTAT
744 LD A,(V_NEWS)
745 OR A
746 RET Z
747 JP SNOVER
748

```

```

749 ; -----
750 ; HELPSCR: how to play. Any key returns.
751 HELPSCR: CALL KEYRAW
752 LD (V_KEY),A
753 CALL CLRSCR
754 LD HL,HELPTX
755 LD D,0
756 HELP1: LD A,(HL) ; row; strings run in order, one per row,
757 CP 0FFH ; first byte = column (FF ends)
758 JR Z,HELP9
759 LD E,A
760 HL INC
761 PUSH DE
762 PUSH HL
763 LD C,XWONF
764 CALL TXTAT
765 HL POP
766 HELP2: LD A,(HL) ; skip past the NUL
767 INC HL
768 OR A
769 JR NZ,HELP2
770 DE POP
771 INC D

```

```

772 JR HELP1
773 HELP9: CALL KEYSCN
774 OR A
775 JR Z,HELP9
776 RET
777
778 HELPTX: DB 16,"FUJITZEE",0
779 DB 0,"",0
780 DB 1,"ROLL FIVE DICE, UP TO THREE",0
781 DB 1,"ROLLS A TURN. TRIGGER MARKS",0
782 DB 1,"DICE TO HOLD, THEN PICK ONE",0
783 DB 1,"SCORE ROW: RED SHOWS WHAT AN",0
784 DB 1,"OPEN ROW WOULD EARN. BEST",0
785 DB 1,"TOTAL AFTER 13 ROUNDS WINS.",0
786 DB 0,"",0
787 DB 1,"STICK MOVES TRIGGER SELECTS",0
788 DB 1,"UP ENTERS THE SCORE ROWS",0
789 DB 1,"TRIGGER READIES IN THE LOBBY",0
790 DB 1,"CE LEAVES . THIS SCREEN",0
791 DB 0,"",0
792 DB 1,"FUJITZEE.CARR-DESIGNS.COM",0
793 DB 0FFH
794
795 TCHOOS: DB "CHOOSE A TABLE",0
796 TJOIN: DB "JOINING ...",0
797 TREADY: DB "READY",0
798 TVIEWW: DB "VIEW",0
799 TDOT: DB " ",0
800 TTURN: DB "TURN",0
801 TYOURT: DB "YOUR TURN - ROLL OR SCORE",0
802 TPAGE: DB "< > VIEW",0
803 TROUND: DB "RD",0
804 TOF13: DB "/13",0
805 TGOVER: DB "G A M E O V E R",0
806 TGOK: DB "CE LEAVES . NEXT GAME SOON",0
807 TLOBK1: DB "TRIGGER = READY ON/OFF",0
808 TLOBK2: DB "CE LEAVES . HELP 0 REFRESH",0
809 TBANG: DB "!",0
810

```

TURN.INC

```

1 ; turn.inc -- your turn. Two cursor modes over one bounded input slice:
2 ; DICE mode walks the ROLL tile and the five dice (trigger holds a die
3 ; or submits the roll); SCORE mode walks the open scorecard rows
4 ; (trigger takes the row). Stick-up crosses from dice to scores; down
5 ; from the bottom of a column drops back to the dice while rolls
6 ; remain.
7 ;
8 ; Like battleship's TARGET, TURNIN runs one bounded ~5 s slice and
9 ; returns: GAMELP a repolls and re-enters while it is still our turn, so
10 ; a game that ends (or a table that empties) is noticed within a slice.
11 ; UIMODE/DCUR/SCUR/KEEPPBF persist across slices; a keypress restarts
12 ; the slice, and KEYSN's edge detection means holding a key cannot
13 ; spin it.
14 ;
15 ; The countdown reseeds from the wire's moveTime on every entry, so the
16 ; calibrated-delay clock self-corrects each poll; at zero it bails --
17 ; the server has already force-scored, and only the next poll can
18 ; discover where the game went (the Intellivision lesson).
19 ;
20 ; V_FLAG on return: 0 slice out / timeout, 1 roll staged (KEEPPBF),
21 ; 2 score staged (SCOREIX), 3 leave, 4 help.
22 ;
23 TURNIN: XOR      A
24             LD      (V_FLAG),A
25             LD      (TGFRM),A
26             LD      HL,GOMVTIM
27             CALL    RXGETB
28             LD      (TGLEFT),A
29             LD      B,250
30             ; ~5 s slice of 20 ms ticks
31 TILP:  PUSH      BC
32             CALL    KEYSN
33             POP      BC
34             OR      A
35             JP      NZ,TIKEY
36             PUSH     BC
37             LD      DE,2500
38             ; ~20 ms
39 TIDLY: DEC      DE
40             LD      A,D
41             OR      E
42             JR      NZ,TIDLY
43             LD      HL,TGFRM
44             LD      HL,(HL)
45             ; the local second hand
46             INC     HL
47             LD      A,(HL)
48             CP      50
49             ; 50 x 20 ms = 1 s
50             JR      C,TINX
51             LD      (HL),0
52             LD      A,(TGLEFT)
53             OR      A
54             ; already zero: bail now
55             JR      Z,TIOUT
56             DEC     A
57             LD      (TGLEFT),A
58             PUSH     AF
59             LD      L,A
60             ; repaint the clock field
61             LD      H,0
62             CALL    U16STR
63             LD      HL,LINBUF
64             LD      DE,14*256+37
65             C,XRONF
66             LD      B,3
67             CALL    TXTN
68             POP      AF
69             OR      A
70             JR      Z,TIOUT
71             ; hit zero: the server has skipped us
72             CP      6
73             ; the last five seconds tick
74             CALL    C,SNDTICK
75             POP      BC
76
77 DJNZ      TILP
78 RET
79
80 TIOUT:  POP      BC
81 RET
82
83 TIRST:  LD      B,250
84 JP      TILP
85
86 ; --- Key dispatch ---
87 TIKEY:  LD      C,A
88             CP      'E'
89             ; leave -- handed back to GAMELP so the
90             ; stack stays balanced
91             NZ,TIK0
92             LD      A,3
93             LD      (V_FLAG),A
94             RET
95
96 TIK0:   CP      ' '
97             ; help, same hand-back
98             JR      NZ,TIK1
99             LD      A,4
100            LD      (V_FLAG),A
101            RET
102
103 TIK1:   LD      A,(UIMODE)
104            OR      A
105            JP      NZ,TISCR
106
107 ; --- DICE mode ---
108 LD      A,C
109 CP      'L'
110 JR      NZ,TID1
111 LD      A,(DCUR)
112 OR      A
113 JR      NZ,TID1
114 LD      A,(DCUR)
115 OR      A
116 JR      NZ,TID1
117 LD      A,6
118 TID1:   DEC     A
119 LD      (DCUR),A
120 TIDMV:  CALL    SNDKEY
121             CALL    DICEHUD
122             JP      TIRST
123
124 TID1:   CP      'R'
125 JR      NZ,TID2
126 LD      A,(DCUR)
127 INC     A
128 CP      6
129 JR      C,TIDR1
130 XOR     A
131 TIDR1:  LD      (DCUR),A
132             LD      TIDMV
133
134 TID2:   CP      'U'
135 JR      NZ,TID3
136 LD      HL,GODICE
137 CALL    RXGETB
138 OR      A
139 JP      Z,TIBAD
140 LD      A,1
141 LD      (UIMODE),A
142 CALL    SCURFX
143 CALL    SNDSEL
144 CALL    CARDRW
145 CALL    DICEHUD
146 JP      TIRST
147
148 TID3:   CP      'T'
149 JR      NZ,TIRST
150 LD      A,(DCUR)
151 OR      A
152 JR      Z,TIDROL
153 DEC     A
154 LD      HL,KEEPPBF
155 ADD     A,L
156
157 ; slice out: V_FLAG stays 0
158 ; discard the slice counter
159 ; a press restarts the slice

```

```

133 LD L,A
134 JR NC,TIDT1
135 INC H
136 TIDT1: LD A,(HL)
137 XOR 1 ; '0' <-> '1'
138 LD (HL),A
139 CALL SNDHLD
140 CALL DICEHUD
141 JP TIRST
142 TIDROL: LD HL,GORLEFT ; the ROLL tile: submit the roll
143 CALL RXGETB
144 OR A
145 JP Z,TIBAD ; no rolls left
146 LD HL,KEEPBF ; at least one die must move
147 LD B,5
148 TIDR2: LD A,(HL)
149 CP '1'
150 JR Z,TIDGO
151 INC HL
152 DJNZ TIDR2
153 JP TIBAD ; everything held: nothing to roll
154 TIDGO: LD A,1
155 LD (V_FLAG),A
156 JP SNDROLL
157
158 ; ---- SCORE mode ----
159 TISCR: LD A,C
160 CP 'U'
161 JR NZ,TIS1
162 LD A,(SCUR)
163 CALL SKPUP
164 JP C,TIRST ; already at the top
165 TISMV: LD (SCUR),A
166 CALL SNDKEY
167 CALL CARDRW
168 JP TIRST
169 TIS1: CP 'D'
170 JR NZ,TIS2
171 LD A,(SCUR)
172 CALL SKPDN
173 JR NC,TISMV
174 LD HL,GORLEFT ; off the bottom: back to the dice --
175 CALL RXGETB ; unless the rolls are spent
176 OR A
177 JP Z,TIBAD
178 XOR A
179 LD (UIMODE),A
180 CALL SNDSEL
181 CALL CARDRW ; clear the score cursor
182 CALL DICEHUD ; show the dice cursor
183 JP TIRST
184 TIS2: CP 'L'
185 JR Z,TISCOL
186 CP 'R'
187 JR NZ,TIS3
188 TISCOL: LD A,(SCUR) ; the matching row of the other column
189 CP 8
190 JR NC,TISC1
191 ADD A,8
192 JR TISC2
193 TISC1: SUB 8
194 CP 6 ; slot 14 maps beside the always-closed
195 JR C,TISC2 ; upper total: settle on the sixes row
196 LD A,5
197 TISC2: LD E,A
198 CALL VALIDC
199 CP 0FFH
200 LD A,E
201 JR NZ,TISMV ; open: land there
202 CALL SKPDN ; closed: nearest open below, then above
203 JR NC,TISMV

```

```

204 LD A,E
205 CALL SKPUP
206 JR NC,TISMV
207 JP TIBAD ; the whole column is closed
208 CP 'T'
209 TIS3: JP NZ,TIRST
210 LD A,(SCUR)
211 CALL VALIDC
212 CP 0FFH
213 JP Z,TIBAD ; a closed row (should be unreachable)
214 LD E,A
215 LD A,(SCUR)
216 LD (SCOREIX),A
217 LD A,2
218 LD (V_FLAG),A
219 LD A,(SCUR) ; the fanfare for a real fujitzee
220 CP SCFUJI
221 JR NZ,TISSC
222 LD A,E
223 OR A
224 JR Z,TISSC
225 JP SNDFUJI
226 TISSC: JP SNDSOR
227
228 ; A rejected press earns only a short slice: an accepted key restarting
229 ; the full one is an aiming player, but a trigger bounced off a dead
230 ; ROLL tile must not be able to starve the poll loop.
231 TIBAD: CALL SNDBAD
232 LD B,50 ; ~1 s
233 JP TILP
234
235 ; ----
236 ; VALIDC: A = validScores[slot A]. FFH means the row is closed (or the
237 ; server's own). Preserves BC,DE. Clobbers HL.
238 VALIDC: ADD A,GOVALID
239 LD L,A
240 LD H,0
241 JP RXGETB
242
243 ; SKPDN: from slot A, the next open row down the same column; carry set
244 ; when there is none. SKPUP: the same, upward. The always-closed slots
245 ; (the upper total and the bonus) skip like any other closed row.
246 ; Clobber B,C,HL.
247 SKPDN: CP 8
248 LD B,8 ; the left column ends before slot 8
249 JR C,SKD0
250 LD B,15
251 SKD0: LD C,A
252 SKD1: INC C
253 LD A,C
254 CP B
255 JR NC,SKDF
256 CALL VALIDC
257 CP 0FFH
258 JR Z,SKD1
259 LD A,C
260 OR A ; clear carry
261 RET
262 SKDF: SCF
263 RET
264
265 SKPUP: CP 8
266 LD B,0 ; the left column starts at slot 0
267 JR C,SKU0
268 LD B,8
269 SKU0: LD C,A
270 SKU1: LD A,C
271 CP B
272 JR Z,SKUF
273 DEC C
274 LD A,C

```

```

275     CALL    VALIDC
276     CP      0FFH
277     JR      Z,SKU1
278     LD      A,C
279     OR      A
280     RET
281 SKUF:    SCF
282     RET
283
284 ; PICKBEST: the score cursor onto the highest-paying open row (first
285 ; wins a tie) -- the resting place once the rolls run out. Clobbers all.
286 PICKBEST: LD    B,0
287     LD      C,0
288     LD      E,0FFH      ; no candidate yet
289     LD      A,B
290     CP      15
291     JR      NC,PKB9
292     CALL    VALIDC
293     CP      0FFH
294     JR      Z,PKB2
295     LD      D,A
296     LD      A,E
297     CP      0FFH
298     JR      Z,PKBT      ; the first open row is the baseline
299     LD      A,D
300     CP      E
301     JR      C,PKB2
302     JR      Z,PKB2
303     LD      E,D
304     LD      C,B
305     INC     B
306     JR      PKB1
307     LD      A,C
308     LD      (SCUR),A
309     RET
310
311 ; SCURFX: keep the score cursor on an open row when entering the mode.
312 SCURFX: LD      A,(SCUR)
313     CALL    VALIDC
314     CP      0FFH
315     RET     NZ
316     JP      PICKBEST
317
318 ; -----
319 ; KEEPST: the mask back to roll-everything. KEEPSYN: restage it from
320 ; the wire's echo after a roll lands (so the next submit re-rolls the
321 ; same dice unless the player retunes it); an empty echo falls back to
322 ; roll-everything.
323 KEEPST: LD      HL,KEEPBF
324     LD      B,5
325     LD      (HL),'1'
326     INC     HL
327     DJNZ    KRS1
328     LD      (HL),0
329     RET
330
331 KEEPSYN: LD      HL,GOKEEP
332     LD      DE,KEEPBF
333     LD      B,5
334     CALL    RXSTRN
335     LD      A,(KEEPBF)
336     OR      A
337     RET     NZ
338     JR      KEEPST
339

```

```

1 ; card.inc -- the scorecard and the standings panel.
2 ;
3 ; The card is the intv port's compact two-column form: the viewed
4 ; player's 16 slots in the left 15 columns (labels from cardst.inc,
5 ; values here) with the reclaimed right half of the screen as an
6 ; always-visible standings panel -- every seat's name, live running
7 ; total, and a TURN tag. One card is shown at a time; GLWAIT pages it
8 ; with left/right while it is someone else's turn.
9 ;
10 ; Everything renders straight out of the reply window. The only
11 ; arithmetic in the whole client lives here: RUNITOT sums score slots
12 ; 6-14, because the wire's grand total (slot 15) is empty until game
13 ; over.
14 ;
15 ; PLBASE: HL = wire offset of player record A (GOPLYRS + A*42).
16 ; Clobbers B,DE.
17 PLBASE: LD HL,GOPLYRS
18 OR A
19 RET Z
20 LD B,A
21 LD DE,PLSTRID
22 PLB1: ADD HL,DE
23 DJNZ PLB1
24 RET
25 ;
26 ; RDISCOR: DE = score word C (0-15) of the player record at wire offset
27 ; HL. Preserves HL, C.
28 RDISCOR: PUSH HL
29 LD A,C
30 ADD A,A
31 ADD A,PLSCORS
32 LD E,A
33 LD D,0
34 ADD HL,DE
35 CALL RXGETW
36 POP HL
37 RET
38 ;
39 ; RUNITOT: HL = player A's running total: the sum of score slots 6-14
40 ; (upper total, and the seven lower rows -- all kept current by
41 ; the server, so the sum tracks the true score and equals the wire's
42 ; grand total once the game ends). Any FFXH word is an unset or
43 ; spectator marker, never a score: skipped. Clobbers everything.
44 RUNITOT: CALL PLBASE
45 LD (V_BASE),HL
46 LD HL,0
47 LD (V_TOT),HL
48 LD C,SCUPTOT
49 RTT1: LD A,C
50 CP 15
51 JR NC,RTT9
52 LD HL,(V_BASE)
53 CALL RDISCOR
54 LD A,D
55 CP OFFH
56 JR Z,RTT3
57 LD HL,(V_TOT)
58 ADD HL,DE
59 LD (V_TOT),HL
60 RTT3: INC C
61 JR RTT1
62 RTT9: LD HL,(V_TOT)
63 RET
64 ;
65 ; DECFLD: HL as decimal, right-justified space-padded into a B-cell
66 ; field at LINBUF (NUL-terminated). A value wider than the field is
67 ; left as-is (TXTN's cell count truncates it). Clobbers everything.
68 DECFLD: PUSH BC
69 CALL UI6STR
70 POP BC
71 LD HL,LINBUF

```

```

72 LD C,0
73 DFL1: LD A,(HL)
74 OR A
75 JR Z,DFL2
76 INC HL
77 INC C
78 JR DFL1
79 DFL2: LD A,B ; the shift = width - length
80 SUB C
81 RET Z
82 RET C
83 LD (V_ST),A
84 LD E,A ; slide the digits (and the NUL) right
85 LD D,0
86 PUSH HL
87 ADD HL,DE
88 EX DE,HL ; DE = destination end
89 POP HL ; HL = source end (the NUL)
90 LD B,0
91 INC C ; the length plus the NUL
92 LDDR
93 LD A,(V_ST) ; space-fill the vacated front
94 LD B,A
95 LD HL,LINBUF
96 DFL3: LD (HL),A
97 INC HL
98 DJNZ DFL3
99 RET
100 ;
101 ; CARDWR: the viewed player's 16 value cells. Potentials (what an open
102 ; row would earn, in red, from validScores) appear only on your own
103 ; card, on your turn, once the dice exist -- GOVALID is all zeros in
104 ; the lobby and meaningless on anyone else's card.
105 CARDWR: LD A,(VIEWIDX)
106 CALL PLBASE
107 LD (V_BASE),HL
108 XOR A ; V_TM2 = potentials mode
109 LD (V_TM2),A
110 LD A,(VIEWIDX)
111 OR A
112 LD A,(VIEWIDX)
113 JR NZ,CDW0
114 LD A,(V_MYTRN)
115 OR A
116 JR Z,CDW0
117 LD HL,GODICE
118 CALL RXGETB
119 OR A
120 JR Z,CDW0
121 LD A,1
122 LD (V_TM2),A
123 CDW0: XOR A
124 LD (V_CAT),A
125 CDW1: LD A,(V_CAT)
126 CP 16
127 RET NC
128 CALL SCELL
129 LD A,(V_CAT)
130 INC A
131 LD (V_CAT),A
132 JR CDW1
133 ;
134 ; SCELL: one value cell for slot (V_CAT). Filled scores in white; an
135 ; open row's potential in red (a zero only under the cursor); closed or
136 ; unavailable rows as a black dash; the score cursor's cell inverse.
137 ; Slot 15 shows the running total instead of the wire's empty word.
138 SCELL: LD A,(V_CAT)
139 CP 15
140 JR Z,SCTOT
141 LD HL,(V_BASE)
142 LD C,A

```

```

143 CALL RDSCOR
144 LD A,D
145 CP 0FFH
146 JR Z,SCUNS
147 EX DE,HL ; filled: the value, white
148 LD B,3
149 CALL DECFLD
150 LD C,XWONF
151 JR SCDRW
152 SCUNS: LD A,(V_TM2) ; unset: a potential, or a dash
153 OR A
154 JR Z,SCDASH
155 LD A,(V_CAT)
156 CP SCUPTOT ; 6 and 7 are the server's rows: never
157 JR Z,SCDASH ; selectable, no potential to show
158 CP SCBONUS
159 JR Z,SCDASH
160 CALL VALIDC
161 CP 0FFH
162 JR Z,SCDASH
163 LD E,A
164 OR A
165 JR NZ,SCPOT ; a zero: visible only under the cursor
166 CALL SCCURQ
167 NZ,SCBLNK
168 SCPOT: LD L,E
169 LD H,0
170 LD B,3
171 CALL DECFLD
172 LD C,XRONF
173 JR SCDRW
174 SCBLNK: LD HL,TNUL
175 LD C,XWONF
176 JR SCDRW2
177 SCDASH: LD HL,TDASH
178 LD C,XKONF
179 JR SCDRW2
180 SCTOT: LD A,(VIEWIDX)
181 CALL RUNTOT
182 LD B,3
183 CALL DECFLD
184 LD C,XWONF
185 SCDRW: LD HL,LINBUF
186 SCDRW2: CALL SCCURQ ; the cursor's cell draws inverse
187 NZ,SCDRW3
188 LD C,XFONW
189 SCDRW3: LD A,(V_CAT)
190 PUSH HL ; CATPOS clobbers HL, the string
191 CALL CATPOS
192 POP HL
193 LD B,3
194 JP TXTN
195
196 ; SCCURQ: Z set when the score cursor is visible and sits on (V_CAT).
197 DE Preserves HL, DE, BC
198 SCCURQ: LD A,(V_TM2) ; potentials mode implies my card, my turn
199 OR A
200 JR Z,SCCQN
201 LD A,(UIMODE)
202 OR A
203 JR Z,SCCQN
204 LD A,(SCUR)
205 PUSH HL
206 LD HL,V_CAT
207 CP (HL)
208 POP HL
209 RET
210 SCCQN: OR 1 ; force NZ
211 RET
212
213 ; -----

```

```

214 ; PANEL: the standings, rows 1-6, char cols 17-39: name (inverse when
215 ; that card is on screen, black for a spectator), running total, and a
216 ; red TURN tag on the active seat. Absent seats blank themselves via
217 ; the fixed-width fields; row 7 counts any seats past the six shown.
218 PANEL: XOR A
219 LD (V_TMP),A
220 PANL: LD A,(V_TMP)
221 CP 6
222 JP NC,PANVW
223 LD HL,V_FC
224 CP (HL)
225 JP NC,PANBLK
226 CALL PLBASE
227 PUSH HL
228 LD DE,LINBUF
229 LD B,9
230 CALL RXSTRN
231 POP HL
232 LD C,0 ; scores[0] = FFEH marks a spectator
233 CALL RDSCOR
234 LD A,D
235 CP 0FFH
236 JR NZ,PANP1
237 LD A,E
238 CP 0FEH
239 JR NZ,PANP1
240 LD A,(V_TMP) ; spectator: black name, nothing else
241 INC A
242 LD D,A
243 LD E,17
244 LD HL,LINBUF
245 LD C,XKONF
246 LD B,23
247 CALL TXTN
248 JP PANNXT
249 PANP1: LD A,(V_TMP) ; the name; the viewed card's owner inverse
250 LD HL,VIEWIDX
251 CP (HL)
252 LD C,XFONW
253 JR Z,PANP2
254 LD C,XWONF
255 PANP2: LD A,(V_TMP)
256 INC A ; panel rows 1-6
257 LD D,A
258 LD E,17
259 LD HL,LINBUF
260 LD B,9
261 CALL TXTN
262 LD A,(V_TMP) ; the running total
263 CALL RUNTOT
264 LD B,4
265 CALL DECFLD
266 LD A,(V_TMP)
267 INC A
268 LD D,A
269 LD E,27
270 LD HL,LINBUF
271 LD C,XWONF
272 LD B,4
273 CALL TXTN
274 LD A,(V_TMP) ; the TURN tag
275 LD HL,V_ACT
276 CP (HL)
277 LD HL,TTURN
278 JR Z,PANT1
279 LD HL,TNUL
280 PANT1: LD A,(V_TMP)
281 INC A
282 LD D,A
283 LD E,32
284 LD C,XRONF

```

```

285     LD      B,4
286     CALL   TXTN
287     JR      PANNXT
288 PANBLK: LD  A,(V_TMP)      ; absent seat: blank the row
289     INC    A
290     LD      D,A
291     LD      E,17
292     LD      HL,TNUL
293     LD      C,XWONF
294     LD      B,23
295     CALL   TXTN
296 PANNXT: LD  A,(V_TMP)
297     INC    A
298     LD      (V_TMP),A
299     JP      PANIL
300 PANVW: LD  A,(V_PC)        ; row 7: "+N VIEW" when seats overflow
301     SUB    6
302     JR      C,PANVB
303     JR      Z,PANVB
304     ADD    A,'0'
305     LD      (LINBUF+1),A
306     LD      A,'+'
307     LD      (LINBUF),A
308     LD      HL,TVIEW2
309     LD      DE,LINBUF+2
310 PANVC: LD  A,(HL)
311     LD      (DE),A
312     INC    HL
313     INC    DE
314     OR     A
315     JR      NZ,PANVC
316     LD      HL,LINBUF
317     JR      PANVD
318 PANVB: LD  HL,TNUL
319 PANVD: LD  DE,7*256+17
320     LD      C,XWONF
321     LD      B,8
322     JP      TXTN
323
324 TVIEW2: DB  " VIEW",0
325 TDASH:  DB  " -",0
326

```

CARDST.INC

```

1 ; cardst.inc -- the scorecard's static geometry: the 16 row labels
2 ; (drawn once per class entry, and by the DEMO screen), and the
3 ; slot -> screen position map the value cells share.
4 ;
5 ; The card is two columns of eight rows: slots 0-7 (ones..sixes, the
6 ; upper total, the lower total), slots 8-15 (the seven lower
7 ; rows and the running total) down the right. Labels sit at cols 0 and
8 ; 8; values at cols 4 and 12.
9
10 ; CATPOS: slot A (0-15) -> D = screen row, E = value column. Preserves
11 ; BC, clobbers A,HL.
12 CATPOS: PUSH BC
13     LD      D,C,A
14     LD      B,0
15     LD      HL,CATROW
16     ADD    HL,BC
17     LD      D,(HL)
18     LD      E,4
19     LD      A,C
20     CP     8
21     JR      C,CATP9
22     LD      E,12
23 CATP9: POP  BC

```

```

24     RET
25
26 ; CARDST: the 16 labels, in slot order.
27 CARDST: LD  HL,CATLBL
28     XOR     A
29     LD      (V_CAT),A
30 CST1: LD  A,(V_CAT)
31     CP     16
32     RET    NC
33     PUSH   HL
34     CALL   CATPOS
35     LD      A,E
36     SUB    4                ; the label column: 4 -> 0, 12 -> 8
37     LD      E,A
38     POP     HL
39     LD      C,XWONF
40     CALL   TXTAT            ; leaves HL at the NUL
41     INC    HL
42     LD      A,(V_CAT)
43     INC    A
44     LD      (V_CAT),A
45     JR      CST1
46

```



```

47 CATROW: DB      1,2,3,4,5,6,7,8
48          DB      1,2,3,4,5,6,7,8
49
50 CATLBL: DB      "ONE",0,"TWO",0,"THR",0,"FOU",0
51          DB      "FIV",0,"SIX",0,"UP",0,"BON",0
52          DB      "SET3",0,"SET4",0,"HOU",0,"SRUN",0
53          DB      "LRUN",0,"CNT",0,"FUJI",0,"TOT",0
54

```

DICE.INC

```

1 ; dice.inc -- the dice strip: five 12x12 dice, the ROLL tile, the hold
2 ; markers, and the roll animation.
3 ;
4 ; A die is 3 bytes x 12 lines at 2bpp, sitting exactly on char rows
5 ; 10-11 (pixel lines 60-71), so the ROLL tile and the hold markers are
6 ; ordinary text on the same grid and everything stays byte-aligned. The
7 ; faces are not stored as bitmaps: each face is three 3-bit pip masks
8 ; (top / middle / bottom row of pips) and the twelve lines are built
9 ; from a skeleton, so all six faces cost 18 bytes of data.
10 ;
11 ; The hold markers live in the text row UNDER the dice ('HLD' when a
12 ; die will be kept), and the dice-mode cursor is that field -- or the
13 ; ROLL tile -- drawn inverse. No blinking: the family redraws on every
14 ; keypress and that is signal enough.
15 ;
16 DIETOP EQU      60      ; the strip's first pixel line
17 PIPB EQU      0D7H      ; one row byte: white,black,black,white
18 WHTB EQU      0FFH      ; one row byte: solid white
19
20 ; DIEADDR: DE = screen address of die A (0-4). Clobbers A,HL.
21 DIEADDR: ADD     A,A
22          LD      HL,DIEPOS
23          ADD     A,L
24          LD      L,A
25          JR      NC,DIEA1
26          INC     H
27 DIEA1: LD      E,(HL)
28          INC     HL
29          LD      D,(HL)
30          RET
31
32 ; DIEDRW: face A (0 = empty tray, 1-6 = pips) at screen address DE.
33 ; Clobbers everything.
34 DIEDRW: OR      A
35          JR      NZ,DIED1
36          EX      DE,HL      ; empty: back to felt
37          LD      B,12
38          LD      C,3
39          LD      D,0
40          JP      FILLRC
41 DIED1: DEC     A      ; the face's three pip masks
42          LD      C,A
43          ADD     A,A
44          ADD     A,C
45          LD      C,A      ; C = (face-1)*3
46          LD      HL,ROWSKEL
47          LD      B,12
48 DIEDL: LD      A,(HL)
49          INC     HL
50          PUSH    A
51          CP      0FFH
52          JR      NZ,DIEDP
53          XOR     A      ; a white line: no pips
54          JR      DIEDM
55 DIEDP: BC      PUSH    B
56          ADD     A,C
57          LD      L,A

```

```

58          LD      H,0
59          LD      BC,FACETAB
60          ADD     HL,BC
61          LD      A,(HL)
62          POP     BC
63 DIEDM: PUSH    BC      ; A = mask, bit 2 left .. bit 0 right
64          PUSH    DE
65          LD      C,A
66          LD      B,3
67 DIEDB: LD      A,C
68          AND     4
69          LD      A,WHTB
70          JR      Z,DIEDB2
71          LD      A,PIPB
72          LD      (DE),A
73 DIEDB2: INC     DE
74          SLA     C
75          DJNZ    DIEDB
76          POP     DE
77          POP     BC
78          LD      A,E      ; down one screen line
79          ADD     A,BYTEPL
80          LD      E,A
81          JR      NC,DIEDN
82          INC     D
83 DIEDN: POP     HL
84          DJNZ    DIEDL
85          RET
86
87 ; -----
88 ; DICEROW: the five dice from the wire. An out-of-range byte (or the
89 ; NULs of a turn not yet rolled) draws the empty tray.
90 DICEROW: XOR     A
91          LD      (V_TM2),A
92 DICR1: LD      A,(V_TM2)
93          CP      5
94          RET     NC
95          LD      HL,GODICE
96          LD      E,A
97          LD      D,0
98          ADD     HL,DE
99          CALL    RXGETB
100          SUB     '0'      ; '1'-'6' -> 1-6; anything else -> 0
101          C, DICR2
102          CP      7
103          JR      C, DICR3
104 DICR2: XOR     A
105 DICR3: LD      (V_FACE),A
106          LD      A,(V_TM2)
107          CALL    DIEADDR
108          LD      A,(V_FACE)
109          CALL    DIEDRW
110          LD      A,(V_TM2)
111          INC     A
112          LD      (V_TM2),A
113          JR      DICR1
114

```

```

115 ; -----
116 ; DICEHUD: the ROLL tile (rows 10-11, cols 1-4) and the five hold
117 ; markers (row 12, under the dice). The mask shown is the local staging
118 ; buffer on your turn, the wire's echo otherwise; '0' means kept. The
119 ; dice-mode cursor inherits its field.
120 DICEHUD: LD HL,GORLEFT
121 CALL RXGETB
122 LD (V_FACE),A ; rollsLeft, reused below
123 LD HL,TROLL ; the tile: dead black at 0 rolls,
124 LD C,XKONF ; inverse under the cursor, else white
125 OR A
126 JR Z,DIH1
127 LD C,XWONF
128 CALL DCURQ0
129 NZ,DIH1
130 LD C,XFONW
131 DIH1: LD DE,10*256+1
132 LD B,4
133 PUSH BC
134 CALL TXTN
135 POP BC
136 LD A,(V_FACE) ; "xN" under it, same color
137 ADD A,'0'
138 LD (LINBUF+2),A
139 LD A,'1'
140 LD (LINBUF),A
141 LD A,'X'
142 LD (LINBUF+1),A
143 XOR A
144 LD (LINBUF+3),A
145 LD HL,LINBUF
146 LD DE,11*256+1
147 LD B,4
148 CALL TXTN
149 XOR A ; the five markers
150 LD (V_TM2),A
151 DIH2: LD A,(V_TM2)
152 CP 5
153 RET NC
154 CALL KEEPGET ; A = this die's mask char
155 CP '0'
156 LD HL,THLD
157 JR Z,DIH3
158 LD HL,THUL
159 DIH3: LD C,XRONF
160 LD A,(V_TM2)
161 INC A ; cursor 1-5 = dice 0-4
162 CALL DCURQ
163 JR NZ,DIH4
164 LD C,XFONW
165 DIH4: LD A,(V_TM2) ; field col = 8 + i*5
166 ADD A,A
167 ADD A,A
168 LD E,A
169 LD A,(V_TM2)
170 ADD A,E
171 ADD A,8
172 LD E,A
173 LD D,12
174 LD B,3
175 CALL TXTN
176 LD A,(V_TM2)
177 INC A
178 LD (V_TM2),A
179 JR DIH2
180
181 ; KEEPGET: A = mask char for die (V_TM2): the staging buffer on your
182 ; turn, the wire otherwise. Preserves BC,DE... clobbers A,HL.
183 KEEPGET: LD A,(V_MYTRN)
184 OR A
185 JR Z,KPGW

```

```

186 LD A,(V_TM2)
187 LD HL,KEEPBF
188 ADD A,L
189 LD L,A
190 JR NC,KPG1
191 INC H
192 KPG1: LD A,(HL)
193 RET
194 KPGW: PUSH DE
195 LD A,(V_TM2)
196 LD HL,G0KEEP
197 LD E,A
198 LD D,0
199 ADD HL,DE
200 CALL RXGETB
201 POP DE
202 RET
203
204 ; DCURQ0 DCURQ: Z set when the dice-mode cursor is visible and equals
205 ; 0 (the ROLL tile) / A. Preserves BC,DE,HL.
206 DCURQ0: XOR A
207 DCURQ: PUSH HL
208 LD HL,V_MYTRN
209 BIT 0,(HL)
210 JR Z,DCQN
211 LD HL,UIMODE
212 BIT 0,(HL)
213 JR NZ,DCQN
214 LD HL,DCUR
215 CP (HL)
216 POP HL
217 RET
218 DCQN: POP HL ; force NZ
219 OR 1
220 RET
221
222 ; -----
223 ; ROLLANN: a roll landed -- rattle the re-rolled dice. Which dice is the
224 ; wire's own answer: the server echoes the mask the roll was made with,
225 ; so G0EEP names the moving dice for your rolls and everyone else's
226 ; alike. Eight flickers of LFSR faces with a tick each, then the caller
227 ; repats the truth on the ordinary render path.
228 ROLLANN: LD B,8
229 RANM1: PUSH BC
230 XOR A
231 LD (V_TM2),A
232 RANM2: LD A,(V_TM2)
233 CP 5
234 JR NC,RANM3
235 LD HL,G0KEEP
236 LD E,A
237 LD D,0
238 ADD HL,DE
239 CALL RXGETB
240 CP '1'
241 JR NZ,RANM4 ; kept: it never moves
242 LD A,6
243 CALL RNDMOD
244 INC A
245 LD (V_FACE),A
246 LD A,(V_TM2)
247 CALL DIEADDR
248 LD A,(V_FACE)
249 CALL DIEDRW
250 RANM4: LD A,(V_TM2)
251 INC A
252 LD (V_TM2),A
253 JR RANM2
254 RANM3: CALL SNTICK
255 LD DE,6000 ; ~50 ms between flickers
256 RANMD: DEC DE

```

```

257 LD A,D
258 OR E
259 JR NZ,RANMD
260 POP BC
261 DJNZ RANM1
262 RET
263
264 ;-----
265 ROWSKEL: DB 0FFH,0FFH ; the twelve lines of a face: white
266 DB 0,0 ; margin, pip row, gap, pip row, gap,
267 DB 0FFH ; pip row, margin
268 DB 1,1
269 DB 0FFH
270 DB 2,2
271 DB 0FFH,0FFH
272
273 ; Three pip masks per face (top, middle, bottom); bit 2 = left,
274 ; bit 1 = centre, bit 0 = right.
275 FACETAB: DB 0,2,0 ; 1
276 DB 4,0,1 ; 2
277 DB 4,5,1 ; 3
278 DB 5,0,5 ; 4
279 DB 5,2,5 ; 5
280 DB 5,5,5 ; 6
281
282 ; The five dice: byte columns 8,13,18,23,28 on pixel line 60.
283 DIEPOS: DW NORMEM+DIETOP+BYTEPL+8
284 DW NORMEM+DIETOP+BYTEPL+13
285 DW NORMEM+DIETOP+BYTEPL+18
286 DW NORMEM+DIETOP+BYTEPL+23
287 DW NORMEM+DIETOP+BYTEPL+28
288
289 TROLL: DB "ROLL",0
290 THLD: DB "HLD",0
291 THUL: DB 0
292

```

NAMENT.INC

```

1 ; nament.inc -- the name-entry screen.
2 ;
3 ; Eight slots, edited with the stick (or the keypad's arrow keys): up and
4 ; down turn the character wheel A-Z 0-9 then '-', left and right move the
5 ; cursor, trigger accepts. '-' is the blank -- the wheel has no space, so
6 ; a name can never need the '+' URL escape -- and everything from the
7 ; first '-' on is dropped at accept. An all-blank name keeps the default.
8 ;
9 NAMESCR: CALL KEYRAW ; no leaking presses from the OS menu
10 LD (V_KEY),A
11 CALL CLRSCR
12 SYSSUK STRDIS
13 DB 16 ; "FUJITZEE": 8 chars x 16px, centred
14 DB 0
15 DB 4CH
16 DW TTITLE
17 LD HL,TNAME
18 LD DE,6*256+15
19 LD C,XWONF
20 CALL TXTAT
21 LD HL,TACCP
22 LD DE,12*256+5
23 LD C,XWONF
24 CALL TXTAT
25 LD HL,PLNBUF ; edit buffer = name padded with blanks
26 LD DE,NAMEED
27 LD B,8
28 NEINI: LD A,(HL)
29 OR A

```

```

30 JR Z,NEINI2
31 LD (DE),A
32 INC HL
33 INC DE
34 DJNZ NEINI
35 JR NEINI9
36 A, '-'
37 NEINI3: LD (DE),A
38 INC DE
39 DJNZ NEINI3
40 NEINI9: XOR A
41 LD (V_NCUR),A
42 CALL NEDRAW
43 NEKEY: CALL KEYSN
44 OR A
45 JR Z,NEKEY
46 CP 'U'
47 JR NZ,NEK1
48 LD E,1 ; wheel forward
49 JR NEWHL
50 NEK1: CP 'D'
51 JR NZ,NEK2
52 LD E,36 ; wheel back (mod 37)
53 JR NEWHL
54 NEK2: CP 'L'
55 JR NZ,NEK3
56 LD A,(V_NCUR)
57 OR A
58 JR Z,NEKEY

```

```

59      DEC      A
60      LD        (V_NCUR),A
61      JR        NEDRW
62      NEK3:    CP      'R'
63      JR        NZ,NEK4
64      LD        A,(V_NCUR)
65      CP        7
66      JR        NC,NEKEY
67      INC      A
68      LD        (V_NCUR),A
69      JR        NEDRW
70      NEK4:    CP      'T'
71      JR        NZ,NEKEY
72      LD        HL,NAMEED      ; accept: copy up to the first '-'
73      DE,PLNBUF
74      LD        B,8
75      NEACC:   LD        A,(HL)
76      CP        '-'
77      JR        Z,NEACC9
78      LD        (DE),A
79      INC      HL
80      INC      DE
81      DJNZ     NEACC
82      NEACC9:  LD        A,E      ; empty name: keep what PLNBUF had
83      CP        PLNBUF-4E00H      ; (its low byte; the buffer is in 4Exx)
84      JR        Z,NEKEY
85      XOR      A
86      LD        (DE),A
87      JP        SNDSEL      ; confirm cue, then back to the caller
88
89      NEWHL:   LD        A,(V_NCUR)
90      LD        HL,NAMEED      ; step the wheel at the cursor by E
91      PUSH     DE
92      LD        E,A
93      LD        D,0
94      ADD      HL,DE
95      POP      DE
96      LD        A,(HL)
97      PUSH     HL
98      LD        HL,WHEEL      ; find its wheel index
99      LD        B,0
100     NEWH1:   CP        (HL)
101     JR        Z,NEWH2
102     INC      HL
103     INC      B
104     JR        NEWH1
105     NEWH2:   LD        A,B
106     ADD      A,E      ; +1 or +36
107     NEWH3:   CP        37
108     JR        C,NEWH4
109     SUB      37
110     JR        NEWH3
111     NEWH4:   LD        HL,WHEEL
112     LD        E,A
113     LD        D,0
114     ADD      HL,DE
115     LD        A,(HL)
116     POP      HL
117     LD        (HL),A
118     NEDRW:   CALL     SNDKEY
119     CALL     NEDRAW
120     JP        NEKEY
121
122     ; NEDRAW: the eight slots at row 8, cols 16-23, cursor cell inverse.
123     NEDRAW:  XOR      A
124     LD        (V_TMP),A
125     NED1:    LD        A,(V_TMP)
126     CP        8
127     RET      NC
128     LD        HL,NAMEED
129     LD        E,A

```

```

130     LD        D,0
131     ADD      HL,DE
132     LD        A,(HL)
133     LD        (LINBUF),A
134     XOR      A
135     LD        (LINBUF+1),A
136     LD        A,(V_TMP)
137     LD        HL,V_NCUR
138     CP        (HL)
139     LD        C,XFONW
140     JR        Z,NED2
141     LD        C,XWONF
142     LD        A,(V_TMP)
143     ADD      A,16
144     LD        E,A
145     LD        D,8
146     LD        HL,LINBUF
147     LD        B,1
148     CALL     TXTN
149     LD        A,(V_TMP)
150     INC      A
151     LD        (V_TMP),A
152     JR        NED1
153
154     WHEEL:   DB      "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789-"
155     TNAME:   DB      "YOUR NAME",0
156     TACCP:   DB      "STICK EDITS - TRIGGER ACCEPTS",0
157

```

SOUND.INC

```

1 ; sound.inc -- cues, written straight to the music processor.
2 ;
3 ; Tone A only, plus the noise generator for the dice rattle -- the first
4 ; use of the noise channel in this family of ports. Everything blocks:
5 ; every caller is already in an input loop, and the longest cue is under
6 ; half a second.
7
8 TONEA EQU 11H ; tone A period (TONMO+1)
9
10 ; BEEP: A = tone A period (higher = lower pitch), B = duration, ~10 ms
11 ; units. Clobbers A,B,DE.
12 BEEP: PUSH BC
13 LD C,A
14 LD A,20H
15 OUT (TONMO),A
16 LD A,C
17 OUT (TONEA),A
18 LD A,08H
19 OUT (VOLAB),A
20 POP BC
21 BEEPW: LD DE,600
22 BEEP: DEC DE
23 LD A,D
24 OR E
25 JR NZ,BEEP
26 DJNZ BEEP
27 XOR A
28 OUT (VOLAB),A
29 RET
30
31 SNDKEY: LD A,18H ; key click
32 LD B,2
33 JP BEEP
34
35 SNDSEL: LD A,28H ; select / join / confirm
36 LD B,5
37 JP BEEP
38
39 SNDHLD: LD A,14H ; a die kept (or released): a short
40 LD B,2 ; high click, distinct from the key
41 JP BEEP
42
43 SNDTRN: LD A,30H ; your turn: low then high
44 LD B,6
45 CALL BEEP
46 LD A,18H
47 LD B,8
48 JP BEEP
49
50 SNDSCOR: LD A,30H ; a row scored: rising chirp
51 LD B,3
52 CALL BEEP
53 LD A,20H
54 LD B,4
55 JP BEEP
56
57 SNDFUJI: LD A,40H ; FUJITZEE! five rising notes
58 LD B,4
59 CALL BEEP
60 LD A,30H
61 LD B,4
62 CALL BEEP
63 LD A,28H
64 LD B,4
65 CALL BEEP
66 LD A,20H
67 LD B,4
68 CALL BEEP
69 LD A,18H
70 LD B,10
71 JP BEEP
72
73 ; SNDROLL: the dice rattle -- a decaying burst of noise. Noise mode is
74 ; bit 5 of VOLC (the tone C volume / noise control register); the noise
75 ; level itself rides VOLN. Clobbers A,B,C,DE.
76 SNDROLL: LD A,20H
77 OUT (TONMO),A ; master oscillator clocks the noise LFSR
78 LD A,20H
79 OUT (VOLC),A ; noise mode on, tone C volume 0
80 LD B,8 ; eight ~35 ms bursts, fading
81 LD C,0F0H
82 SNROL1: LD A,C
83 OUT (VOLN),A
84 LD DE,4400 ; ~35 ms
85 SNROL2: DEC DE
86 LD A,D
87 OR E
88 JR NZ,SNROL2
89 LD A,C ; fade a step
90 SUB 20H
91 LD C,A
92 DJNZ SNROL1
93 XOR A
94 OUT (VOLN),A
95 OUT (VOLC),A ; noise mode off
96 RET
97
98 SNDBAD: LD A,70H ; the rejection buzz
99 LD B,5
100 JP BEEP
101
102 SNTICK: LD A,10H ; the last-seconds tick
103 LD B,1
104 JP BEEP
105
106 SNDOVER: LD A,40H ; game over
107 LD B,5
108 CALL BEEP
109 LD A,30H
110 LD B,5
111 CALL BEEP
112 LD A,20H
113 LD B,10
114 JP BEEP
115

```

DEMO.INC

```

1 ; demo.inc -- static mock play screen, assembled when DEMO=1.
2 ;
3 ; M1 scaffolding: exercises every drawing path with zero network -- the
4 ; play-screen sketch from the plan, populated with a four-seat game in
5 ; progress. This is where the palette, the dice, and the column
6 ; geometry get tuned by screenshot; the live screens (screens.inc)
7 ; replace it from M2 on.
8
9 DEMOSCR:
10 CALL CLRSCR
11 CALL CARDST ; the real label geometry
12
13 ; Row 0: the viewed player and the round.
14 LD HL,DMYOU
15 LD DE,0*256+0
16 LD C,XWONF
17 CALL TXTAT
18 LD HL,DMRD
19 LD DE,0*256+10
20 LD C,XWONF
21 CALL TXTAT
22
23 ; The left column's values (col 4): white filled, black dashes, one
24 ; red potential with its cell inverse two rows further down.
25 LD HL,DMVALS ; (row, col, color, 3 chars + NUL) x N,
26 DMV1: LD A,(HL) ; FF ends
27 CP 0FFH
28 JR Z,DMV9
29 LD D,A
30 INC HL
31 LD E,(HL)
32 INC HL
33 LD C,(HL)
34 INC HL
35 LD B,3
36 CALL TXTN ; leaves HL at the NUL
37 INC HL
38 JR DMV1
39 DMV9:
40
41 ; The standings panel: four seats, the third on turn.
42 LD HL,DMNAM1
43 LD DE,1*256+17
44 LD C,XFONW ; the viewed card's owner
45 CALL TXTAT
46 LD HL,DMNAM2
47 LD DE,2*256+17
48 LD C,XWONF
49 CALL TXTAT
50 LD HL,DMNAM3
51 LD DE,3*256+17
52 LD C,XWONF
53 CALL TXTAT
54 LD HL,DMNAM4
55 LD DE,4*256+17
56 LD C,XWONF
57 CALL TXTAT
58 LD HL,DMTOT1
59 LD DE,1*256+27
60 LD C,XWONF
61 CALL TXTAT
62 LD HL,DMTOT2
63 LD DE,2*256+27
64 LD C,XWONF
65 CALL TXTAT
66 LD HL,DMTOT3
67 LD DE,3*256+27
68 LD C,XWONF
69 CALL TXTAT
70 LD HL,DMTOT4
71 LD DE,4*256+27
72 LD C,XWONF
73 CALL TXTAT
74 LD HL,DMTURN
75 LD DE,3*256+32
76 LD C,XRONF
77 CALL TXTAT
78
79 ; Row 9: the your-turn banner.
80 LD HL,DMBANR
81 LD DE,9*256+6
82 LD C,XRONF
83 CALL TXTAT
84
85 ; The dice strip: ROLL tile, five faces, hold markers.
86 LD HL,DMROLL
87 LD DE,10*256+1
88 LD C,XFONW ; the dice cursor parked on ROLL
89 CALL TXTAT
90 LD HL,DMX2
91 LD DE,11*256+1
92 LD C,XWONF
93 CALL TXTAT
94 XOR A
95 LD (V_TM2),A
96 DMD1: LD A,(V_TM2)
97 CP 5
98 JR NC,DMD9
99 LD HL,DMFACE
100 LD E,A
101 LD D,0
102 ADD HL,DE
103 LD A,(HL)
104 LD (V_FACE),A
105 LD A,(V_TM2)
106 CALL D1EADDR
107 LD A,(V_FACE)
108 CALL D1EDRW
109 LD A,(V_TM2)
110 INC A
111 LD (V_TM2),A
112 JR DMD1
113 DMD9: LD HL,DMHLD
114 LD DE,12*256+13
115 LD C,XRONF
116 CALL TXTAT
117 LD HL,DMHLD
118 LD DE,12*256+23
119 LD C,XRONF
120 CALL TXTAT
121 LD HL,DMHLD
122 LD DE,12*256+28
123 LD C,XRONF
124 CALL TXTAT
125
126 ; Row 14: the prompt and the clock.
127 LD HL,DMPRM
128 LD DE,14*256+0
129 LD C,XWONF
130 CALL TXTAT
131 LD HL,DMCLK
132 LD DE,14*256+37

```

```

133 LD C,XRONF
134 CALL TXTAT
135 RET
136
137 DMYOU: DB "ASTRO",0
138 DMRD: DB "RD 5/13",0
139 DMBANR: DB "YOUR TURN - ROLL OR SCORE",0
140 DMROLL: DB "ROLL",0
141 DMX2: DB "X2",0
142 DMHLD: DB "HLD",0
143 DMPRM: DB "ROLL 2. KEEP OR SCORE",0
144 DMCLK: DB "45",0
145 DMNAM1: DB "ASTRO",0
146 DMNAM2: DB "MOZZWALD",0
147 DMNAM3: DB "ERIC",0
148 DMNAM4: DB "THOM",0
149 DMTOT1: DB "86",0
150 DMTOT2: DB "92",0
151 DMTOT3: DB "71",0
152 DMTOT4: DB "64",0
153 DMTURN: DB "TURN",0
154 DMFACE: DB 3,5,2,6,1
155
156 ; (row, col, color, three cells) per value cell; FF ends.
157 DMVALS: DB 1,4,XWONF," 3",0
158 DB 2,4,XWONF," 6",0
159 DB 3,4,XRONF," 15",0 ; an open row's potential
160 DB 4,4,XWONF," 12",0
161 DB 5,4,XKONF," -",0
162 DB 6,4,XWONF," 18",0
163 DB 7,4,XWONF," 39",0
164 DB 8,4,XKONF," -",0
165 DB 1,12,XWONF," 24",0
166 DB 2,12,XFONW," 17",0 ; the score cursor's cell
167 DB 3,12,XWONF," 25",0
168 DB 4,12,XKONF," -",0
169 DB 5,12,XWONF," 40",0
170 DB 6,12,XKONF," -",0
171 DB 7,12,XKONF," -",0
172 DB 8,12,XWONF,"128",0
173 DB 0FFH
174

```

RND.INC

```

1 ; rnd.inc -- a 16-bit Galois LFSR for the placement seeds.
2 ;
3 ; The state is a pure maximal LFSR (poly mask 2DH, period 65535),
4 ; stepped a full 8 bits per call so consecutive outputs are independent
5 ; PRNG bytes. The R register is folded into the OUTPUT only, never the
6 ; state: R advances every opcode fetch, so it varies w/wh input
7 ; and network timing and makes runs differ -- but XORing it into the
8 ; state (the first cut here) correlated consecutive calls whenever the
9 ; code path between them was fixed, and every placement seeded dir=0,
10 ; x=0. State out, output in.
11 ;
12 ; RNDNXT: A = the next pseudo-random byte. Clobbers B,HL.
13 RNDNXT: LD HL,(RNDST)
14 LD B,8
15 RNDST: ADD HL,HL ; one LFSR bit: shift left, tap on carry
16 JR NC,RNDST2
17 LD A,L
18 XOR 2DH ; maximal 16-bit polynomial
19 LD L,A
20 RNDST2: DJNZ RNDST
21 LD A,H
22 OR L
23 JR NZ,RNDST3

```

```

24 INC L ; never park on zero
25 RNDST3: LD (RNDST),HL
26 LD A,R
27 XOR L
28 RET
29
30 ; RNDMOD: A = a random value 0 .. A-1 (A >= 1). Clobbers B,C,HL.
31 RNDMOD: LD C,A
32 CALL RNDNXT
33 SUB C
34 JR NC,RNDM1
35 ADD A,C
36 RET
37

```

C.7 CONFIG

CONFIG.ASM

```
1 ; config.asm -- FujiNet CONFIG for the Bally Astrocade.
2
3 ; The full-feature CONFIG client: WiFi setup (scan / custom SSID / on-screen
4 ; password entry / connect), the 8 host slots (list, mount, rename), directory
5 ; browsing (paging, subfolders, .cfg suppression, filtering), copying a file
6 ; between hosts, an info screen, and boot with a progress bar -- the Astrocade
7 ; counterpart of fujinet-config/intv, grown from the bring-up's testrom/
8 ; fujicfg.asm and speaking the same cartridge mailbox (fujilib.inc).
9
10 ; RESET on the console restarts this program but not the cart, which is why
11 ; sequence numbers always derive from ACKSEQ (see fujilib.inc) and why no
12 ; screen assumes RAM survived.
13
14 ; Modules (INCLUDEd below, one translation unit):
15 ; fujicmd.inc command wrappers on the fujilib transport
16 ; input.inc hand controller + 24-key keypad polling, repeat, events
17 ; font.inc byte-aligned 5x7 blitter; BIOS glyphs + lowercase set
18 ; ui.inc rows, chevron cursor, footer, decimal, memset
19 ; edit.inc on-screen character-grid text editor
20 ; wifi.inc ST_CHECK_WIFI / ST_CONNECT_WIFI / ST_SET_WIFI
21 ; hosts.inc host slots: list / mount / rename / lobby / copy skin
22 ; browse.inc ST_SELECT_FILE: paging, subdirs, filter, copy hooks
23 ; copy.inc COPY_FILE with .cfg sibling, return-to-source
24 ; info.inc adapter config display
25 ; boot.inc SET_DEVICE_FULLPATH + MOUNT_IMAGE w/ progress, swap
26
27 INCLUDE "HVGLIB.H"
28
29 ;----- geometry -----
30 ; LINES=80 frees 4C80H-4FFFH (896 bytes) for variables while keeping ten
31 ; 8-pixel text rows: title, eight list rows, footer.
32 LINES EQU 80
33 NROWS EQU 8
34 TITLE EQU 1 ; y of the title row
35 LISTY0 EQU 10 ; y of list row 0
36 FOOTY EQU 73 ; y of the footer row
37 LISTX EQU 8 ; x (pixels) where list text starts
38 NAMELEN EQU 24 ; display width of a directory entry
39 LISTLEN EQU 36 ; READ_DIR_ENTRY maxlen for listings: enough
40 ; that the .cfg/.bin/" suffix survives the
41 ; crunch, and (being != 31) keeps the firmware
42 ; from prepending icon bytes
43 FULLLEN EQU 120 ; re-read width for paths that must open
44 DEVSLOT EQU 0
45
46 ;----- RAM map (screen RAM above the 80 visible lines) -----
47 ; The V_SRC/V_SSID union is safe by construction: the WiFi screens are
48 ; unreachable once ST_HOSTS has been entered, and copy (the V_SRC user)
49 ; can only start from the file browser.
50 V_PATH EQU 4C80H ; 192 current dir path, NUL-term, leading '/'
51 V_SRC EQU 4D40H ; 224 copy-source full path / boot path
52 V_SSID EQU 4D40H ; 33 \ union with V_SRC (wifi phase only)
53 V_PASS EQU 4D61H ; 64 /
54 V_ENTRY EQU 4E20H ; 128 editor accumulator / full-name re-read
55 V_FILT EQU 4EA8H ; 32 directory filter ("*" = none)
56 V_EPOS EQU 4EC0H ; 16 per-row absolute dir position, 8 x 2 LE
57 V_EDIR EQU 4ED0H ; 8 per-row type: 0 file / 1 dir / 2 cart
58 V_PSTK EQU 4ED8H ; 16 page-start stack, 8 x 2 LE
59 ; Persistent cursor bits / input state bits just below the stack, the way the
60 ; bring-up's fujicfg keeps its state at 4F40H+; the big buffers stay lower.
61 ; 4EE8-4FFF is unused spare.
62 V_TMP EQU 4F00H ; 8 scratch (rebuild each use)
63 LINBUF EQU 4F08H ; 40 display line being built
64 HEXBUF EQU 4F30H ; 8
65 V_HOST EQU 4F38H ; selected host slot
66 V_CUR EQU 4F39H ; cursor row
67 V_CNT EQU 4F3AH ; entries on this page
68 V_DPOS EQU 4F3BH ; 2 dir position of the top row
69 V_DNEXT EQU 4F3DH ; 2 dir position after the last row
70 V_PDEP EQU 4F3FH ; page stack depth
71 V_CMODE EQU 4F40H ; 0 normal, 1 = picking a copy destination
72 V_HOSTS EQU 4F41H ; copy-source host slot
73 V_NNET EQU 4F42H ; networks found by the last scan
74 V_GX EQU 4F43H ; editor grid cursor
75 V_GY EQU 4F44H ;
76 V_GLEN EQU 4F45H ; editor value length
77 V_GMAX EQU 4F46H ;
78 V_GCASE EQU 4F47H ; 0 upper, 1 lower
79 V_PDIR EQU 4F48H ; input: last emitted direction bit
80 V_PTRG EQU 4F49H ; input: previous trigger state
81 V_PKP EQU 4F4AH ; 4 input: previous keypad columns
82 V_RPT EQU 4F4EH ; input: auto-repeat countdown
83 ; 4F4FH-4F5FH spare; stack floor 4F60H (96-byte budget, shallow call tree)
84 STACK EQU 4FC0H
85 STUB EQU 4FE0H ; boot swap stub (6 bytes, runs from RAM)
86
87 ; STDIS/CHDIS option bytes: bits 2-3 = fg color, bits 0-1 = bg color.
88 OPTFB EQU 0CH ; fg 3 (white) on bg 0
89 OPTHI EQU 04H ; fg 1 (accent)
90 OPTDIM EQU 08H ; fg 2 (dim)
91
92 ; Blitter color specs: bits 1-0 = fg pixel code, bits 3-2 = bg pixel code.
93 GCNORM EQU 03H ; white on black
94 GCINV EQU 0CH ; black on white (cursor cell)
95
96 ORG FIRSTC
97 DB 55H ; menued-cartridge sentinel
98 DW MENUST ; chain to the on-board SELECT GAME list
99 DW PRGNAM
100 DW PRGSTR
101 PRGNAM: DB "FUJINET CONFIG"
102 DB 0
103
104 PRGSTR: DI
105 LD SP, STACK
106 INTFC
107 DO SYSTEM
108 DB LINES*2
109 DB 0
110 DB 8
111 DO COLSET
112 DW PALET
113 DO FILL ; clear past the visible lines through the
114 DW NORMEM ; variable region (0x4000-0x4EFF): the vars
115 DW 0F00H ; live in screen RAM above the display, where
116 DB 0 ; the SELECT GAME menu leaves text that would
117 EXIT ; otherwise read back as stale values. Stops
118 ; short of 0x4F00 so the running interpreter's
119 ; stack (near 0x4FC0) is left intact.
120
121 XOR A
122 LD (V_FILT),A
123 LD (V_CMODE),A
124 LD (V_HOST),A
```



```

125     LD      (V_PDIR),A
126     LD      (V_PTRG),A
127     LD      (V_RPT),A
128     LD      (V_PKP),A
129     LD      (V_PKP+1),A
130     LD      (V_PKP+2),A
131     LD      (V_PKP+3),A
132     LD      HL,V_PATH      ; path = "/"
133     LD      (HL),'/'
134     INC     HL
135     LD      (HL),A
136
137     CALL    FNCHECK
138     JP      Z,CKWIFI
139
140 ; No mailbox: the EPROM-diagnostic path. Show what we read and halt.
141 NOCARD: LD    HL,TNOCART
142         CALL   UTITLE
143         LD     A,(FNMAGF)
144         LD     DE,LISTY0*256+40
145         CALL   HEXAT
146         LD     A,(FNMAGN)
147         LD     DE,LISTY0*256+56
148         CALL   HEXAT
149 HALTE:  JR     HALTE
150
151 ; ---- shared failure helpers -----
152 ; UFAIL: footer = message at HL + the code in A as hex, pause ~2 s.
153 ; Returns to the caller, which decides where to go next.
154 UFAIL:  PUSH  AF
155         CALL   UFOOT
156         POP    AF
157         LD     DE,FOOTY*256+140
158         CALL   HEXAT
159         JR     UPZ2
160
161 ; UMSG: footer = message at HL, pause ~2 s (input skips).
162 UMSG:   CALL   UFOOT
163         LD     DE,200
164         JP     UPAUSE
165
166 TNOCART: DB     "NO FUJINET CART",0
167
168 PALET:  DB      0E7H,0F4H,7AH,00H      ; COL3..COL0: white/amber/blue/black
169         DB      0E7H,0F4H,7AH,00H
170
171 INCLUDE "fujicmd.inc"
172 INCLUDE "input.inc"
173 INCLUDE "font.inc"
174 INCLUDE "ui.inc"
175 INCLUDE "edit.inc"
176 INCLUDE "wifi.inc"
177 INCLUDE "hosts.inc"
178 INCLUDE "browse.inc"
179 INCLUDE "copy.inc"
180 INCLUDE "info.inc"
181 INCLUDE "boot.inc"
182 INCLUDE "fujilib.inc"
183 INCLUDE "fujidisp.inc"

```

FUJICMD.INC -- THE FUJI-DEVICE WRAPPERS

```

1 ; fujicmd.inc -- Fuji-device (70H) command wrappers on the fujilib transport.
2 ;
3 ; Payload shapes matter more than they look (learned the hard way on INTV,
4 ; recorded in fujinet-config/intv/fujicmd.bas and honored here):
5 ; - The firmware's transaction_get() FAILS a short read, so OPEN_DIRECTORY
6 ;   and SET_DEVICE_FULLPATH always send a full 256-byte NUL-padded payload.

```

```

7 ; - COPY_FILE is the exception: it reads the payload as a string, so it is
8 ;   sent at EXACT length -- a padding NUL would land inside the destination
9 ;   filename. Its host slot params are 1-BASED.
10 ; - SET_SSID needs nparam >= 1 (value ignored) and exactly 97 bytes:
11 ;   ssid[33] + password[64].
12 ; - Directory EOF is exactly two 7FH bytes; directories get a trailing '/'

```

```

13 ; from the firmware; maxlen != 31 keeps icon bytes out of the reply.
14 ; - CONFIG_BOOT (0D9H) is deliberately never sent: CONFIG lives in the
15 ; cart's own flash, and the flag would only unmount device slot 0.
16
17 ; ---- command ids beyond fujilib.inc's set ----
18 FCWHS EQU 0F3H ; WRITE_HOST_SLOTS
19 FCSSID EQU 0FBH ; SET_SSID
20 FCSCAN EQU 0FDH ; SCAN_NETWORKS
21 FCSCRES EQU 0FCH ; GET_SCAN_RESULT
22 FOWSTAT EQU 0FAH ; GET_WIFI_STATUS (3 = connected)
23 FCWEN EQU 0EAH ; GET_WIFI_ENABLED
24 FCSSID EQU 0FEH ; GET_SSID (ssid[33] + pass[64])
25 FCCOPY EQU 0D8H ; COPY_FILE ("src|dst", 1-based hosts)
26
27 ; CGO: commit (8 quanta) and validate. Success: carry clear (and Z).
28 ; Failure of any kind -- timeout, transport error, NAK -- returns carry set
29 ; with A = the FNERR code (0FFH for timeout). Clobbers everything.
30 CGO: LD B,8
31 CGOB: CALL FNCOMMIT
32 ; JR CGOT
33 OR A
34 JR NZ,CGOF
35 LD A,(FNREPLY)
36 CP FCACK
37 RET Z
38 LD A,(FNERR)
39 CGOF: SCF
40 RET
41 CGOT: LD A,0FFH
42 SCF
43 RET
44
45 ; CD00: run the 0-param command E start to finish. Carry set on failure.
46 CD00: LD A,FNDFUJI
47 LD L,0
48 CALL FNBEGIN
49 JR CGO
50
51 ; CD01: run command E with the single 1-byte param A. Carry set on failure.
52 CD01: PUSH AF
53 LD A,FNDFUJI
54 LD L,1
55 CALL FNBEGIN
56 POP AF
57 CALL FNPARB
58 JR CGO
59
60 ; FNPARB: append a 2-byte parameter with value HL. Clobbers A,D,E.
61 FNPARB: LD A,2
62 CALL FNTXBYT
63 LD A,L
64 CALL FNTXBYT
65 LD A,H
66 JP FNTXBYT
67
68 ; CTXSTR: stream the NUL-terminated string at HL, adding its length to C.
69 ; Clobbers A,D,E,HL.
70 CTXSTR: LD A,(HL)
71 OR A
72 RET Z
73 CALL FNTXBYT
74 INC HL
75 INC C
76 JR CTXSTR
77
78 ; CTXFIX: stream exactly B bytes from HL; once a NUL is hit the remainder
79 ; is streamed as zeros. Clobbers A,B,D,E,HL.
80 CTXFIX: LD A,(HL)
81 OR A
82 JR Z,CTXF0
83 CALL FNTXBYT

```

```

84 INC HL
85 DJNZ CTXFIX
86 RET
87 CTXF0: XOR A
88 CALL FNTXBYT
89 DJNZ CTXF0
90 RET
91
92 ; CPAD: pad the payload with zeros until C bytes total reach 256.
93 ; (C = 0 means "none sent yet" and pads the full 256.) Clobbers A,B,D,E.
94 CPAD: LD B,C
95 CPAD1: XOR A
96 CALL FNTXBYT
97 INC B
98 JR NZ,CPAD1
99 RET
100
101 ; CMOUNT: MOUNT_HOST slot A. Carry set on failure.
102 CMOUNT: LD E,FCMHST
103 JR CD01
104
105 ; CODIR: OPEN_DIRECTORY on host A with V_PATH + V_FILTER, padded to 256.
106 ; Carry set on failure.
107 CODIR: PUSH AF
108 LD A,FNDFUJI
109 LD E,FCODIR
110 LD L,1
111 CALL FNBEGIN
112 POP AF
113 CALL FNPARB
114 LD C,0
115 LD HL,V_PATH
116 CALL CTXSTR
117 XOR A
118 CALL FNTXBYT
119 INC C
120 LD HL,V_FILTER
121 CALL CTXSTR
122 CALL CPAD
123 JP CGO
124
125 ; CCDIR: CLOSE_DIRECTORY. Carry set on failure.
126 CCDIR: LD E,FCODIR
127 JR CD00
128
129 ; CDPOS: SET_DIRECTORY_POSITION HL (16-bit). Carry set on failure/NAK --
130 ; a position past the end NAKs, which callers treat as end-of-directory.
131 CDPOS: PUSH HL
132 LD A,FNDFUJI
133 LD E,FCSDPS
134 LD L,1
135 CALL FNBEGIN
136 POP HL
137 CALL FNPARB
138 JP CGO
139
140 ; CDENT: READ_DIR_ENTRY with maxlen A. Carry set on failure or on the
141 ; 7FH,7FH end-of-directory marker; clear with the name in the reply window.
142 CDENT: PUSH AF
143 LD A,FNDFUJI
144 LD E,FCRDIR
145 LD L,2
146 CALL FNBEGIN
147 POP AF
148 CALL FNPARB
149 XOR A
150 CALL FNPARB
151 CALL CGO
152 RET C
153 LD A,(FNADATA)
154 CP 7FH

```

```

155      JR      NZ,CDEOK
156      LD      A,(FNRDATA+1)
157      CP      7FH
158      JR      NZ,CDEOK
159      SCF
160      RET
161 CDEOK: OR      A
162      RET
163
164 ; CDE16: position HL, then read the entry at maxlen A. Carry set on
165 ; failure or end of directory.
166 CDE16: PUSH   AF
167      CALL   CDPOS
168      JR      C,CDE16F
169      POP    AF
170      JP      CDENT
171 CDE16F: POP    AF
172      SCF
173      RET
174

```

WIFI.INC

```

1 ; wifi.inc -- ST_CHECK_WIFI / ST_CONNECT_WIFI / ST_SET_WIFI.
2 ;
3 ; Mirrors the INTV flow: at power-on decide between going straight to the
4 ; host list (wifi disabled or already connected), waiting out a connect on
5 ; the stored SSID, or running the scan / select / password / save chain.
6 ; The WiFi screens are unreachable once ST_HOSTS has been entered, which
7 ; is what makes the V_SSID/V_PASS union with V_SRC safe.
8
9 ;---- ST_CHECK_WIFI ----
10 CKWIFI: LD      HL,TTITLE
11      CALL   UTITLE
12      LD      HL,TWAIT
13      CALL   UFOOT
14      LD      DE,1500      ; ~15 s for the ESP32 link; input skips
15 CKW1:  LD      A,(FNSTATS)
16      RRA
17      JR      C,CKW2
18      PUSH   DE
19      CALL   INPOLL
20      POP    DE
21      OR      A
22      JR      NZ,CKW2
23      DEC    DE
24      LD      A,D
25      OR      E
26      JR      NZ,CKW1
27 CKW2:  LD      E,FCWEN
28      CALL   CD00
29      JR      C,CKWH      ; can't even ask: let the host page cope
30      LD      A,(FNRDATA)
31      OR      A
32      JR      Z,CKWH      ; wifi disabled
33      LD      E,FCWSTAT
34      CALL   CD00
35      JR      C,CKWH
36      LD      A,(FNRDATA)
37      CP      3
38      JR      Z,CKWH      ; already connected
39      LD      E,FCGSSID
40      CALL   CD00
41      JR      C,CKWH
42      LD      A,(FNRDATA)
43      OR      A
44      JP      Z,WSCAN      ; nothing stored: set one up
45      LD      HL,FNRDATA      ; stored SSID: wait for the connect

```

```

46      LD      DE,V_SSID
47      LD      BC,33
48      LDIR
49      JR      WCONN
50 CKWH:  JP      HOSTPG
51
52 ;---- ST_CONNECT_WIFI ----
53 WCONN: LD      HL,TCONN
54      CALL   UTITLE
55      LD      HL,V_SSID
56      LD      DE,LISTV0*256+0
57      LD      C,GCNORM
58      LD      B,20
59      CALL   DRWSTR
60      LD      HL,TABORT
61      CALL   UFOOT
62      LD      B,20      ; ~20 x 2 s
63      PUSH   BC
64      LD      E,FCWSTAT
65      CALL   CD00
66      POP    BC
67      JR      C,WCN2
68      LD      A,(FNRDATA)
69      CP      3
70      JP      Z,HOSTPG
71      WCN2:  PUSH   BC
72      LD      DE,2000
73      CALL   UPAUSE
74      POP    BC
75      OR      A
76      JR      NZ,WSCAN      ; user abort: pick a network instead
77      LD      HL,TCFAIL
78      CALL   UMSG
79      JR      WSCAN
80
81 ;---- ST_SET_WIFI ----
82 WSCAN: LD      HL,TSELN
83      CALL   UTITLE
84      LD      HL,TSCAN
85      CALL   UFOOT
86      LD      E,FCSCAN
87      CALL   CD00
88      JR      C,WSC0
89      LD      A,(FNRDATA)
90

```

```

91 CP NROWS-1 ; 7 networks + the OTHER row fill the list
92 JR C,WSC1
93 LD A,NROWS-1
94 JR WSC1
95 WSC0: XOR A
96 WSC1: LD (V_NNET),A
97 LD B,0
98 WSD1: LD A,(V_NNET)
99 CP B
100 JR Z,WSDOTH
101 PUSH BC
102 LD A,B
103 LD E,FCSCRES
104 CALL CDO1
105 POP BC
106 JR C,WSDN ; failed row stays blank
107 PUSH BC
108 LD HL,FNRDATA
109 LD B,20
110 CALL CPYSAN
111 POP BC
112 PUSH BC
113 LD C,OPTFB
114 CALL USHOWR
115 POP BC
116 PUSH BC
117 LD A,(FNRDATA+33) ; RSSI, negative dBm
118 LD HL,TBAR1
119 CP 0B8H ; worse than -70
120 JR C,WSB1
121 LD HL,TBAR2
122 CP 0CAH ; worse than -55
123 JR C,WSB1
124 LD HL,TBAR3
125 WSB1: POP BC
126 PUSH BC
127 CALL UROWY
128 LD E,138
129 LD C,OPTDIM
130 SYSTEM STRDIS
131 POP BC
132 WSDN: INC B
133 JR WSD1
134 WSDOTH: CALL UROWY
135 LD E,LISTX
136 LD C,OPTHI
137 LD HL,TOTHER
138 SYSTEM STRDIS
139 LD HL,TWSFOOT
140 CALL UFOOT
141 XOR A
142 LD (V_CUR),A
143 LD B,0
144 CALL UCROFF
145 WLOOP: CALL TFWAIT
146 CP EVUP
147 JR Z,WSMU
148 CP EVDOWN
149 JR Z,WSMD
150 CP EVTRIP
151 JR Z,WSEL
152 CP EVCE
153 JP Z,WSCAN
154 JR WLOOP
155
156 WSMU: LD A,(V_CUR)
157 OR A
158 JR Z,WLOOP
159 LD B,A
160 CALL UCROFF
161 LD A,(V_CUR)

```

```

162 DEC A
163 WSET: LD (V_CUR),A
164 LD B,A
165 CALL UCROFF
166 WLOOP: JR WLOOP
167 WSDM: LD A,(V_NNET)
168 LD C,A
169 LD A,(V_CUR)
170 CP C
171 JR NC,WLOOP ; already on OTHER
172 LD B,A
173 CALL UCROFF
174 LD A,(V_CUR)
175 INC A
176 WSET: JR WSET
177
178 WSEL: LD A,(V_NNET)
179 LD C,A
180 LD A,(V_CUR)
181 CP C
182 JR Z,WSDOTH
183 LD E,FCSCRES ; re-fetch the chosen network's name
184 CALL CDO1
185 JP C,WSCAN
186 LD HL,FNRDATA
187 LD DE,V_SSID
188 LD BC,33
189 LDIR
190 JR WPASS
191 WSDOTH: XOR A
192 LD (V_ENTRY),A
193 LD HL,TESSID
194 LD A,32
195 CALL EDIT
196 JP C,WSCAN
197 LD HL,V_ENTRY
198 LD DE,V_SSID
199 LD BC,33
200 LDIR
201
202 WPASS: XOR A
203 LD (V_ENTRY),A
204 LD HL,TEPASS
205 LD A,63
206 CALL EDIT
207 JP C,WSCAN
208 LD HL,V_ENTRY
209 LD DE,V_PASS
210 LD BC,64
211 LDIR
212
213 ; WS_DONE: SET_SSID needs nparam >= 1 (value ignored) and exactly
214 ; ssid[33] + password[64] = 97 payload bytes.
215 LD HL,TSAVING
216 CALL UTITLE
217 LD A,FNDFUJI
218 LD E,FCSSID
219 LD L,1
220 CALL FNBEGIN
221 XOR A
222 CALL FNPARB
223 LD HL,V_SSID
224 LD B,33
225 CALL CTXFIX
226 LD HL,V_PASS
227 LD B,64
228 CALL CTXFIX
229 CALL CGO
230 JP NC,WCONN
231 LD HL,TSFAIL
232 CALL UMSG

```

```

233 JP WSCAN
234
235 TTITLE: DB "FUJINET CONFIG",0
236 TWAIT: DB "WAITING FOR NETWORK LINK",0
237 TCONN: DB "CONNECTING",0
238 TABORT: DB "ANY INPUT CANCELS",0
239 TCFAIL: DB "CONNECT FAILED",0
240 TSELN: DB "SELECT NETWORK",0
241 TSCAN: DB "SCANNING...",0
242 TOTHER: DB "OTHER (ENTER SSID)",0
243 TWSFOOT: DB "T:PICK CE:RESCAN",0
244 TESSID: DB "ENTER SSID",0
245 TEPASS: DB "ENTER PASSWORD",0
246 TSAVING: DB "SAVING...",0
247 TSFAIL: DB "SAVE FAILED",0
248 TBAR1: DB "##",0
249 TBAR2: DB "***",0
250 TBAR3: DB "****",0
251

```

HOSTS.INC

```

1 ; hosts.inc -- the host slot page: list, mount, rename, lobby.
2 ;
3 ; The list is drawn straight out of the READ_HOST_SLOTS reply window (8
4 ; records x 32 bytes = 256 = exactly slice 0); nothing is buffered. That
5 ; same stay-painted property powers the rename and the lobby's slot claim:
6 ; WRITE_HOST_SLOTS is streamed byte-for-byte FROM the reply window (with
7 ; the edited slot substituted from V_ENTRY), so no 256-byte RAM mirror
8 ; ever exists. The one rule: no transaction may run between the freshness
9 ; READ and the stream'd WRITE -- the editor is transaction-free, and
10 ; HWRITE re-reads immediately before writing.
11 ;
12 ; In copy mode (V_CMODE=1) the page re-skins itself as the destination
13 ; host picker, exactly like the INTV port.
14
15 HOSTPG: LD A,(V_CMODE)
16 OR A
17 LD HL,THOSTS
18 JR Z,HP1
19 LD HL,TCPVH
20 HP1: CALL UTITLE
21 LD E,FCRHS
22 CALL CDOB
23 JR NC,HP2
24 LD HL,EHOSTS
25 CALL UFAL
26 JR HOSTPG ; keep trying; the ESP32 may still be booting
27 HP2: LD B,0
28 HPD1: PUSH BC
29 LD A,B ; slot digit, accent colour
30 ADD A,'1'
31 LD (HEXBUF),A
32 XOR A
33 LD (HEXBUF+1),A
34 CALL UROWY
35 LD E,LISTX
36 LD C,OPTHI
37 LD HL,HEXBUF
38 SYSTEM STRDIS
39 POP BC
40 PUSH BC
41 LD A,B ; name from the reply window, slot*32
42 RRCA
43 RRCA
44 RRCA
45 LD L,A
46 LD H,0

```

```

47 LD DE,FNRDATA
48 ADD HL,DE
49 LD A,(HL)
50 OR A
51 JR NZ,HPD2
52 LD HL,EMPTY
53 HPD2: LD B,NAMELEN-2
54 CALL CPYSAN
55 POP BC
56 PUSH BC
57 CALL UROWY
58 LD E,LISTX+12
59 LD C,OPTFB
60 LD HL,LINBUF
61 SYSTEM STRDIS
62 POP BC
63 INC B
64 LD A,B
65 CP B
66 JR C,HPD1
67 LD A,(V_CMODE)
68 OR A
69 LD HL,THFOOT
70 JR Z,HP3
71 LD HL,THCFOOT
72 HP3: CALL UFOOT
73 LD A,(V_HOST)
74 AND Z ; RESET leaves RAM arbitrary
75 LD (V_HOST),A
76 LD B,A
77 CALL UCRO
78 HKEY: CALL INWAIT
79 CP
80 JR Z,HKU
81 CP
82 JR Z,HKD
83 CP
84 JR Z,HKT
85 CP
86 JR Z,HKE
87 CP
88 JR Z,HKC
89 CP
90 JR C,HKEY
91 CP
92 JR NC,HKEY

```

```

93 SUB EVDD
94 JP HKDIG
95 HKU: LD A,(V_HOST)
96 OR A
97 JR Z,HKEY
98 LD B,A
99 CALL UCROFF
100 LD A,(V_HOST)
101 DEC A
102 HKSET: LD (V_HOST),A
103 LD B,A
104 CALL UCRON
105 JR HKEY
106 HKD: LD A,(V_HOST)
107 CP 7
108 JR NC,HKEY
109 LD B,A
110 CALL UCROFF
111 LD A,(V_HOST)
112 INC A
113 JR HKSET
114
115 ; keypad digits: 1-8 jump to a slot; 9 = info, 0 = lobby (normal mode).
116 HKDIG: LD C,A
117 LD A,(V_CMODE)
118 OR A
119 JR NZ,HKDIG1
120 LD A,C
121 CP 9
122 JP Z,INFOPG
123 OR A
124 JP Z,LOBBY
125 HKDIG1: LD A,C
126 OR A
127 JR Z,HKEY
128 CP 9
129 JR NC,HKEY
130 DEC A
131 PUSH AF
132 LD A,(V_HOST)
133 LD B,A
134 CALL UCROFF
135 POP AF
136 JR HKSET
137
138 ; trigger: mount the slot and browse its root.
139 HKT: LD A,(V_HOST)
140 CALL CMOUNT
141 JR NC,HKT1
142 LD HL,EHOST
143 CALL UFAIL
144 JP HOSTPG
145 HKT1: CALL SRESET
146 JP SFDRAW
147
148 ; = : rename the slot through the grid editor (normal mode only).
149 HKE: LD A,(V_CMODE)
150 OR A
151 JP NZ,HKEY
152 LD A,(V_HOST)
153 RRCA
154 RRCA
155 RRCA
156 LD L,A
157 LD H,0
158 LD DE,FNRDATA
159 ADD HL,DE
160 LD DE,V_ENTRY
161 LD BC,32
162 LDIR
163 XOR A
164 LD (V_ENTRY+32),A
165 LD HL,TEHOST
166 LD A,31
167 CALL EDIT
168 JP C,HOSTPG ; cancel leaves the slot untouched
169 LD A,(V_HOST)
170 CALL HWRITE
171 JP NC,HOSTPG
172 LD HL,EHWRT
173 CALL UFAIL
174 JP HOSTPG
175
176 ; CE: cancel an in-flight copy.
177 HKC: LD A,(V_CMODE)
178 OR A
179 JP Z,HKEY
180 XOR A
181 LD (V_CMODE),A
182 JP HOSTPG
183
184 ; HWRITE: replace slot A with V_ENTRY and write all 8 slots back.
185 ; Fresh READ_HOST_SLOTS immediately first, then WRITE_HOST_SLOTS streamed
186 ; from the reply window with slot A substituted (NUL-padded to 32).
187 ; Carry set on failure.
188 HWRITE: LD (V_TMP+2),A
189 LD E,FCRHS
190 CALL CD00
191 RET C
192 LD A,FNDFUJI
193 LD E,FCWHS
194 LD L,0
195 CALL FNBEGIN
196 LD C,0
197 HWL: LD A,(V_TMP+2) ; slot being streamed
198 CP C
199 JR Z,HWD
200 LD A,C
201 RRCA
202 RRCA
203 RRCA
204 LD L,A
205 LD H,0
206 LD DE,FNRDATA
207 ADD HL,DE
208 LD B,32
209 HWC: LD A,(HL)
210 CALL FNTXBYT
211 INC HL
212 DJNZ HWC
213 HWN: LD HL,V_ENTRY
214 LD B,32
215 CALL CTXFIX
216 INC C
217 LD A,C
218 CP 8
219 JR C,HWL
220 JP CGO
221
222 ; keypad 0: find (or claim slot 8 for) the lobby host, mount it, boot the
223 ; lobby ROM.
224 LOBBY: LD HL,TLOBBY
225 CALL UTITLE
226 LD E,FCRHS
227 CALL CD00 ; fresh read; the title repaint cost nothing
228 LD NC,LB0
229 LD HL,EHOSTS
230 CALL UFAIL
231 JP HOSTPG
232 LB0: LD C,0
233 LB1: PUSH BC

```

```

235     LD      A,C
236     RRCA
237     RRCA
238     RRCA
239     LD      L,A
240     LD      H,0
241     LD      DE,FNRDATA
242     ADD     HL,DE
243     LD      DE,TLOBH
244     CALL    PCMPI
245     POP     BC
246     JR      Z,LBFND
247     INC     C
248     LD      A,C
249     CP      8
250     JR      C,LB1
251     LD      HL,TLOBH      ; not present: claim slot 8
252     LD      DE,V_ENTRY
253     CALL    USTRCP
254     LD      A,7
255     CALL    HWRITE
256     JR      NC,LB2
257     LD      HL,EHWRT
258     CALL    UFAIL
259     JP      HOSTPG
260 LB2:   LD      C,7
261 LBFND: LD      A,C
262     PUSH    BC
263     CALL    CMOUNT
264     POP     BC
265     JR      NC,LB3
266     LD      HL,EMHOST
267     CALL    UFAIL
268     JP      HOSTPG
269 LB3:   LD      A,C
270     LD      (V_HOST),A
271     LD      HL,TLOBP
272     LD      DE,V_SRC
273     CALL    USTRCP
274     LD      A,C
275     JP      BOOTGO
276
277 ; USTRCP: copy the NUL-terminated string at HL to DE, NUL included.
278 ; DE ends just past the NUL. Clobbers A,HL,DE.
279 USTRCP: LD      A,(HL)
280     LD      (DE),A
281     INC     HL
282     INC     DE
283     OR      A
284     JR      NZ,USTRCP
285     RET
286
287 ; PCMPI: case-insensitive compare of the NUL-terminated strings at HL
288 ; and DE. Z set when equal. Clobbers A,B,HL,DE.
289 PCMPI: LD      A,(DE)
290     CALL    PFOLD
291     LD      B,A
292     LD      A,(HL)
293     CALL    PFOLD
294     CP      B
295     RET     NZ
296     OR      A
297     RET     Z
298     INC     HL
299     INC     DE
300     JR      PCMPI
301
302 ; PFOLD: fold a-z to A-Z.
303 PFOLD: CP      'a'
304     RET     C
305     CP      'z'+1

```

```

306     RET     NC
307     SUB     20H
308     RET
309
310 THOSTS: DB      "FUJINET HOSTS",0
311 TCPVH:  DB      "COPY TO HOST",0
312 TEMPTY: DB      "-----",0
313 THFOOT: DB      "T:OPEN =:EDIT 9:INF 0:LOB",0
314 THCFOOT: DB     "T:PICK DEST CE:CANCEL",0
315 TEHOST: DB      "HOST NAME",0
316 TLOBBY: DB      "GAME LOBBY",0
317 TLOBH:  DB      "ec.tnfs.io",0
318 TLOBP:  DB      "/astrocade/lobby.bin",0
319 EHOSTS: DB      "HOSTS ERR",0
320 EMHOST: DB      "HOST ERR",0
321 EHWRT:  DB      "SAVE ERR",0
322

```

BROWSE.INC

```

1 ; browse.inc -- ST_SELECT_FILE: the directory browser.
2 ;
3 ; Stateless by construction, like the INTV port: every page draw is
4 ; MOUNT_HOST + OPEN_DIRECTORY(path+filter) + one SET_DIRECTORY_POSITION /
5 ; READ_DIR_ENTRY pair per row + CLOSE_DIRECTORY, drawn straight from the
6 ; reply window. Only per-row metadata survives: the absolute directory
7 ; position (16-bit -- big directories exist) and the entry type. *.cfg
8 ; entries are suppressed client-side; directories arrive with a trailing
9 ; '/' from the firmware (there is no separate flag) and get the accent
10 color; .bin carts stay white; everything else is dimmed.
11 ;
12 ; Display names are crunched to NAMELEN, so anything that must actually
13 ; OPEN (subdir descent, boot, copy) re-reads the row at FULLLEN first.
14 ;
15 ; SRESET: back to the root of the mounted host, paging cleared.
16 SRESET: LD HL,V_PATH
17 LD (HL),'/'
18 INC HL
19 LD (HL),0
20 SRST1: LD HL,0
21 LD (V_DPOS),HL
22 XOR A
23 LD (V_PDEP),A
24 LD (V_CUR),A
25 RET
26 ;
27 ; SFRSET: paging cleared (path kept), then redraw.
28 SFRSET: CALL SRST1
29 JP SFDRAW
30 ;
31 ; ---- page draw -----
32 SFDRAW: CALL UCLRA
33 LD HL,V_PATH ; title row = tail of the path
34 LD B,26
35 CALL UTAIL
36 LD HL,LINBUF
37 LD DE,TITLE*256+2
38 LD C,OPTDIM
39 SYSTEM
40 LD A,(V_HOST)
41 CALL CMOUNT
42 JP C,SFERR
43 LD A,(V_HOST)
44 CALL CODIR
45 JP C,SFERR
46 XOR A
47 LD (V_CNT),A
48 LD HL,(V_DPOS)
49 LD (V_VNEXT),HL ; running directory position
50 SFD1: LD A,(V_CNT)
51 CP NROWS
52 JR NC,SFDE
53 LD HL,(V_DNEXT)
54 LD A,LISTLEN
55 CALL CDE16
56 JR C,SFDE ; end of directory (or error)
57 CALL NISCFG
58 JR NZ,SFD2
59 LD HL,(V_DNEXT) ; a .cfg: suppress and move on
60 INC HL
61 LD (V_DNEXT),HL
62 JR SFD1
63 SFD2: LD A,(V_CNT) ; V_EPOS[row] = position
64 ADD A,A
65 LD E,A
66 LD D,0
67 LD HL,V_EPOS
68 ADD HL,DE
69 LD DE,(V_DNEXT)
70 LD (HL),E
71 INC HL
72 LD (HL),D
73 CALL STYPE ; V_EDIR[row] = type
74 LD E,A
75 LD A,(V_CNT)
76 LD C,A
77 LD B,0
78 LD HL,V_EDIR
79 ADD HL,BC
80 LD (HL),E
81 LD A,E ; row color by type
82 LD C,OPTHI
83 CP 1
84 JR Z,SFD3
85 LD C,OPTFB
86 CP 2
87 JR Z,SFD3
88 LD C,OPTDIM
89 SFD3: PUSH BC
90 LD HL,FNRDATA
91 LD B,NAMELEN
92 CALL CPYSAN
93 POP BC
94 LD A,(V_CNT)
95 LD B,A
96 CALL USHOWR
97 LD A,(V_CNT)
98 INC A
99 LD (V_CNT),A
100 LD HL,(V_DNEXT)
101 INC HL
102 LD (V_DNEXT),HL
103 JP SFD1
104 SFDE: CALL CODIR ; stateless: close what we opened
105 LD A,(V_CNT)
106 OR A
107 JR NZ,SFDF
108 LD HL,(V_DPOS) ; empty page
109 LD A,H
110 OR L
111 JR NZ,SFPOPE ; paged past the end: step back
112 LD A,(V_PATH+1) ; empty at position 0:
113 OR A
114 JR NZ,SFEMPT ; in a subdir, or
115 LD A,(V_FILT)
116 OR A
117 JR NZ,SFEMPT ; filtered -- otherwise the root is
118 JP HOSTPG ; genuinely empty: back to hosts
119 SFEMPT: LD HL,TEMPTD
120 CALL UFOOT
121 JR SFLOOP
122 SFPOPE: CALL SPOPPG
123 JP SFDRAW
124 SFDF: LD A,(V_CMODE)
125 OR A
126 LD HL,TSFOOT
127 JR Z,SFD4
128 LD HL,TSCFOOT
129 SFD4: CALL UFOOT
130 LD A,(V_CNT)
131 LD B,A
132 LD A,(V_CUR)

```



```

133 CP B
134 JR C,SFD5
135 LD A,B
136 DEC A
137 LD (V_CUR),A
138 SFD5: LD A,(V_CUR)
139 LD B,A
140 CALL UCRON
141
142 ; ----- input loop -----
143 SFLOOP: CALL INWAIT
144 CP EVUP
145 JR Z,SKU
146 CP EVDOWN
147 JR Z,SKD
148 CP EVLEFT
149 JR Z,SKPGB
150 CP EVRIGHT
151 JR Z,SKPGF
152 CP EVTRIG
153 JP Z,SKT
154 CP EVCE
155 JP Z,SKCE
156 CP EVD0+1
157 JP Z,SKUPD
158 CP EVD0+2
159 JR Z,SKPGB
160 CP EVD0+3
161 JR Z,SKPGF
162 CP EVD0+4
163 JP Z,SKFLT
164 CP EVD0+5
165 JP Z,SKCPY
166 JR SFLOOP
167
168 SKU: LD A,(V_CUR)
169 OR A
170 JR Z,SFLOOP
171 LD B,A
172 CALL UCROFF
173 LD A,(V_CUR)
174 DEC A
175 SKSET: LD (V_CUR),A
176 LD B,A
177 CALL UCRON
178 JR SFLOOP
179 SKD: LD A,(V_CNT)
180 OR A
181 JR Z,SFLOOP
182 DEC A
183 LD B,A
184 LD A,(V_CUR)
185 CP B
186 JR NC,SFLOOP
187 LD B,A
188 CALL UCROFF
189 LD A,(V_CUR)
190 INC A
191 JR SKSET
192
193 SKPGB: LD A,(V_PDEP)
194 OR A
195 JR Z,SFLOOP
196 CALL SPOPPG
197 XOR A
198 LD (V_CUR),A
199 JP SFDRAW
200 SKPGF: LD A,(V_CNT)
201 CP NROWS
202 JR C,SFLOOP
203 LD A,(V_PDEP)
204 CP 8
205 JP NC,SFLOOP
206 ADD A,A
207 LD E,A
208 LD D,0
209 LD HL,V_PSTK
210 ADD HL,DE
211 LD DE,(V_DPOS)
212 LD (HL),E
213 INC HL
214 LD (HL),D
215 LD A,(V_PDEP)
216 INC A
217 LD (V_PDEP),A
218 LD HL,(V_DNEXT)
219 LD (V_DPOS),HL
220 XOR A
221 LD (V_CUR),A
222 JP SFDRAW
223
224 ; SPOPPG: pop the page stack into V_DPOS (or land on 0).
225 SPOPPG: LD A,(V_PDEP)
226 OR A
227 JR Z,SPP0
228 DEC A
229 LD (V_PDEP),A
230 ADD A,A
231 LD E,A
232 LD D,0
233 LD HL,V_PSTK
234 ADD HL,DE
235 LD A,(HL)
236 INC HL
237 LD H,(HL)
238 LD L,A
239 LD (V_DPOS),HL
240 RET
241 SPP0: LD HL,0
242 LD (V_DPOS),HL
243 RET
244
245 ; keypad 1: up one directory (the path always ends in '/').
246 SKUPD: LD A,(V_PATH+1)
247 OR A
248 JP Z,SFLOOP ; at the root
249 LD HL,V_PATH
250 CALL USTRLN
251 HL ; the trailing '/'
252 SKUP1: DEC HL
253 LD A,(HL)
254 CP A, '/'
255 JR NZ,SKUP1
256 INC HL
257 LD (HL),0
258 JPSFRSET
259
260 ; trigger: descend into a directory, or act on a file.
261 SKT: LD A,(V_CNT)
262 OR A
263 JP Z,SFLOOP
264 LD A,(V_CUR)
265 LD E,A
266 LD D,0
267 LD HL,V_EDIR
268 ADD HL,DE
269 LD A,(HL)
270 CP 1
271 JR Z,SADV
272 LD A,(V_CMODE)
273 OR A
274 JP NZ,CPHERE ; mid-copy: a file pick means "copy here"

```

; only page forward off a full page

```

275 JP BOOTSEL
276
277 ; descend: full-length re-read, append (the firmware's trailing '/')
278 ; included), restart paging.
279 SADV: CALL SFULL
280 JP C,SFDRAW
281 LD HL,V_PATH
282 CALL USTRLN
283 LD B,C
284 HL PUSH ; the path's NUL
285 LD HL,V_ENTRY
286 CALL USTRLN
287 POP DE
288 LD A,B
289 ADD A,C
290 JR C,SADVF
291 CP 190
292 JR NC,SADVF
293 LD HL,V_ENTRY
294 CALL USTRCP
295 JP SFRSET
296 SADV: LD HL,TPLONG
297 CALL UMSG
298 JP SFDRAW
299
300 ; keypad 4: edit the filter (a scratch copy -- ESC must not be
301 ; destructive). A new pattern rennumbers everything: paging restarts.
302 SKFLT: LD HL,V_FILT
303 LD DE,V_ENTRY
304 LD BC,32
305 LDIR
306 LD HL,TEFILT
307 LD A,31
308 CALL EDIT
309 JP C,SFRSET
310 LD HL,V_ENTRY
311 LD DE,V_FILT
312 LD BC,32
313 LDIR
314 JP SFRSET
315
316 ; keypad 5: mark a copy source / perform the copy (copy.inc).
317 SKCPY: LD A,(V_CMODE)
318 OR A
319 JP NZ,CPHERE
320 JP CPMARK
321
322 ; CE: back to the hosts (cancels an in-flight copy).
323 SKCE: XOR A
324 LD (V_CMODE),A
325 JP HOSTPG
326
327 SFERR: LD HL,EODIR
328 CALL UFAIL
329 JP HOSTPG
330
331 ; ---- helpers -----
332 ; SFULL: re-read the selected row's name at FULLLEN into V_ENTRY.
333 ; The directory is opened and closed here (stateless). Carry set on failure.
334 SFULL: LD A,(V_HOST)
335 CALL CODIR
336 RET C
337 LD A,(V_CUR)
338 ADD A,A
339 LD E,A
340 LD D,0
341 LD HL,V_EPOS
342 ADD HL,DE
343 LD A,(HL)
344 INC HL
345 LD H,(HL)

346 LD L,A
347 LD A,FULLLEN
348 CALL CODE16
349 JR C,SFULLF
350 LD HL,FNRDATA
351 LD DE,V_ENTRY
352 LD BC,FULLLEN
353 LDIR
354 XOR A
355 LD (V_ENTRY+FULLLEN),A
356 CALL CCDIR
357 OR A
358 RET
359 SFULLF: CALL CCDIR
360 SCF
361 RET
362
363 ; SISCFG: Z set when the name in the reply window ends ".cfg" (any case).
364 SISCFG: LD HL,FNRDATA
365 CALL USTRLN
366 LD A,C
367 CP 4
368 JR C,SNCFG
369 DEC HL
370 DEC HL
371 DEC HL
372 DEC HL
373 LD A,(HL)
374 CP '.'
375 JR NZ,SNCFG
376 INC HL
377 LD A,(HL)
378 CALL PFOLD
379 CP 'C'
380 JR NZ,SNCFG
381 INC HL
382 LD A,(HL)
383 CALL PFOLD
384 CP 'F'
385 JR NZ,SNCFG
386 INC HL
387 LD A,(HL)
388 CALL PFOLD
389 CP 'G'
390 RET
391 SNCFG: LD A,1
392 OR A
393 RET
394
395 ; STYPE: A = type of the name in the reply window:
396 ; 1 = directory (trailing '/'), 2 = .bin cart, 0 = anything else.
397 STYPE: LD HL,FNRDATA
398 CALL USTRLN
399 LD A,C
400 OR A
401 JR Z,STY0
402 DEC HL
403 LD A,(HL)
404 CP '/'
405 JR NZ,STY1
406 LD A,1
407 RET
408 STY1: LD A,C
409 CP 4
410 JR C,STY0
411 DEC HL
412 DEC HL
413 LD A,(HL)
414 CP '.'
415 JR NZ,STY0

```

```

417 INC HL
418 LD A,(HL)
419 CALL PFOLD
420 CP 'B'
421 JR NZ,STY0
422 INC HL
423 LD A,(HL)
424 CALL PFOLD
425 CP 'I'
426 JR NZ,STY0
427 INC HL
428 LD A,(HL)
429 CALL PFOLD
430 CP 'N'
431 JR NZ,STY0
432 LD A,2
433 RET
434 STY0: XOR A
435 RET
436
437 TEMPTD: DB "<EMPTY> 4:FLT 1:UP CE",0
438 TSFOOT: DB "1:UP 2/3:PG 4:FLT 5:CPY CE",0
439 TSCFOOT: DB "5:COPY HERE CE:CANCEL",0
440 TEFILT: DB "FILTER (!NAME-SUBDIRS)",0
441 TPLONG: DB "PATH TOO LONG",0
442 EODIR: DB "DIR ERR",0
443

```

COPY.INC

```

1 ; copy.inc -- COPY_FILE between hosts, with the .cfg sibling.
2
3 ; Ports intv/st_copy.bas: keypad 5 on a file records its full path in
4 ; V_SRC and re-skins the host page as "COPY TO HOST"; picking a slot
5 ; drops into that host's root; 5 again (or a trigger on any file) copies
6 ; into the directory being browsed. The copySpec is "srcfulpath|destdir"
7 ; basename(src)" -- basename only, so a "lname" recursive-filter hit like
8 ; "roms/a/game.bin" never asks the destination for directories it doesn't
9 ; have -- sent at EXACT length (no NUL, no padding) with 1-BASED host
10 ; slots.
11
12 ; After a copy of a ROM with a 4-character extension the copySpec is sent
13 ; again with both extensions swapped to ".cfg" (destination lowercase --
14 ; the spelling the firmware tries first), retrying the SOURCE as ".CFG"
15 ; for case-sensitive hosts. A miss is normal, not a failure. Unlike INTV,
16 ; which patches its TX buffer in place, the TX stream here is write-only,
17 ; so each attempt rebuilds the spec (CPSEND).
18
19 ; keypad 5, normal mode: mark the selected file as the copy source.
20 CPMARK: LD A,(V_CNT)
21 OR A
22 JP Z,SFLOOP
23 LD A,(V_CUR)
24 LD E,A
25 LD D,0
26 LD HL,V_EDIR
27 ADD HL,DE
28 LD A,(HL)
29 CP 1
30 JR NZ,CPM1
31 LD HL,TNODIR
32 CALL UMSG
33 JP SFDRAW
34 CPM1: CALL SFULL
35 JP C,SFDRAW
36 CALL SBLDSRC
37 JP C,SFDRAW
38 LD A,(V_HOST)

```

```

39 LD (V_CHOST),A
40 LD A,1
41 LD (V_CMODE),A
42 JP HOSTPG
43
44 ; SBLDSRC: V_SRC = V_PATH + V_ENTRY (also the boot path builder).
45 ; Carry set -- message already shown -- when it wouldn't fit.
46 SBLDSRC: LD HL,V_PATH
47 CALL USTRLN
48 LD B,C
49 LD HL,V_ENTRY
50 CALL USTRLN
51 LD A,B
52 ADD A,C
53 JR C,SBLONG
54 CP 224
55 JR NC,SBLONG
56 LD HL,V_PATH
57 LD DE,V_SRC
58 CALL USTRCP
59 DEC DE
60 LD HL,V_ENTRY
61 CALL USTRCP
62 OR A
63 RET
64 SBLONG: LD HL,TPLONG
65 CALL UMSG
66 SCF
67 RET
68
69 ; the copy itself, into the directory being browsed.
70 CPHERE: LD HL,V_SRC
71 CALL USTRLN
72 LD A,C
73 LD (V_TMP),A ; source length
74 DEC HL ; find the basename (V_SRC begins '/')
75 LD A,(HL)
76 CP '/'

```

```

77 JR NZ,CPH1
78 INC HL
79 LD (V_TMP+2),HL ; basename pointer
80 CALL USTRLN
81 LD A,C
82 LD (V_TMP+4),A ; basename length
83 LD HL,V_PATH
84 CALL USTRLN ; C = dest dir length
85 LD A,(V_TMP) ; spec length = src + 1 + destdir + base
86 ADD A,C
87 JR C,CPLONG
88 INC A
89 JR C,CPLONG
90 LD B,A
91 LD A,(V_TMP+4)
92 ADD A,B
93 JR C,CPLONG
94 CP 250
95 JR NC,CPLONG
96 LD HL,TCFPG
97 CALL UFOOT
98 XOR A ; the real copy, original extensions
99 CALL CPSEND
100 JR C,CPFAIL
101 LD A,(V_TMP) ; .cfg sibling only for ".xxx" sources
102 CP S
103 JR C,CPOK
104 LD E,A
105 LD D,0
106 LD HL,V_SRC
107 ADD HL,DE
108 DEC HL
109 DEC HL
110 DEC HL
111 LD HL
112 LD A,(HL)
113 CP ','
114 JR NZ,CPOK
115 LD A,(V_TMP+4)
116 CP S ; ".bin" alone has no basename
117 JR C,CPOK
118 LD A,1
119 CALL CPSEND
120 JR NC,CPOK
121 LD A,2
122 CALL CPSEND ; case-sensitive host: source as .CFG
123 CPOK: LD HL,TCPOK
124 CALL UMSG
125 JR CPBACK
126 CPLONG: LD HL,TPLONG
127 CALL UMSG
128 JP SFDRAW
129 CPFAIL: LD HL,ECOPY
130 CALL UFAIL
131
132 ; back to where the copy started: source host, source directory,
133 ; unfiltered (the filter belonged to the destination browse).
134 CPBACK: XOR A
135 LD (V_CMODE),A
136 LD (V_FFLT),A
137 LD A,(V_CHOST)
138 LD (V_HOST),A
139 LD HL,(V_TMP+2)
140 LD DE,V_SRC
141 OR A
142 SBC HL,DE ; HL = basename offset
143 LD A,H
144 OR A
145 JR NZ,CPROOT
146 LD A,L
147 CP 191
148 JR NC,CPROOT ; deeper than V_PATH holds: land on root
149 LD C,A
150 LD B,0
151 LD HL,V_SRC
152 LD DE,V_PATH
153 LDIR
154 XOR A
155 LD (DE),A
156 JP SFRSET
157 LD HL,V_PATH
158 LD (HL), '/'
159 INC HL
160 LD (HL),0
161 JP SFRSET
162
163 ; CPSEND: send one COPY_FILE. A = extension mode: 0 = as recorded,
164 ; 1 = both ends ".cfg", 2 = source ".CFG", destination ".cfg".
165 ; Carry set on failure.
166 CPSEND: LD (V_TMP+5),A
167 LD A,FNDUJ1
168 LD E,FCCOPY
169 LD L,2
170 CALL FNBEGIN
171 LD A,(V_CHOST) ; 1-BASED slots, source then destination
172 INC A
173 CALL FNPARB
174 LD A,(V_HOST)
175 INC A
176 CALL FNPARB ; source path
177 LD A,(V_TMP)
178 LD B,A
179 LD HL,V_SRC
180 LD A,(V_TMP+5)
181 OR A
182 JR Z,CPS1
183 LD A,B
184 SUB 3
185 LD B,A
186 CALL CTXN
187 LD A,(V_TMP+5)
188 LD HL,TCFGL
189 CP 2
190 JR NZ,CPS2
191 LD HL,TCFGL
192 CALL FNTXSTR
193 JR CPS3
194 CPS1: CALL CTXN
195 CPS3: LD A,'|'|
196 CALL FNTXBYT
197 LD HL,V_PATH ; destination directory (ends '/')
198 CALL FNTXSTR
199 LD HL,(V_TMP+2) ; basename
200 LD A,(V_TMP+5)
201 OR A
202 JR Z,CPS4
203 LD A,(V_TMP+4)
204 SUB 3
205 LD B,A
206 CALL CTXN
207 LD HL,TCFGL ; destination always lowercase
208 CALL FNTXSTR
209 JR CPS5
210 CPS4: CALL FNTXSTR
211 CPS5: JP CGO
212
213 ; CTXN: stream exactly B raw bytes from HL. Clobbers A,B,D,E,HL.
214 CTXN: LD A,(HL)
215 CALL FNTXBYT
216 INC HL
217 DJNZ CTXN
218 RET

```

```

219
220 TNODIR: DB      "CANNOT COPY A FOLDER",0
221 TCPING: DB      "COPYING...",0
222 TCPOK:  DB      "COPIED",0
223 ECOPY:  DB      "COPY FAILED",0
224 TCFGU:  DB      "CFG",0
225 TCFGU:  DB      "CFG",0
226

```

BOOT.INC

```

1 ; boot.inc -- SET_DEVICE_FULLPATH + MOUNT_IMAGE with a progress bar,
2 ; then the swap.
3
4 ; MOUNT_IMAGE makes the ESP32 pull the file and push it to the cartridge
5 ; over the DBC state-channel WHILE the transaction is still outstanding, so
6 ; the commit loop here is FNCOMMIT's proven shape (SEQ from ACKSEQ, never
7 ; a local counter) with one addition: it watches FN_R_BOOT_PCT and paints
8 ; the bar. Only passive status-window reads happen while the transaction
9 ; is in flight -- no hotspots.
10
11 ; On success the cart publishes BOOT_READY; we arm BOOTLOCK, copy the
12 ; 6-byte stub to the top of screen RAM and jump: the cart flips to the
13 ; staged image between the stub's swap read and the JP 0, and the OS
14 ; cold-starts into it. A staged image carrying the "FUJI" claim keeps the
15 ; mailbox after tcpv swap; a plain commercial dump runs as a plain ROM
16 ; with the mailbox dead. Both are firmware policy -- nothing to do here.
17 ; CONFIG_BOOT (0D9H) is never sent.
18
19 BARY EQU 40 ; progress bar: y, raw start byte, width
20 BARB EQU 8
21 BARW EQU 25 ; 25 bytes = 100 px = 4 px per percent
22
23 ; boot the browser's selection: V_SRC = V_PATH + full entry name.
24 BOOTSEL:CALL SFULL
25 ; C,SFDRAW
26 CALL SBLDSRC
27 JP C,SFDRAW
28 LD A,(V_HOST)
29
30 ; BOOTGO: boot the file at V_SRC from host slot A.
31 BOOTGO: LD (V_TMP+6),A
32 LD HL,TBOOT
33 CALL UTITLE
34 LD HL,V_SRC
35 LD B,26
36 CALL UTAIL
37 LD B,0
38 LD C,OPTDIM
39 CALL USHOWR
40 LD HL,TNPOW
41 CALL UFOOT
42 LD A,FNDFUJI
43 LD E,FCSDFP
44 LD L,3
45 CALL FNBEGIN
46 LD A,DEVSL0T
47 CALL FNPARB
48 LD A,(V_TMP+6)
49 CALL FNPARB
50 LD A,FMREAD
51 CALL FNPARB
52 LD C,0
53 LD HL,V_SRC
54 CALL CTXSTR
55 CALL CPAD
56 CALL CGO
57 JR NC,BGMNT

```

```

58 LD HL,EPATH
59 BGFAIL: CALL UFALL
60 JP HOSTPG
61 BGMNT: CALL BARBG
62 LD A,FNDFUJI
63 LD E,FCMING
64 LD L,2
65 CALL FNBEGIN
66 LD A,DEVSL0T
67 CALL FNPARB
68 LD A,FMREAD
69 CALL FNPARB
70 CALL BCOMMIT
71 JR NC,BGRDY
72 PUSH AF
73 LD A,(FNBSTAT)
74 CP FBFAIL
75 JR NC,BGBERR2
76 POP AF
77 LD HL,EMOUNT
78 JR BGFAIL
79 BGBERR2: JP AF
80 BGBERR: CALL BERRNAM
81 LD A,(FNBERR)
82 JR BGFAIL
83 BGRDY: LD BC,3000 ; ~30 s for the stage to settle
84 BW1: LD A,(FNBSTAT)
85 CP FBREADY
86 JR Z,BREADY
87 CP FBFAIL
88 JR NC,BGBERR
89 PUSH BC
90 CALL DELAY10
91 POP BC
92 DEC BC
93 LD A,B
94 OR C
95 JR NZ,BW1
96 LD HL,ESTAGE
97 LD A,BFFH
98 JR BGFAIL
99 BREADY: LD A,100
100 CALL BARUPD
101 LD C,FRBOOTL
102 LD A,FNBLMAG
103 CALL FNRGWR
104 LD HL,BSTUB
105 LD DE,STUB
106 LD BC,BSTUBL
107 LDIR
108 JP STUB
109 BSTUB: LD A,(FNSWAP)
110 JP 0
111 BSTUBL EQU $-BSTUB
112
113 ; BCOMMIT: commit the staged MOUNT_IMAGE, painting FN_R_BOOT_PCT into the
114 ; bar while the transaction is outstanding. ~120 s budget. Carry set on

```

```

115 ; timeout / transport error / NAK, exactly like CG0.
116 BCOMMIT: LD A,0FFH
117 LD (V_TMP),A ; last percent drawn
118 LD A,(FNACKSQ)
119 INC A
120 JR NZ,BC1
121 INC A ; 255 wraps past the reserved 0
122 BC1: LD H,A
123 LD C,FRSEQ
124 CALL FNRGWR
125 LD B,60 ; 60 x ~2 s
126 BCO: LD DE,0
127 BCI: LD A,(FNACKSQ)
128 CP H
129 JR Z,BCOK
130 LD A,(FNBPCCT)
131 LD L,A
132 LD A,(V_TMP)
133 CP L
134 JR Z,BCSKIP
135 LD A,L
136 LD (V_TMP),A
137 PUSH HL
138 PUSH DE
139 BC PUSH
140 CALL BARUPD
141 POP BC
142 POP DE
143 POP HL
144 BCSKIP: DEC DE
145 LD A,D
146 OR E
147 JR NZ,BCI
148 DJNZ BCO
149 SCF
150 RET
151 BCOK: LD A,(FNERR)
152 OR A
153 JR NZ,BCF
154 LD A,(FNREPLY)
155 CP FCACK
156 RET Z
157 LD A,(FNERR)
158 BCF: SCF
159 RET
160
161 ; BARBG: paint the empty bar (8 lines of color-1 pixels).
162 BARBG: LD HL,NORMEM+BARY+BYTEPL+BARB
163 LD B,8
164 BB1: PUSH BC
165 PUSH HL
166 LD BC,BARW
167 LD A,55H
168 CALL MSET
169 HL POP
170 LD DE,BYTEPL
171 ADD HL,DE
172 POP BC
173 DJNZ BB1
174 RET
175
176 ; BARUPD: A = percent 0-100: fill pct/4 bytes per line, percent text under.
177 BARUPD: PUSH AF
178 SRL A
179 SRL A
180 OR A
181 JR Z,BAR2
182 LD C,A
183 LD HL,NORMEM+BARY+BYTEPL+BARB
184 LD B,8
185 BU1: PUSH BC

```

```

186 PUSH HL
187 LD B,0
188 LD A,0FFH
189 CALL MSET
190 HL POP
191 LD DE,BYTEPL
192 ADD HL,DE
193 POP BC
194 DJNZ BU1
195 BAR2: POP AF
196 CALL UPCT
197 LD HL,HEXBUF
198 LD DE,(BARY+12)*256+72
199 LD C,OPTFB
200 SYSTEM STRDIS
201 RET
202
203 ; BERRNAM: HL = the message for the FN_R_BOOT_ERR code.
204 BERRNAM: LD A,(FNBERR)
205 LD HL,EBTOOB
206 CP 1
207 RET Z
208 LD HL,EBTRUNC
209 CP 2
210 RET Z
211 LD HL,EBNOMAP
212 CP 3
213 RET Z
214 LD HL,ESTAGE
215 RET
216
217 TBOOT: DB "BOOTING",0
218 TNOPOW: DB "DO NOT POWER OFF",0
219 EPATH: DB "PATH ERR",0
220 EMOUNT: DB "MOUNT ERR",0
221 ESTAGE: DB "STAGE ERR",0
222 EBTOOB: DB "ROM TOO BIG",0
223 EBTRUNC: DB "TRUNCATED XFÉR",0
224 EBNOMAP: DB "NO MAPPING",0
225

```

INFO.INC

```
1 ; info.inc -- keypad 9: SSID / IP / firmware version.
2 ;
3 ; One GET_ADAPTERCONFIG_EXTENDED; the 240-byte reply fits slice 0 and is
4 ; rendered straight from the window: ssid at +0 (drawn with the lowercase-
5 ; capable blitter -- SSIDs are case-sensitive), fn_version at +125,
6 ; sLocalIP at +140.
7
8 INFOFG: LD      HL,TINFO
9          CALL   UTITLE
10         LD      E,FCACFGX
11         CALL   CD00
12         JR      NC,IN1
13         LD      HL,EINFO
14         CALL   UFAIL
15         JP      HOSTPG
16 IN1:     LD      B,0
17         LD      HL,TLSSID
18         CALL   INLBL
19         LD      HL,FNRDATA
20         LD      DE,(LISTY0+8)*256+1
21         LD      C,GCNORM
22         LD      B,20
23         CALL   DRWSTR
24         LD      B,3
25         LD      HL,TLIP
26         CALL   INLBL
27         LD      HL,FNRDATA+140
28         LD      B,16
29         CALL   CPYSAN
30         LD      B,4
31         CALL   INVAL
32         LD      B,6
33         LD      HL,TLVER
34         CALL   INLBL
35         LD      HL,FNRDATA+125
36         LD      B,15
37         CALL   CPYSAN
38         LD      B,7
39         CALL   INVAL
40         LD      HL,TIFOOT
41         CALL   UFOOT
42 INLOOP:  CALL   INWAIT
43         CP      EVTRIG
44         JR      Z,INBACK
45         CP      EVCE
46         JR      NZ,INLOOP
47 INBACK:  JP      HOSTPG
48
49 ; INLBL: label at HL on list row B. INVAL: LINBUF on list row B.
50 INLBL:   PUSH   HL
51         CALL   UROWY
52         POP    HL
53         LD      E,LISTX
54         LD      C,OPTDIM
55         SYSTEM STRDIS
56         RET
57 INVAL:   LD      C,OPTFB
58         JP      USHOWR
59
60 TINFO:   DB      "FUJINET INFO",0
61 TLSSID:  DB      "SSID",0
62 TLIP:    DB      "IP",0
63 TLVER:   DB      "VERSION",0
64 TIFOOT:  DB      "T:BACK",0
65 EINFO:   DB      "INFO ERR",0
66
```

EDIT.INC

```

1 ; edit.inc -- the on-screen character-grid text editor.
2 ;
3 ; Ports the INTV grid entry: a 16x4 grid of 20H-5FH, a CASE toggle that
4 ; turns the letter rows lowercase (drawn for real -- font.inc), a value
5 ; line showing the tail of the buffer with an inverse-video cursor cell,
6 ; and an action row (CASE SPC DEL OK ESC). Keypad shortcuts: digits type
7 ; directly, . types a dot, CE backspaces, = accepts. Cancel is the ESC
8 ; cell only, so a reflexive CE while backspacing can't discard the edit.
9 ;
10 ; Contract: caller preloads V_ENTRY (NUL-terminated) and passes the title
11 ; in HL and the max length in A. The editor runs NO transactions, so a
12 ; reply window the caller is holding stays valid throughout. Returns
13 ; carry set on cancel. V_ENTRY is mutated either way; callers editing
14 ; durable data copy out only on accept.
15 ;
16 EVALY EQU 12 ; value line y
17 EGRDX EQU 2 ; grid origin, x byte pair
18 EGRDY EQU 28 ; grid origin, y
19 EGPITCH EQU 10 ; grid row pitch
20 ;
21 ; EDIT: HL = title, A = max length (<= 127). Carry set on cancel.
22 EDIT: LD (V_GMAX),A
23 PUSH HL
24 LD HL,V_ENTRY
25 CALL USTRLN
26 LD A,C
27 LD (V_GLEN),A
28 XOR A
29 LD (V_GCASE),A
30 LD (V_GX),A
31 LD A,2 ; start on the A-0 row
32 LD (V_GY),A
33 POP HL
34 CALL UTITLE
35 CALL EVDRAW
36 CALL EGDRAW
37 CALL EADRAW
38 ELOOP: CALL INWAIT
39 CP EVUP
40 JP Z,EMVU
41 CP EVDOWN
42 JP Z,EMVD
43 CP EVLEFT
44 JP Z,EMVL
45 CP EVRIGHT
46 JP Z,EMVR
47 CP EVTRIG
48 JP Z,EACT
49 CP EVCE
50 JP Z,EDELK
51 CP EVEQ
52 JP Z,EOK
53 CP EVDOT
54 JR NZ,ELP1
55 LD A,' '
56 JP EAPP
57 ELP1: CP EVD0
58 JR C,ELOOP
59 CP EVD0*10
60 JR NC,ELOOP
61 SUB EVD0
62 ADD A,'0'
63 JP EAPP
64 ; ---- movement -----
65 EMVU: LD A,(V_GY)
66
67 OR A
68 JP Z,ELOOP
69 CP 4
70 JR Z,EMVU4
71 CALL ECNRM
72 LD HL,V_GY
73 DEC (HL)
74 CALL ECINV
75 ELOOP
76 EMVU4: LD A,3 ; leave the action row: gx = gx+3+1
77 LD (V_GY),A
78 LD A,(V_GX)
79 LD B,A
80 ADD A,A
81 ADD A,B
82 INC A
83 LD (V_GX),A
84 CALL EADRAW
85 CALL ECINV
86 JP ELOOP
87
88 EMVD: LD A,(V_GY)
89 CP 4
90 JP Z,ELOOP
91 CP 3
92 JR Z,EMVD4
93 CALL ECNRM
94 LD HL,V_GY
95 INC (HL)
96 CALL ECINV
97 JP ELOOP
98 EMVD4: CALL ECNRM ; enter the action row: gx = gx+5/16
99 LD A,4
100 LD (V_GY),A
101 LD A,(V_GX)
102 LD B,A
103 ADD A,A
104 ADD A,A
105 ADD A,B
106 SRL A
107 SRL A
108 SRL A
109 LD (V_GX),A
110 CALL EADRAW
111 JP ELOOP
112
113 EMVL: LD A,(V_GX)
114 OR
115 JP Z,ELOOP
116 LD A,(V_GY)
117 CP 4
118 JR Z,EMVL4
119 CALL ECNRM
120 LD HL,V_GX
121 DEC (HL)
122 CALL ECINV
123 JP ELOOP
124 EMVL4: LD HL,V_GX
125 DEC (HL)
126 CALL EADRAW
127 JP ELOOP
128
129 EMVR: LD A,(V_GY)
130 CP 4
131 JR Z,EMVR4

```



```

133 LD A,(V_GX)
134 CP 15
135 JP NC,ELOOP
136 CALL ECNRM
137 LD HL,V_GX
138 INC (HL)
139 CALL ECINV
140 JP ELOOP
141 EMVR4: LD A,(V_GX)
142 CP 4
143 JP NC,ELOOP
144 LD HL,V_GX
145 INC (HL)
146 CALL EADRAW
147 JP ELOOP
148
149 ; ---- actions -----
150 EACT: LD A,(V_GY)
151 CP 4
152 JR Z,EACT4
153 RLCA ; grid char: 20H + gy*16 + gx
154 RLCA
155 RLCA
156 LD B,A
157 LD A,(V_GX)
158 ADD A,B
159 ADD A,20H
160 CALL EMAPC
161 EAPP: JR A,(V_GX)
162 EACT4: LD A,(V_GX)
163 OR A
164 JR Z,ECASE
165 CP 1
166 LD A,1
167 JR Z,EAPP
168 LD A,(V_GX)
169 CP 2
170 JR Z,EDELK
171 CP 3
172 JR Z,EOK
173 SCF
174 RET ; ESC
175
176 EOK: OR A
177 RET
178
179 ECASE: LD A,(V_GCASE)
180 XOR 1
181 LD (V_GCASE),A
182 CALL EGDRAW
183 JP ELOOP
184
185 EAPP: LD B,A
186 LD A,(V_GMAX)
187 LD C,A
188 LD A,(V_GLEN)
189 CP C
190 JP NC,ELOOP
191 LD E,A
192 LD D,0
193 LD HL,V_ENTRY
194 ADD HL,DE
195 LD (HL),B
196 INC HL
197 LD (HL),0
198 INC A
199 LD (V_GLEN),A
200 CALL EVDRAW
201 JP ELOOP
202
203
204 EDELK: LD A,(V_GLEN)
205 OR A
206 JP Z,ELOOP
207 DEC A
208 LD (V_GLEN),A
209 LD E,A
210 LD D,0
211 LD HL,V_ENTRY
212 ADD HL,DE
213 LD (HL),0
214 CALL EVDRAW
215 JP ELOOP
216
217 ; EMAPC: apply the CASE toggle to grid character A.
218 EMAPC: PUSH BC
219 LD B,A
220 LD A,(V_GCASE)
221 OR A
222 LD A,B
223 JR Z,EMC9
224 CP 'A'
225 JR C,EMC9
226 CP 'Z'+1
227 JR NC,EMC9
228 ADD A,20H
229 EMC9: POP BC
230 RET
231
232 ; ---- drawing -----
233 ; ECINV / ECNRM: draw the cell under the grid cursor (no-op on the
234 ; action row, which EADRAW owns).
235 ECINV: LD C,GCINV
236 JR ECDRW
237 ECNRM: LD C,GCNORM
238 ECDRW: LD A,(V_GY)
239 CP 4
240 RET NC
241 LD B,A
242 ADD A,A
243 ADD A,A
244 ADD A,B
245 ADD A,A
246 ADD A,EGRDY
247 LD D,A
248 LD A,B
249 RLCA
250 RLCA
251 RLCA
252 LD B,A
253 LD A,(V_GX)
254 LD E,A
255 ADD A,B
256 ADD A,20H
257 CALL EMAPC
258 PUSH AF
259 LD A,E
260 ADD A,EGRDX
261 LD E,A
262 POP AF
263 JP DRWCH
264
265 ; EGDRAW: the whole 16x4 grid, cursor cell inverse.
266 EGDRAW: LD B,0
267 EGD1: LD C,0
268 EGD2: PUSH BC
269 LD A,B
270 ADD A,A
271 ADD A,A
272 ADD A,B
273 ADD A,A

```

```

275 ADD A,EGRDY
276 LD D,A
277 LD A,C
278 ADD A,EGRDX
279 LD E,A
280 LD A,B
281 RLCA
282 RLCA
283 RLCA
284 RLCA
285 ADD A,C
286 ADD A,20H
287 CALL EMAPC
288 PUSH AF
289 LD A,(V_GY)
290 CP B
291 JR NZ,EGDN
292 LD A,(V_GX)
293 CP C
294 JR NZ,EGDN
295 LD C,GCINV
296 JR EGDC
297 EGDN: LD C,GCNORM
298 EGDC: POP AF
299 CALL DRWCH
300 POP BC
301 INC C
302 LD A,C
303 CP 16
304 JR C,EGD2
305 INC B
306 LD A,B
307 CP 4
308 JR C,EGD1
309 RET
310
311 ; EVDRAW: the value line -- tail window of V_ENTRY plus the cursor cell.
312 EVDRAW: LD A,(V_GLEN)
313 SUB 19
314 JR NC,EVD1
315 XOR A
316 EVD1: LD (V_TMP+1),A ; window start
317 LD B,A
318 LD A,(V_GLEN)
319 SUB B
320 LD (V_TMP),A ; cursor cell
321 LD B,0 ; cell
322 EVD2: PUSH BC
323 LD A,(V_TMP+1)
324 ADD A,B
325 LD HL,V_GLEN
326 CP (HL)
327 JR NC,EVDSP
328 LD E,A
329 LD D,0
330 LD HL,V_ENTRY
331 ADD HL,DE
332 LD A,(HL)
333 JR EVDC
334 EVDSP: LD A,' '
335 EVDC: LD C,GCNORM
336 PUSH AF
337 LD A,(V_TMP)
338 CP B
339 JR NZ,EVD3
340 LD C,GCINV
341 EVD3: POP AF
342 LD D,EVALY
343 LD E,B
344 CALL DRWCH
345 POP BC
346 INC B
347 LD A,B
348 CP 20
349 JR C,EVD2
350 RET
351
352 ; EADRAW: the action row, selected word highlighted.
353 EADRAW: CALL UCLRF
354 LD B,0
355 EAD1: PUSH BC
356 LD A,B
357 ADD A,A
358 LD E,A
359 LD D,0
360 LD HL,EAWTAB
361 ADD HL,DE
362 LD A,(HL)
363 INC HL
364 LD H,(HL)
365 LD L,A
366 PUSH HL
367 LD HL,EAXTAB
368 LD E,B
369 LD D,0
370 ADD HL,DE
371 LD E,(HL)
372 POP HL
373 LD D,FOOTY
374 LD A,(V_GY)
375 CP 4
376 JR NZ,EADN
377 LD A,(V_GX)
378 CP B
379 JR NZ,EADN
380 LD C,OPTHI
381 JR EADC
382 EADN: LD C,OPTFB
383 EADC: LD SYSTEM
384 POP BC
385 INC B
386 LD A,B
387 CP 5
388 JR C,EAD1
389 RET
390
391 EAWTAB: DW TCASE,TSPC,TDEL,TOK,TESC
392 EAXTAB: DB 2,38,66,92,116
393 TCASE: DB "CASE",0
394 TSPC: DB "SPC",0
395 TDEL: DB "DEL",0
396 TOK: DB "OK",0
397 TESC: DB "ESC",0
398

```

UI.INC

```

1 ; ui.inc -- screen scaffolding shared by every state.
2 ;
3 ; The layout is ten 8-pixel rows: title (y=1), eight list rows starting at
4 ; LISTY0, footer (y=FOOTY). List text goes through LINBUF + STRDIS at its
5 ; native 6-pixel pitch (26 columns); the selection cursor is a chevron in
6 ; the byte-aligned gutter column, drawn with the font.inc blitter so moving
7 ; it never needs the row's text again (directory rows would otherwise cost
8 ; a mailbox round trip apiece to repaint).
9
10 ; MSET: fill BC bytes at HL with A. Clobbers everything.
11 MSET: LD (HL),A
12 DEC BC
13 LD A,B
14 OR C
15 RET Z
16 LD D,H
17 LD E,L
18 INC DE
19 LDIR
20 RET
21
22 ; Clear the whole screen / the list area / the footer band.
23 UCLRA: LD HL,NORMEM
24 BC,LINES*BYTEPL
25 JR UCLR0
26 UCLRL: LD HL,NORMEM+9*BYTEPL
27 LD BC,64*BYTEPL
28 JR UCLR0
29 UCLRF: LD HL,NORMEM+72*BYTEPL
30 LD BC,8*BYTEPL
31 UCLR0: XOR A
32 JP MSET
33
34 ; UTITLE: clear the screen and show the title string at HL.
35 UTITLE: PUSH HL
36 CALL UCLRA
37 POP HL
38 LD DE,TITLEY*256+2
39 LD C,OPTFB
40 SYSTEM STRDIS
41 RET
42
43 ; UFOOT: replace the footer with the string at HL.
44 UFOOT: PUSH HL
45 CALL UCLRF
46 POP HL
47 LD DE,FOOTY*256+2
48 LD C,OPTFB
49 SYSTEM STRDIS
50 RET
51
52 ; UROWY: D = y scanline of list row B.
53 UROWY: LD A,B
54 ADD A,A
55 ADD A,A
56 ADD A,A
57 ADD A,LISTY0
58 LD D,A
59 RET
60
61 ; USHOWR: display LINBUF on list row B with STRDIS options C.
62 USHOWR: PUSH BC
63 CALL UROWY
64 POP BC
65 LD E,LISTX
66 LD HL,LINBUF
67 SYSTEM STRDIS
68 RET
69
70 ; UCRON / UCROFF: draw / erase the chevron cursor on list row B.
71 UCRON: LD A,'>'
72 JR UCRI
73 UCROFF: LD A,' '
74 UCRI: PUSH AF
75 CALL UROWY
76 POP AF
77 LD E,0
78 LD C,GCNORM
79 JP DRWCH
80
81 ; USTRLN: C = length of the NUL-terminated string at HL (HL ends on the
82 ; NUL). Clobbers A.
83 USTRLN: LD C,0
84 USL1: LD A,(HL)
85 OR A
86 RET Z
87 INC HL
88 INC C
89 JR USL1
90
91 ; UTAIL: copy the LAST (up to) B characters of the string at HL into
92 ; LINBUF, sanitized -- for paths too long to fit a row. Clobbers everything.
93 ; USTRLN returns the length in C and leaves B alone, so no save is needed
94 ; (and saving would clobber the length we just measured).
95 UTAIL: CALL USTRLN ; C = length, HL -> the NUL, B kept = max
96 LD A,C
97 B
98 JR C,UTAI1
99 LD A,B
100 UTAI1: LD E,A
101 LD D,0
102 OR A
103 SBC HL,DE
104 LD B,A
105 JP CPYSAN
106
107 ; UPECT: render A (0-100) as "NN%" into HEXBUF, NUL-terminated, no
108 ; leading zeros. Clobbers A,B,D,E,H.
109 UPECT: LD DE,HEXBUF
110 CP 100
111 JR C,UPC1
112 LD A,'1'
113 LD (DE),A
114 INC DE
115 LD A,'0'
116 LD (DE),A
117 INC DE
118 LD (DE),A
119 INC DE
120 JR UPC9
121 UPC1: LD B,'0'
122 UPC2: CP 10
123 JR C,UPC3
124 SUB 10
125 INC B
126 JR UPC2
127 UPC3: LD H,A
128 LD A,B
129 CP '0'
130 JR Z,UPC4
131 LD (DE),A
132 INC DE

```

```

133 UPC4: LD A,H
134 ADD A,'0'
135 LD (DE),A
136 INC DE
137 UPC9: LD A,'%'
138 LD (DE),A
139 INC DE
140 XOR A
141 LD (DE),A
142 RET
143

```

FONT.INC -- THE 5X7 BLITTER

```

1 ; font.inc -- byte-aligned 5x7 character blitter, with lowercase.
2 ;
3 ; The BIOS font (5x7, 7 bytes per glyph, pattern in the top 5 bits) covers
4 ; only 20H-63H -- no lowercase -- but SSIDs and Wifi passwords are case-
5 ; sensitive, so this module draws characters itself: BIOS glyph data for
6 ; 20H-63H, its own 26-glyph set for a-z. Cells are 8 pixels wide and drawn
7 ; at byte-aligned positions (2 bytes per scanline at 2bpp), which is what
8 ; makes the blit shift-free; STRDIS keeps its 6-pixel pitch everywhere that
9 ; doesn't need lowercase or inverse-video cells.
10 ;
11 ; Color spec byte: bits 1-0 = foreground pixel code, bits 3-2 = background
12 ; (GCNORM = white on black, GCINV = black on white for the cursor cell).
13
14 BIOSFNT EQU 08E4H ; system ROM font base ('!' = 08EBH etc.)
15
16 ; GADDR: HL = screen address of cell (D = y scanline, E = x byte pair).
17 ; Clobbers B,C.
18 GADDR: LD H,0
19 LD L,D
20 ADD HL,HL
21 ADD HL,HL
22 ADD HL,HL ; y*8
23 LD B,H
24 LD C,L
25 ADD HL,HL
26 ADD HL,HL ; y*32
27 ADD HL,BC ; y*40
28 LD B,0
29 LD C,E
30 ADD HL,BC
31 ADD HL,BC ; + x*2
32 LD BC,NORMEM
33 ADD HL,BC
34 RET
35
36 ; GLYPHA: HL = glyph address for character A (7 pattern bytes).
37 ; Unprintables become '.'. Clobbers A,D,E.
38 GLYPHA: CP 'a'
39 JR C,GLYB
40 CP 'z'+1
41 JR NC,GLYB
42 SUB 'a'
43 LD HL,LCFONT
44 JR GLYI
45 GLYB: CP 20H
46 JR C,GLYDOT
47 CP 64H
48 JR C,GLYOK
49 GLYDOT: LD A','
50 GLYOK: SUB 20H
51 LD HL,BIOSFNT
52 GLYI: LD E,A ; HL += A*7
53 LD D,0
54 PUSH HL

```

```

55 LD L,E
56 LD H,0
57 ADD HL,HL
58 ADD HL,HL
59 ADD HL,HL ; *8
60 OR A
61 SBC HL,DE ; *7
62 POP DE
63 ADD HL,DE
64 RET
65
66 ; EXPAND: pattern byte A -> D,E (8 pixels at 2bpp) using color spec C.
67 ; Preserves C. Clobbers A,B.
68 EXPAND: PUSH BC
69 LD B,8
70 LD DE,0
71 EXPL: SLA E
72 RL D
73 SLA E
74 RL D
75 RLCA
76 PUSH AF
77 LD A,C
78 JR C,EXPF
79 SRL A
80 SRL A ; background code
81 AND 3
82 OR E
83 LD E,A
84 POP AF
85 DJNZ EXPL
86 POP BC
87 RET
88
89 ; DRWCH: draw character A at cell (E = x byte pair 0-19, D = y scanline)
90 ; with color spec C. Draws 8 scanlines (glyph + one background line).
91 ; Clobbers everything incl. IX.
92 DRWCH: PUSH BC
93 PUSH DE
94 CALL GLYPHA
95 PUSH HL
96 POP IX
97 POP DE
98 CALL GADDR
99 POP BC
100 LD B,8
101 LD A,B
102 DRWR: DEC A
103 JR Z,DRWBL ; 8th line: background only
104 LD A,(IX+0)
105 INC IX
106 JR DRWX
107 DRWBL: XOR A
108 DRWX: PUSH BC

```

```

109 CALL EXPAND
110 POP BC
111 LD (HL),D
112 INC HL
113 LD (HL),E
114 LD DE,39
115 ADD HL,DE
116 DJNZ DRWR
117 RET
118
119 ; DRWSTR: draw the NUL-terminated string at HL from cell (E,D), color C,
120 ; at most B characters. Clobbers everything.
121 DRWSTR: LD A,(HL)
122 OR A
123 RET Z
124 PUSH HL
125 PUSH BC
126 PUSH DE
127 CALL DRWCH
128 POP DE
129 POP BC
130 POP HL
131 INC HL
132 INC E
133 DJNZ DRWSTR
134 RET
135
136 ; ---- lowercase glyphs, same format as the BIOS font ----
137 LCFONT: DB 000H,000H,070H,008H,078H,008H,078H ; a
138 DB 000H,000H,0F0H,008H,008H,008H,0F0H ; b
139 DB 000H,000H,078H,000H,008H,008H,078H ; c
140 DB 008H,008H,078H,008H,008H,008H,078H ; d
141 DB 000H,000H,070H,008H,0F8H,008H,070H ; e
142 DB 030H,048H,040H,0E0H,040H,040H,040H ; f
143 DB 000H,078H,008H,008H,078H,008H,070H ; g
144 DB 008H,008H,0F0H,008H,008H,008H,008H ; h
145 DB 020H,000H,060H,020H,020H,020H,070H ; i
146 DB 010H,000H,030H,010H,010H,090H,060H ; j
147 DB 080H,008H,090H,0A0H,0C0H,0A0H,090H ; k
148 DB 060H,020H,020H,020H,020H,020H,070H ; l
149 DB 000H,000H,0D0H,0A8H,0A8H,0A8H,0A8H ; m
150 DB 000H,000H,0F0H,008H,008H,008H,008H ; n
151 DB 000H,000H,070H,008H,008H,008H,070H ; o
152 DB 000H,0F0H,008H,008H,0F0H,008H,008H ; p
153 DB 000H,078H,008H,008H,078H,008H,008H ; q
154 DB 000H,000H,0B0H,0C8H,008H,008H,008H ; r
155 DB 000H,000H,078H,008H,070H,008H,0F0H ; s
156 DB 040H,040H,0E0H,040H,040H,048H,030H ; t
157 DB 000H,000H,008H,008H,008H,098H,068H ; u
158 DB 000H,000H,008H,008H,008H,050H,020H ; v
159 DB 000H,000H,008H,0A8H,0A8H,0A8H,050H ; w
160 DB 000H,000H,008H,050H,020H,050H,008H ; x
161 DB 000H,008H,008H,008H,078H,008H,070H ; y
162 DB 000H,000H,0F8H,010H,020H,040H,0F8H ; z
163

```

INPUT.INC (CONFIG'S OWN)

```

1 ; input.inc -- player 1 hand controller + the 24-key keypad, as events.
2 ;
3 ; The clients run DI/poll and the vertical-address feedback register is
4 ; lightpen-latched (useless as a frame source), so pacing is a calibrated
5 ; ~10 ms delay loop and all state is edge-detected against previous-state
6 ; bytes in RAM.
7 ;
8 ; Directions merge the stick with the keypad's physical arrow keys and
9 ; auto-repeat (~300 ms delay, ~80 ms rate). A diagonal keeps only the
10 ; lowest set bit, and -- the INTV port's lesson -- a direction CHANGE

```

```

11 ; without release goes through the repeat gate instead of firing at once,
12 ; so a thumb parked on a boundary cannot machine-gun the list.
13 ; Trigger and keypad keys are edge-only.
14
15 EVNONE EQU 0
16 EVUP EQU 1
17 EVDOWN EQU 2
18 EVLEFT EQU 3
19 EVRIGHT EQU 4
20 EVTRIG EQU 5

```

```

21 EVCE EQU 6 ; CE key (clear / cancel / backspace)
22 EVEQ EQU 7 ; = key (accept / edit)
23 EVDOT EQU 8 ; . key
24 EVD0 EQU 10H ; digits: EVD0+n
25
26 BUP EQU 01H ; SW0 bits, active high
27 BDOWN EQU 02H
28 BLEFT EQU 04H
29 BRIGHT EQU 08H
30 BTRIG EQU 10H
31
32 REPDEL EQU 30 ; auto-repeat: initial delay, ticks
33 REPRATE EQU 8 ; auto-repeat: rate, ticks
34
35 ; DELAY10: ~10 ms at 1.789 Mhz. Clobbers A,D,E.
36 DELAY10:
37 LD DE,745
38 DLY1: DEC DE
39 LD A,D
40 OR E
41 JR NZ,DLY1
42 RET
43
44 ; INPOLL: one input tick. Returns A = event (EVNONE if nothing).
45 ; Clobbers everything.
46 INPOLL: CALL DELAY10
47 IN A,(SW0)
48 LD B,A ; keep for the trigger bit
49 AND 0FH
50 LD C,A
51 IN A,(KEY2)
52 AND 1 ; keypad ^ key
53 JR Z,IPK1
54 SET 0,C
55 IN A,(KEY1)
56 AND 1 ; keypad v key
57 JR Z,IPK2
58 SET 1,C
59 IPK2: LD A,C
60 OR A
61 JR Z,IPNOD
62 LD E,A ; lowest set bit only
63 NEG
64 AND E
65 LD A,E
66 LD A,(V_PDIR)
67 CP E
68 JR Z,IPHELD
69 OR A ; anything held before?
70 LD A,E
71 LD (V_PDIR),A
72 LD A,REPDEL
73 LD (V_RPT),A
74 JR NZ,IPTRG
75 LD A,E ; changed without release: gate, no event
76 ; fresh press: emit
77 IPHELD: LD A,(V_RPT)
78 DEC A
79 LD (V_RPT),A
80 JR NZ,IPTRG
81 LD A,REPRATE
82 LD (V_RPT),A
83 LD A,E
84 IPDIR: LD D,EVUP ; bit 0..3 -> EVUP..EVRIGHT
85 IPDB: RRA
86 JR C,IPDG
87 INC D
88 JR IPDB
89 IPDG: LD A,D
90 RET
91

```

```

92 IPNOD: XOR A
93 LD (V_PDIR),A
94
95 ; ---- trigger, edge only -----
96 IPTRG: LD A,B
97 AND BTRIG
98 LD E,A
99 LD A,(V_PTRG)
100 LD D,A
101 LD A,E
102 LD (V_PTRG),A
103 OR A
104 JR Z,IPKEYS
105 LD A,D
106 OR A
107 JR NZ,IPKEYS
108 LD A,EVTRIG
109 RET
110
111 ; ---- keypad matrix, edge only -----
112 IPKEYS: LD HL,V_PKP
113 LD D,0 ; column index: KEY0..KEY3
114 IPKL: LD A,D
115 ADD A,KEY0
116 LD C,A
117 LD B,0
118 IN A,(C)
119 AND 3FH
120 LD E,A ; current
121 LD A,(HL)
122 CPL
123 AND E ; rising edges
124 LD (HL),E
125 LD C,A
126 LD A,D
127 CP 1 ; the arrow keys (KEY1/KEY2 bit 0) are
128 JR C,IPKM ; direction inputs, not keypad edges
129 CP 3
130 JR NC,IPKM
131 RES 0,C
132 IPKM: LD A,C
133 OR A
134 JR NZ,IPKDEC
135 INC HL
136 INC D
137 LD A,D
138 CP 4
139 JR C,IPKL
140 XOR A ; EVNONE
141 RET
142
143 IPKDEC: LD E,A ; lowest edge bit
144 NEG
145 AND E
146 LD B,0 ; bit index
147 IPKB: RRA
148 JR C,IPKG
149 INC B
150 JR IPKB
151 LD A,D ; table index = column*6 + bit
152 ADD A,A
153 ADD A,D
154 ADD A,A
155 ADD A,B
156 LD E,A
157 LD D,0
158 LD HL,KPTAB
159 ADD HL,DE
160 LD A,(HL)
161 RET
162

```

```

163 ; Keypad decode: MAME astrohome.cpp's matrix, active high.
164 ; Column KEY0 (rightmost) .. KEY3 (leftmost), bit 0 (bottom) .. 5 (top).
165 KPTAB: DB 0,0,0,0,0,EVEQ ; KEY0: % / x - + =
166 DB 0,0,EVD0+9,EVD0+6,EVD0+3,EVDOT ; KEY1: v CH 9 6 3 .
167 DB 0,0,EVD0+8,EVD0+5,EVD0+2,EVD0 ; KEY2: ^ MS 8 5 2 0
168 DB 0,0,EVD0+7,EVD0+4,EVD0+1,EVCE ; KEY3: C MR 7 4 1 CE
169
170 ; INWAIT: block until an event arrives. Returns it in A.
171 INWAIT: CALL INPOLL
172 OR A
173 JR Z,INWAIT
174 RET
175
176 ; UPAUSE: wait DE ticks (~10 ms each); any input ends the wait early.
177 ; Returns A = the event, or EVNONE if the wait ran out.
178 UPAUSE: PUSH DE
179 CALL INPOLL
180 POP DE
181 OR A
182 RET NZ
183 DEC DE
184 LD A,D
185 OR E
186 JR NZ,UPAUSE
187 XOR A
188 RET
189

```

FUJIDISP.INC

```

1 ; fujidisp.inc -- shared screen helpers for the FujiNet clients.
2 ;
3 ; Vendorsd copy of fujinet-firmware/pico/astrocade/testrom/fujidisp.inc.
4 ; Expects LINBUF/HEXBUF/OPTFB from the including file. STRDIS register form
5 ; (Nutting p.37): HL = string, C = options, D = Y, E = X.
6 ;
7 ; Copy B bytes from HL into LINBUF, 0-terminated, sanitized for STRDIS:
8 ; lowercase folds to uppercase, anything else outside 20H-63H becomes '.'.
9 ; (STRDIS treats 64H-7FH as control codes). Stops early at a 0 byte.
10 CPYSAN: LD DE,LINBUF
11 CPYS1: LD A,(HL)
12 OR A
13 JR Z,CPYS9
14 CP 61H ; 'a'
15 JR C,CPYS2
16 CP 7BH ; 'z'+1
17 JR NC,CPYS2
18 SUB 20H
19 CPYS2: CP 20H
20 JR C,CPYS3
21 CP 64H
22 JR C,CPYS4
23 CPYS3: LD A,','
24 CPYS4: LD (DE),A
25 INC HL
26 INC DE
27 DJNZ CPYS1
28 CPYS9: XOR A
29 LD (DE),A
30 RET
31
32 ; Display LINBUF at x=2, y=A.
33 SHOWLIN:
34 LD D,A
35 LD E,2
36 LD C,OPTFB
37 LD HL,LINBUF
38 SYSTEM STRDIS

```

```

39 RET
40
41 ; Display A as two hex digits at x=E, y=D.
42 HEXAT: PUSH DE
43 LD C,A
44 RRCA
45 RRCA
46 RRCA
47 CALL HEXDIG (HEXBUF),A
48 LD A,C
49 CALL HEXDIG (HEXBUF+1),A
50 LD A,C
51 CALL HEXDIG (HEXBUF+2),A
52 LD A,C
53 XOR A
54 LD A,C
55 POP DE
56 LD C,OPTFB
57 LD HL,HEXBUF
58 SYSTEM STRDIS
59 RET
60
61 HEXDIG: AND 0FH
62 ADD A,'0'
63 CP '9'+1
64 RET C
65 ADD A,'A'-'9'-1
66 RET
67

```


APPENDIX D: THE C DEMO

The complete `c-demo` project of Chapter 6, from `listings/c-demo/` — built with `z88dk's +astrocde` target and verified end to end under MAME against a live fujinet-pc (the transaction log read `dev=70 cmd=C4 ... err=0 reply=06 rxlen=240`). The build recipe is printed in Chapter 6; `checkrom.py` is the same layout checker the assembly clients vendor.

FUJINET.H

```
1 /* fujinet.h -- the FujiNet cartridge mailbox for Z88DK C.
2 *
3 * A C rendering of the contract in fujilib.inc, which in turn mirrors the
4 * single source of truth, fujinet-firmware/pico/astrocade/firmware/
include/
5 * fujii_mailbox.h. Everything here is a READ: the Astrocade cartridge edge
6 * has no write strobe, so the console talks by reading hotspot pages where
7 * the low address byte IS the payload, and the cartridge answers by
8 * repainting the ROM bytes it serves.
9 *
10 * All addresses are console addresses (cart offset + 0x2000).
11 */
12
13 #ifndef FUJINET_H
14 #define FUJINET_H
15
16 /* ---- reply bytes (repainted ROM) ----- */
17
18 #define FN_RDATA ((volatile unsigned char *)0x3b00) /* 256-byte slice */
19
20 /* Reply byte at a CONSTANT offset. sccz80 (z88dk 3bd06cad, 2026-07)
21 * miscompiles a constant subscript on a cast-constant pointer --
22 * FN_RDATA[2] becomes the constant 2, silently -- while a variable
23 * subscript and this dereference-of-a-sum form both compile correctly.
24 * Use FN_RDATA[i] only when i is a variable.
25 */
26 #define FN_RDATA_B(i) (*(volatile unsigned char *) (0x3b00 + (i)))
27
28 #define FN_ACKSEQ ((volatile unsigned char *)0x3c00) /* echoes SEQ
29 */
30 #define FN_STATUS ((volatile unsigned char *)0x3c01) /* bit0 link up
31 */
32 #define FN_ERRBYTE ((volatile unsigned char *)0x3c02) /* last
33 fb_status */
34 #define FN_REPLY ((volatile unsigned char *)0x3c03) /* 0x06 ACK/0x15
35 */
36 #define FN_RXLEN_LO ((volatile unsigned char *)0x3c04) /* reply length
37 */
38 #define FN_RXLEN_HI ((volatile unsigned char *)0x3c05)
39 #define FN_BOOT_STATE ((volatile unsigned char *)0x3c06)
40 #define FN_BOOT_PCT ((volatile unsigned char *)0x3c07) /* 0-100
41 */
42 #define FN_BOOT_ERR ((volatile unsigned char *)0x3c08)
43 #define FN_MAGIC0 ((volatile unsigned char *)0x3c09) /* 'F'
44 */
45 #define FN_MAGIC1 ((volatile unsigned char *)0x3c0a) /* 'N'
46 */
47 #define FN_PROTO ((volatile unsigned char *)0x3c0b) /* version, 2
48 */
49
50 #define FN_SLICE_ECHO ((volatile unsigned char *)0x3c0c) /* published
51 last */
52
53 #define FN_STATUS_LINK 0x01
54 #define FN_STATUS_BUSY 0x02
55
56 /* ---- hotspot pages (console -> cart; reading is writing) ----- */
57
58 #define FN_REGSEL ((volatile unsigned char *)0x3d00) /* +reg: arm
59 */
60 #define FN_REGDATA ((volatile unsigned char *)0x3e00) /* +val: deliver
61 */
62 #define FN_DATA ((volatile unsigned char *)0x3f00) /* +val: TX byte
63 */
64 #define FN_SWAP ((volatile unsigned char *)0x3dfe) /* armed only
65 */
66 #define FN_BANKSEL ((volatile unsigned char *)0x3d80) /* +page (v2)
67 */
68
69 /* ---- mailbox registers ----- */
70
71 #define FN_REG_DEVICE 0x00 /* FujiBus device id
72 */
73 #define FN_REG_CMD 0x01 /* FujiBus command id
74 */
75 #define FN_REG_NPARAM 0x02 /* parameters in the TX stream
76 */
77 #define FN_REG_DATA_RST 0x05 /* any value: rewind the TX pointer
78 */
79 #define FN_REG_RXSLICE 0x06 /* which reply slice FN_RDATA shows
80 */
81 #define FN_REG_SEQ 0x10 /* nonzero, != ACKSEQ: launch transaction
82 */
83 #define FN_REG_BOOTLOCK 0x11 /* 0xB5 arms the ROM swap
84 */
85 #define FN_BOOTLOCK_MAGIC 0xb5
86
87 /* ---- devices and limits ----- */
88
89 #define FN_DEV_FUJII 0x70 /* the FujiNet itself
90 */
91 #define FN_DEV_NET 0x71 /* network unit N1:
92 */
93
94 #define FN_TX_MAX 320 /* TX stream: params + payload
95 */
96 #define FN_RX_MAX 1024 /* reply: 4 slices of 256
```

```

*/
72
73 /* ---- transaction error codes (FN_ERRBYTE / fn_commit result) ----- */
74
75 #define FN_ERR_OK          0
76 #define FN_ERR_NOLINK      1
77 #define FN_ERR_TIMEOUT     2
78 #define FN_ERR_BADFRAME    3
79 #define FN_ERR_TOOBIG      4
80 #define FN_ERR_CLIENT_TIMEOUT 0xff /* fn_commit gave up waiting */
81
82 #define FN_ACK              0x06
83 #define FN_NAK              0x15
84
85 /* ---- Fuji device (0x70) commands used here ----- */
86
87 #define FUJI_GET_ADAPTERCONFIG_EXTENDED 0xc4
88
89 /* ---- network device (0x71) commands ----- */
90
91 #define NET_OPEN            0x4f /* 'O' */
92 #define NET_CLOSE          0x43 /* 'C' */
93 #define NET_READ           0x52 /* 'R' */
94 #define NET_STATUS         0x53 /* 'S' */
95 #define NET_WRITE          0x57 /* 'W' */
96
97 #define NET_MODE_RDWR 0x0c /* also HTTP GET */
98 #define NET_TRANS_NONE 0
99
100 /* ---- the transport (fujinet.c) -----
101  * fn_commit and fn_slice return FN_ERR_OK, an FN_ERR_* code, or
102  * FN_ERR_CLIENT_TIMEOUT. A timeout quantum is 65,536 polls of the ACK
103  * byte -- about two seconds in the assembly library, a little longer in
104  * C. The sequence number is always derived from the cartridge's own
105  * FN_ACKSEQ, never from a local counter: console RESET starts this
106  * program but not the cartridge.
107  */
108
109 unsigned char fn_check(void); /* 1 if the 'F','N' magic is there */
110
111 void fn_regwr(unsigned char reg, unsigned char val);
112 void fn_tx_byte(unsigned char v);
113 void fn_tx_parb(unsigned char v); /* one-byte parameter */
114 void fn_tx_parw(unsigned int v); /* two-byte parameter */
115 void fn_tx_str(const char *s); /* payload, no NUL */
116 void fn_begin(unsigned char device, unsigned char command,
117               unsigned char nparam);
118 unsigned char fn_commit(unsigned char quanta);
119 unsigned char fn_slice(unsigned char n);
120
121 /* flat 0-1023 view over the four reply slices */
122 void rx_home(void); /* call after every transaction */
123 unsigned char rx_getb(unsigned int off);
124 unsigned int rx_getw(unsigned int off);
125 void rx_strn(char *dst, unsigned int off, unsigned char max);
126
127 /* N: device round trip, netcat's shape */
128 unsigned char net_open(const char *devicespec, unsigned char mode,
129                       unsigned char translation);
130 unsigned char net_close(void);
131 unsigned char net_status(void); /* fills the three cells below */
132 extern unsigned int net_avail;

```

```

132 extern unsigned char net_connected;
133 extern unsigned char net_devstatus;
134 unsigned char net_read(unsigned int len); /* reply lands in FN_RDATA */
135 extern unsigned int net_rxlen; /* captured after net_read */
136 unsigned char net_write(const unsigned char *buf, unsigned char len);
137
138 #endif
139

```

FUJINET.C

```
1 /* fujinet.c -- the mailbox transport, in C.
2  *
3  * Every routine is a direct rendering of its fujilib.inc counterpart; the
4  * comments there apply here too. The one C-specific trick is fn_sink: a
5  * hotspot is "written" by READING it, and assigning the volatile read to a
6  * volatile byte guarantees the compiler emits the load and never elides
7  * or reorders it. Instruction fetches in between are harmless -- the
8  * program executes from 0x2000-0x3aff and the hotspot pages start at
9  * 0x3d00, so ordinary code never trips them.
10 */
11
12 #include "fujinet.h"
13
14 static volatile unsigned char fn_sink;
15
16 #define FN_HOT(page, v) (fn_sink = (page)[v])
17
18 unsigned int net_avail;
19 unsigned char net_connected;
20 unsigned char net_devstatus;
21 unsigned int net_rxlen;
22
23 static unsigned char cur_slice;
24
25 unsigned char fn_check(void)
26 {
27     return FN_MAGIC0 == 'F' && FN_MAGIC1 == 'N';
28 }
29
30 void fn_regwr(unsigned char reg, unsigned char val)
31 {
32     FN_HOT(FN_REGSEL, reg); /* arm the register */
33     FN_HOT(FN_REGDATA, val); /* deliver the value */
34 }
35
36 void fn_tx_byte(unsigned char v)
37 {
38     FN_HOT(FN_DATA, v);
39 }
40
41 void fn_tx_parb(unsigned char v)
42 {
43     fn_tx_byte(1); /* size byte */
44     fn_tx_byte(v);
45 }
46
47 void fn_tx_parw(unsigned int v)
48 {
49     fn_tx_byte(2);
50     fn_tx_byte(v & 0xff); /* little-endian */
51     fn_tx_byte(v >> 8);
52 }
53
54 void fn_tx_str(const char *s)
55 {
56     while (*s)
57         fn_tx_byte(*s++);
58 }
59
60 void fn_begin(unsigned char device, unsigned char command,
61               unsigned char nparam)
62 {
63     fn_regwr(FN_REG_DATA_RST, 0);
64     fn_regwr(FN_REG_DEVICE, device);
65     fn_regwr(FN_REG_CMD, command);
66     fn_regwr(FN_REG_NPARAM, nparam);
67 }
68
69 /* Commit the transaction and wait for the cartridge to acknowledge.
70  * The next sequence number is the cart's own persisted ACKSEQ + 1,
71  * wrapping 255 -> 1 (0 is reserved as "never used").
72  */
73 unsigned char fn_commit(unsigned char quanta)
74 {
75     unsigned char want;
76     unsigned int spin;
77
78     want = FN_ACKSEQ + 1;
79     if (want == 0)
80         want = 1;
81     fn_regwr(FN_REG_SEQ, want);
82     while (quanta-- > 0) {
83         spin = 0;
84         do {
85             if (FN_ACKSEQ == want)
86                 return FN_ERRBYTE;
87             } while (--spin != 0); /* 65,536 polls per quantum */
88     }
89     return FN_ERR_CLIENT_TIMEOUT;
90 }
91
92 /* Select reply slice n and wait for the repaint: the cart publishes
93  * FN_SLICE_ECHO last, so the slice is whole once it echoes.
94  */
95 unsigned char fn_slice(unsigned char n)
96 {
97     unsigned int spin;
98
99     fn_regwr(FN_REG_RXSLICE, n);
100     spin = 0;
101     do {
102         if (FN_SLICE_ECHO == n)
103             return FN_ERR_OK;
104     } while (--spin != 0);
105     return FN_ERR_CLIENT_TIMEOUT;
106 }
107
108 /* ---- flat reply window ----- */
109
110 /* Every transaction makes the cart republish slice 0; call rx_home after
111  * each one so the cache agrees.
112  */
113 void rx_home(void)
114 {
115     cur_slice = 0;
116 }
```

```

117
118 unsigned char rx_getb(unsigned int off)
119 {
120     unsigned char s = off >> 8;
121
122     if (s != cur_slice) {
123         cur_slice = s;
124         fn_slice(s);
125     }
126     return FN_RDATA[off & 0xff];
127 }
128
129 unsigned int rx_getw(unsigned int off)
130 {
131     unsigned char lo = rx_getb(off);
132
133     return lo | ((unsigned int)rx_getb(off + 1) << 8);
134 }
135
136 void rx_strn(char *dst, unsigned int off, unsigned char max)
137 {
138     unsigned char c;
139
140     while (max--) {
141         c = rx_getb(off++);
142         if (c == 0)
143             break;
144         *dst++ = c;
145     }
146     *dst = 0;
147 }
148
149 /* ---- N: device round trip (netcat's shape) -----
*/
150
151 unsigned char net_open(const char *devicespec, unsigned char mode,
152                        unsigned char translation)
153 {
154     unsigned char r;
155
156     fn_begin(FN_DEV_NET, NET_OPEN, 2);
157     fn_tx_parb(mode);
158     fn_tx_parb(translation);
159     fn_tx_str(devicespec);
160     r = fn_commit(8); /* 16 s: HTTPS handshakes are slow */
161     rx_home();
162     if (r != FN_ERR_OK)
163         return r;
164     return FN_REPLY == FN_ACK ? FN_ERR_OK : FN_NAK;
165 }
166
167 unsigned char net_close(void)
168 {
169     unsigned char r;
170
171     fn_begin(FN_DEV_NET, NET_CLOSE, 0);
172     r = fn_commit(3);
173     rx_home();
174     return r;
175 }
176
177 unsigned char net_status(void)
178 {
179     unsigned char r;
180
181     fn_begin(FN_DEV_NET, NET_STATUS, 2);
182     fn_tx_parb(0);
183     fn_tx_parb(0);
184     r = fn_commit(4);
185     rx_home();
186     if (r != FN_ERR_OK)
187         return r;
188     if (FN_REPLY != FN_ACK)
189         return FN_NAK;
190     net_avail = FN_RDATA_B(0) | ((unsigned int)FN_RDATA_B(1) << 8);
191     net_connected = FN_RDATA_B(2);
192     net_devstatus = FN_RDATA_B(3);
193     return FN_ERR_OK;
194 }
195
196 unsigned char net_read(unsigned int len)
197 {
198     unsigned char r;
199
200     fn_begin(FN_DEV_NET, NET_READ, 1);
201     fn_tx_byte(2); /* one two-byte parameter: the length */
202     fn_tx_byte(len & 0xff);
203     fn_tx_byte(len >> 8);
204     r = fn_commit(8);
205     rx_home();
206     if (r != FN_ERR_OK)
207         return r;
208     if (FN_REPLY != FN_ACK)
209         return FN_NAK;
210     /* capture NOW: the length bytes belong to this transaction only */
211     net_rxlen = FN_RXLEN_LO | ((unsigned int)FN_RXLEN_HI << 8);
212     return FN_ERR_OK;
213 }
214
215 unsigned char net_write(const unsigned char *buf, unsigned char len)
216 {
217     unsigned char r;
218     unsigned char n = len;
219
220     if (n == 0)
221         return FN_ERR_OK;
222     fn_begin(FN_DEV_NET, NET_WRITE, 1);
223     fn_tx_byte(2); /* the count rides twice: as this */
224     fn_tx_byte(len); /* parameter, and as the payload the */
225     fn_tx_byte(0); /* cartridge measures */
226     while (n--)
227         fn_tx_byte(*buf++);
228     r = fn_commit(4);
229     rx_home();
230     if (r != FN_ERR_OK)
231         return r;
232     return FN_REPLY == FN_ACK ? FN_ERR_OK : FN_NAK;
233 }
234

```

TEXT.H

```
1 /* text.h -- byte-aligned 5x7 text on the Astrocade screen, in C.
2 *
3 * The C rendering of CONFIG's font.inc: BIOS glyph data for 0x20-0x63
4 * plus a 26-glyph lowercase set, drawn in 8-pixel cells at byte-aligned
5 * positions (two bytes per scanline at 2bpp), which is what makes the
6 * blit shift-free. 20 columns; a text row is 8 scanlines.
7 */
8
9 #ifndef TEXT_H
10 #define TEXT_H
11
12 void txt_clear(void); /* also the screen RAM init */
13 void txt_putc(unsigned char x, unsigned char y, char c);
14 void txt_puts(unsigned char x, unsigned char y, const char *s);
15 void txt_hex(unsigned char x, unsigned char y, unsigned char v);
16
17 #endif
18
```

TEXT.C

```
1 /* text.c -- byte-aligned 5x7 text, white (color 3) on black (color 0). */
2
3 #include "text.h"
4
5 #define SCREEN ((unsigned char *)0x4000)
6 #define BYTEPL 40 /* bytes per scanline */
7 #define LINES 80 /* visible scanlines: 10 rows */
8 #define BIOSFNT ((const unsigned char *)0x08e4) /* 5x7, 7 bytes/glyph */
9
10 /* Lowercase glyphs, same format as the BIOS font (from CONFIG's
11 * font.inc: pattern in the top 5 bits, 7 rows).
12 */
13 static const unsigned char lcfnt[26 * 7] = {
14     0x00,0x00,0x70,0x08,0x78,0x88,0x78, /* a */
15     0x80,0x80,0xf0,0x88,0x88,0x88,0xf0, /* b */
16     0x00,0x00,0x78,0x80,0x80,0x80,0x78, /* c */
17     0x08,0x08,0x78,0x88,0x88,0x88,0x78, /* d */
18     0x00,0x00,0x70,0x88,0xf8,0x80,0x70, /* e */
19     0x30,0x48,0x40,0xe0,0x40,0x40,0x40, /* f */
20     0x00,0x78,0x88,0x88,0x78,0x08,0x70, /* g */
21     0x80,0x80,0xf0,0x88,0x88,0x88,0x88, /* h */
22     0x20,0x00,0x60,0x20,0x20,0x20,0x70, /* i */
23     0x10,0x00,0x30,0x10,0x10,0x90,0x60, /* j */
24     0x80,0x80,0x90,0xa0,0xc0,0xa0,0x90, /* k */
25     0x60,0x20,0x20,0x20,0x20,0x20,0x70, /* l */
26     0x00,0x00,0xd0,0xa8,0xa8,0xa8,0xa8, /* m */
27     0x00,0x00,0xf0,0x88,0x88,0x88,0x88, /* n */
28     0x00,0x00,0x70,0x88,0x88,0x88,0x70, /* o */
29     0x00,0xf0,0x88,0x88,0xf0,0x80,0x80, /* p */
30     0x00,0x78,0x88,0x88,0x78,0x08,0x08, /* q */
31     0x00,0x00,0xb0,0xc8,0x50,0x80,0x80, /* r */
32     0x00,0x00,0x78,0x80,0x70,0x08,0xf0, /* s */
33     0x40,0x40,0xe0,0x40,0x40,0x48,0x30, /* t */
34     0x00,0x00,0x88,0x88,0x88,0x98,0x68, /* u */
35     0x00,0x00,0x88,0x88,0x88,0x50,0x20, /* v */

```

```
36     0x00,0x00,0x88,0xa8,0xa8,0xa8,0x50, /* w */
37     0x00,0x00,0x88,0x50,0x20,0x50,0x88, /* x */
38     0x00,0x88,0x88,0x88,0x78,0x08,0x70, /* y */
39     0x00,0x00,0xf8,0x10,0x20,0x40,0xf8, /* z */
40 };
41
42 void txt_clear(void)
43 {
44     unsigned char *p = SCREEN;
45     unsigned int n = LINES * BYTEPL;
46
47     while (n--)
48         *p++ = 0;
49 }
50
51 /* Expand the top 5 bits of a pattern byte to 2bpp, foreground color 3. */
52 static unsigned int expand(unsigned char pat)
53 {
54     unsigned int px = 0;
55     unsigned char b;
56
57     for (b = 0; b < 8; ++b) {
58         px <= 2;
59         if (pat & 0x80)
60             px |= 3;
61         pat <<= 1;
62     }
63     return px;
64 }
65
66 void txt_putc(unsigned char x, unsigned char y, char c)
67 {
68     const unsigned char *glyph;
69     unsigned char *cell;
70     unsigned char r;

```

```

71 unsigned int px;
72
73 if (c >= 'a' && c <= 'z')
74     glyph = lcfont + (c - 'a') * 7;
75 else {
76     if (c < 0x20 || c >= 0x64)
77         c = '.';
78     glyph = BIOSFONT + (c - 0x20) * 7;
79 }
80 cell = SCREEN + (unsigned int)y * 8 * BYTEPL + x * 2;
81 for (r = 0; r < 8; ++r) {
82     px = (r < 7) ? expand(glyph[r]) : 0;
83     cell[0] = px >> 8;
84     cell[1] = px & 0xff;
85     cell += BYTEPL;
86 }
87 }
88
89 void txt_puts(unsigned char x, unsigned char y, const char *s)
90 {
91     while (*s && x < 20)
92         txt_putc(x++, y, *s++);
93 }
94
95 void txt_hex(unsigned char x, unsigned char y, unsigned char v)
96 {
97     static const char digits[] = "0123456789ABCDEF";
98
99     txt_putc(x, y, digits[v >> 4]);
100    txt_putc(x + 1, y, digits[v & 0xf]);
101 }
102

```

MAIN.C

```

1 /* main.c -- FUJINET C DEMO for the Bally Astrocade.
2 *
3 * The C rendering of "first contact": check the cartridge magic, run one
4 * GET_ADAPTERCONFIG_EXTENDED transaction, and show the adapter's SSID,
5 * firmware version and IP address on screen. Built with z88dk's
6 * +astrocde target -- see the Makefile for the recipe and the handbook's
7 * C chapter for the caveats.
8 *
9 * The crt0 runs with interrupts disabled (CRT_ENABLE_EIDI=1) and they
10 * stay off for the program's whole life: with I = 0, Z80 refresh strays
11 * land in the on-board ROM and never touch the mailbox hotspots.
12 */
13
14 #include "fujinet.h"
15 #include "text.h"
16
17 void port_out(unsigned char port, unsigned char val); /* support.asm */
18
19 #define LINES 80 /* visible scanlines: 10 text rows */
20
21 /* COLSET order, descending ports 7..0: color 3,2,1,0 for the left
22 * palette, then the same for the right. 3 white, 2 red, 1 gray,
23 * 0 black; byte = (hue << 3) | luminance.
24 */
25 static const unsigned char palette[8] = {

```

```

26     0x07, 0x52, 0x03, 0x00,
27     0x07, 0x52, 0x03, 0x00,
28 };
29
30 static void screen_init(void)
31 {
32     unsigned char i;
33
34     for (i = 0; i < 8; ++i)
35         port_out(7 - i, palette[i]);
36     port_out(0x09, 0); /* HORCB: one palette all the way over */
37     port_out(0x0a, LINES * 2); /* VERBL: blank below the text */
38     port_out(0x0e, 8); /* INMOD */
39     txt_clear();
40 }
41
42 static char buf[33];
43
44 int main(void)
45 {
46     unsigned char r;
47
48     screen_init();
49     txt_puts(0, 0, "FUJINET C DEMO");
50

```

```

51     if (!fn_check()) {
52         txt_puts(0, 2, "NO FUJINET CART");
53         for (;;) ;
54     }
55
56     txt_puts(0, 1, "PROTO");
57     txt_hex(6, 1, FN_PROTO);
58     txt_puts(9, 1, (FN_STATUS & FN_STATUS_LINK) ? "LINK UP" : "NO LINK");
59
60     /* One whole transaction: no parameters, no payload, 240-byte reply. */
61     fn_begin(FN_DEV_FUJI, FUJI_GET_ADAPTERCONFIG_EXTENDE, 0);
62     r = fn_commit(8);
63     rx_home();
64     if (r != FN_ERR_OK) {
65         txt_puts(0, 3, "ERR");
66         txt_hex(4, 3, r);
67         for (;;) ;
68     }
69     if (FN_REPLY != FN_ACK) {
70         txt_puts(0, 3, "NAK");
71         for (;;) ;
72     }
73
74     rx_strn(buf, 0, 20);      /* ssid[33] at offset 0          */
75     txt_puts(0, 3, "SSID");
76     txt_puts(0, 4, buf);
77
78     rx_strn(buf, 125, 15);    /* fn_version[15] at offset 125 */
79     txt_puts(0, 5, "FW");
80     txt_puts(3, 5, buf);
81
82     rx_strn(buf, 140, 16);    /* sLocalIP[16] at offset 140    */
83     txt_puts(0, 6, "IP");
84     txt_puts(3, 6, buf);
85
86     txt_puts(0, 8, "OK");
87     for (;;) ;
88     return 0;
89 }
90

```

SUPPORT.ASM

```

1 ; support.asm -- the two things C cannot say on this target.
2 ;
3 ; 1. fputc_cons_native: z88dk's classic runtime insists every target has a
4 ;   console driver; the incomplete +astrocde target ships none, and this
5 ;   demo paints screen RAM itself, so a stub satisfies the linker.
6 ; 2. port_out: sccz80 has no port intrinsics here. Arguments are pushed
7 ;   left to right and widened to 16 bits, so at entry the value sits at
8 ;   SP+2 and the port at SP+4; the caller cleans up.
9
10     SECTION code_clib
11
12     PUBLIC fputc_cons_native
13 fputc_cons_native:
14     ret
15
16     PUBLIC _port_out
17 _port_out:

```

```

18     ld     hl,2
19     add    hl,sp
20     ld     a,(hl)      ; value (low byte of the widened arg)
21     inc    hl
22     inc    hl
23     ld     c,(hl)      ; port
24     out    (c),a
25
26     ret

```

BUILD.SH

```
1 #!/usr/bin/env bash
2 # build.sh -- compile the FUJINET C DEMO with z88dk's +astrocde target.
3 #
4 # Usage: ./build.sh
5 #
6 # Output: build/cdemo.bin, always exactly 8192 bytes: the compiled code,
7 # zero-padded, mailbox pages clear, "FUJI" claim signature at 0x1CFC so
8 # the mailbox stays alive when the image is booted over the network.
9 #
10 # Environment:
11 # Z88DK=path          z88dk checkout (default ~/Workspace/z88dk); zcc is
12 #                      taken from $Z88DK/bin and ZCCCFG derived from it
13 #
14 # The +astrocde target is marked incomplete upstream: it ships a crt0 and
15 # HVGLIB.H but no target clib, so the build links the generic z80 clib and
16 # support.asm supplies the console-driver stub the runtime insists on.
17 # The pragmas place BSS at 0x4D00 (above the 80 visible scanlines) and the
18 # stack at 0x4FC0, and CRT_ENABLE_EIDI=1 makes the crt0 run D1 first --
19 # the mailbox contract.
20 #
21 set -euo pipefail
22 cd "$(dirname "$0")"
23 #
24 WINDOW=8192
25 CLAIM_OFF=$((0x1CFC))
26 ROM_TOP=$((0x1B00))
27 #
28 Z88DK=${Z88DK:-$HOME/Workspace/z88dk}
29 export ZCCCFG="$Z88DK/lib/config"
30 export PATH="$Z88DK/bin:$PATH"
31 ZCC="$Z88DK/bin/zcc"
32 [ -x "$ZCC" ] || { echo "build.sh: no zcc at $ZCC; set Z88DK=" >&2; exit
33 1; }
34 #
35 mkdir -p build
36 #
37 "$ZCC" +astrocde main.c text.c fujinet.c support.asm \
38 -o build/cdemo.raw -m \
39 -pragma-define:CRT_ENABLE_EIDI=1 \
40 -pragma-define:REGISTER_SP=0x4fc0 \
41 -pragma-define:CRT_ORG_BSS=0x4d00 \
42 -pragma-define:CRT_MODEL=1
43 #
44 size=$(wc -c < build/cdemo.raw)
45 if [ "$size" -gt "$ROM_TOP" ]; then
46     echo "build.sh: cdemo is $size bytes, over the $ROM_TOP ROM top" >&2
47     exit 1
48 fi
49 #
50 # BSS and stack must stay inside 0x4D00-0x4FBF.
51 bss_end=$(awk '/__BSS_END/ { print strtonum("0x" $3); exit }' \
52 build/cdemo.map 2>/dev/null || echo 0)
53 if [ "$bss_end" -gt "$(0x4FC0)" ]; then
54     printf "build.sh: BSS ends at %x, past the 0x4FC0 stack floor\n" \
55         "$bss_end" >&2
56     exit 1
57 fi
```

```
58 # Pad to the full window, clear above ROM_TOP, stamp the claim.
59 {
60     cat build/cdemo.raw
61     head -c $((WINDOW - size)) /dev/zero
62 } > build/cdemo.bin
63 printf 'FUJI' | dd of=build/cdemo.bin bs=1 seek=$CLAIM_OFF \
64     conv=notrunc status=none
65 rm -f build/cdemo.raw
66 #
67 python3 tools/checkrom.py build/cdemo.bin
68 echo "build.sh: code+data $size of $ROM_TOP bytes"
69 #
```




A COMMUNITY PRODUCT OF THE FUJINET PROJECT